

My LeetCode Submissions - @rahul-chilukuri

[Download PDF](#)[Follow @TheShubham99 on GitHub](#)[Star on GitHub](#)[View Source Code](#)

210 Course Schedule II (link)

Description

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `[0,1]`

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0.

Example 2:

Input: `numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`

Output: `[0,2,1,3]`

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both course 1 and 2. So one correct course order is `[0,1,2,3]`. Another correct ordering is `[0,2,1,3]`.

Example 3:

Input: `numCourses = 1, prerequisites = []`

Output: `[0]`

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= numCourses * (numCourses - 1)`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `ai != bi`
- All the pairs `[ai, bi]` are **distinct**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        if(prerequisites.length < 1) {
            return IntStream.range(0,numCourses).toArray();
        }

        List<Integer>[] graph = new ArrayList[numCourses];
        for(int i=0; i < numCourses; i++) {
            graph[i]=new ArrayList<>();
        }
        for(int[] prerequisite: prerequisites) {
            graph[prerequisite[1]].add(prerequisite[0]);
        }

        int[] visited = new int[numCourses]; // 0 = unvisited, 1 = visiting, 2 = visited
        List<Integer> courseOrder = new LinkedList<>();

        for(int i=0; i < numCourses; i++) {
            if(visited[i] == 2) continue;
            if(hasCycle(graph, visited, courseOrder, i)) {
                return new int[0];
            }
        }
        return courseOrder.stream().mapToInt(i -> i).toArray();
    }

    private boolean hasCycle(List<Integer>[] graph, int[] visited, List<Integer> order, int course) {
        if(visited[course] == 1) return true;
        if(visited[course] == 2) return false;
        visited[course] = 1;
        for(int neighbor: graph[course]) {
            if(hasCycle(graph, visited, order, neighbor)) return true;
        }
        visited[course] = 2;
        order.addFirst(course);
        return false;
    }
}
```

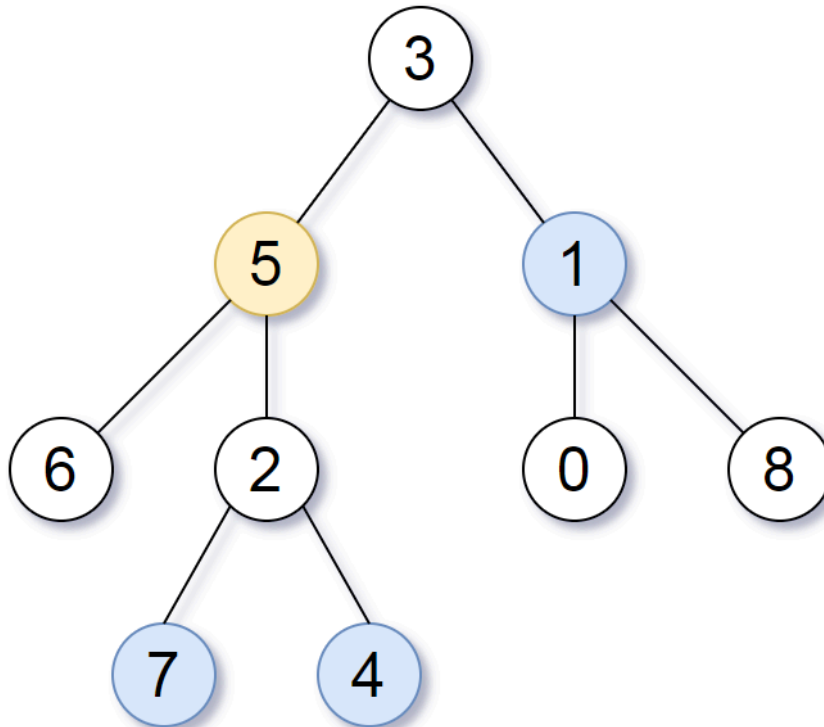
893 All Nodes Distance K in Binary Tree (link)

Description

Given the `root` of a binary tree, the value of a target node `target`, and an integer `k`, return *an array of the values of all nodes that have a distance k from the target node*.

You can return the answer in **any order**.

Example 1:



Input: `root = [3,5,1,6,2,0,8,null,null,7,4]`, `target = 5`, `k = 2`

Output: `[7,4,1]`

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1

Example 2:

Input: `root = [1]`, `target = 1`, `k = 3`

Output: `[]`

Constraints:

- The number of nodes in the tree is in the range $[1, 500]$.
- $0 \leq \text{Node.val} \leq 500$
- All the values `Node.val` are **unique**.
- `target` is the value of one of the nodes in the tree.
- $0 \leq k \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 */
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { val = x; }
}

class Solution {
    Map<TreeNode, TreeNode> parentMap = new HashMap<>();
    Set<TreeNode> visited = new HashSet<>();

    TreeNode target = null;
    List<Integer> result = new ArrayList<>();
    public List<Integer> distanceK(TreeNode root, TreeNode target, int k) {
        this.target = target;
        dfs(root, null, k, false);
        return result;
    }

    void dfs(TreeNode root, TreeNode parent, int distance, boolean found) {
        if(root == null) return;
        parentMap.put(root, parent);

        if(found) {
            if(visited.contains(root) || distance < 0) return;
            if(distance == 0) {
                result.add(root.val);
                return;
            }
            visited.add(root);
            dfs(root.left, root, distance-1, true);
            dfs(root.right, root, distance-1, true);
            if (parent != null)
                dfs(parent, parentMap.get(parent), distance-1, true);
        } else {
            if(root == target) {
                dfs(root, parent, distance, true);
            } else {
                dfs(root.left, root, distance, false);
                dfs(root.right, root, distance, false);
            }
        }
    }
}
```

417 Pacific Atlantic Water Flow (link)

Description

There is an $m \times n$ rectangular island that borders both the **Pacific Ocean** and **Atlantic Ocean**. The **Pacific Ocean** touches the island's left and top edges, and the **Atlantic Ocean** touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the **height above sea level** of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is **less than or equal to** the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a **2D list of grid coordinates** `result` where `result[i] = [ri, ci]` denotes that rain water can flow from cell (r_i, c_i) to **both** the Pacific and Atlantic oceans.

Example 1:

Pacific Ocean					
Pacific Ocean	1	2	2	3	5
	3	2	3	4	4
	2	4	5	3	1
	6	7	1	4	5
	5	1	1	2	4
Atlantic Ocean					

Input: `heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]`

Output: `[[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]`

Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:

[0,4]: [0,4] -> Pacific Ocean
[0,4] -> Atlantic Ocean
[1,3]: [1,3] -> [0,3] -> Pacific Ocean
[1,3] -> [1,4] -> Atlantic Ocean
[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean
[1,4] -> Atlantic Ocean
[2,2]: [2,2] -> [1,2] -> [0,2] -> Pacific Ocean
[2,2] -> [2,3] -> [2,4] -> Atlantic Ocean
[3,0]: [3,0] -> Pacific Ocean
[3,0] -> [4,0] -> Atlantic Ocean
[3,1]: [3,1] -> [3,0] -> Pacific Ocean
[3,1] -> [4,1] -> Atlantic Ocean
[4,0]: [4,0] -> Pacific Ocean

[4,0] -> Atlantic Ocean
Note that there are other possible paths for these cells to flow to the Pacific and Atlantic oceans.

Example 2:

Input: heights = [[1]]

Output: [[0,0]]

Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights}[r].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights}[r][c] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    private final int[][] DIRS = new int[][]{{-1,0},{1,0},{0,-1},{0,1}};
    private int rows;
    private int cols;
    boolean[][] atlantic;
    boolean[][] pacific;
    List<List<Integer>> bothReachable = new ArrayList<>();

    public List<List<Integer>> pacificAtlantic(int[][] heights) {
        List<List<Integer>> result = new ArrayList<>();
        if(heights.length == 0 || heights[0].length == 0) return result;

        rows = heights.length;
        cols = heights[0].length;
        pacific = new boolean[rows][cols];
        atlantic = new boolean[rows][cols];

        //check pacific
        for(int i=0; i < rows; i++) dfs(heights, i, 0, pacific, false);
        for(int i=0; i < cols; i++) dfs(heights, 0, i, pacific, false);

        //check atlantic
        for(int i=0; i < rows; i++) dfs(heights, i, cols-1, atlantic, true);
        for(int i=0; i < cols; i++) dfs(heights, rows-1, i, atlantic, true);

        return bothReachable;
    }

    void dfs(int[][] landHeights, int row, int col, boolean[][] visited, boolean checkPacific) {
        if (visited[row][col]) return;
        visited[row][col] = true;
        if(checkPacific && pacific[row][col]) {
            bothReachable.add(List.of(row, col));
        }

        for(int[] dir: DIRS) {
            int nrow = row+dir[0];
            int ncol = col+dir[1];

            if((nrow < 0 || nrow >= rows || ncol < 0 || ncol >= cols)
                || visited[nrow][ncol]
                || landHeights[nrow][ncol] < landHeights[row][col])
                continue;

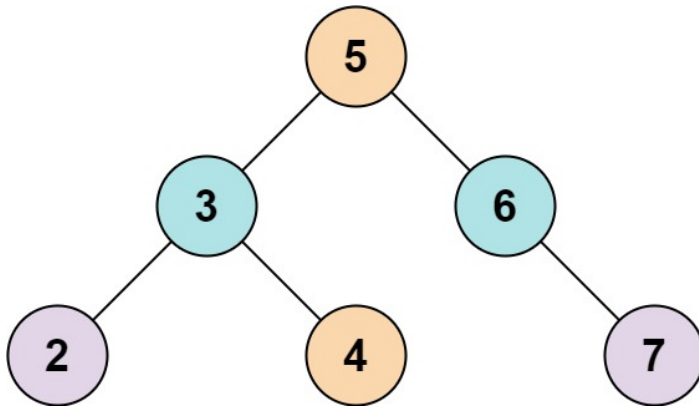
            dfs(landHeights, nrow, ncol, visited, checkPacific);
        }
    }
}
```

653 Two Sum IV - Input is a BST (link)

Description

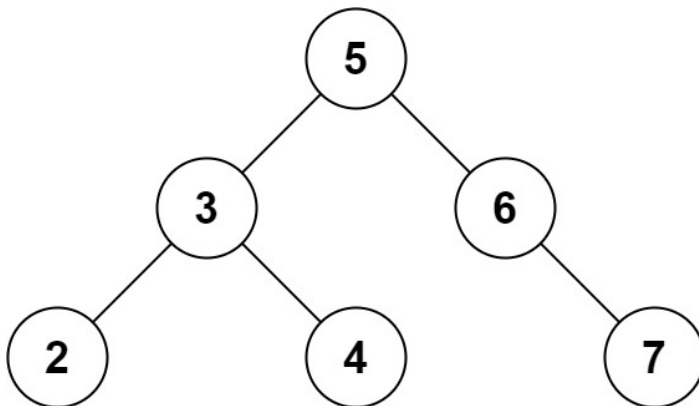
Given the root of a binary search tree and an integer k , return `true` if there exist two elements in the BST such that their sum is equal to k , or `false` otherwise.

Example 1:



Input: root = [5,3,6,2,4,null,7], k = 9
Output: true

Example 2:



Input: root = [5,3,6,2,4,null,7], k = 28
Output: false

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$
- root is guaranteed to be a **valid** binary search tree.
- $-10^5 \leq k \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean findTargetII(TreeNode root, int target) {
        Stack<Integer> stack = new Stack<>();
        inOrder(root, stack);
        int left=0, right=stack.size()-1;
        while(left < right) {
            int sum = stack.get(left)+stack.get(right);
            if(sum == target)
                return true;
            else if(sum > target)
                right--;
            else
                left++;
        }
        return false;
    }

    void inOrder(TreeNode root, Stack<Integer> st) {
        if(root == null)
            return;
        inOrder(root.left, st);
        st.push(root.val);
        inOrder(root.right, st);
    }

    public boolean findTarget(TreeNode root, int target) {
        if(root == null) return false;
        Stack<TreeNode> front = new Stack<>();
        Stack<TreeNode> back = new Stack<>();
        push(front, root, true);
        push(back, root, false);

        int left = next(front, true);
        int right = next(back, false);
        while(left < right) {
            int temp = left+right;
            if(temp < target) {
                left = next(front, true);
            } else if(temp > target) {
                right = next(back, false);
            } else {
                return true;
            }
        }
        return false;
    }

    int next(Stack<TreeNode> stack, boolean left) {
        TreeNode node = stack.pop();
        TreeNode newNode = left ? node.right: node.left;
    }
}
```

```
        push(stack, newNode, left);
        return node.val;
    }

    void push(Stack<TreeNode> stack, TreeNode node, boolean left) {
        while(node != null) {
            stack.push(node);
            node = left ? node.left: node.right;
        }
    }
}
```


310 Minimum Height Trees (link)

Description

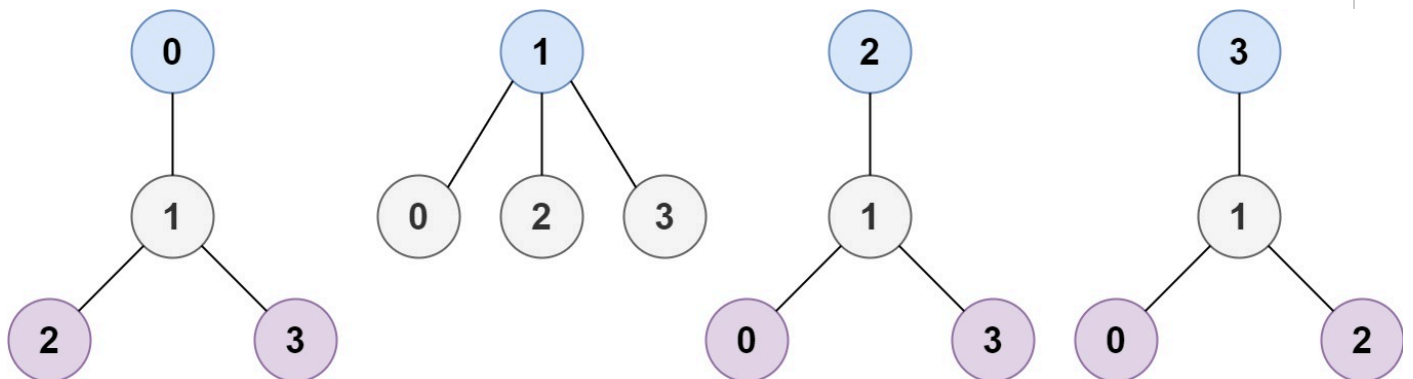
A tree is an undirected graph in which any two vertices are connected by *exactly* one path. In other words, any connected graph without simple cycles is a tree.

Given a tree of n nodes labelled from 0 to $n - 1$, and an array of $n - 1$ edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root. When you select a node x as the root, the result tree has height h . Among all possible rooted trees, those with minimum height (i.e. $\min(h)$) are called **minimum height trees** (MHTs).

Return a list of all **MHTs'** root labels. You can return the answer in **any order**.

The **height** of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:

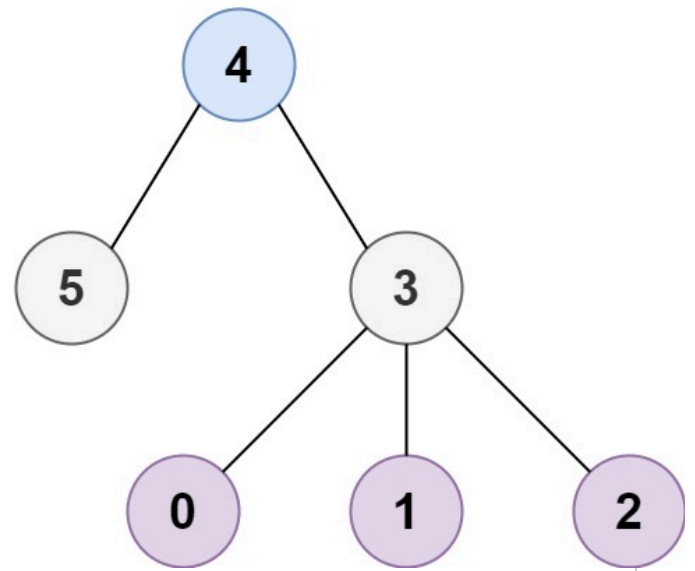
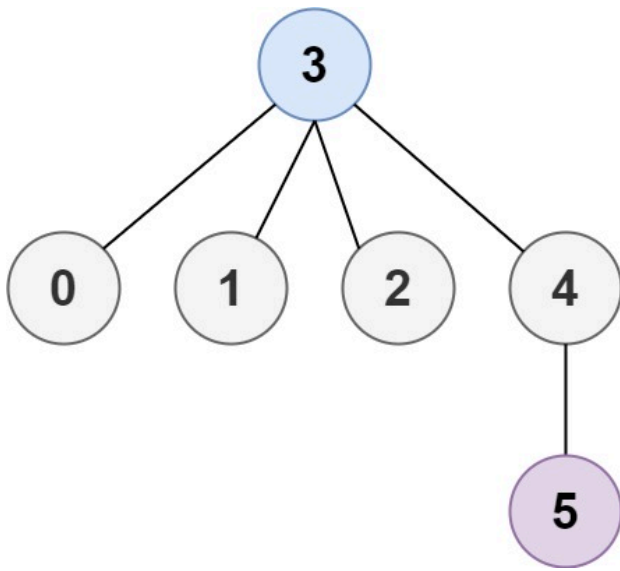


Input: $n = 4$, $\text{edges} = [[1,0],[1,2],[1,3]]$

Output: $[1]$

Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the center of the tree.

Example 2:



Input: $n = 6$, $edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]$
Output: $[3,4]$

Constraints:

- $1 \leq n \leq 2 * 10^4$
- $edges.length == n - 1$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is **guaranteed** to be a tree and there will be **no repeated** edges.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findMinHeightTrees(int n, int[][] edges) {
        if(n <= 2)
            return IntStream.range(0,n).boxed().collect(Collectors.toList());

        List<List<Integer>> graph = new ArrayList<>();
        int[] degree = new int[n];
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

        for(int[] edge: edges){
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
            degree[edge[0]]++;
            degree[edge[1]]++;
        }

        Queue<Integer> leaves = new ArrayDeque<>();
        for(int i=0; i<n;i++)
            if(degree[i] == 1)
                leaves.offer(i);

        int remainingNodes = n;
        while(remainingNodes > 2) {
            int size = leaves.size();
            remainingNodes -= size;

            for(int i=0; i < size; i++){
                int leaf = leaves.poll();
                for(int neighbor: graph.get(leaf)) {
                    if(--degree[neighbor] == 1)
                        leaves.offer(neighbor);
                }
            }
        }
        return new ArrayList<>(leaves);
    }
}
```

695 Max Area of Island (link)

Description

You are given an $m \times n$ binary matrix `grid`. An island is a group of 1's (representing land) connected **4-directionally** (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The **area** of an island is the number of cells with a value 1 in the island.

Return *the maximum area of an island in grid*. If there is no island, return 0.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: `grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,0,0,0,0,0,0,0,0,0]]`
Output: 6
Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: `grid = [[0,0,0,0,0,0,0,0]]`
Output: 0

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- `grid[i][j]` is either 0 or 1.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int m=0;
    int n=0;
    boolean[][] seen;
    int[][] grid;

    public int maxAreaOfIslandI(int[][] grid) {
        this.grid = grid;
        this.m = grid.length;
        this.n = grid[0].length;
        this.seen = new boolean[m][n];

        int maxArea=0;
        for(int r=0; r<m; r++)
            for(int c=0; c<n; c++)
                maxArea = Math.max(maxArea, areaOfAnIsland(r,c));
        return maxArea;
    }

    private int areaOfAnIsland(int r, int c) {
        if(r<0 || r>=m || c<0 || c>=n || grid[r][c] == 0 || seen[r][c])
            return 0;

        seen[r][c] = true;
        return 1+areaOfAnIsland(r+1, c)
            +areaOfAnIsland(r-1, c)
            +areaOfAnIsland(r, c-1)
            +areaOfAnIsland(r, c+1);
    }

    public int maxAreaOfIsland(int[][] grid) {
        int[][] DIRS = {{-1,0}, {1,0}, {0, 1}, {0,-1}};
        int maxArea = 0;
        int m = grid.length;
        int n = grid[0].length;
        Queue<Integer> qu = new ArrayDeque();

        for(int i=0; i < m; i++) {
            for(int j=0; j < n; j++) {
                if(grid[i][j] == 0) continue;
                qu.offer(i*n+j);
                grid[i][j] = 0;

                int localMax = 1;
                while(!qu.isEmpty()) {
                    int coords = qu.poll();
                    int r = coords/n;
                    int c = coords%n;

                    for(int[] dir: DIRS) {
                        int nr = r+dir[0];
                        int nc = c+dir[1];

                        if(nr >= m || nr < 0 || nc >= n || nc < 0 || grid[nr][nc] == 0) continue;
                        qu.offer(nr*n+nc);
                        grid[nr][nc] = 0;
                        localMax++;
                    }
                }
                maxArea = Math.max(localMax, maxArea);
            }
        }
        return maxArea;
    }
}
```


[1923 Sentence Similarity III \(link\)](#)

Description

You are given two strings `sentence1` and `sentence2`, each representing a **sentence** composed of words. A sentence is a list of **words** that are separated by a **single** space with no leading or trailing spaces. Each word consists of only uppercase and lowercase English characters.

Two sentences `s1` and `s2` are considered **similar** if it is possible to insert an arbitrary sentence (*possibly empty*) inside one of these sentences such that the two sentences become equal. **Note** that the inserted sentence must be separated from existing words by spaces.

For example,

- `s1 = "Hello Jane"` and `s2 = "Hello my name is Jane"` can be made equal by inserting "my name is" between "Hello" and "Jane" in `s1`.
- `s1 = "Frog cool"` and `s2 = "Frogs are cool"` are **not** similar, since although there is a sentence "s are" inserted into `s1`, it is not separated from "Frog" by a space.

Given two sentences `sentence1` and `sentence2`, return **true** if `sentence1` and `sentence2` are **similar**. Otherwise, return **false**.

Example 1:

Input: `sentence1 = "My name is Haley"`, `sentence2 = "My Haley"`

Output: true

Explanation:

`sentence2` can be turned to `sentence1` by inserting "name is" between "My" and "Haley".

Example 2:

Input: `sentence1 = "of"`, `sentence2 = "A lot of words"`

Output: false

Explanation:

No single sentence can be inserted inside one of the sentences to make it equal to the other.

Example 3:

Input: `sentence1 = "Eating right now"`, `sentence2 = "Eating"`

Output: true

Explanation:

`sentence2` can be turned to `sentence1` by inserting "right now" at the end of the sentence.

Constraints:

- $1 \leq \text{sentence1.length}, \text{sentence2.length} \leq 100$
- `sentence1` and `sentence2` consist of lowercase and uppercase English letters and spaces.
- The words in `sentence1` and `sentence2` are separated by a single space.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean areSentencesSimilar(String sentence1, String sentence2) {
        String[] arr1 = sentence1.split(" ");
        String[] arr2 = sentence2.split(" ");
        int n1 = arr1.length;
        int n2 = arr2.length;
        if(n1 < n2) return areSentencesSimilar(sentence2, sentence1);

        int l = -1;
        int r1 = n1, r2 = n2;

        //front and back
        do l++;while(l < n2 && arr1[l].equals(arr2[l]));
        do {r1--; r2--;} while(r2 > -1 && arr1[r1].equals(arr2[r2]));

        return r2 < l;
    }
}
```

[207 Course Schedule \(link\)](#)

Description

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`
Output: `true`
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output: `false`
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<Integer>[] graph = new ArrayList[numCourses];
        for(int i=0; i < numCourses; i++) {
            graph[i]=new ArrayList<>();
        }
        for(int[] prerequisite: prerequisites) {
            graph[prerequisite[1]].add(prerequisite[0]);
        }

        boolean[] visited = new boolean[numCourses];
        boolean[] path = new boolean[numCourses];

        for(int i=0; i < numCourses; i++) {
            if(!visited[i]) {
                if(hasCycle(graph, visited, path, i)) {
                    return false;
                }
            }
        }
        return true;
    }

    boolean hasCycle(List<Integer>[] graph, boolean[] visited, boolean[] path, int course) {
        if(path[course]) return true;
        if(visited[course]) return false;

        path[course] = true;
        for(int childCourse: graph[course]) {
            if(hasCycle(graph, visited, path, childCourse)) {
                return true;
            }
        }
        path[course] = false;
        visited[course] = true;
        return false;
    }
}
```

[1582 Design Browser History \(link\)](#)

Description

You have a **browser** of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history **at most** steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history **at most** steps.

Example:

Input:

```
["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]  
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],
```

Output:

```
[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetc
```

Explanation:

```
BrowserHistory browserHistory = new BrowserHistory("leetcode.com");  
browserHistory.visit("google.com");      // You are in "leetcode.com". Visit "google.com"  
browserHistory.visit("facebook.com");    // You are in "google.com". Visit "facebook.com"  
browserHistory.visit("youtube.com");     // You are in "facebook.com". Visit "youtube.com"  
browserHistory.back(1);                  // You are in "youtube.com", move back to "facebook.com" return  
browserHistory.back(1);                  // You are in "facebook.com", move back to "google.com" return  
browserHistory.forward(1);               // You are in "google.com", move forward to "facebook.com" ret  
browserHistory.visit("linkedin.com");    // You are in "facebook.com". Visit "linkedin.com"  
browserHistory.forward(2);               // You are in "linkedin.com", you cannot move forward any step  
browserHistory.back(2);                  // You are in "linkedin.com", move back two steps to "facebook  
browserHistory.back(7);                  // You are in "google.com", you can move back only one step to
```

Constraints:

- 1 <= homepage.length <= 20
- 1 <= url.length <= 20
- 1 <= steps <= 100
- homepage and url consist of '.' or lower case English letters.
- At most 5000 calls will be made to visit, back, and forward.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class BrowserHistory {

    Stack<String> history;
    Stack<String> future;
    public BrowserHistory(String homepage) {
        history = new Stack<>();
        history.push(homepage);
        future = new Stack<>();
    }

    public void visit(String url) {
        history.push(url);
        future.clear();
    }

    public String back(int steps) {
        while(steps > 0 && history.size() > 1) {
            future.push(history.pop());
            steps--;
        }
        return history.peek();
    }

    public String forward(int steps) {
        while(steps > 0 && future.size() > 0) {
            history.push(future.pop());
            steps--;
        }
        return history.peek();
    }
}

/**
 * Your BrowserHistory object will be instantiated and called as such:
 * BrowserHistory obj = new BrowserHistory(homepage);
 * obj.visit(url);
 * String param_2 = obj.back(steps);
 * String param_3 = obj.forward(steps);
 */
```

[438 Find All Anagrams in a String \(link\)](#)

Description

Given two strings s and p , return an array of all the start indices of p 's anagrams in s . You may return the answer in **any order**.

Example 1:

Input: $s = \text{"cbaebabacd"}, p = \text{"abc"}$

Output: $[0,6]$

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: $s = \text{"abab"}, p = \text{"ab"}$

Output: $[0,1,2]$

Explanation:

The substring with start index = 0 is "ab", which is an anagram of "ab".

The substring with start index = 1 is "ba", which is an anagram of "ab".

The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

- $1 \leq s.length, p.length \leq 3 * 10^4$
- s and p consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        int[] pcounts = new int[26];
        int[] scounts = new int[26];

        char[] parr = p.toCharArray();
        char[] sarr = s.toCharArray();

        int sLen = sarr.length;
        int pLen = parr.length;

        List<Integer> result = new ArrayList<>();
        if(pLen > sLen) return result;

        for(int i=0; i < pLen; i++) {
            pcounts[parr[i]-'a']++;
            scounts[sarr[i]-'a']++;
        }

        if(areSame(pcounts, scounts)) result.add(0);

        for(int i = pLen; i < sLen; i++) {
            scounts[sarr[i-pLen]-'a']--;
            scounts[sarr[i]-'a']++;
            if(areSame(pcounts, scounts))
                result.add(i-pLen+1);
        }
        return result;
    }

    private boolean areSame(int[] x, int[] y){
        for(int i = 0; i < 26; i++){
            if(x[i] != y[i])
                return false;
        }

        return true;
    }
}
```

[128 Longest Consecutive Sequence \(link\)](#)

Description

Given an unsorted array of integers `nums`, return *the length of the longest consecutive elements sequence*.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`

Output: 9

Example 3:

Input: `nums = [1,0,1,2]`

Output: 3

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    //key - num in nums
    //value[0] - root of key
    //value[1] - rank of root
    Map<Integer, int[]> parent = null;

    public int longestConsecutive(int[] nums) {
        if(nums.length < 2)
            return nums.length;
        parent = new HashMap<>();

        int result = 1;
        for(int num: nums) {
            Integer r = 1;
            if(!parent.containsKey(num)) {
                find(num);
                if( parent.containsKey(num-1)){
                    r = Math.max(r , union(num, num-1));
                }
                if( parent.containsKey(num+1)) {
                    r = Math.max(r , union(num, num+1));
                }

                if(r == 1) continue;

                //find parent of num and get it's rank
                result = Math.max(r, result);
            }
        }

        return result;
    }

    private int find(int x) {
        int[] xParent = parent.get(x);
        if(xParent == null || x == xParent[0]) {
            if(xParent == null)
                parent.put(x, new int[]{x, 1});
            return x;
        } else {
            xParent[0] = find(xParent[0]);
            return xParent[0];
        }
    }

    private int union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if( rootX != rootY) {
            int[] rx = parent.get(rootX);
            int[] ry = parent.get(rootY);
            if(rx[1] >= ry[1]) {
                rx[1] += ry[1];
                parent.put(rootY, rx);
                return rx[1];
            } else {
                ry[1] += rx[1];
                parent.put(rootX, ry);
                return ry[1];
            }
        }
        return parent.get(rootX)[1];
    }
}
```

```
}  
  
// class Solution {  
//     public int longestConsecutive(int[] nums) {  
//         if(nums.length == 0)  
//             return 0;;  
  
//         Arrays.sort(nums);  
//         int longest = 1;  
//         int current = 1;  
//         for(int i=1; i < nums.length; i++) {  
//             if(nums[i] != nums[i-1]) {  
//                 if(nums[i] == nums[i-1]+1) {  
//                     current++;  
//                     longest = Math.max(current, longest);  
//                 } else {  
//                     current = 1;  
//                 }  
//             }  
//         }  
//         return longest;  
//     }  
// }
```

187 Repeated DNA Sequences (link)

Description

The **DNA sequence** is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'.

- For example, "ACGAATTCCG" is a **DNA sequence**.

When studying **DNA**, it is useful to identify repeated sequences within the DNA.

Given a string s that represents a **DNA sequence**, return all the **10-letter-long** sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in **any order**.

Example 1:

```
Input: s = "AAAAACCCCCAAAAACCCCCCAAAAGGGTTT"
Output: ["AAAAACCCCC", "CCCCCAAAA"]
```

Example 2:

```
Input: s = "AAAAAAAAAAAA"
Output: ["AAAAAAAAAA"]
```

Constraints:

- $1 \leq s.length \leq 10^5$
- $s[i]$ is either 'A', 'C', 'G', or 'T'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<String> findRepeatedDnaSequences(String s) {
        int L=10, n = s.length();
        Set<String> sequences = new HashSet<>();
        Set<String> result = new HashSet<>();
        for(int i =0; i < n-9; i++) {
            String sequence = s.substring(i, i+10);
            if(!sequences.add(sequence)) {
                result.add(sequence);
            }
        }
        return new ArrayList<>(result);
    }
}
```

460 LFU Cache (link)

Description

Design and implement a data structure for a [Least Frequently Used \(LFU\)](#) cache.

Implement the LFUCache class:

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the **least frequently used** key before inserting a new item. For this problem, when there is a **tie** (i.e., two or more keys with the same frequency), the **least recently used** key would be invalidated.

To determine the least frequently used key, a **use counter** is maintained for each key in the cache. The key with the smallest **use counter** is the least frequently used key.

When a key is first inserted into the cache, its **use counter** is set to 1 (due to the put operation). The **use counter** for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]
```

Explanation

```
// cnt(x) = the use counter for key x
// cache=[] will show the last used order for tiebreakers (leftmost element is most recent)
LFUCache lfu = new LFUCache(2);
lfu.put(1, 1); // cache=[1,_], cnt(1)=1
lfu.put(2, 2); // cache=[2,1], cnt(2)=1, cnt(1)=1
lfu.get(1);    // return 1
               // cache=[1,2], cnt(2)=1, cnt(1)=2
lfu.put(3, 3); // 2 is the LFU key because cnt(2)=1 is the smallest, invalidate 2.
               // cache=[3,1], cnt(3)=1, cnt(1)=2
lfu.get(2);    // return -1 (not found)
lfu.get(3);    // return 3
               // cache=[3,1], cnt(3)=2, cnt(1)=2
lfu.put(4, 4); // Both 1 and 3 have the same cnt, but 1 is LRU, invalidate 1.
               // cache=[4,3], cnt(4)=1, cnt(3)=2
lfu.get(1);    // return -1 (not found)
lfu.get(3);    // return 3
               // cache=[3,4], cnt(4)=1, cnt(3)=3
lfu.get(4);    // return 4
               // cache=[4,3], cnt(4)=2, cnt(3)=3
```

Constraints:

- $1 \leq \text{capacity} \leq 10^4$
- $0 \leq \text{key} \leq 10^5$
- $0 \leq \text{value} \leq 10^9$
- At most $2 * 10^5$ calls will be made to get and put.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.*;

class DLL {
    public DLL prev, next;
    public int key, val, freq;

    DLL(int key, int val) {
        this.key = key;
        this.val = val;
        this.freq = 1;
    }
}

class DLinkedList {
    DLL head, tail;
    int size;

    DLinkedList() {
        head = new DLL(0, 0); // dummy head
        tail = new DLL(0, 0); // dummy tail
        head.next = tail;
        tail.prev = head;
        size = 0;
    }

    // Add node right after head (most recent)
    void addToFront(DLL node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
        size++;
    }

    // Remove a given node
    void remove(DLL node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
        size--;
    }

    // Remove the least recently used node (just before tail)
    DLL removeTail() {
        if (size > 0) {
            DLL node = tail.prev;
            remove(node);
            return node;
        }
        return null;
    }

    boolean isEmpty() {
        return size == 0;
    }
}

public class LFUCache {
    int capacity;
    int minFreq;
    Map<Integer, DLL> cache; // key -> node
    Map<Integer, DLinkedList> freqMap; // freq -> list of nodes with this freq

    public LFUCache(int capacity) {
        this.capacity = capacity;
        this.minFreq = 0;
    }
}
```

```

        cache = new HashMap<>();
        freqMap = new HashMap<>();
    }

    public int get(int key) {
        DLL node = cache.get(key);
        if (node == null) return -1;
        updateFrequency(node);
        return node.val;
    }

    public void put(int key, int value) {
        if (capacity == 0) return;

        if (cache.containsKey(key)) {
            DLL node = cache.get(key);
            node.val = value;
            updateFrequency(node);
        } else {
            if (cache.size() == capacity) {
                DLinkedList minFreqList = freqMap.get(minFreq);
                DLL toRemove = minFreqList.removeTail();
                cache.remove(toRemove.key);
            }

            DLL newNode = new DLL(key, value);
            cache.put(key, newNode);
            minFreq = 1;
            freqMap.putIfAbsent(1, new DLinkedList());
            freqMap.get(1).addToFront(newNode);
        }
    }

    private void updateFrequency(DLL node) {
        int oldFreq = node.freq;
        DLinkedList oldList = freqMap.get(oldFreq);
        oldList.remove(node);

        if (oldList.isEmpty()) {
            freqMap.remove(oldFreq);
            if (minFreq == oldFreq) minFreq++;
        }

        node.freq++;
        freqMap.putIfAbsent(node.freq, new DLinkedList());
        freqMap.get(node.freq).addToFront(node);
    }
}

```


[91 Decode Ways \(link\)](#)

Description

You have intercepted a secret message encoded as a string of numbers. The message is **decoded** via the following mapping:

```
"1" -> 'A'
"2" -> 'B'
...
"25" -> 'Y'
"26" -> 'Z'
```

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode.

Given a string *s* containing only digits, return the **number of ways** to **decode** it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a **32-bit** integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: *s* = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- $1 \leq s.length \leq 100$
- s contains only digits and may contain leading zero(s).

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int numDecodings(String s) {
        if(s == null || s.trim().length() == 0 || s.trim().charAt(0) == '0')
            return 0;

        int length = s.length();
        char prevDigit=s.charAt(0), digit;
        int first = 1;
        int second = 1;

        for(int i=2; i <= length; i++) {
            int third = 0;
            digit = s.charAt(i-1);
            if(digit != '0') {
                third +=second;
            }

            if(prevDigit != '0') {
                int pc = (digit-'0')+(prevDigit-'0')*10;
                if(pc > 9 && pc < 27)
                    third += first;
            }
            prevDigit = digit;
            first = second;
            second = third;
        }
        return second;
    }
}
```

[121 Best Time to Buy and Sell Stock \(link\)](#)

Description

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: `prices = [7,6,4,3,1]`

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxProfit(int[] prices) {
        //from day 1 check the max number in feature
        //only traverse forward, so min - max
        int maxProfit = 0;
        int minPrice = prices[0];
        for(int i = 1 ; i < prices.length; i++) {
            minPrice = Math.min(minPrice, prices[i]);
            maxProfit = Math.max(prices[i] - minPrice, maxProfit);
        }
        return maxProfit;
    }
}
```

79 Word Search [\(link\)](#)

Description

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` *if* `word` *exists in the grid*.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: `board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "ABCCED"`
Output: `true`

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: `board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "SEE"`
Output: `true`

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCB"
Output: false

Constraints:

- m == board.length
- n = board[i].length
- 1 <= m, n <= 6
- 1 <= word.length <= 15
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean exist(char[][] board, String word) {
        int rows = board.length;
        int cols = board[0].length;

        char[] arr = word.toCharArray();
        int wordLength = arr.length;

        //board has less characters than the word
        //each cell can only be used once
        if(wordLength > rows*cols) return false;

        int[] counts = new int[128];
        for(int i=0; i < wordLength; i++)
            counts[arr[i]]++;

        List<int[]> listCounts = new ArrayList<>();
        for(int i=0; i < 128; i++) {
            if(counts[i] == 0) {
                listCounts.add(new int[]{i, -2}); //to avoid
                continue;
            };
            listCounts.add(new int[]{i, counts[i]});
        }
        listCounts.sort((a,b)->(a[1]-b[1]));
        for(int i=0; i < wordLength; i++) {
            if(listCounts.get(arr[i])[1] == -2) continue;
            listCounts.get(arr[i])[1]--;
            if(listCounts.get(i)[1] < 0) return false;
        }

        for(int i=0; i < rows; i++)
            for(int j=0; j < cols; j++)
                if(board[i][j] == arr[0] && backtrack(board, arr, wordLength, rows, cols, i,j, 0))
                    return true;
        return false;
    }

    private boolean backtrack(char[][] board, char[] arr, int wordLength, int rows, int cols, int row, int col, int index) {
        if(index >= wordLength)
            return true;
        if( row < 0 || row >= rows || col < 0 || col >= cols || arr[index] != board[row][col])
            return false;

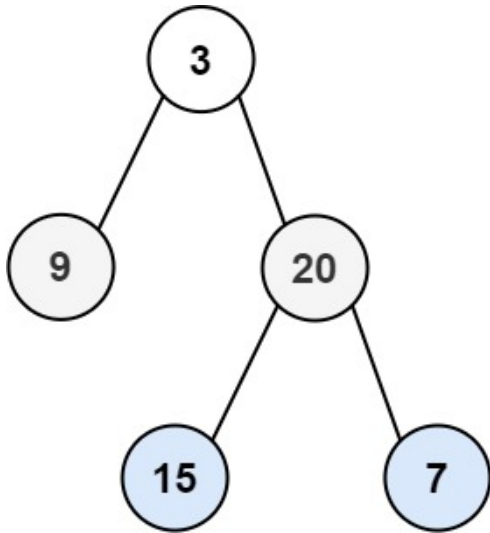
        board[row][col]='*';
        boolean found = false;
        if(backtrack(board, arr, wordLength, rows, cols, row, col-1, index+1)
            || backtrack(board, arr, wordLength, rows, cols, row, col+1, index+1)
            || backtrack(board, arr, wordLength, rows, cols, row-1, col, index+1)
            || backtrack(board, arr, wordLength, rows, cols, row+1, col, index+1)
        ) {
            return true;
        }
        board[row][col]=arr[index];
        return found;
    }
}
```


107 Binary Tree Level Order Traversal II [\(link\)](#)

Description

Given the `root` of a binary tree, return *the bottom-up level order traversal of its nodes' values*. (i.e., from left to right, level by level from leaf to root).

Example 1:



Input: `root = [3,9,20,null,null,15,7]`
Output: `[[15,7],[9,20],[3]]`

Example 2:

Input: `root = [1]`
Output: `[[1]]`

Example 3:

Input: `root = []`
Output: `[]`

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-1000 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> levelOrderBottom(TreeNode root) {
        LinkedList<List<Integer>> result = new LinkedList<>();
        if(root == null) return result;
        dfs(root, 0, result);
        return result;
    }

    void dfs(TreeNode root, int level, LinkedList<List<Integer>> result) {
        if(root == null) return;
        if(result.size() == level) {
            result.addFirst(new ArrayList<>());
        }

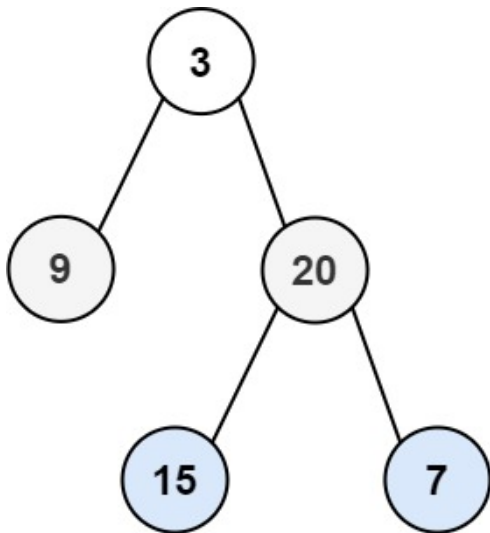
        int index = result.size()-level-1;
        result.get(index).add(root.val);
        dfs(root.left, level+1, result);
        dfs(root.right, level+1, result);
    }
}
```

[102 Binary Tree Level Order Traversal \(link\)](#)

Description

Given the `root` of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:



Input: `root = [3,9,20,null,null,15,7]`
Output: `[[3],[9,20],[15,7]]`

Example 2:

Input: `root = [1]`
Output: `[[1]]`

Example 3:

Input: `root = []`
Output: `[]`

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-1000 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> levels = new ArrayList<>();
        if(root == null) return levels;

        Queue<TreeNode> qu = new ArrayDeque<>();
        qu.offer(root);
        //bfs(qu, levels);
        recursiveBFS(qu, levels);
        return levels;
    }

    private void bfs(Queue<TreeNode> queue, List<List<Integer>> levels) {
        while(!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>(size);
            while(size-->0) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if(node.left != null)
                    queue.offer(node.left);
                if(node.right != null)
                    queue.offer(node.right);
            }
            levels.add(level);
        }
    }

    private void recursiveBFS(Queue<TreeNode> qu, List<List<Integer>> levels) {
        if(qu.isEmpty()) return;
        List<Integer> level = new ArrayList<>();
        int size = qu.size();
        while(size-- > 0) {
            TreeNode node = qu.poll();
            level.add(node.val);
            if(node.left != null)
                qu.offer(node.left);
            if(node.right != null)
                qu.offer(node.right);
        }
        levels.add(level);
        recursiveBFS(qu, levels);
    }
}
```

25 Reverse Nodes in k-Group (link)

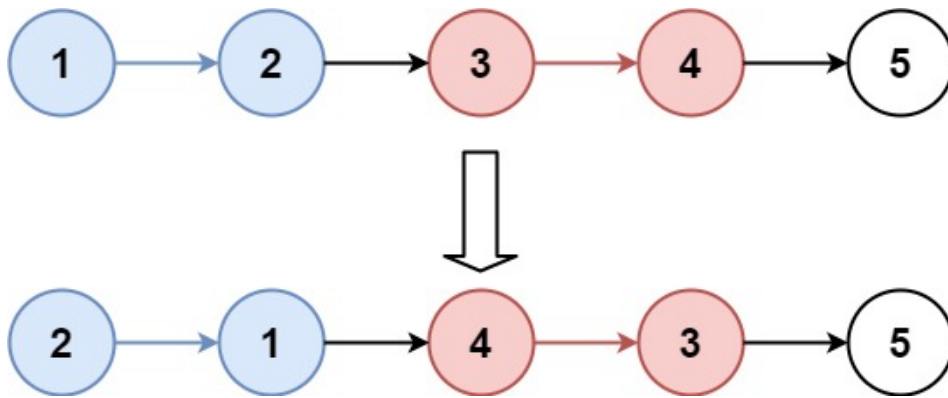
Description

Given the head of a linked list, reverse the nodes of the list k at a time, and return *the modified list*.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

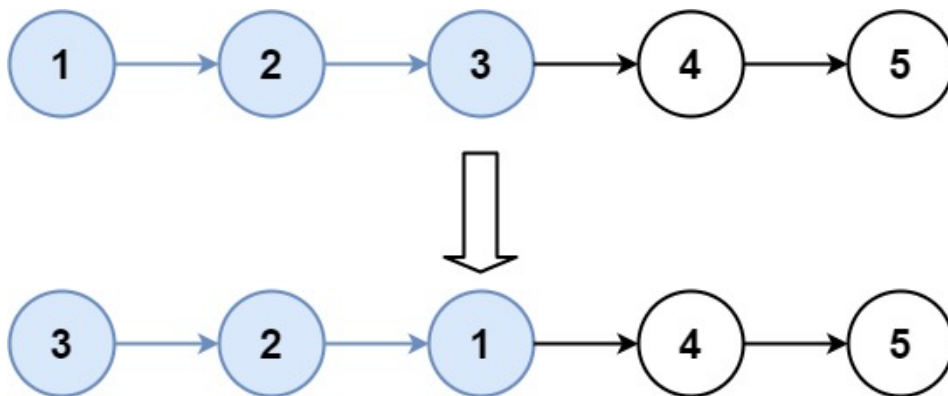
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        if(head == null ) return head;

        int count = 0;
        ListNode curr = head;
        while(count < k && curr != null) {
            curr = curr.next;
            count++;
        }

        if(count < k ) return head;

        ListNode newHead = reverse(head, curr);
        head.next = reverseKGroup(curr, k);
        return newHead;
    }

    ListNode reverse(ListNode first, ListNode term) {
        ListNode prev = null, temp;
        while(first != term) {
            temp = first.next;
            first.next = prev;

            prev = first;
            first = temp;
        }
        return prev;
    }
}
```

[29 Divide Two Integers \(link\)](#)

Description

Given two integers `dividend` and `divisor`, divide two integers **without** using multiplication, division, and mod operator.

The integer division should truncate toward zero, which means losing its fractional part. For example, 8.345 would be truncated to 8, and -2.7335 would be truncated to -2.

Return *the quotient after dividing* `dividend` *by* `divisor`.

Note: Assume we are dealing with an environment that could only store integers within the **32-bit** signed integer range: $[-2^{31}, 2^{31} - 1]$. For this problem, if the quotient is **strictly greater than** $2^{31} - 1$, then return $2^{31} - 1$, and if the quotient is **strictly less than** -2^{31} , then return -2^{31} .

Example 1:

Input: `dividend = 10, divisor = 3`
Output: 3
Explanation: $10/3 = 3.33333..$ which is truncated to 3.

Example 2:

Input: `dividend = 7, divisor = -3`
Output: -2
Explanation: $7/-3 = -2.33333..$ which is truncated to -2.

Constraints:

- $-2^{31} \leq \text{dividend}, \text{divisor} \leq 2^{31} - 1$
- `divisor` $\neq 0$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int divide(int dividend, int divisor) {
        int HALF_INT_MIN = -1073741824;
        if(divisor == 1) return dividend;
        if(dividend == Integer.MIN_VALUE && divisor== -1)
            return Integer.MAX_VALUE;
        int negatives = 0;
        if(dividend > 0) {
            negatives++;
            dividend = -dividend;
        }

        if(divisor > 0) {
            negatives++;
            divisor = -divisor;
        }

        int highestDouble = divisor;
        int highestPowerOfTwo = -1;
        while (highestDouble >= HALF_INT_MIN && dividend <= highestDouble + highestDouble) {
            highestPowerOfTwo += highestPowerOfTwo;
            highestDouble += highestDouble;
        }

        int quotient = 0;
        while(dividend <= divisor) {
            if(dividend <= highestDouble) {
                quotient += highestPowerOfTwo;
                dividend -= highestDouble;
            }
            highestPowerOfTwo >>=1;
            highestDouble >>=1;
        }

        return negatives == 1? quotient:-quotient;
    }
}
```

[23 Merge k Sorted Lists \(link\)](#)

Description

You are given an array of k linked-lists `lists`, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
merging them into one sorted linked list:
1->1->2->3->4->4->5->6
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

```
Input: lists = [[]]
Output: []
```

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 10^4$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-10^4 \leq \text{lists}[i][j] \leq 10^4$
- `lists[i]` is sorted in **ascending order**.
- The sum of `lists[i].length` will not exceed 10^4 .

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        if(lists == null || lists.length == 0) return null;
        return divide(lists, 0, lists.length-1);
    }

    public ListNode divide(ListNode[] lists, int left, int right) {
        if(left < right) {
            int mid = left+(right-left)/2;
            ListNode l = divide(lists, left, mid);
            ListNode r = divide(lists, mid+1, right);
            return mergeTwoLists(l, r);
        }
        return lists[left]; //left never greater than right as per calling methods
    }

    public ListNode mergeTwoLists(ListNode n1, ListNode n2) {
        if(n1 == null) return n2;
        if(n2 == null) return n1;
        if(n1.val < n2.val) {
            n1.next = mergeTwoLists(n1.next, n2);
            return n1;
        } else {
            n2.next = mergeTwoLists(n1, n2.next);
            return n2;
        }
    }
}
```

2691 Count Vowel Strings in Ranges (link)

Description

You are given a **0-indexed** array of strings `words` and a 2D array of integers `queries`.

Each query `queries[i] = [li, ri]` asks us to find the number of strings present at the indices ranging from `li` to `ri` (both **inclusive**) of `words` that start and end with a vowel.

Return *an array* `ans` *of size* `queries.length`, *where* `ans[i]` *is the answer to the* `ith` *query*.

Note that the vowel letters are 'a', 'e', 'i', 'o', and 'u'.

Example 1:

```
Input: words = ["aba","bcb","ece","aa","e"], queries = [[0,2],[1,4],[1,1]]
Output: [2,3,0]
Explanation: The strings starting and ending with a vowel are "aba", "ece", "aa" and "e".
The answer to the query [0,2] is 2 (strings "aba" and "ece").
to query [1,4] is 3 (strings "ece", "aa", "e").
to query [1,1] is 0.
We return [2,3,0].
```

Example 2:

```
Input: words = ["a","e","i"], queries = [[0,2],[0,1],[2,2]]
Output: [3,2,1]
Explanation: Every string satisfies the conditions, so we return [3,2,1].
```

Constraints:

- $1 \leq \text{words.length} \leq 10^5$
- $1 \leq \text{words}[i].\text{length} \leq 40$
- `words[i]` consists only of lowercase English letters.
- $\sum(\text{words}[i].\text{length}) \leq 3 * 10^5$
- $1 \leq \text{queries.length} \leq 10^5$
- $0 \leq l_i \leq r_i < \text{words.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    static boolean[] VMAP = new boolean[128];
    static {
        VMAP['a'] = VMAP['e'] = VMAP['i'] = VMAP['o'] = VMAP['u'] = true;
    }
    public int[] vowelStrings(String[] words, int[][] queries) {
        int n = words.length;
        int[] rangedq = new int[n+1];
        char s, e;
        for(int i=0; i < n; i++) {
            rangedq[i+1] = rangedq[i];
            s = words[i].charAt(0);
            e = words[i].charAt(words[i].length()-1);
            if(VMAP[s] && VMAP[e]) {
                rangedq[i+1]++;
            }
        }
        int[] ans = new int[queries.length];
        for(int i=0; i < queries.length; i++) {
            ans[i] = rangedq[queries[i][1]+1] - rangedq[queries[i][0]];
        }
        return ans;
    }
}
```

[1787 Sum of Absolute Differences in a Sorted Array \(link\)](#)

Description

You are given an integer array `nums` sorted in **non-decreasing** order.

Build and return an integer array `result` with the same length as `nums` such that `result[i]` is equal to the **summation of absolute differences** between `nums[i]` and all the other elements in the array.

In other words, `result[i]` is equal to $\sum(|\text{nums}[i] - \text{nums}[j]|)$ where $0 \leq j < \text{nums.length}$ and $j \neq i$ (**0-indexed**).

Example 1:

```
Input: nums = [2,3,5]
Output: [4,3,5]
Explanation: Assuming the arrays are 0-indexed, then
result[0] = |2-2| + |2-3| + |2-5| = 0 + 1 + 3 = 4,
result[1] = |3-2| + |3-3| + |3-5| = 1 + 0 + 2 = 3,
result[2] = |5-2| + |5-3| + |5-5| = 3 + 2 + 0 = 5.
```

Example 2:

```
Input: nums = [1,4,6,8,10]
Output: [24,15,13,15,21]
```

Constraints:

- $2 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq \text{nums}[i + 1] \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    static {
        for (int i = 0; i < 200; i++) {
            getSumAbsoluteDifferences(new int[0]);
        }
    }
    public static int[] getSumAbsoluteDifferences(int[] nums) {
        int n = nums.length;
        int[] prefix = new int[n];

        int rightSum = 0;
        for(int i=0; i < n; i++) {
            rightSum += nums[i];
        }

        int[] result = new int[n];
        int leftSum = 0;
        for(int i=0; i < n; i++) {
            rightSum -= nums[i];

            int leftTotal = i*nums[i] - leftSum;
            int rightTotal = rightSum - (n-1-i)*nums[i];

            result[i] = leftTotal+rightTotal;
            leftSum += nums[i];
        }
        return result;
    }
}
```

2714 Left and Right Sum Differences [\(link\)](#)

Description

You are given a **0-indexed** integer array `nums` of size `n`.

Define two arrays `leftSum` and `rightSum` where:

- `leftSum[i]` is the sum of elements to the left of the index `i` in the array `nums`. If there is no such element, `leftSum[i] = 0`.
- `rightSum[i]` is the sum of elements to the right of the index `i` in the array `nums`. If there is no such element, `rightSum[i] = 0`.

Return an integer array `answer` of size `n` where `answer[i] = |leftSum[i] - rightSum[i]|`.

Example 1:

Input: `nums = [10,4,8,3]`

Output: `[15,1,11,22]`

Explanation: The array `leftSum` is `[0,10,14,22]` and the array `rightSum` is `[15,11,3,0]`.
The array `answer` is `[|0 - 15|, |10 - 11|, |14 - 3|, |22 - 0|] = [15,1,11,22]`.

Example 2:

Input: `nums = [1]`

Output: `[0]`

Explanation: The array `leftSum` is `[0]` and the array `rightSum` is `[0]`.
The array `answer` is `[|0 - 0|] = [0]`.

Constraints:

- `1 <= nums.length <= 1000`
- `1 <= nums[i] <= 105`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] leftRightDifference(int[] nums) {
        int n = nums.length;

        int rightSum = 0;
        for(int i=0; i < n; i++) {
            rightSum += nums[i];
        }

        int leftSum = 0;
        int[] result = new int[n];
        for(int i=0; i < n; i++) {
            rightSum -= nums[i];
            result[i] = Math.abs(leftSum - rightSum);
            leftSum += nums[i];
        }
        return result;
    }
}
```

[303 Range Sum Query - Immutable \(link\)](#)

Description

Given an integer array `nums`, handle multiple queries of the following type:

1. Calculate the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `int sumRange(int left, int right)` Returns the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Example 1:

Input

```
["NumArray", "sumRange", "sumRange", "sumRange"]  
[[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
```

Output

```
[null, 1, -1, -3]
```

Explanation

```
NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);  
numArray.sumRange(0, 2); // return (-2) + 0 + 3 = 1  
numArray.sumRange(2, 5); // return 3 + (-5) + 2 + (-1) = -1  
numArray.sumRange(0, 5); // return (-2) + 0 + 3 + (-5) + 2 + (-1) = -3
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^5 \leq \text{nums}[i] \leq 10^5$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most 10^4 calls will be made to `sumRange`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class NumArray {

    int[] arr;

    public NumArray(int[] nums) {
        arr = nums;
        for(int i=1; i < nums.length; i++) {
            arr[i] += arr[i-1];
        }
    }

    public int sumRange(int left, int right) {
        return arr[right]-(left == 0 ? 0 : arr[left-1]);
    }
}

/**
 * Your NumArray object will be instantiated and called as such:
 * NumArray obj = new NumArray(nums);
 * int param_1 = obj.sumRange(left,right);
 */
```

3195 Separate Black and White Balls [\(link\)](#)

Description

There are n balls on a table, each ball has a color black or white.

You are given a **0-indexed** binary string s of length n , where 1 and 0 represent black and white balls, respectively.

In each step, you can choose two adjacent balls and swap them.

Return the **minimum** number of steps to group all the black balls to the right and all the white balls to the left.

Example 1:

```
Input: s = "101"
Output: 1
Explanation: We can group all the black balls to the right in the following way:
- Swap s[0] and s[1], s = "011".
Initially, 1s are not grouped together, requiring at least 1 step to group them to the right.
```

Example 2:

```
Input: s = "100"
Output: 2
Explanation: We can group all the black balls to the right in the following way:
- Swap s[0] and s[1], s = "010".
- Swap s[1] and s[2], s = "001".
It can be proven that the minimum number of steps needed is 2.
```

Example 3:

```
Input: s = "0111"
Output: 0
Explanation: All the black balls are already grouped to the right.
```

Constraints:

- $1 \leq n \leq s.length \leq 10^5$
- $s[i]$ is either '0' or '1'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public long minimumSteps(String s) {
        char white = '0';
        char[] arr = s.toCharArray();
        int blacks = 0;
        long counter = 0;
        for(char ch: arr) {
            if(ch == white) {
                counter += blacks;
            } else {
                blacks += 1;
            }
        }
        return counter;
    }
}
```

[443 String Compression \(link\)](#)

Description

Given an array of characters `chars`, compress it using the following algorithm:

Begin with an empty string `s`. For each group of **consecutive repeating characters** in `chars`:

- If the group's length is 1, append the character to `s`.
- Otherwise, append the character followed by the group's length.

The compressed string `s` **should not be returned separately**, but instead, be stored **in the input character array** `chars`. Note that group lengths that are 10 or longer will be split into multiple characters in `chars`.

After you are done **modifying the input array**, return *the new length of the array*.

You must write an algorithm that uses only constant extra space.

Example 1:

Input: `chars = ["a","a","b","b","c","c","c"]`

Output: Return 6, and the first 6 characters of the input array should be: `["a","2","b","2","c","3"]`

Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".

Example 2:

Input: `chars = ["a"]`

Output: Return 1, and the first character of the input array should be: `["a"]`

Explanation: The only group is "a", which remains uncompressed since it's a single character.

Example 3:

Input: `chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]`

Output: Return 4, and the first 4 characters of the input array should be: `["a","b","1","2"]`.

Explanation: The groups are "a" and "bbbbbbbbbbb". This compresses to "ab12".

Constraints:

- `1 <= chars.length <= 2000`
- `chars[i]` is a lowercase English letter, uppercase English letter, digit, or symbol.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    private int[] DIVS = {1000,100,10,1};
    public int compress(char[] chars) {
        int last = 0;
        int length = chars.length;
        int start = 0, end = 0;
        while(end < length) {
            while(end < length && chars[start] == chars[end]){
                end++;
            }
            chars[last++] = chars[start];
            int diff = end-start;
            if(diff > 1 && diff < 10) {
                chars[last++] = (char)('0'+diff);
            } else if(diff > 9) {
                last = addCount(chars, last, diff);
            }
            start = end;
        }
        return last;
    }

    int addCount(char[] arr, int last, int count) {
        for(int i=0; i < DIVS.length; i++) {
            if(count < DIVS[i]) continue;
            if(count%DIVS[i] == 0) {
                for(char ch: String.valueOf(count).toCharArray()) {
                    arr[last++] = ch;
                }
                break;
            } else {
                arr[last++] = (char) ('0'+count/DIVS[i]);
                count %= DIVS[i];
            }
        }
        return last;
    }
}
```

[1850 Minimum Length of String After Deleting Similar Ends \(link\)](#)

Description

Given a string s consisting only of characters 'a', 'b', and 'c'. You are asked to apply the following algorithm on the string any number of times:

1. Pick a **non-empty** prefix from the string s where all the characters in the prefix are equal.
2. Pick a **non-empty** suffix from the string s where all the characters in this suffix are equal.
3. The prefix and the suffix should not intersect at any index.
4. The characters from the prefix and suffix must be the same.
5. Delete both the prefix and the suffix.

Return the **minimum length** of s after performing the above operation any number of times (possibly zero times).

Example 1:

Input: $s = "ca"$
Output: 2
Explanation: You can't remove any characters, so the string stays as is.

Example 2:

Input: $s = "cabaabac"$
Output: 0
Explanation: An optimal sequence of operations is:
- Take prefix = "c" and suffix = "c" and remove them, $s = "abaaba"$.
- Take prefix = "a" and suffix = "a" and remove them, $s = "baab"$.
- Take prefix = "b" and suffix = "b" and remove them, $s = "aa"$.
- Take prefix = "a" and suffix = "a" and remove them, $s = ""$.

Example 3:

Input: $s = "aabccabba"$
Output: 3
Explanation: An optimal sequence of operations is:
- Take prefix = "aa" and suffix = "a" and remove them, $s = "bccabb"$.
- Take prefix = "b" and suffix = "bb" and remove them, $s = "cca"$.

Constraints:

- $1 \leq s.length \leq 10^5$
- s only consists of characters 'a', 'b', and 'c'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minimumLength(String s) {
        char[] arr = s.toCharArray();
        int length = arr.length;
        int left = 0, right = length-1;
        while(left < right && arr[left] == arr[right]) {
            char c = arr[left];
            while(left <= right && c == arr[left]) {
                left++;
            }
            while(left <= right && c == arr[right]) {
                right--;
            }
        }
        return right-left+1;
    }
}
```

696 Count Binary Substrings [\(link\)](#)

Description

Given a binary string s , return the number of non-empty substrings that have the same number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped consecutively.

Substrings that occur multiple times are counted the number of times they occur.

Example 1:

Input: $s = "00110011"$

Output: 6

Explanation: There are 6 substrings that have equal number of consecutive 1's and 0's: "0011", "01", "1100", "10", "0011", "1100". Notice that some of these substrings repeat and are counted the number of times they occur. Also, "00110011" is not a valid substring because all the 0's (and 1's) are not grouped together.

Example 2:

Input: $s = "10101"$

Output: 4

Explanation: There are 4 substrings: "10", "01", "10", "01" that have equal number of consecutive 1's and 0's.

Constraints:

- $1 \leq s.length \leq 10^5$
- $s[i]$ is either '0' or '1'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int countBinarySubstrings(String s) {
        char[] arr = s.toCharArray();
        int length = arr.length;
        int counter = 1;
        int prevCounter = 0;
        int result = 0;
        for(int i = 1; i < length; i++) {
            if(arr[i-1] == arr[i]) {
                counter++;
            } else {
                result += Math.min(prevCounter, counter);
                prevCounter = counter;
                counter = 1;
            }
        }
        return result+Math.min(prevCounter, counter);
    }
}
```

3018 Make String a Subsequence Using Cyclic Increments ([link](#))

Description

You are given two **0-indexed** strings `str1` and `str2`.

In an operation, you select a **set** of indices in `str1`, and for each index `i` in the set, increment `str1[i]` to the next character **cyclically**. That is 'a' becomes 'b', 'b' becomes 'c', and so on, and 'z' becomes 'a'.

Return `true` *if it is possible to make* `str2` *a subsequence of* `str1` *by performing the operation* **at most once**, *and* `false` *otherwise*.

Note: A subsequence of a string is a new string that is formed from the original string by deleting some (possibly none) of the characters without disturbing the relative positions of the remaining characters.

Example 1:

```
Input: str1 = "abc", str2 = "ad"
Output: true
Explanation: Select index 2 in str1.
Increment str1[2] to become 'd'.
Hence, str1 becomes "abd" and str2 is now a subsequence. Therefore, true is returned.
```

Example 2:

```
Input: str1 = "zc", str2 = "ad"
Output: true
Explanation: Select indices 0 and 1 in str1.
Increment str1[0] to become 'a'.
Increment str1[1] to become 'd'.
Hence, str1 becomes "ad" and str2 is now a subsequence. Therefore, true is returned.
```

Example 3:

```
Input: str1 = "ab", str2 = "d"
Output: false
Explanation: In this example, it can be shown that it is impossible to make str2 a subsequence of str1.
Therefore, false is returned.
```

Constraints:

- $1 \leq \text{str1.length} \leq 10^5$
- $1 \leq \text{str2.length} \leq 10^5$
- `str1` and `str2` consist of only lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canMakeSubsequence(String str1, String str2) {
        int n2 = str2.length();
        int n1 = str1.length();
        if(n2 > n1 ) return false;
        int j=0;
        for(int i = 0; i < n1 && j < n2;i++) {
            int c1 = str1.charAt(i);
            int c2 = str2.charAt(j);
            if(c1 == c2 || c2 == (c1==122 ? 97: (c1+1)) ) {
                j++;
            }
        }
        return j == n2;
    }
}
```

189 Rotate Array (link)

Description

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- $0 \leq k \leq 10^5$

Follow up:

- Try to come up with as many solutions as you can. There are at least **three** different ways to solve this problem.
- Could you do it in-place with $O(1)$ extra space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
// class Solution {
//     public void rotate(int[] A, int K) {
//         int N = A.length;
//         if(N == 0) return;

//         K %= N;
//         if(K == 0);

//         reverse(A, 0, N-1);
//         reverse(A, 0, K-1);
//         reverse(A, K, N-1);
//     }

//     private void reverse(int[] A, int left, int right) {
//         int tmp;
//         while(left < right) {
//             tmp = A[left];
//             A[left++] = A[right];
//             A[right--] = tmp;
//         }
//     }
// }
// }

class Solution {
    public void rotate(int[] nums, int k) {
        int n = nums.length;
        k %= n;
        if(k == 0) return;
        int count=0;
        int startIdx = 0;
        while(count < n) {
            int currentIdx = startIdx;
            int prev = nums[startIdx];
            do {
                int nextIdx = (currentIdx+k)%n;
                int temp = nums[nextIdx];
                nums[nextIdx] = prev;

                prev = temp;
                currentIdx = nextIdx;
                count++;
            } while(startIdx != currentIdx);
            startIdx++;
        }
    }
}
```

[1880 Largest Merge Of Two Strings \(link\)](#)

Description

You are given two strings `word1` and `word2`. You want to construct a string `merge` in the following way: while either `word1` or `word2` are non-empty, choose **one** of the following options:

- If `word1` is non-empty, append the **first** character in `word1` to `merge` and delete it from `word1`.
 - For example, if `word1` = "abc" and `merge` = "dv", then after choosing this operation, `word1` = "bc" and `merge` = "dva".
- If `word2` is non-empty, append the **first** character in `word2` to `merge` and delete it from `word2`.
 - For example, if `word2` = "abc" and `merge` = "", then after choosing this operation, `word2` = "bc" and `merge` = "a".

Return *the lexicographically **largest** merge you can construct*.

A string `a` is lexicographically larger than a string `b` (of the same length) if in the first position where `a` and `b` differ, `a` has a character strictly larger than the corresponding character in `b`. For example, "abcd" is lexicographically larger than "abcc" because the first position they differ is at the fourth character, and `d` is greater than `c`.

Example 1:

Input: `word1` = "cabaa", `word2` = "bcaaa"

Output: "cbcabaaaaa"

Explanation: One way to get the lexicographically largest merge is:

- Take from `word1`: `merge` = "c", `word1` = "abaa", `word2` = "bcaaa"
- Take from `word2`: `merge` = "cb", `word1` = "abaa", `word2` = "caaa"
- Take from `word2`: `merge` = "cbc", `word1` = "abaa", `word2` = "aaa"
- Take from `word1`: `merge` = "cbca", `word1` = "baa", `word2` = "aaa"
- Take from `word1`: `merge` = "cbcab", `word1` = "aa", `word2` = "aaa"
- Append the remaining 5 a's from `word1` and `word2` at the end of `merge`.

Example 2:

Input: `word1` = "abcabc", `word2` = "abdcaba"

Output: "abdcabcbcab"

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 3000$
- `word1` and `word2` consist only of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String largestMerge(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();
        char[] arr = new char[m+n];
        int i = 0, j = 0, k = 0;
        while(i < m || j < n) {
            if(i == m) {
                while(j < n) {
                    arr[k++] = word2.charAt(j++);
                }
            } else if (j == n) {
                while(i < m) {
                    arr[k++] = word1.charAt(i++);
                }
            } else {
                char achar = word1.charAt(i);
                char bchar = word2.charAt(j);

                if(achar > bchar) {
                    i++;
                    arr[k++] = achar;
                } else if(bchar > achar) {
                    arr[k++] = bchar;
                    j++;
                } else {
                    boolean w1Higher = false;
                    int ii = i+1, jj = j+1;
                    while(ii < m && jj < n && word1.charAt(ii) == word2.charAt(jj)) {
                        ii++; jj++;
                    }

                    w1Higher = ii == m ? false : jj == n ? true: word1.charAt(ii) > word2.charAt(jj);
                    if(w1Higher){
                        arr[k++] = achar;i++;
                    } else {
                        arr[k++] = bchar;j++;
                    }
                }
            }
        }

        return String.valueOf(arr);
    }
}
```

979 DI String Match (link)

Description

A permutation perm of $n + 1$ integers of all the integers in the range $[0, n]$ can be represented as a string s of length n where:

- $s[i] == 'I'$ if $\text{perm}[i] < \text{perm}[i + 1]$, and
- $s[i] == 'D'$ if $\text{perm}[i] > \text{perm}[i + 1]$.

Given a string s , reconstruct the permutation perm and return it. If there are multiple valid permutations perm , return **any of them**.

Example 1:

```
Input: s = "IDID"
Output: [0,4,1,3,2]
```

Example 2:

```
Input: s = "III"
Output: [0,1,2,3]
```

Example 3:

```
Input: s = "DDI"
Output: [3,2,0,1]
```

Constraints:

- $1 \leq s.length \leq 10^5$
- $s[i]$ is either 'I' or 'D'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] diStringMatch(String s) {
        int left = 0, length = s.length(), right = length;
        int[] result = new int[length+1];
        for(int i=0; i < length ; i++) {
            result[i] = s.charAt(i) == 'I' ? left++: right--;
        }
        result[length] = right;
        return result;
    }
}
```

841 Shortest Distance to a Character (link)

Description

Given a string s and a character c that occurs in s , return *an array of integers* answer *where* $\text{answer.length} == s.length$ *and* $\text{answer}[i]$ *is the distance from index* i *to the closest occurrence of character* c *in* s .

The **distance** between two indices i and j is $\text{abs}(i - j)$, where abs is the absolute value function.

Example 1:

```
Input: s = "loveleetcode", c = "e"
Output: [3,2,1,0,1,0,0,1,2,2,1,0]
Explanation: The character 'e' appears at indices 3, 5, 6, and 11 (0-indexed).
The closest occurrence of 'e' for index 0 is at index 3, so the distance is  $\text{abs}(0 - 3) = 3$ .
The closest occurrence of 'e' for index 1 is at index 3, so the distance is  $\text{abs}(1 - 3) = 2$ .
For index 4, there is a tie between the 'e' at index 3 and the 'e' at index 5, but the distance is still 2.
The closest occurrence of 'e' for index 8 is at index 6, so the distance is  $\text{abs}(8 - 6) = 2$ .
```

Example 2:

```
Input: s = "aaab", c = "b"
Output: [3,2,1,0]
```

Constraints:

- $1 \leq s.length \leq 10^4$
- $s[i]$ and c are lowercase English letters.
- It is guaranteed that c occurs at least once in s .

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] shortestToChar(String s, char c) {
        int length = s.length();
        char[] arr = s.toCharArray();
        int[] result = new int[length];
        List<Integer> indices = new ArrayList<>();

        int counter=0;
        for(int i=0; i < length; i++) {
            if(arr[i] == c) {
                result[i] = 0;
                indices.add(i);
            } else {
                result[i] = -1;
            }
            counter += result[i];
        }

        if(counter == 0) return result;

        for(int i=0; i < length; i++) {
            if(result[i] == 0) continue;
            result[i] = Math.abs(binarySearch(indices, i));
        }

        return result;
    }

    int binarySearch(List<Integer> nums, int pos) {
        int size = nums.size();
        int lo = 0, hi = size-1;
        while(lo <= hi) {
            int mid = (lo+hi)/2;
            int midPos = nums.get(mid);
            int diff = Math.abs(pos - midPos);
            if(diff == 1) return 1;
            if(midPos > pos) {
                hi = mid-1;
            } else {
                lo = mid+1;
            }
        }
        if(hi < 0) return Math.abs(pos-nums.get(0));
        if(lo == size) return Math.abs(pos-nums.get(size-1));
        return Math.min(Math.abs(nums.get(hi)-pos), Math.abs(nums.get(lo)-pos));
    }
}
```

[2816 Lexicographically Smallest Palindrome \(link\)](#)

Description

You are given a string s consisting of **lowercase English letters**, and you are allowed to perform operations on it. In one operation, you can **replace** a character in s with another lowercase English letter.

Your task is to make s a **palindrome** with the **minimum number of operations** possible. If there are **multiple palindromes** that can be made using the **minimum** number of operations, make the **lexicographically smallest** one.

A string a is lexicographically smaller than a string b (of the same length) if in the first position where a and b differ, string a has a letter that appears earlier in the alphabet than the corresponding letter in b .

Return *the resulting palindrome string*.

Example 1:

Input: $s = \text{"egcfe"}$

Output: "efcfe"

Explanation: The minimum number of operations to make "egcfe" a palindrome is 1, and the lexicographically

Example 2:

Input: $s = \text{"abcd"}$

Output: "abba"

Explanation: The minimum number of operations to make "abcd" a palindrome is 2, and the lexicographically

Example 3:

Input: $s = \text{"seven"}$

Output: "neven"

Explanation: The minimum number of operations to make "seven" a palindrome is 1, and the lexicographically

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of only lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String makeSmallestPalindrome(String s) {
        char[] arr = s.toCharArray();
        int length = arr.length;
        int left=0, right = length-1;
        while(left < right) {
            if(arr[left] != arr[right]) {
                char c = (char) Math.min(arr[left], arr[right]);
                arr[left] = c;
                arr[right] = c;
            }
            left++; right--;
        }
        return String.valueOf(arr);
    }
}
```

680 Valid Palindrome II (link)

Description

Given a string `s`, return `true` *if the `s` can be palindrome after deleting **at most one** character from it.*

Example 1:

```
Input: s = "aba"
Output: true
```

Example 2:

```
Input: s = "abca"
Output: true
Explanation: You could delete the character 'c'.
```

Example 3:

```
Input: s = "abc"
Output: false
```

Constraints:

- $1 \leq s.length \leq 10^5$
- `s` consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean validPalindrome(String s) {
        char[] arr = s.toCharArray();
        int length = arr.length;
        int left=0, right = length-1;
        return validPalindrome(arr, left, right, true);
    }

    boolean validPalindrome(char[] arr, int left, int right, boolean allowedOnce) {
        while(left < right) {
            if(arr[left] != arr[right]) {
                if(!allowedOnce) return false;
                else {
                    return validPalindrome(arr, left+1, right, false)
                        || validPalindrome(arr, left, right-1, false);
                }
            }
            left++; right--;
        }
        return true;
    }
}
```

557 Reverse Words in a String III (link)

Description

Given a string s , reverse the order of characters in each word within a sentence while still preserving whitespace and initial word order.

Example 1:

```
Input: s = "Let's take LeetCode contest"
Output: "s'teL ekat edoCteeL tsetnoc"
```

Example 2:

```
Input: s = "Mr Ding"
Output: "rM gniD"
```

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- s contains printable **ASCII** characters.
- s does not contain any leading or trailing spaces.
- There is **at least one** word in s .
- All the words in s are separated by a single space.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reverseWords(String s) {
        int right=0,left=0;
        char[] arr = s.toCharArray();
        while(right < arr.length) {
            do right++; while(right < arr.length && arr[right] != ' ');
            int stoppedAt = right--;

            while(left < right) {
                char c = arr[left];
                arr[left] = arr[right];
                arr[right] = c;
                left++; right--;
            }

            left = stoppedAt+1;
            right = left;
        }
        return String.valueOf(arr);
    }
}
```

345 Reverse Vowels of a String[\(link\)](#)

Description

Given a string `s`, reverse only all the vowels in the string and return it.

The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in both lower and upper cases, more than once.

Example 1:

Input: `s = "IceCreAm"`

Output: `"AceCreIm"`

Explanation:

The vowels in `s` are ['I', 'e', 'e', 'A']. On reversing the vowels, `s` becomes `"AceCreIm"`.

Example 2:

Input: `s = "leetcode"`

Output: `"leotcede"`

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- `s` consist of **printable ASCII** characters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reverseVowels(String s) {
        int len = s.length();
        if(len < 2) return s;

        boolean[] vowelMap = new boolean[128];
        for(int ch: "AaEeIiOoUu".toCharArray())
            vowelMap[ch] = true;

        int left = -1, right = len;
        char[] arr = s.toCharArray();
        while(true) {
            do left++; while(left < len && !vowelMap[arr[left]]);
            do right--; while(right > -1 && !vowelMap[arr[right]]);
            if(left > right) break;
            char t = arr[left];
            arr[left] = arr[right];
            arr[right] = t;
        }
        return String.valueOf(arr);
    }
}
```

[344 Reverse String \(link\)](#)

Description

Write a function that reverses a string. The input string is given as an array of characters *s*.

You must do this by modifying the input array [in-place](#) with $O(1)$ extra memory.

Example 1:

```
Input: s = ["h","e","l","l","o"]  
Output: ["o","l","l","e","h"]
```

Example 2:

```
Input: s = ["H","a","n","n","a","h"]  
Output: ["h","a","n","n","a","H"]
```

Constraints:

- $1 \leq s.length \leq 10^5$
- $s[i]$ is a [printable ascii character](#).

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public void reverseString(char[] s) {  
        int left = 0, right = s.length-1;  
        while(left < right) {  
            char t = s[left];  
            s[left] = s[right];  
            s[right] = t;  
            right--;  
            left++;  
        }  
    }  
}
```

[2707 Merge Two 2D Arrays by Summing Values \(link\)](#)

Description

You are given two **2D** integer arrays `nums1` and `nums2`.

- `nums1[i] = [idi, vali]` indicate that the number with the id `idi` has a value equal to `vali`.
- `nums2[i] = [idi, vali]` indicate that the number with the id `idi` has a value equal to `vali`.

Each array contains **unique** ids and is sorted in **ascending** order by id.

Merge the two arrays into one array that is sorted in ascending order by id, respecting the following conditions:

- Only ids that appear in at least one of the two arrays should be included in the resulting array.
- Each id should be included **only once** and its value should be the sum of the values of this id in the two arrays. If the id does not exist in one of the two arrays, then assume its value in that array to be 0.

Return *the resulting array*. The returned array must be sorted in ascending order by id.

Example 1:

Input: `nums1 = [[1,2],[2,3],[4,5]]`, `nums2 = [[1,4],[3,2],[4,1]]`

Output: `[[1,6],[2,3],[3,2],[4,6]]`

Explanation: The resulting array contains the following:

- id = 1, the value of this id is 2 + 4 = 6.
- id = 2, the value of this id is 3.
- id = 3, the value of this id is 2.
- id = 4, the value of this id is 5 + 1 = 6.

Example 2:

Input: `nums1 = [[2,4],[3,6],[5,5]]`, `nums2 = [[1,3],[4,3]]`

Output: `[[1,3],[2,4],[3,6],[4,3],[5,5]]`

Explanation: There are no common ids, so we just include each id with its value in the resulting list.

Constraints:

- `1 <= nums1.length, nums2.length <= 200`
- `nums1[i].length == nums2[j].length == 2`
- `1 <= idi, vali <= 1000`
- Both arrays contain unique ids.
- Both arrays are in strictly ascending order by id.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] mergeArrays(int[][] nums1, int[][] nums2) {
        int m = nums1.length;
        int n = nums2.length;
        int i = 0, j = 0;
        List<int[]> list = new ArrayList<>();
        while(i < m || j < n) {
            if(i == m) {
                list.add(nums2[j++]);
            } else if(j == n) {
                list.add(nums1[i++]);
            } else {
                if(nums1[i][0] == nums2[j][0]) {
                    int[] tmp = new int[2];
                    tmp[0] = nums1[i][0];
                    tmp[1] += nums1[i++][1] + nums2[j++][1];
                    list.add(tmp);
                } else if(nums1[i][0] < nums2[j][0]) {
                    list.add(nums1[i++]);
                } else {
                    list.add(nums2[j++]);
                }
            }
        }
        return list.toArray(new int[0][2]);
    }
}
```

556 Next Greater Element III (link)

Description

Given a positive integer n , find *the smallest integer which has exactly the same digits existing in the integer n and is greater in value than n* . If no such positive integer exists, return -1 .

Note that the returned integer should fit in **32-bit integer**, if there is a valid answer but it does not fit in **32-bit integer**, return -1 .

Example 1:

Input: $n = 12$
Output: 21

Example 2:

Input: $n = 21$
Output: -1

Constraints:

- $1 \leq n \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int nextGreaterElement(int n) {
        if(n < 10) return -1;
        String ns = String.valueOf(n);
        char[] nums = ns.toCharArray();
        int l = nums.length;
        int pivot = -1;
        for(int i=l-2; i > -1; i--) {
            if(nums[i] < nums[i+1]) {
                pivot = i;
                break;
            }
        }

        if(pivot == -1) return -1;
        for(int i=l-1; i > pivot; i--) {
            if(nums[i] > nums[pivot]) {
                char t = nums[i];
                nums[i] = nums[pivot];
                nums[pivot] = t;
                break;
            }
        }
        reverse(nums, pivot+1, l-1);
        long val = Long.parseLong(String.valueOf(nums));
        return val > Integer.MAX_VALUE ? -1: (int) val;
    }

    void reverse(char[] nums, int i, int j) {
        while(i < j) {
            char t = nums[i];
            nums[i] = nums[j];
            nums[j] = t;
            i++; j--;
        }
    }
}
```

2228 Watering Plants II (link)

Description

Alice and Bob want to water n plants in their garden. The plants are arranged in a row and are labeled from 0 to $n - 1$ from left to right where the i^{th} plant is located at $x = i$.

Each plant needs a specific amount of water. Alice and Bob have a watering can each, **initially full**. They water the plants in the following way:

- Alice waters the plants in order from **left to right**, starting from the 0^{th} plant. Bob waters the plants in order from **right to left**, starting from the $(n - 1)^{\text{th}}$ plant. They begin watering the plants **simultaneously**.
- It takes the same amount of time to water each plant regardless of how much water it needs.
- Alice/Bob **must** water the plant if they have enough in their can to **fully** water it. Otherwise, they **first** refill their can (instantaneously) then water the plant.
- In case both Alice and Bob reach the same plant, the one with **more** water currently in his/her watering can should water this plant. If they have the same amount of water, then Alice should water this plant.

Given a **0-indexed** integer array `plants` of n integers, where `plants[i]` is the amount of water the i^{th} plant needs, and two integers `capacityA` and `capacityB` representing the capacities of Alice's and Bob's watering cans respectively, return *the number of times they have to refill to water all the plants*.

Example 1:

Input: `plants = [2,2,3,3]`, `capacityA = 5`, `capacityB = 5`

Output: 1

Explanation:

- Initially, Alice and Bob have 5 units of water each in their watering cans.
 - Alice waters plant 0, Bob waters plant 3.
 - Alice and Bob now have 3 units and 2 units of water respectively.
 - Alice has enough water for plant 1, so she waters it. Bob does not have enough water for plant 2, so he refills his can and waters it.
- So, the total number of times they have to refill to water all the plants is $0 + 0 + 1 + 0 = 1$.

Example 2:

Input: `plants = [2,2,3,3]`, `capacityA = 3`, `capacityB = 4`

Output: 2

Explanation:

- Initially, Alice and Bob have 3 units and 4 units of water in their watering cans respectively.
 - Alice waters plant 0, Bob waters plant 3.
 - Alice and Bob now have 1 unit of water each, and need to water plants 1 and 2 respectively.
 - Since neither of them have enough water for their current plants, they refill their cans and then water the plants.
- So, the total number of times they have to refill to water all the plants is $0 + 1 + 1 + 0 = 2$.

Example 3:

Input: `plants = [5]`, `capacityA = 10`, `capacityB = 8`

Output: 0

Explanation:

- There is only one plant.
 - Alice's watering can has 10 units of water, whereas Bob's can has 8 units. Since Alice has more water, she waters the plant.
- So, the total number of times they have to refill is 0.

Constraints:

- `n == plants.length`
- `1 <= n <= 105`
- `1 <= plants[i] <= 106`
- `max(plants[i]) <= capacityA, capacityB <= 109`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minimumRefill(int[] plants, int capacityA, int capacityB) {
        int length = plants.length;
        int left = 0, right = length-1;
        int counter=0;
        int afill = capacityA, bfill = capacityB;
        boolean exit = false;
        while(left < right) {
            if(afill < plants[left]) {
                afill = capacityA;
                counter++;
            }
            if(bfill < plants[right]) {
                bfill = capacityB;
                counter++;
            }
            afill -= plants[left++];
            bfill -= plants[right--];
            if(left == right && Math.max(afill, bfill) < plants[left]) {
                counter++;
            }
        }
        return counter;
    }
}
```

[2271 Rearrange Array Elements by Sign \(link\)](#)

Description

You are given a **0-indexed** integer array `nums` of **even** length consisting of an **equal** number of positive and negative integers.

You should return the array of `nums` such that the the array follows the given conditions:

1. Every **consecutive pair** of integers have **opposite signs**.
2. For all integers with the same sign, the **order** in which they were present in `nums` is **preserved**.
3. The rearranged array begins with a positive integer.

Return *the modified array after rearranging the elements to satisfy the aforementioned conditions*.

Example 1:

Input: `nums = [3,1,-2,-5,2,-4]`

Output: `[3,-2,1,-5,2,-4]`

Explanation:

The positive integers in `nums` are `[3,1,2]`. The negative integers are `[-2,-5,-4]`.

The only possible way to rearrange them such that they satisfy all conditions is `[3,-2,1,-5,2,-4]`.

Other ways such as `[1,-2,2,-5,3,-4]`, `[3,1,2,-2,-5,-4]`, `[-2,3,-5,1,-4,2]` are incorrect because they do not

Example 2:

Input: `nums = [-1,1]`

Output: `[1,-1]`

Explanation:

1 is the only positive integer and -1 the only negative integer in `nums`.

So `nums` is rearranged to `[1,-1]`.

Constraints:

- $2 \leq \text{nums.length} \leq 2 * 10^5$
- `nums.length` is **even**
- $1 \leq |\text{nums}[i]| \leq 10^5$
- `nums` consists of **equal** number of positive and negative integers.

It is not required to do the modifications in-place.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] rearrangeArray(int[] nums) {
        int pos=0, neg=1;
        int n = nums.length;
        int[] newArr = new int[n];
        for(int start = 0; start < n ; start++) {
            if(nums[start] > 0) {
                newArr[pos] = nums[start];
                pos += 2;
            } else {
                newArr[neg] = nums[start];
                neg += 2;
            }
        }
        return newArr;
    }
}
```


15 3Sum (link)

Description

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.`

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.`

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.`

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: `nums = [0,1,1]`

Output: `[]`

Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: `nums = [0,0,0]`

Output: `[[0,0,0]]`

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-10^5 \leq \text{nums}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        return new AbstractList<List<Integer>>(){
            private List<List<Integer>> triplets;
            int length=nums.length;
            public List<Integer> get(int i){
                init();
                return triplets.get(i);
            }

            public int size(){
                init();
                return triplets.size();
            }

            void init() {
                if(triplets != null) return;
                Arrays.sort(nums);
                triplets = new ArrayList<>();
                for(int i=0; i < length-2; i++) {
                    if(i > 0 && nums[i-1] == nums[i]) continue;
                    int sum, left=i+1, right= length -1;
                    while(left < right) {
                        sum = nums[i]+nums[left]+nums[right];
                        if(sum == 0) {
                            triplets.add(List.of(nums[i], nums[left], nums[right]));
                            do left++;while(left < right && nums[left] == nums[left-1]);
                            do right--;while(left < right && nums[right] == nums[right+1]);
                        } else if(sum > 0) {
                            right--;
                        } else {
                            left++;
                        }
                    }
                }
            }
        };
    }
}
```

[18 4Sum \(link\)](#)

Description

Given an array `nums` of n integers, return *an array of all the **unique quadruplets*** `[nums[a], nums[b], nums[c], nums[d]]` such that:

- $0 \leq a, b, c, d < n$
- $a, b, c,$ and d are **distinct**.
- `nums[a] + nums[b] + nums[c] + nums[d] == target`

You may return the answer in **any order**.

Example 1:

Input: `nums = [1,0,-1,0,-2,2]`, `target = 0`
Output: `[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]`

Example 2:

Input: `nums = [2,2,2,2,2]`, `target = 8`
Output: `[[2,2,2,2]]`

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.*;
class Solution {
    public List<List<Integer>> fourSum(int[] nums, int target) {
        return new AbstractList<List<Integer>>(){
            private List<List<Integer>> result;
            int length=nums.length;
            public List<Integer> get(int i){
                init();
                return result.get(i);
            }

            public int size(){
                init();
                return result.size();
            }

            void init() {
                if(result != null) return;
                Arrays.sort(nums);
                result=new ArrayList<>();
                for(int i=0; i < length-3; i++) {
                    if(i > 0 && nums[i-1] == nums[i]) continue;
                    for(int j=i+1; j < length-2; j++) {
                        if(j > i+1 && nums[j] == nums[j-1]) continue;
                        int left = j+1, right = length-1;
                        while(left < right) {
                            long sum = 0L+nums[i]+nums[j]+nums[left]+nums[right];
                            if(sum == target) {
                                result.add(List.of(nums[i], nums[j], nums[left], nums[right]));
                                do left++; while(left < right && nums[left] == nums[left-1]);
                                do right--; while(left < right && nums[right] == nums[right+1]);
                            } else if(sum > target) {
                                right--;
                            } else {
                                left++;
                            }
                        }
                    }
                }
            }
        };
    }
}
```

2320 Find All K-Distant Indices in an Array [\(link\)](#)

Description

You are given a **0-indexed** integer array `nums` and two integers `key` and `k`. A **k-distant index** is an index `i` of `nums` for which there exists at least one index `j` such that $|i - j| \leq k$ and `nums[j] == key`.

Return a list of all *k-distant indices sorted in increasing order*.

Example 1:

Input: `nums = [3,4,9,1,3,9,5]`, `key = 9`, `k = 1`

Output: `[1,2,3,4,5,6]`

Explanation: Here, `nums[2] == key` and `nums[5] == key`.

- For index 0, $|0 - 2| > k$ and $|0 - 5| > k$, so there is no `j` where $|0 - j| \leq k$ and `nums[j] == key`. Thus 0 is not a k-distant index.
 - For index 1, $|1 - 2| \leq k$ and `nums[2] == key`, so 1 is a k-distant index.
 - For index 2, $|2 - 2| \leq k$ and `nums[2] == key`, so 2 is a k-distant index.
 - For index 3, $|3 - 2| \leq k$ and `nums[2] == key`, so 3 is a k-distant index.
 - For index 4, $|4 - 5| \leq k$ and `nums[5] == key`, so 4 is a k-distant index.
 - For index 5, $|5 - 5| \leq k$ and `nums[5] == key`, so 5 is a k-distant index.
 - For index 6, $|6 - 5| \leq k$ and `nums[5] == key`, so 6 is a k-distant index.
- Thus, we return `[1,2,3,4,5,6]` which is sorted in increasing order.

Example 2:

Input: `nums = [2,2,2,2,2]`, `key = 2`, `k = 2`

Output: `[0,1,2,3,4]`

Explanation: For all indices `i` in `nums`, there exists some index `j` such that $|i - j| \leq k$ and `nums[j] == key`. Hence, we return `[0,1,2,3,4]`.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $1 \leq \text{nums}[i] \leq 1000$
- `key` is an integer from the array `nums`.
- $1 \leq k \leq \text{nums.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findKDistantIndices(int[] nums, int key, int k) {
        List<Integer> kDistantIndices = new ArrayList<>();
        List<Integer> keyIndices = new ArrayList<>();

        for(int i=0; i < nums.length; i++) {
            if(nums[i] == key) {
                keyIndices.add(i);
            }
        }

        for(int i=0; i < nums.length; i++) {
            for(int idx: keyIndices) {
                if(Math.abs(idx-i) <= k) {
                    kDistantIndices.add(i);
                    break;
                }
            }
        }
        return kDistantIndices;
    }
}
```

[2551 Apply Operations to an Array \(link\)](#)

Description

You are given a **0-indexed** array `nums` of size `n` consisting of **non-negative** integers.

You need to apply `n - 1` operations to this array where, in the i^{th} operation (**0-indexed**), you will apply the following on the i^{th} element of `nums`:

- If `nums[i] == nums[i + 1]`, then multiply `nums[i]` by 2 and set `nums[i + 1]` to 0. Otherwise, you skip this operation.

After performing **all** the operations, **shift** all the 0's to the **end** of the array.

- For example, the array `[1,0,2,0,0,1]` after shifting all its 0's to the end, is `[1,2,1,0,0,0]`.

Return *the resulting array*.

Note that the operations are applied **sequentially**, not all at once.

Example 1:

Input: `nums = [1,2,2,1,1,0]`

Output: `[1,4,2,0,0,0]`

Explanation: We do the following operations:

- `i = 0`: `nums[0]` and `nums[1]` are not equal, so we skip this operation.
 - `i = 1`: `nums[1]` and `nums[2]` are equal, we multiply `nums[1]` by 2 and change `nums[2]` to 0. The array becomes `[1,4,0,1,1,0]`.
 - `i = 2`: `nums[2]` and `nums[3]` are not equal, so we skip this operation.
 - `i = 3`: `nums[3]` and `nums[4]` are equal, we multiply `nums[3]` by 2 and change `nums[4]` to 0. The array becomes `[1,4,0,2,0,0]`.
 - `i = 4`: `nums[4]` and `nums[5]` are equal, we multiply `nums[4]` by 2 and change `nums[5]` to 0. The array becomes `[1,4,0,2,0,0]`.
- After that, we shift the 0's to the end, which gives the array `[1,4,2,0,0,0]`.

Example 2:

Input: `nums = [0,1]`

Output: `[1,0]`

Explanation: No operation can be applied, we just shift the 0 to the end.

Constraints:

- `2 <= nums.length <= 2000`
- `0 <= nums[i] <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] applyOperations(int[] nums) {
        int n = nums.length;
        for(int i=1; i < n; i++) {
            if(nums[i-1] == nums[i]) {
                nums[i-1] *= 2;
                nums[i] = 0;
            }
        }

        int k = 0;
        for(int i=0; i < n; i++) {
            if(nums[i] != 0) {
                nums[k++] = nums[i];
            }
        }

        for(int i=k; i < n; i++)
            nums[i] = 0;
        return nums;
    }
}
```


[2265 Partition Array According to Given Pivot \(link\)](#)

Description

You are given a **0-indexed** integer array `nums` and an integer `pivot`. Rearrange `nums` such that the following conditions are satisfied:

- Every element less than `pivot` appears **before** every element greater than `pivot`.
- Every element equal to `pivot` appears **in between** the elements less than and greater than `pivot`.
- The **relative order** of the elements less than `pivot` and the elements greater than `pivot` is maintained.
 - More formally, consider every p_i, p_j where p_i is the new position of the i^{th} element and p_j is the new position of the j^{th} element. If $i < j$ and **both** elements are smaller (*or larger*) than `pivot`, then $p_i < p_j$.

Return `nums` *after the rearrangement*.

Example 1:

Input: `nums = [9,12,5,10,14,3,10]`, `pivot = 10`

Output: `[9,5,3,10,10,12,14]`

Explanation:

The elements 9, 5, and 3 are less than the pivot so they are on the left side of the array.

The elements 12 and 14 are greater than the pivot so they are on the right side of the array.

The relative ordering of the elements less than and greater than pivot is also maintained. `[9, 5, 3]` and

Example 2:

Input: `nums = [-3,4,3,2]`, `pivot = 2`

Output: `[-3,2,4,3]`

Explanation:

The element -3 is less than the pivot so it is on the left side of the array.

The elements 4 and 3 are greater than the pivot so they are on the right side of the array.

The relative ordering of the elements less than and greater than pivot is also maintained. `[-3]` and `[4,`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^6 \leq \text{nums}[i] \leq 10^6$
- `pivot` equals to an element of `nums`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] pivotArray(int[] nums, int pivot) {
        int n = nums.length;
        int[] lesser = new int[n];
        int[] greater = new int[n];
        int counter = 0;
        int l=0, g = 0;
        for(int num: nums) {
            if(num < pivot) lesser[l++]=num;
            else if(num > pivot) greater[g++]=num;
            else counter++;
        }

        for(int i=0; i < l; i++)
            nums[i]=lesser[i];

        for(int i=0; i < counter; i++)
            nums[i+l]=pivot;

        for(int i=0; i < g; i++)
            nums[l+counter+i]=greater[i];
        return nums;
    }
}
```

958 Sort Array By Parity II (link)

Description

Given an array of integers `nums`, half of the integers in `nums` are **odd**, and the other half are **even**.

Sort the array so that whenever `nums[i]` is odd, `i` is **odd**, and whenever `nums[i]` is even, `i` is **even**.

Return *any answer array that satisfies this condition*.

Example 1:

Input: `nums = [4,2,5,7]`

Output: `[4,5,2,7]`

Explanation: `[4,7,2,5]`, `[2,5,4,7]`, `[2,7,4,5]` would also have been accepted.

Example 2:

Input: `nums = [2,3]`

Output: `[2,3]`

Constraints:

- $2 \leq \text{nums.length} \leq 2 * 10^4$
- `nums.length` is even.
- Half of the integers in `nums` are even.
- $0 \leq \text{nums}[i] \leq 1000$

Follow Up: Could you solve it in-place?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] sortByParityII(int[] nums) {
        int n = nums.length;
        int even=0;
        int odd=1;
        while(odd < n && even < n ) {
            if((nums[even]&1) == 0) even += 2;
            else if((nums[odd]&1) > 0) odd += 2;
            else {
                int t = nums[odd];
                nums[odd] = nums[even];
                nums[even] = t;
                odd += 2;
                even += 2;
            }
        }
        return nums;
    }
}
```

2128 Reverse Prefix of Word (link)

Description

Given a **0-indexed** string `word` and a character `ch`, **reverse** the segment of `word` that starts at index 0 and ends at the index of the **first occurrence** of `ch` (**inclusive**). If the character `ch` does not exist in `word`, do nothing.

- For example, if `word` = "abcdefd" and `ch` = "d", then you should **reverse** the segment that starts at 0 and ends at 3 (**inclusive**). The resulting string will be "dcbaefd".

Return *the resulting string*.

Example 1:

Input: `word` = "abcdefd", `ch` = "d"

Output: "dcbaefd"

Explanation: The first occurrence of "d" is at index 3.

Reverse the part of `word` from 0 to 3 (inclusive), the resulting string is "dcbaefd".

Example 2:

Input: `word` = "xyzzxe", `ch` = "z"

Output: "zyxxe"

Explanation: The first and only occurrence of "z" is at index 3.

Reverse the part of `word` from 0 to 3 (inclusive), the resulting string is "zyxxe".

Example 3:

Input: `word` = "abcd", `ch` = "z"

Output: "abcd"

Explanation: "z" does not exist in `word`.

You should not do any reverse operation, the resulting string is "abcd".

Constraints:

- $1 \leq \text{word.length} \leq 250$
- `word` consists of lowercase English letters.
- `ch` is a lowercase English letter.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reversePrefix(String word, char ch) {
        int index=-1;
        char[] charr = word.toCharArray();
        for(char ch1: charr) {
            if(ch == ch1) break;
            else index++;
        }

        if(index == -1 || index == word.length()-1) return word;
        index++;

        int left=0, right = index;
        while(left < right) {
            char t = charr[left];
            charr[left++] = charr[right];
            charr[right--] = t;
        }
        return String.valueOf(charr);
    }
}
```

3451 String Compression III (link)

Description

Given a string `word`, compress it using the following algorithm:

- Begin with an empty string `comp`. While `word` is **not** empty, use the following operation:
 - Remove a maximum length prefix of `word` made of a *single character* `c` repeating **at most** 9 times.
 - Append the length of the prefix followed by `c` to `comp`.

Return the string `comp`.

Example 1:

Input: `word = "abcde"`

Output: `"1a1b1c1d1e"`

Explanation:

Initially, `comp = ""`. Apply the operation 5 times, choosing "a", "b", "c", "d", and "e" as the prefix in each operation.

For each prefix, append "1" followed by the character to `comp`.

Example 2:

Input: `word = "aaaaaaaaaaaaabb"`

Output: `"9a5a2b"`

Explanation:

Initially, `comp = ""`. Apply the operation 3 times, choosing "aaaaaaaa", "aaaaa", and "bb" as the prefix in each operation.

- For prefix "aaaaaaaa", append "9" followed by "a" to `comp`.
- For prefix "aaaaa", append "5" followed by "a" to `comp`.
- For prefix "bb", append "2" followed by "b" to `comp`.

Constraints:

- $1 \leq \text{word.length} \leq 2 * 10^5$
- `word` consists only of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String compressedString(String word) {
        StringBuilder result = new StringBuilder();
        int start = 0, end = 0;
        while(end < word.length()) {
            while(end < word.length() && word.charAt(start) == word.charAt(end)){
                end++;
            }

            //end points to new char
            //window -> right - left +1
            //int numTimes = (end-1)-start+1;
            int numTimes = end-start;
            char ch = word.charAt(start);
            int repeat = numTimes/9;
            int remaining = numTimes%9;
            while(repeat > 0) {
                result.append(9).append(ch);
                repeat--;
            }
            if(remaining != 0) {
                result.append(remaining).append(ch);
            }
            start = end;
        }
        return result.toString();
    }
}
```


540 Single Element in a Sorted Array (link)

Description

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once.

Return *the single element that appears only once*.

Your solution must run in $O(\log n)$ time and $O(1)$ space.

Example 1:

```
Input: nums = [1,1,2,3,3,4,4,8,8]
Output: 2
```

Example 2:

```
Input: nums = [3,3,7,7,10,11,11]
Output: 10
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $0 \leq \text{nums}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int singleNonDuplicate(int[] nums) {
        if(nums.length == 1) return nums[0];
        int lo = 0, n = nums.length, hi = n-1;
        while(lo <= hi) {
            int mid = lo +(hi-lo)/2;
            if(mid == 0) {
                if(nums[mid] != nums[mid+1]) return nums[mid];
                else lo = mid+1;
            } else if (mid == n-1) {
                if(nums[mid] != nums[mid-1]) return nums[mid];
                else hi = mid-1;
            } else {
                if(nums[mid] != nums[mid-1] && nums[mid] != nums[mid+1]) {
                    return nums[mid];
                } else if(nums[mid-1] == nums[mid]) {
                    if((mid&1) == 0) hi = mid;
                    else lo = mid+1;
                } else {
                    if((mid&1) == 0) lo = mid;
                    else hi = mid-1;
                }
            }
        }
        return -1;
    }
}
```

5 Longest Palindromic Substring [\(link\)](#)

Description

Given a string s , return *the longest palindromic substring* in s .

Example 1:

```
Input: s = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.
```

Example 2:

```
Input: s = "cbbd"
Output: "bb"
```

Constraints:

- $1 \leq s.length \leq 1000$
- s consist of only digits and English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String longestPalindrome(String s) {
        int n = s.length();
        int start = 0, end = 0;
        boolean[] dp = new boolean[n];
        for(int i=n-2; i > -1; i--) {
            boolean[] temp = new boolean[n];
            temp[i] = true;
            for(int j=i+1; j < n; j++) {
                if(s.charAt(i) == s.charAt(j) && (j-i == 1 || dp[j-1])) {
                    temp[j] = true;
                    if(j-i > end-start) {
                        end = j;
                        start = i;
                    }
                }
            }
            dp = temp;
        }
        return s.substring(start, end+1);
    }

    public String longestPalindromeII(String s) {
        int n = s.length();
        if(n == 1) return s;
        int start=0, end = 0;

        boolean[] dp = new boolean[n];
        for(int i=n-2; i > -1; i--) {
            boolean[] temp = new boolean[n];
            temp[i] = true;
            for(int j=i+1; j < n; j++) {
                if(s.charAt(i) == s.charAt(j) && (j-i == 1 || dp[j-1])) {
                    temp[j] = true;
                    if(j-i > end-start) {
                        end = j;
                        start = i;
                    }
                }
            }
            dp = temp;
        }
        return s.substring(start, end+1);
    }

    public String longestPalindromeI(String s) {
        int n = s.length();
        if(n == 1) return s;
        int start=0, end=0;

        for(int idx=0; idx < n; idx++) {
            int i = idx, j = idx;
            while(j < n-1 && s.charAt(j) == s.charAt(j+1)) j++;
            while(i >= 0 && j < n && s.charAt(i) == s.charAt(j)) {
                i--;
                j++;
            }

            i++;
            j--;

            if(j-i > end-start) {
                start=i;
                end=j;
            }
        }
    }
}
```

```
        return s.substring(start, end+1);  
    }  
}
```

[200 Number of Islands \(link\)](#)

Description

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return *the number of islands*.

An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` is '0' or '1'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int m = 0, n = 0;
    int[][] DIRS = {{-1,0}, {1,0}, {0,-1}, {0,1}};
    public int numIslands(char[][] grid) {
        m = grid.length;
        n = grid[0].length;
        int numIslands = 0;

        for(int i = 0; i < m; i++) {
            for(int j=0; j < n; j++) {
                if(grid[i][j] == '1'){
                    dfs(grid, i, j);
                    numIslands++;
                }
            }
        }
        return numIslands;
    }

    void dfs(char[][] grid, int i, int j) {
        if(i < 0 || i >=m || j < 0 || j >= n || grid[i][j] == '0') {
            return;
        }

        grid[i][j]='0';
        dfs(grid, i+1, j);
        dfs(grid, i-1, j);
        dfs(grid, i, j+1);
        dfs(grid, i, j-1);
    }
}
```

394 Decode String [\(link\)](#)

Description

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like $3a$ or $2[4]$.

The test cases are generated so that the length of the output will never exceed 10^5 .

Example 1:

```
Input: s = "3[a]2[bc]"
Output: "aaabcbc"
```

Example 2:

```
Input: s = "3[a2[c]]"
Output: "accaccacc"
```

Example 3:

```
Input: s = "2[abc]3[cd]ef"
Output: "abcbcccdcdcdcd"
```

Constraints:

- $1 \leq s.length \leq 30$
- s consists of lowercase English letters, digits, and square brackets '['].
- s is guaranteed to be a **valid** input.
- All the integers in s are in the range $[1, 300]$.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String decodeString(String s) {
        int length = s.length();
        Stack<Integer> kStack = new Stack<>();
        Stack<StringBuilder> builderStack = new Stack<>();
        builderStack.push(new StringBuilder());

        int i=0;
        while(i < length) {
            char ch = s.charAt(i);
            if(ch == '[') {
                builderStack.push(new StringBuilder());
            } else if(ch == ']') {
                StringBuilder temp = builderStack.pop();
                int k = kStack.pop();
                while(k-- > 0) {
                    builderStack.peek().append(temp);
                }
                temp.setLength(0);
            } else if(ch > 47 && ch < 58) {
                int digit = ch - '0', j=i+1;
                for(; j < length && s.charAt(j) != '['; j++){
                    digit = 10*digit + s.charAt(j) - '0';
                }
                kStack.push(digit);
                i = j;
                continue;
            } else {
                builderStack.peek().append(ch);
            }
            i++;
        }
        return builderStack.pop().toString();
    }
}
```

[75 Sort Colors \(link\)](#)

Description

Given an array `nums` with n objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- `nums[i]` is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public void sortColors(int[] nums) {
        int curr=0, left = 0, right = nums.length -1;
        if(right == 0) return;

        while(curr <= right) {
            if(nums[curr] == 0) {
                swap(nums, curr++, left++);
            } else if(nums[curr] == 2) {
                while(right > curr && nums[right] == 2) right--;
                swap(nums, curr, right--);
            } else {
                curr++;
            }
        }
    }

    void swap(int[] arr, int i, int j) {
        int t = arr[j];
        arr[j] = arr[i];
        arr[i] =t;
    }
}
```

[69 Sqrt\(x\) \(link\)](#)

Description

Given a non-negative integer x , return *the square root of x rounded down to the nearest integer*. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

Input: $x = 4$
Output: 2
Explanation: The square root of 4 is 2, so we return 2.

Example 2:

Input: $x = 8$
Output: 2
Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is



Constraints:

- $0 \leq x \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int mySqrt(int x) {
        if(x < 2) return x;
        long longX = x;

        long hi=x/2, lo=2,mid,sq;
        while(lo <= hi) {
            mid = lo+(hi-lo)/2;
            sq = mid*mid;
            if(sq == longX) return (int)mid;

            if(sq > x) hi = mid-1;
            else lo = mid+1;
        }
        return (int) hi;
    }
}
```

62 Unique Paths (link)

Description

There is a robot on an $m \times n$ grid. The robot is initially located at the **top-left corner** (i.e., $\text{grid}[0][0]$). The robot tries to move to the **bottom-right corner** (i.e., $\text{grid}[m - 1][n - 1]$). The robot can only move either down or right at any point in time.

Given the two integers m and n , return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: $m = 3, n = 7$
Output: 28

Example 2:

Input: $m = 3, n = 2$
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down

Constraints:

- $1 \leq m, n \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    char[] keyChars = new char[2];
    public int uniquePathsII(int m, int n) {
        int[][] dp = new int[m][n];

        for(int i=0; i < m; i++) {
            for(int j=0; j < n; j++) {
                if(i==0 || j==0) {
                    dp[i][j]=1;
                    continue;
                }
                dp[i][j]=dp[i-1][j]+dp[i][j-1];
            }
        }
        return dp[m-1][n-1];
    }
    public int uniquePaths(int m, int n) {
        return uniquePaths(m,n, new HashMap<>());
    }

    private int uniquePaths(int m, int n, Map<String, Integer> map) {
        keyChars[0] = (char) m;
        keyChars[1] = (char) n;
        String key = String.valueOf(keyChars);
        if(map.containsKey(key))
            return map.get(key);
        if(m==1 && n ==1)
            return 1;
        if(m == 0 || n ==0)
            return 0;

        int noOfPaths = uniquePaths(m-1,n, map)+uniquePaths(m,n-1,map);
        map.put(key, noOfPaths);
        return noOfPaths;
    }
}
```

50 Pow(x, n).(link)

Description

Implement [pow\(x, n\)](#), which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31}-1$
- n is an integer.
- Either x is not zero or $n > 0$.
- $-10^4 \leq x^n \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public double myPow(double x, int n) {
        double result = recurse(x, Math.abs(n));
        return n < 0 ? 1/result:result;
    }

    private double recurse(double x, int n) {
        if(x == 0) return 0;
        if(n == 0) return 1;
        if(n == 1) return x;

        double result = recurse(x*x, n/2);
        if((n&1) == 0) return result;
        else return x*result;
    }
}
```

[35 Search Insert Position \(link\)](#)

Description

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5
Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2
Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7
Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- nums contains **distinct** values sorted in **ascending** order.
- $-10^4 \leq \text{target} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int lo = 0, hi = nums.length;
        while(lo < hi) {
            int mid = (lo+hi)/2;
            if(target == nums[mid])
                return mid;
            if(target < nums[mid])
                hi = mid;
            else
                lo = mid+1;
        }
        return lo;
    }

    public int searchInsertII(int[] nums, int target) {
        int lo = 0, hi = nums.length -1;
        while(lo <= hi) {
            int mid = (lo+hi)/2;
            if(target == nums[mid])
                return mid;
            if(target < nums[mid])
                hi = mid -1;
            else if(target > nums[mid])
                lo = mid +1;
        }
        return lo;
    }
}
```

20 Valid Parentheses (link)

Description

Given a string s containing just the characters '(', ')', '{', '}', '[', and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: $s = "()"$

Output: true

Example 2:

Input: $s = "()[]{}"$

Output: true

Example 3:

Input: $s = "()["$

Output: false

Example 4:

Input: $s = "([])"$

Output: true

Constraints:

- $1 \leq s.length \leq 10^4$
- s consists of parentheses only '()[]{}'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<Character>();
        for(char ch: s.toCharArray()) {
            if(!stack.isEmpty()) {
                char pch = stack.peek();
                if((ch == ')' && pch == '(')
                || (ch == ']' && pch == '[')
                || (ch == '}' && pch == '{')) {
                    stack.pop();
                    continue;
                }
            }
            stack.push(ch);
        }
        return stack.isEmpty();
    }
}
```

1319 Unique Number of Occurrences (link)

Description

Given an array of integers `arr`, return `true` *if the number of occurrences of each value in the array is unique* or `false` *otherwise*.

Example 1:

Input: `arr = [1,2,2,1,1,3]`

Output: `true`

Explanation: The value 1 has 3 occurrences, 2 has 2 and 3 has 1. No two values have the same number of occurrences.

Example 2:

Input: `arr = [1,2]`

Output: `false`

Example 3:

Input: `arr = [-3,0,1,-3,1,1,1,-3,10,0]`

Output: `true`

Constraints:

- `1 <= arr.length <= 1000`
- `-1000 <= arr[i] <= 1000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean uniqueOccurrences(int[] arr) {
        int[] map = new int[2001];
        for(int num:arr)
            map[num+1000]++;

        int[] counts = new int[2001];
        for(int count: map) {
            if(count == 0) continue;
            if(counts[count] > 0)
                return false;
            counts[count] = 1;
        }
        return true;
    }
}
```

[1392 Find the Difference of Two Arrays \(link\)](#)

Description

Given two **0-indexed** integer arrays `nums1` and `nums2`, return *a list answer of size 2 where:*

- `answer[0]` *is a list of all **distinct** integers in `nums1` which are **not** present in `nums2`.*
- `answer[1]` *is a list of all **distinct** integers in `nums2` which are **not** present in `nums1`.*

Note that the integers in the lists may be returned in **any** order.

Example 1:

Input: `nums1 = [1,2,3]`, `nums2 = [2,4,6]`

Output: `[[1,3],[4,6]]`

Explanation:

For `nums1`, `nums1[1] = 2` is present at index 0 of `nums2`, whereas `nums1[0] = 1` and `nums1[2] = 3` are not present in `nums2`.
For `nums2`, `nums2[0] = 2` is present at index 1 of `nums1`, whereas `nums2[1] = 4` and `nums2[2] = 6` are not present in `nums1`.

Example 2:

Input: `nums1 = [1,2,3,3]`, `nums2 = [1,1,2,2]`

Output: `[[3],[]]`

Explanation:

For `nums1`, `nums1[2]` and `nums1[3]` are not present in `nums2`. Since `nums1[2] == nums1[3]`, their value is only once in the answer list.
Every integer in `nums2` is present in `nums1`. Therefore, `answer[1] = []`.

Constraints:

- $1 \leq \text{nums1.length}, \text{nums2.length} \leq 1000$
- $-1000 \leq \text{nums1}[i], \text{nums2}[i] \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> findDifference(int[] nums1, int[] nums2) {
        int MIN_VALUE=-1000;
        int[] temp = new int[2001];

        for(int num: nums2) {
            temp[num-MIN_VALUE] = 1;
        }
        List<Integer> l1 = new ArrayList<>();
        for(int num: nums1) {
            if(temp[num-MIN_VALUE] == 1) continue;
            temp[num-MIN_VALUE] = 1;
            l1.add(num);
        }

        Arrays.fill(temp, 0);
        for(int num: nums1) {
            temp[num-MIN_VALUE] = 1;
        }
        List<Integer> l2 = new ArrayList<>();
        for(int num: nums2) {
            if(temp[num-MIN_VALUE] == 1) continue;
            temp[num-MIN_VALUE] = 1;
            l2.add(num);
        }

        return List.of(l1,l2);
    }
}
```

735 Asteroid Collision (link)

Description

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

Input: `asteroids = [5,10,-5]`

Output: `[5,10]`

Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.

Example 2:

Input: `asteroids = [8,-8]`

Output: `[]`

Explanation: The 8 and -8 collide exploding each other.

Example 3:

Input: `asteroids = [10,2,-5]`

Output: `[10]`

Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.

Constraints:

- $2 \leq \text{asteroids.length} \leq 10^4$
- $-1000 \leq \text{asteroids}[i] \leq 1000$
- $\text{asteroids}[i] \neq 0$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] asteroidCollision(int[] asteroids) {
        int[] stack = new int[asteroids.length];
        int top = -1;

        for (int asteroid : asteroids) {
            boolean push = true;
            while (top > -1 && asteroid < 0 && stack[top] > 0) {
                int collision = stack[top] + asteroid;
                if (collision < 0) {
                    top--;
                    continue;
                } else if (collision == 0) {
                    top--;
                }
                push = false;
                break;
            }
            if (push) {
                stack[++top] = asteroid;
            }
        }

        return Arrays.copyOf(stack, top+1);
    }
}
```

[1833 Find the Highest Altitude \(link\)](#)

Description

There is a biker going on a road trip. The road trip consists of $n + 1$ points at different altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n where `gain[i]` is the **net gain in altitude** between points i and $i + 1$ for all $(0 \leq i < n)$. Return *the highest altitude of a point*.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- `n == gain.length`
- `1 <= n <= 100`
- `-100 <= gain[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int largestAltitude(int[] gain) {
        int maxAlt = 0, alt = 0, prevAlt = 0, length = gain.length;
        for(int i=0; i < length; i++) {
            alt = prevAlt + gain[i];
            maxAlt = Math.max(alt, maxAlt);
            prevAlt = alt;
        }
        return maxAlt;
    }
}
```

[1586 Longest Subarray of 1's After Deleting One Element](#) ([link](#)).

Description

Given a binary array `nums`, you should delete one element from it.

Return *the size of the longest non-empty subarray containing only 1's in the resulting array*. Return 0 if there is no such subarray.

Example 1:

Input: `nums = [1,1,0,1]`

Output: 3

Explanation: After deleting the number in position 2, `[1,1,1]` contains 3 numbers with value of 1's.

Example 2:

Input: `nums = [0,1,1,1,0,1,1,0,1]`

Output: 5

Explanation: After deleting the number in position 4, `[0,1,1,1,1,0,1]` longest subarray with value of 1

Example 3:

Input: `nums = [1,1,1]`

Output: 2

Explanation: You must delete one element.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- `nums[i]` is either 0 or 1.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int longestSubarray(int[] nums) {
        int longest = 0;
        int left = 0, right = 0;
        int countZero = 0;
        while(right < nums.length) {
            if(nums[right] == 0)
                countZero++;
            while(countZero > 1) {
                if(nums[left++] == 0)
                    countZero--;
            }
            longest = Math.max(longest, right-left);
            right++;
        }
        return longest;
    }
}
```

[459 Repeated Substring Pattern \(link\)](#)

Description

Given a string s , check if it can be constructed by taking a substring of it and appending multiple copies of the substring together.

Example 1:

```
Input: s = "abab"  
Output: true  
Explanation: It is the substring "ab" twice.
```

Example 2:

```
Input: s = "aba"  
Output: false
```

Example 3:

```
Input: s = "abcabcabcabc"  
Output: true  
Explanation: It is the substring "abc" four times or the substring "abcabc" twice.
```

Constraints:

- $1 \leq s.length \leq 10^4$
- s consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {
        int n = s.length();
        StringBuilder substring = new StringBuilder();
        for(int i=n/2; i >=1;i--) {
            if(n%i != 0) continue;
            substring.append(s.substring(0,i));
            int j = n;
            while(j > 0) {
                if(!substring.toString().equals(s.substring(j-i, j))) {
                    break;
                }
                j -= i;
            }
            substring.setLength(0);
            if(j == 0) return true;
        }

        // for(int i=n/2; i > 0;i--) {
        //     if(n%i == 0 && s.substring(0,i).repeat(n/i).equals(s)) {
        //         return true;
        //     }
        // }
        return false;
    }
}
```

1046 Max Consecutive Ones III (link)

Description

Given a binary array `nums` and an integer `k`, return *the maximum number of consecutive 1's in the array if you can flip at most k 0's*.

Example 1:

Input: `nums = [1,1,1,0,0,0,1,1,1,1,0]`, `k = 2`
Output: 6
Explanation: `[1,1,1,0,0,1,1,1,1]`
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Example 2:

Input: `nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1]`, `k = 3`
Output: 10
Explanation: `[0,0,1,1,1,1,1,1,1,1,0,0,0,1,1,1,1]`
Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- `nums[i]` is either 0 or 1.
- $0 \leq k \leq \text{nums.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int longestOnes(int[] nums, int k) {
        int longest = 0, count = 0;
        int left=0, right=0;
        int maxReplacement = 0;
        while(right < nums.length) {
            maxReplacement += nums[right] == 0 ? 1:0;
            if(maxReplacement > k) {
                maxReplacement -= nums[left++] == 0 ? 1:0;
            }
            longest = Math.max(longest, right-left+1);
            right++;
        }
        return longest;
    }
}
```

1567 Maximum Number of Vowels in a Substring of Given Length (link)

Description

Given a string s and an integer k , return *the maximum number of vowel letters in any substring of s with length k* .

Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.

Example 1:

Input: $s = \text{"abciidef"}, k = 3$
Output: 3
Explanation: The substring "iii" contains 3 vowel letters.

Example 2:

Input: $s = \text{"aeiou"}, k = 2$
Output: 2
Explanation: Any substring of length 2 contains 2 vowels.

Example 3:

Input: $s = \text{"leetcode"}, k = 3$
Output: 2
Explanation: "lee", "eet" and "ode" contain 2 vowels.

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists of lowercase English letters.
- $1 \leq k \leq s.length$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxVowels(String s, int k) {
        int[] map = new int[26];
        map['a' - 'a'] = map['e' - 'a'] = map['i' - 'a'] = map['o' - 'a'] = map['u' - 'a'] = 1;
        int maxVowelsInKWindow = 0, vowelCount = 0;
        int l = 0, r = 0;
        while(r < s.length()) {
            char rc = s.charAt(r);
            if(map[rc - 'a'] == 1) {
                vowelCount++;
            }
            if(r - l + 1 == k) {
                maxVowelsInKWindow = Math.max(maxVowelsInKWindow, vowelCount);
                char lc = s.charAt(l);
                if(map[lc - 'a'] == 1) {
                    vowelCount--;
                }
                l++;
            }
            r++;
        }
        return maxVowelsInKWindow;
    }
}
```

[643 Maximum Average Subarray I \(link\)](#)

Description

You are given an integer array `nums` consisting of `n` elements, and an integer `k`.

Find a contiguous subarray whose **length is equal to** `k` that has the maximum average value and return *this value*. Any answer with a calculation error less than 10^{-5} will be accepted.

Example 1:

```
Input: nums = [1,12,-5,-6,50,3], k = 4
Output: 12.75000
Explanation: Maximum average is (12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75
```

Example 2:

```
Input: nums = [5], k = 1
Output: 5.00000
```

Constraints:

- `n == nums.length`
- `1 <= k <= n <= 105`
- `-104 <= nums[i] <= 104`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        double maxSum = Integer.MIN_VALUE;
        int sum=0;

        // for(int i=0; i < k; i++) {
        //     sum += nums[i];
        // }
        // maxSum = sum;
        // for(int i=k; i < nums.length; i++) {
        //     sum += nums[i]-nums[i-k];
        //     maxSum = Math.max(sum, maxSum);
        // }

        int left=0, right = 0;
        while(right < nums.length) {
            sum += nums[right];
            if(right-left+1 == k) {
                maxSum = Math.max(sum, maxSum);
                sum -= nums[left++];
            }
            right++;
        }
        return maxSum/k;
    }
}
```

1146 Greatest Common Divisor of Strings (link)

Description

For two strings s and t , we say " t divides s " if and only if $s = t + t + t + \dots + t + t$ (i.e., t is concatenated with itself one or more times).

Given two strings `str1` and `str2`, return *the largest string x such that x divides both `str1` and `str2`*.

Example 1:

```
Input: str1 = "ABCABC", str2 = "ABC"
Output: "ABC"
```

Example 2:

```
Input: str1 = "ABABAB", str2 = "ABAB"
Output: "AB"
```

Example 3:

```
Input: str1 = "LEET", str2 = "CODE"
Output: ""
```

Constraints:

- $1 \leq \text{str1.length}, \text{str2.length} \leq 1000$
- `str1` and `str2` consist of English uppercase letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String gcdOfStringsI(String str1, String str2) {
        if(!str1.concat(str2).equals(str2.concat(str1))) return "";
        int m = str1.length(), n = str2.length();
        int N = Math.min(m, n);
        for(int i=N; i > 0; i--) {
            //i should split both str1, str2 evenly
            if(m%i == 0 && n%i == 0)
                return str1.substring(0,i);
        }
        return str1.substring(0,1);
    }

    public String gcdOfStrings(String str1, String str2) {
        int m = str1.length(), n = str2.length();
        String result = "";
        int N = Math.min(m,n);
        for(int i=1; i <= N; i++) {
            //i should split both str1, str2 evenly
            if(!(m%i == 0 && n%i == 0)) continue;

            String prefix = str2.substring(0, i);
            if(isValid(str1, str2, prefix)) {
                result = prefix;
            }
        }
        return result;
    }

    boolean isValid(String str1, String str2, String pattern) {
        int len = pattern.length();
        int strLen = str1.length();
        for(int i=0; i < strLen; i+=len) {
            if(!str1.substring(i, i+len).equals(pattern)) {
                return false;
            }
        }
        strLen = str2.length();
        for(int i=0; i < strLen; i+=len) {
            if(!str2.substring(i, i+len).equals(pattern)) {
                return false;
            }
        }
        return true;
    }

    public String gcdIII(String str1, String str2) {
        if(!str1.concat(str2).equals(str2.concat(str1))) return "";
        int m = str1.length(), n = str2.length();
        return str2.substring(0, gcd(m,n));
    }

    int gcd(int a, int b) {
        if(b==0) return a;
        return gcd(b, a%b);
    }
}
```

[3 Longest Substring Without Repeating Characters \(link\)](#)

Description

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

Input: `s = "abcabcbb"`
Output: 3
Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: `s = "bbbbbb"`
Output: 1
Explanation: The answer is "b", with the length of 1.

Example 3:

Input: `s = "pwwkew"`
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- `s` consists of English letters, digits, symbols and spaces.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int lengthOfLongestSubstring(String str) {
        int[] lookup = new int[128];
        Arrays.fill(lookup, -1);
        int r = 0, l = 0, maxLength = 0;

        for(char c: str.toCharArray()) {
            if(lookup[c] > -1) {
                l = Math.max(l, 1+lookup[c]);
            }
            lookup[c] = r;
            maxLength = Math.max(maxLength, r-l+1) ;
            r++;
        }
        return maxLength;
    }
}
```

1798 Max Number of K-Sum Pairs (link)

Description

You are given an integer array `nums` and an integer `k`.

In one operation, you can pick two numbers from the array whose sum equals `k` and remove them from the array.

Return *the maximum number of operations you can perform on the array*.

Example 1:

Input: `nums = [1,2,3,4]`, `k = 5`

Output: 2

Explanation: Starting with `nums = [1,2,3,4]`:

- Remove numbers 1 and 4, then `nums = [2,3]`

- Remove numbers 2 and 3, then `nums = []`

There are no more pairs that sum up to 5, hence a total of 2 operations.

Example 2:

Input: `nums = [3,1,3,4,3]`, `k = 6`

Output: 1

Explanation: Starting with `nums = [3,1,3,4,3]`:

- Remove the first two 3's, then `nums = [1,4,3]`

There are no more pairs that sum up to 6, hence a total of 1 operation.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^9$
- $1 \leq k \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxOperations(int[] nums, int k) {
        int n = nums.length;
        Arrays.sort(nums);

        int left = 0, right = n-1, operations = 0;
        while(left < right) {
            int sum = nums[left] + nums[right];
            if(k == sum) {
                operations++;
                left++;
                right--;
            } else if(k > sum) {
                left++;
            } else {
                right--;
            }
        }
        return operations;
    }
}
```

[605 Can Place Flowers \(link\)](#)

Description

You have a long flowerbed in which some of the plots are planted, and some are not. However, flowers cannot be planted in **adjacent** plots.

Given an integer array `flowerbed` containing 0's and 1's, where 0 means empty and 1 means not empty, and an integer `n`, return `true` if *n new flowers can be planted in the flowerbed without violating the no-adjacent-flowers rule* and `false` otherwise.

Example 1:

```
Input: flowerbed = [1,0,0,0,1], n = 1
Output: true
```

Example 2:

```
Input: flowerbed = [1,0,0,0,1], n = 2
Output: false
```

Constraints:

- $1 \leq \text{flowerbed.length} \leq 2 * 10^4$
- `flowerbed[i]` is 0 or 1.
- There are no two adjacent flowers in `flowerbed`.
- $0 \leq n \leq \text{flowerbed.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canPlaceFlowers(int[] flowerbed, int n) {
        if(n == 0) return true;
        int length = flowerbed.length;
        for(int i=0; i < length; i++) {
            if(flowerbed[i] == 0 && (i==0 || flowerbed[i-1] == 0) && (i==length-1 || flowerbed[i+1] == 0)) {
                if(--n == 0) return true;
                flowerbed[i]=1;
            }
        }
        return n == 0;
    }
}
```

[1528 Kids With the Greatest Number of Candies \(link\)](#)

Description

There are n kids with candies. You are given an integer array `candies`, where each `candies[i]` represents the number of candies the i^{th} kid has, and an integer `extraCandies`, denoting the number of extra candies that you have.

Return a *boolean array* `result` of length n , where `result[i]` is `true` if, after giving the i^{th} kid all the `extraCandies`, they will have the **greatest** number of candies among all the kids, or `false` otherwise.

Note that **multiple** kids can have the **greatest** number of candies.

Example 1:

Input: `candies = [2,3,5,1,3]`, `extraCandies = 3`

Output: `[true,true,true,false,true]`

Explanation: If you give all `extraCandies` to:

- Kid 1, they will have $2 + 3 = 5$ candies, which is the greatest among the kids.
- Kid 2, they will have $3 + 3 = 6$ candies, which is the greatest among the kids.
- Kid 3, they will have $5 + 3 = 8$ candies, which is the greatest among the kids.
- Kid 4, they will have $1 + 3 = 4$ candies, which is not the greatest among the kids.
- Kid 5, they will have $3 + 3 = 6$ candies, which is the greatest among the kids.

Example 2:

Input: `candies = [4,2,1,1,2]`, `extraCandies = 1`

Output: `[true,false,false,false,false]`

Explanation: There is only 1 extra candy.

Kid 1 will always have the greatest number of candies, even if a different kid is given the extra candy.

Example 3:

Input: `candies = [12,1,12]`, `extraCandies = 10`

Output: `[true,false,true]`

Constraints:

- $n == \text{candies.length}$
- $2 \leq n \leq 100$
- $1 \leq \text{candies}[i] \leq 100$
- $1 \leq \text{extraCandies} \leq 50$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Boolean> kidsWithCandies(int[] candies, int extraCandies) {
        List<Boolean> result = new ArrayList<>(candies.length);
        int max = 0;
        for(int candy: candies) {
            max = Math.max(candy, max);
        }
        int niceToHaveExtra = max - extraCandies;
        for(int candy: candies) {
            result.add(candy >= niceToHaveExtra);
        }
        return result;
    }
}
```

2216 Delete the Middle Node of a Linked List (link)

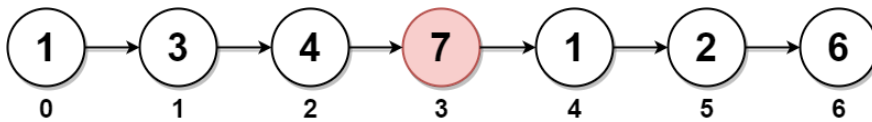
Description

You are given the head of a linked list. **Delete** the **middle node**, and return *the head of the modified linked list*.

The **middle node** of a linked list of size n is the $\lfloor n / 2 \rfloor^{\text{th}}$ node from the **start** using **0-based indexing**, where $\lfloor x \rfloor$ denotes the largest integer less than or equal to x .

- For $n = 1, 2, 3, 4$, and 5 , the middle nodes are $0, 1, 1, 2$, and 2 , respectively.

Example 1:



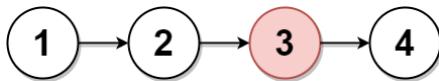
Input: head = [1,3,4,7,1,2,6]

Output: [1,3,4,1,2,6]

Explanation:

The above figure represents the given linked list. The indices of the nodes are written below. Since $n = 7$, node 3 with value 7 is the middle node, which is marked in red. We return the new list after removing this node.

Example 2:



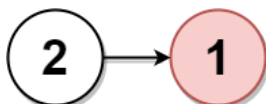
Input: head = [1,2,3,4]

Output: [1,2,4]

Explanation:

The above figure represents the given linked list. For $n = 4$, node 2 with value 3 is the middle node, which is marked in red.

Example 3:



Input: head = [2,1]

Output: [2]

Explanation:

The above figure represents the given linked list. For $n = 2$, node 1 with value 1 is the middle node, which is marked in red. Node 0 with value 2 is the only node remaining after removing node 1.

Constraints:

- The number of nodes in the list is in the range $[1, 10^5]$.
- $1 \leq \text{Node.val} \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode deleteMiddle(ListNode head) {
        if(head.next == null) return null;
        ListNode fast = head, slow = head, prev = null;
        while(fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }

        if(prev != null)
            prev.next = slow.next;
        return head;
    }
}
```

[1894 Merge Strings Alternately \(link\)](#)

Description

You are given two strings `word1` and `word2`. Merge the strings by adding letters in alternating order, starting with `word1`. If a string is longer than the other, append the additional letters onto the end of the merged string.

Return *the merged string*.

Example 1:

```
Input: word1 = "abc", word2 = "pqr"
Output: "apbqcr"
Explanation: The merged string will be merged as so:
word1:  a   b   c
word2:   p   q   r
merged: a p b q c r
```

Example 2:

```
Input: word1 = "ab", word2 = "pqrs"
Output: "apbqrs"
Explanation: Notice that as word2 is longer, "rs" is appended to the end.
word1:  a   b
word2:   p   q   r   s
merged: a p b q   r   s
```

Example 3:

```
Input: word1 = "abcd", word2 = "pq"
Output: "apbqcd"
Explanation: Notice that as word1 is longer, "cd" is appended to the end.
word1:  a   b   c   d
word2:   p   q
merged: a p b q c   d
```

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 100$
- `word1` and `word2` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String mergeAlternately(String word1, String word2) {
        StringBuilder sb = new StringBuilder();
        int m = word1.length();
        int n = word2.length();
        int i=0;
        for(; i < m && i < n; i++) {
            sb.append(word1.charAt(i)).append(word2.charAt(i));
        }
        if(i == n && i < m) sb.append(word1.substring(i));
        if(i == m && i < n) sb.append(word2.substring(i));
        return sb.toString();
    }
}
```

[147 Insertion Sort List \(link\)](#)

Description

Given the head of a singly linked list, sort the list using **insertion sort**, and return *the sorted list's head*.

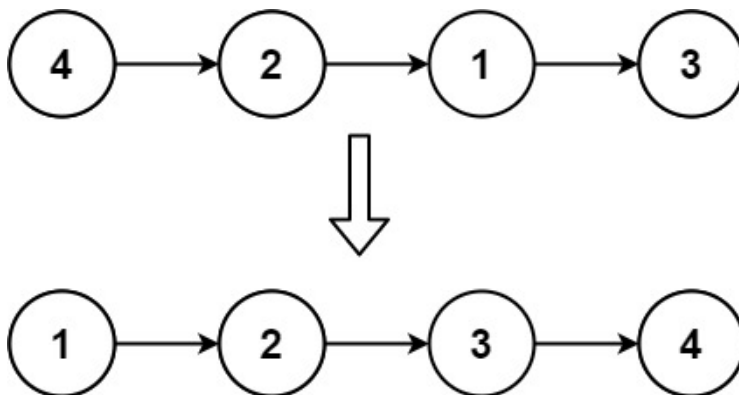
The steps of the **insertion sort** algorithm:

1. Insertion sort iterates, consuming one input element each repetition and growing a sorted output list.
2. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list and inserts it there.
3. It repeats until no input elements remain.

The following is a graphical example of the insertion sort algorithm. The partially sorted list (black) initially contains only the first element in the list. One element (red) is removed from the input data and inserted in-place into the sorted list with each iteration.

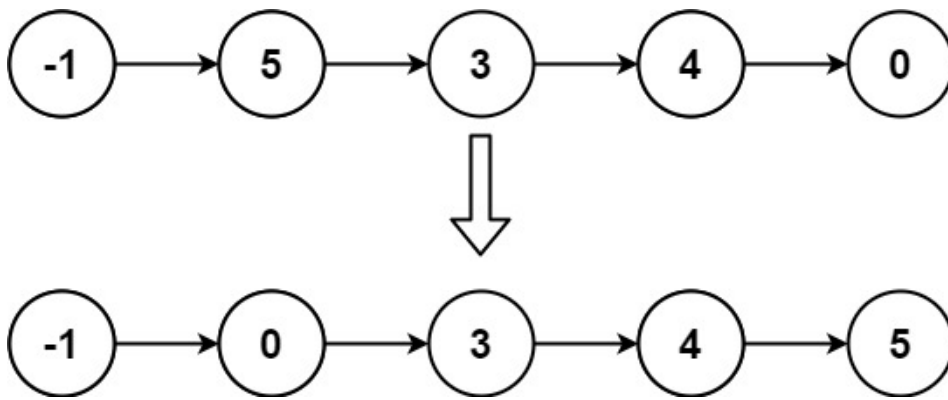
6 5 3 1 8 7 2 4

Example 1:



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Constraints:

- The number of nodes in the list is in the range [1, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode insertionSortList(ListNode head) {
        if(head == null || head.next == null) return head;

        ListNode sentinel = new ListNode(Integer.MIN_VALUE);
        ListNode prev = sentinel;
        while(head != null) {
            ListNode next = head.next;
            if(head.val < prev.val) {
                prev = sentinel;
            }

            while(prev.next != null && prev.next.val < head.val) {
                prev = prev.next;
            }

            head.next = prev.next;
            prev.next = head;
            head = next;
        }
        return sentinel.next;
    }
}
```

[72 Edit Distance \(link\)](#)

Description

Given two strings word1 and word2, return *the minimum number of operations required to convert word1 to word2*.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

```
Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
exection -> execution (insert 'u')
```

Constraints:

- $0 \leq \text{word1.length}, \text{word2.length} \leq 500$
- word1 and word2 consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    private int minimum(int a, int b, int c) {
        return Math.min(a, Math.min(b,c));
    }
    public int minDistance(String word1, String word2) {
        int m = word1.length();
        int n = word2.length();

        if(m == 0) return n;
        if(n == 0) return m;
        if(m == n && word1.equals(word2)) return 0;

        int[][] dp = new int[m+1][n+1];
        // for(int i=0; i<=m; i++)
        //     dp[i][0] = i;
        // for(int i=1; i<=n; i++)
        //     dp[0][i] = i;

        int cost;
        for(int i=1; i<=m;i++) {
            dp[i][0] = i;
            for(int j=1; j<=n; j++) {
                dp[0][j] = j;
                if(word1.charAt(i-1) == word2.charAt(j-1)) cost=0;
                else cost = 1;

                //dp[i-1][j]+1 => deletion
                //dp[i][j-1]+1 => insertion
                //dp[i-1][j-1]+cost => substitution
                dp[i][j] = minimum(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost);
            }
        }
        return dp[m][n];
    }
}
```

300 Longest Increasing Subsequence (link)

Description

Given an integer array `nums`, return *the length of the longest **strictly increasing subsequence***.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int length = nums.length;
        List<Integer> increasingSequence = new ArrayList<>();
        increasingSequence.add(nums[0]);

        int lengthOfLIS=1;
        for(int i=1; i < length; i++) {
            int lastIndex = increasingSequence.size()-1;
            if(increasingSequence.get(lastIndex) < nums[i]) {
                increasingSequence.add(nums[i]);
            } else {
                int position = getPosition(increasingSequence, nums[i]);
                if(increasingSequence.get(position) != nums[i]) {
                    increasingSequence.set(position, nums[i]);
                }
            }
        }
        return increasingSequence.size();
    }

    int getPosition(List<Integer> list, int target) {
        int left = 0, right = list.size()-1;
        while(left <= right) {
            int mid = (left+right)>>1;
            if(list.get(mid) == target)
                return mid;
            if(list.get(mid) < target) {
                left = mid+1;
            } else {
                right = mid-1;
            }
        }
        return left;
    }
}
```

[1250 Longest Common Subsequence \(link\)](#)

Description

Given two strings `text1` and `text2`, return *the length of their longest **common subsequence***. If there is no **common subsequence**, return 0.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

```
Input: text1 = "abcde", text2 = "ace"
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.
```

Example 2:

```
Input: text1 = "abc", text2 = "abc"
Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.
```

Example 3:

```
Input: text1 = "abc", text2 = "def"
Output: 0
Explanation: There is no such common subsequence, so the result is 0.
```

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- `text1` and `text2` consist of only lowercase English characters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2){
        try {
            Callable<Integer> func = ()-> solve(text1, text2);
            FutureTask<Integer> task = new FutureTask(func);
            new Thread(task).start();
            return task.get();
        } catch (Exception ex) {return 0;}
    }
    int solve(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();
        if(m == 0 || n == 0)
            return 0;

        int[] current = new int[n+1];
        int[] prev = new int[n+1];
        for(int i=1; i <= m ; i++) {
            for(int j=1; j <=n; j++) {
                if(text1.charAt(i-1) == text2.charAt(j-1)) {
                    current[j] = 1+prev[j-1];
                } else {
                    current[j] = Math.max(current[j-1], prev[j]);
                }
            }
            System.arraycopy(current, 0, prev, 0, current.length);
        }
        return prev[n];
    }
}
```

28 Find the Index of the First Occurrence in a String (link)

Description

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

```
Input: haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" occurs at index 0 and 6.
The first occurrence is at index 0, so we return 0.
```

Example 2:

```
Input: haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" did not occur in "leetcode", so we return -1.
```

Constraints:

- $1 \leq \text{haystack.length}, \text{needle.length} \leq 10^4$
- `haystack` and `needle` consist of only lowercase English characters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int strStr(String haystack, String needle) {
        int needleLength = needle.length(), haystackLength = haystack.length();
        int maxNeedleIndex = haystackLength - needleLength;

        for(int i=0; i <= maxNeedleIndex; i++) {
            for(int j=0; j < needleLength; j++) {
                if(haystack.charAt(i+j) != needle.charAt(j)) {
                    break;
                }
                if(j == needleLength - 1) return i;
            }
        }
        return -1;
    }
}
```

173 Binary Search Tree Iterator (link)

Description

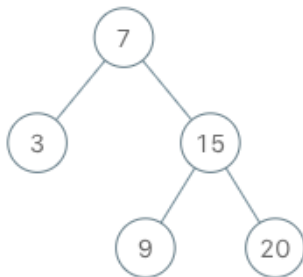
Implement the `BSTIterator` class that represents an iterator over the [in-order traversal](#) of a binary search tree (BST):

- `BSTIterator(TreeNode root)` Initializes an object of the `BSTIterator` class. The `root` of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- `boolean hasNext()` Returns `true` if there exists a number in the traversal to the right of the pointer, otherwise returns `false`.
- `int next()` Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to `next()` will return the smallest element in the BST.

You may assume that `next()` calls will always be valid. That is, there will be at least a next number in the in-order traversal when `next()` is called.

Example 1:



Input

```
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next", "hasNext", "next", "hasNext"]  
[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
```

Output

```
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

Explanation

```
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);  
bSTIterator.next();    // return 3  
bSTIterator.next();    // return 7  
bSTIterator.hasNext(); // return True  
bSTIterator.next();    // return 9  
bSTIterator.hasNext(); // return True  
bSTIterator.next();    // return 15  
bSTIterator.hasNext(); // return True  
bSTIterator.next();    // return 20  
bSTIterator.hasNext(); // return False
```

Constraints:

- The number of nodes in the tree is in the range $[1, 10^5]$.
- $0 \leq \text{Node.val} \leq 10^6$
- At most 10^5 calls will be made to `hasNext`, and `next`.

Follow up:

- Could you implement `next()` and `hasNext()` to run in average $O(1)$ time and use $O(h)$ memory, where h is the height of the tree?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class BSTIterator {
    TreeNode root;
    LinkedList<TreeNode> stack;

    public BSTIterator(TreeNode root) {
        stack = new LinkedList<>();
        this.root = root;
    }

    public int next() {
        while(root != null) {
            stack.push(root);
            root = root.left;
        }
        TreeNode popped = stack.pop();
        root = popped.right;
        return popped.val;
    }

    public boolean hasNext() {
        return !(stack.isEmpty() && root == null);
    }
}

/**
 * Your BSTIterator object will be instantiated and called as such:
 * BSTIterator obj = new BSTIterator(root);
 * int param_1 = obj.next();
 * boolean param_2 = obj.hasNext();
 */
```

297 Serialize and Deserialize Binary Tree (link)

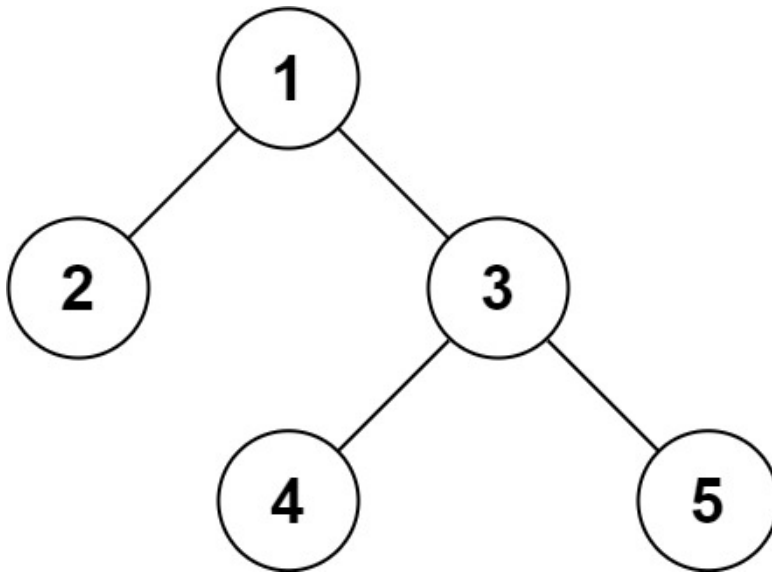
Description

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as [how LeetCode serializes a binary tree](#). You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
public class Codec {
    private char DELIM = ',';
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        serialize(root, sb);
        return sb.toString();
    }

    void serialize(TreeNode root, StringBuilder sb) {
        if(root == null) {
            sb.append("X,");
            return;
        }
        sb.append(root.val).append(DELIM);
        serialize(root.left, sb);
        serialize(root.right, sb);
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        String[] strings = data.split(",");
        return deserialize(strings, new int[]{0});
    }

    TreeNode deserialize(String[] entries, int[] pos){
        if(entries[pos[0]].equals("X")) {
            pos[0]++;
            return null;
        }
        TreeNode node = new TreeNode(Integer.valueOf(entries[pos[0]++]));
        node.left = deserialize(entries, pos);
        node.right = deserialize(entries, pos);
        return node;
    }
}

// Your Codec object will be instantiated and called as such:
// Codec ser = new Codec();
// Codec deser = new Codec();
// TreeNode ans = deser.deserialize(ser.serialize(root));
```

[155 Min Stack \(link\)](#)

Description

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack","push","push","push","getMin","pop","top","getMin"]  
[[],[-2],[0],[-3],[],[],[],[ ]]
```

Output

```
[null,null,null,null,-3,null,0,-2]
```

Explanation

```
MinStack minStack = new MinStack();  
minStack.push(-2);  
minStack.push(0);  
minStack.push(-3);  
minStack.getMin(); // return -3  
minStack.pop();  
minStack.top();    // return 0  
minStack.getMin(); // return -2
```

Constraints:

- $-2^{31} \leq \text{val} \leq 2^{31} - 1$
- Methods pop, top and getMin operations will always be called on **non-empty** stacks.
- At most 3×10^4 calls will be made to push, pop, top, and getMin.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MinStack {
    LinkedList<int[]> stack = null;

    public MinStack() {
        this.stack = new LinkedList<>();
    }

    public void push(int val) {
        int minVal;
        if(stack.isEmpty() || stack.peek()[1] > val) {
            minVal = val;
        } else {
            minVal = stack.peek()[1];
        }
        stack.push(new int[]{val, minVal});
    }

    public void pop() {
        stack.pop();
    }

    public int top() {
        return stack.peek()[0];
    }

    public int getMin() {
        return stack.peek()[1];
    }
}

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack obj = new MinStack();
 * obj.push(val);
 * obj.pop();
 * int param_3 = obj.top();
 * int param_4 = obj.getMin();
 */
```


931 Maximum Frequency Stack (link)

Description

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the `FreqStack` class:

- `FreqStack()` constructs an empty frequency stack.
- `void push(int val)` pushes an integer `val` onto the top of the stack.
- `int pop()` removes and returns the most frequent element in the stack.
 - If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

Input

```
["FreqStack", "push", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]  
[[], [5], [7], [5], [7], [4], [5], [], [], [], []]
```

Output

```
[null, null, null, null, null, null, null, 5, 7, 5, 4]
```

Explanation

```
FreqStack freqStack = new FreqStack();  
freqStack.push(5); // The stack is [5]  
freqStack.push(7); // The stack is [5,7]  
freqStack.push(5); // The stack is [5,7,5]  
freqStack.push(7); // The stack is [5,7,5,7]  
freqStack.push(4); // The stack is [5,7,5,7,4]  
freqStack.push(5); // The stack is [5,7,5,7,4,5]  
freqStack.pop();    // return 5, as 5 is the most frequent. The stack becomes [5,7,5,7,4].  
freqStack.pop();    // return 7, as 5 and 7 is the most frequent, but 7 is closest to the top. The stack  
freqStack.pop();    // return 5, as 5 is the most frequent. The stack becomes [5,7,4].  
freqStack.pop();    // return 4, as 4, 5 and 7 is the most frequent, but 4 is closest to the top. The sta
```

Constraints:

- $0 \leq \text{val} \leq 10^9$
- At most $2 * 10^4$ calls will be made to `push` and `pop`.
- It is guaranteed that there will be at least one element in the stack before calling `pop`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class FreqStack {
    Map<Integer, Integer> freqMap;
    List<LinkedList<Integer>> stacks;
    int maxFreq;

    public FreqStack() {
        freqMap = new HashMap<>();
        stacks = new ArrayList<>();
        stacks.add(new LinkedList<>());
        maxFreq = 0;
    }

    public void push(int val) {
        int freq = freqMap.getOrDefault(val, 0)+1;
        freqMap.put(val, freq);
        maxFreq = Math.max(freq, maxFreq);
        if(stacks.size() == freq)
            stacks.add(new LinkedList<>());
        stacks.get(freq).push(val);
    }

    public int pop() {
        LinkedList<Integer> maxList = stacks.get(maxFreq);
        int val = maxList.pop();

        freqMap.put(val, freqMap.get(val)-1);
        if(maxList.size() == 0) {
            stacks.remove(maxFreq);
            maxFreq--;
        }
        return val;
    }
}

/**
 * Your FreqStack object will be instantiated and called as such:
 * FreqStack obj = new FreqStack();
 * obj.push(val);
 * int param_2 = obj.pop();
 */
```

[146 LRU Cache \(link\)](#)

Description

Design a data structure that follows the constraints of a [Least Recently Used \(LRU\) cache](#).

Implement the LRUCache class:

- LRUCache(int capacity) Initialize the LRU cache with **positive** size capacity.
- int get(int key) Return the value of the key if the key exists, otherwise return -1.
- void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, **evict** the least recently used key.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]  
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Explanation

```
LRUCache lRUCache = new LRUCache(2);  
lRUCache.put(1, 1); // cache is {1=1}  
lRUCache.put(2, 2); // cache is {1=1, 2=2}  
lRUCache.get(1);    // return 1  
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}  
lRUCache.get(2);    // returns -1 (not found)  
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}  
lRUCache.get(1);    // return -1 (not found)  
lRUCache.get(3);    // return 3  
lRUCache.get(4);    // return 4
```

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 10^4$
- $0 \leq \text{value} \leq 10^5$
- At most $2 * 10^5$ calls will be made to get and put.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class DLL {
    public DLL next;
    public DLL prev;
    public int key;
    public int val;

    public DLL(int key, int val) {
        this.key = key;
        this.val = val;
    }
}

class LRUCache {
    private int capacity;
    private DLL tail;
    private DLL head;
    private Map<Integer, DLL> cache;

    LRUCache(int capacity) {
        this.capacity = capacity;
        this.cache = new HashMap<>();

        this.tail = new DLL(0,0);
        this.head = new DLL(0,0);
        this.tail.prev = this.head;
        this.head.next = this.tail;
    }

    private void removeNode(DLL node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void moveToFront(DLL node) {
        removeNode(node);
        addToFront(node);
    }

    private void addToFront(DLL node) {
        node.next = this.head.next;
        this.head.next.prev = node;

        this.head.next = node;
        node.prev = this.head;
    }

    public int get(int key) {
        DLL node = this.cache.get(key);
        if(node == null)
            return -1;
        moveToFront(node);
        return node.val;
    }

    public void put(int key, int value) {
        DLL node = cache.get(key);
        if(node != null) {
            node.val = value;
            moveToFront(node);
            return;
        }

        if(this.cache.size() >= capacity) {
            DLL lru = this.tail.prev;
            this.cache.remove(lru.key);
        }
    }
}
```

```
        removeNode(lru);  
    }  
    DLL newHead = new DLL(key,value);  
    this.cache.put(key, newHead);  
    addToFront(newHead);  
}  
}
```

1023 Time Based Key-Value Store (link)

Description

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the `TimeMap` class:

- `TimeMap()` Initializes the object of the data structure.
- `void set(String key, String value, int timestamp)` Stores the key `key` with the value `value` at the given time `timestamp`.
- `String get(String key, int timestamp)` Returns a value such that `set` was called previously, with `timestamp_prev <= timestamp`. If there are multiple such values, it returns the value associated with the largest `timestamp_prev`. If there are no values, it returns `""`.

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set", "get", "get"]  
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

Explanation

```
TimeMap timeMap = new TimeMap();  
timeMap.set("foo", "bar", 1); // store the key "foo" and value "bar" along with timestamp = 1.  
timeMap.get("foo", 1);       // return "bar"  
timeMap.get("foo", 3);       // return "bar", since there is no value corresponding to foo at timestamp 3.  
timeMap.set("foo", "bar2", 4); // store the key "foo" and value "bar2" along with timestamp = 4.  
timeMap.get("foo", 4);       // return "bar2"  
timeMap.get("foo", 5);       // return "bar2"
```

Constraints:

- $1 \leq \text{key.length}, \text{value.length} \leq 100$
- `key` and `value` consist of lowercase English letters and digits.
- $1 \leq \text{timestamp} \leq 10^7$
- All the timestamps `timestamp` of `set` are strictly increasing.
- At most $2 * 10^5$ calls will be made to `set` and `get`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class TimeMap {
    private Map<String, List<CacheEntry>> cache;

    public TimeMap() {
        cache = new HashMap<>();
    }

    public void set(String key, String value, int timestamp) {
        cache.computeIfAbsent(key, k -> new ArrayList<>()).add(new CacheEntry(value, timestamp));
    }

    public String get(String key, int timestamp) {
        List<CacheEntry> values = cache.get(key);
        if(values == null) return "";

        int left = 0, right = values.size()-1;
        int prevKeyIndex = -1;
        while(left <= right) {
            int mid = (left+right)>>1;
            if(values.get(mid).timestamp <= timestamp) {
                prevKeyIndex = mid;
                left = mid+1;
            } else {
                right = mid-1;
            }
        }

        return prevKeyIndex == -1 ? "" : values.get(prevKeyIndex).value;
    }
}

class CacheEntry {
    String value;
    int timestamp;
    CacheEntry(String _value, int _timestamp) {
        value = _value;
        timestamp = _timestamp;
    }
}

/**
 * Your TimeMap object will be instantiated and called as such:
 * TimeMap obj = new TimeMap();
 * obj.set(key,value,timestamp);
 * String param_2 = obj.get(key,timestamp);
 */
```

[139 Word Break \(link\)](#)

Description

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.
```

Example 3:

```
Input: s = "catsanddog", wordDict = ["cats","dog","sand","and","cat"]
Output: false
```

Constraints:

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` and `wordDict[i]` consist of only lowercase English letters.
- All the strings of `wordDict` are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        int strLength = s.length();

        // boolean[] dp = new boolean[strLength+1];
        // dp[0] = true;
        // for(int i=0; i < strLength; i++) {
        //     if(dp[i]) {
        //         for(String word: wordDict) {
        //             int wl = word.length();
        //             if(i+wl <= strLength && word.equals(s.substring(i, i+wl))) {
        //                 dp[i+wl] = true;
        //             }
        //         }
        //     }
        // }
        // return dp[strLength];

        // Set<Integer> indices=new HashSet<>();
        // indices.add(0);
        // for(int i=0; i< strLength; i++) {
        //     for(String word: wordDict) {
        //         int wl = word.length();
        //         if(i+wl<=strLength && word.equals(s.substring(i, i+wl))) {
        //             indices.add(i+wl);
        //             break;
        //         }
        //     }
        // }
        // }

        TrieNode dict = buildTrie(wordDict);
        Set<Integer> indices=new HashSet<>();
        indices.add(0);
        for(int i=0; i< strLength; i++) {
            if(indices.contains(i)) {
                TrieNode node = dict;
                for(int j=i; j < strLength; j++) {
                    node = node.next[s.charAt(j)-'a'];
                    if(node == null) break;
                    if(node.word) indices.add(j+1);
                }
            }
        }
        return indices.contains(strLength);
    }

    TrieNode buildTrie(List<String> words) {
        TrieNode root = new TrieNode();
        for(String word: words) {
            TrieNode curr = root;
            for(char ch: word.toCharArray()) {
                int i = ch-'a';
                if(curr.next[i] == null)
                    curr.next[i] = new TrieNode();
                curr = curr.next[i];
            }
            curr.word = true;
        }
        return root;
    }
}

class TrieNode {
    public TrieNode[] next = new TrieNode[26];
}
```

```
} public boolean word;
```

143 Reorder List (link)

Description

You are given the head of a singly linked-list. The list can be represented as:

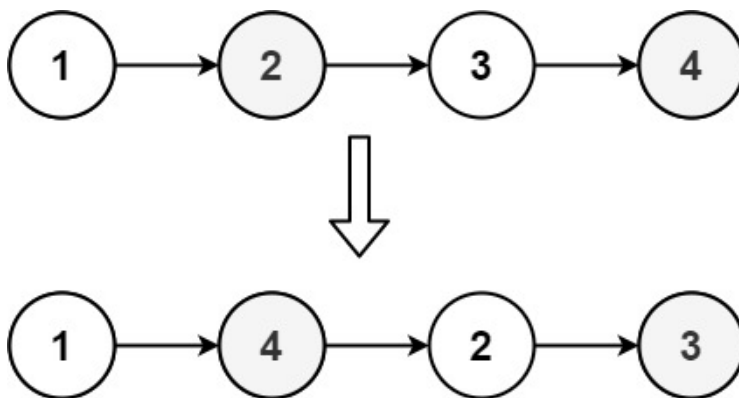
$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

You may not modify the values in the list's nodes. Only nodes themselves may be changed.

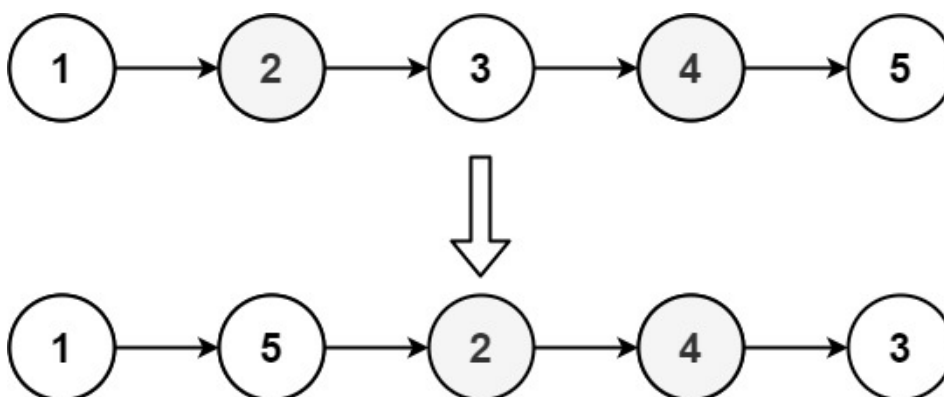
Example 1:



Input: head = [1,2,3,4]

Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]

Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range $[1, 5 * 10^4]$.
- $1 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public void reorderList(ListNode head) {
        if(head.next == null || head.next.next == null)
            return;

        LinkedList<Integer> st = new LinkedList();
        ListNode l1 = head, l2 = head, next = null;
        while(l2 != null && l2.next != null) {
            l2 = l2.next.next;
            l1 = l1.next;
        }

        l2 = l1.next;
        l1.next = null;
        while(l2 != null) {
            st.push(l2.val);
            l2 = l2.next;
        }

        l1 = head;
        l2 = l1;
        while(!st.isEmpty()) {
            next = l1.next;
            l1.next = new ListNode(st.pop(), next);
            l1 = next;
        }
    }
}
```

138 Copy List with Random Pointer (link)

Description

A linked list of length n is given such that each node contains an additional random pointer, which could point to any node in the list, or `null`.

Construct a **deep copy** of the list. The deep copy should consist of exactly n **brand new** nodes, where each new node has its value set to the value of its corresponding original node. Both the `next` and `random` pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. **None of the pointers in the new list should point to nodes in the original list.**

For example, if there are two nodes x and y in the original list, where $x.random \rightarrow y$, then for the corresponding two nodes x and y in the copied list, $x.random \rightarrow y$.

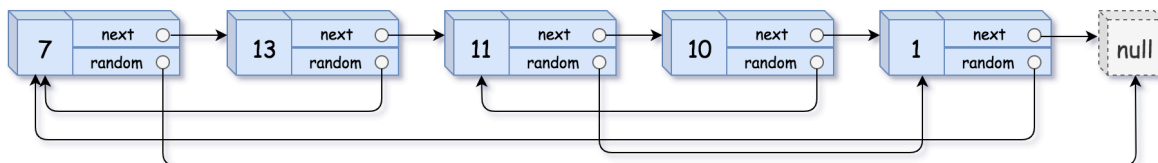
Return *the head of the copied linked list*.

The linked list is represented in the input/output as a list of n nodes. Each node is represented as a pair of `[val, random_index]` where:

- `val`: an integer representing `Node.val`
- `random_index`: the index of the node (range from 0 to $n-1$) that the `random` pointer points to, or `null` if it does not point to any node.

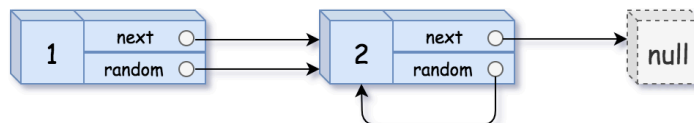
Your code will **only** be given the head of the original linked list.

Example 1:



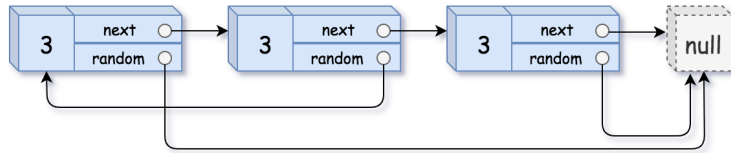
Input: head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
Output: [[7,null],[13,0],[11,4],[10,2],[1,0]]

Example 2:



Input: head = [[1,1],[2,1]]
Output: [[1,1],[2,1]]

Example 3:



Input: head = [[3,null],[3,0],[3,null]]
Output: [[3,null],[3,0],[3,null]]

Constraints:

- $0 \leq n \leq 1000$
- $-10^4 \leq \text{Node.val} \leq 10^4$
- `Node.random` is null or is pointing to some node in the linked list.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    int val;
    Node next;
    Node random;

    public Node(int val) {
        this.val = val;
        this.next = null;
        this.random = null;
    }
}
*/

class Solution {
    public Node copyRandomList(Node head) {
        if(head == null)
            return null;

        Map<Node, Node> map = new HashMap<>();
        Node node = head;
        while(node != null) {
            map.put(node, new Node(node.val));
            node = node.next;
        }

        node = head;
        Node clone = new Node(-1), newHead = map.get(head);
        while(node != null) {
            clone.next = map.get(node);
            if(node.random != null) {
                clone.next.random = map.get(node.random);
            }
            node = node.next;
            clone = clone.next;
        }
        map.clear();
        return newHead;
    }
}
```


[124 Binary Tree Maximum Path Sum \(link\)](#)

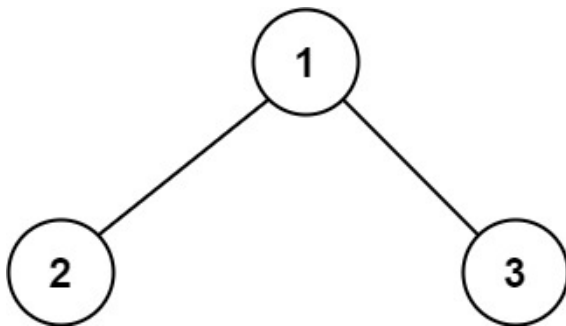
Description

A **path** in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence **at most once**. Note that the path does not need to pass through the root.

The **path sum** of a path is the sum of the node's values in the path.

Given the root of a binary tree, return *the maximum path sum of any non-empty path*.

Example 1:

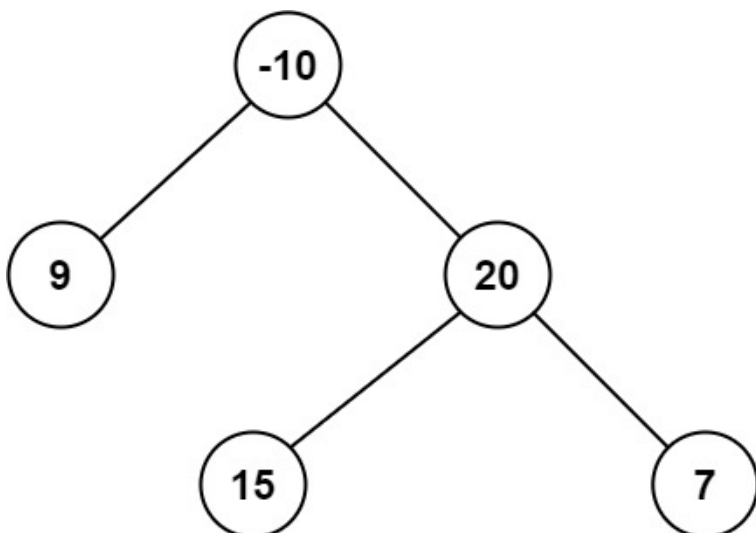


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    private int maxSum=Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) {
        maxSum(root);
        return maxSum;
    }

    int maxSum(TreeNode root) {
        if(root == null) return 0;

        int leftSum = Math.max(0, maxSum(root.left));
        int rightSum = Math.max(0, maxSum(root.right));

        int eitherPathSum = root.val+Math.max(leftSum, rightSum);
        int leftRootRightSum = root.val+leftSum+rightSum;
        maxSum = Math.max(maxSum, Math.max(leftRootRightSum, eitherPathSum));
        return eitherPathSum;
    }
}
```

67 Add Binary (link)

Description

Given two binary strings *a* and *b*, return *their sum as a binary string*.

Example 1:

```
Input: a = "11", b = "1"
Output: "100"
```

Example 2:

```
Input: a = "1010", b = "1011"
Output: "10101"
```

Constraints:

- $1 \leq a.length, b.length \leq 10^4$
- *a* and *b* consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String addBinary(String a, String b) {
        char ZERO='0';
        int la = a.length();
        int lb = b.length();
        if(la == 0) return b;
        if(lb == 0) return a;

        int carry=0, k=Math.max(la,lb)+1, i=la-1, j=lb-1;
        char[] result = new char[k];
        while(i>-1 || j>-1 || carry == 1) {
            int aa = i>-1 ? a.charAt(i--)-ZERO:0;
            int bb = j>-1 ? b.charAt(j--)-ZERO:0;

            int sum = aa+bb+carry;
            carry = (sum&2) > 0 ?1:0;
            sum = sum&1;

            result[--k]=(char) (ZERO+sum);
        }

        String resultStr = String.valueOf(result);
        return k == 0? resultStr:resultStr.substring(1);
    }
}
```

[46 Permutations \(link\)](#)

Description

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 6$
- $-10 \leq \text{nums}[i] \leq 10$
- All the integers of `nums` are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> permute(int[] arr) {
        List<List<Integer>> permutations = new ArrayList<>();
        dfs(arr, 0, permutations);
        return permutations;
    }

    void dfs(int[] arr, int start, List<List<Integer>> permutations) {
        if(arr.length == start) {
            List<Integer> list = new ArrayList<>();
            for(int num: arr) list.add(num);
            permutations.add(list);
            return;
        }

        for(int i=start; i < arr.length; i++) {
            swap(arr, start, i);
            dfs(arr, start+1, permutations);
            swap(arr, start, i);
        }
    }

    void swap(int[] nums, int i, int j) {
        if(i == j) return;
        int t = nums[i];
        nums[i] = nums[j];
        nums[j] = t;
    }
}
```

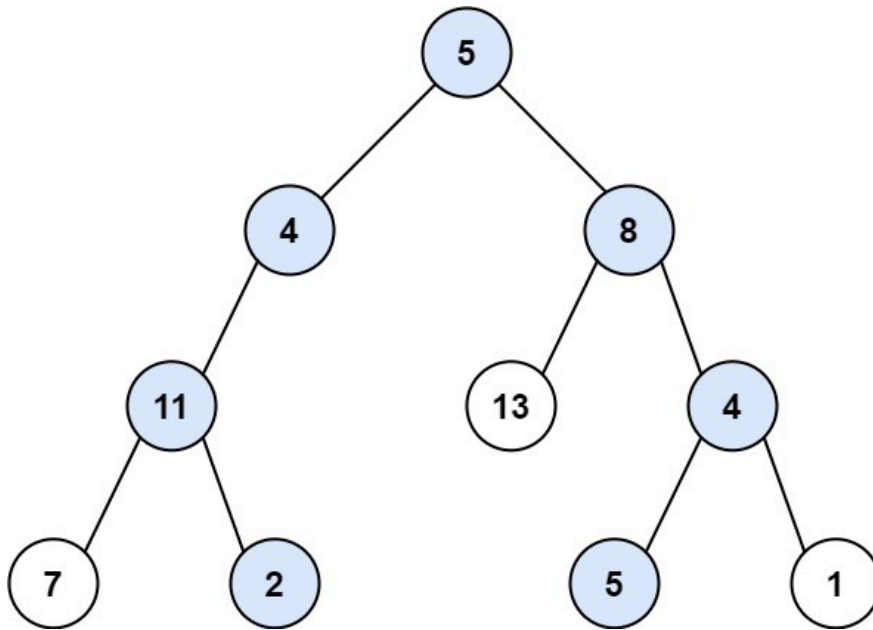
113 Path Sum II (link)

Description

Given the root of a binary tree and an integer `targetSum`, return *all root-to-leaf paths where the sum of the node values in the path equals `targetSum`*. Each path should be returned as a list of the node **values**, not node references.

A **root-to-leaf** path is a path starting from the root and ending at any leaf node. A **leaf** is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

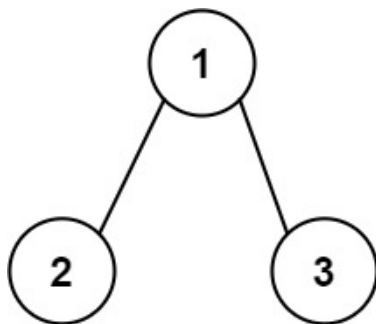
Output: [[5,4,11,2],[5,8,4,5]]

Explanation: There are two paths whose sum equals targetSum:

$5 + 4 + 11 + 2 = 22$

$5 + 8 + 4 + 5 = 22$

Example 2:



Input: root = [1,2,3], targetSum = 5

Output: []

Example 3:

Input: root = [1,2], targetSum = 0
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
        List<List<Integer>> allPaths = new ArrayList<>();
        dfs(root, targetSum, new LinkedList<>(), allPaths);
        return allPaths;
    }

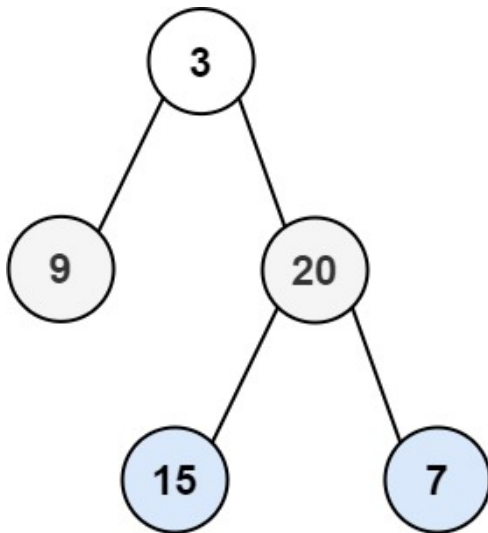
    void dfs(TreeNode root, int targetSum, LinkedList<Integer> stack, List<List<Integer>> result) {
        if(root == null) return;
        stack.addLast(root.val);
        if(root.left == null && root.right == null && targetSum == root.val) {
            result.add(new ArrayList<>(stack));
        } else {
            dfs(root.left, targetSum-root.val, stack, result);
            dfs(root.right, targetSum-root.val, stack, result);
        }
        stack.removeLast();
    }
}
```

[103 Binary Tree Zigzag Level Order Traversal \(link\)](#)

Description

Given the `root` of a binary tree, return *the zigzag level order traversal of its nodes' values*. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: `root = [3,9,20,null,null,15,7]`
Output: `[[3],[20,9],[15,7]]`

Example 2:

Input: `root = [1]`
Output: `[[1]]`

Example 3:

Input: `root = []`
Output: `[]`

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if(root != null) {
            Queue<TreeNode> qu = new LinkedList<>();
            qu.offer(root);

            boolean zigzag=false;
            while(!qu.isEmpty()) {
                int size = qu.size();
                List<Integer> entry = new ArrayList<>();
                while(size-- > 0) {
                    TreeNode node = qu.poll();
                    if(node.left != null)
                        qu.offer(node.left);
                    if(node.right != null)
                        qu.offer(node.right);

                    if(zigzag)
                        entry.addFirst(node.val);
                    else
                        entry.addLast(node.val);
                }
                result.add(entry);
                zigzag = !zigzag;
            }
            return result;
        }
    }
}
```

77 Combinations (link)

Description

Given two integers n and k , return *all possible combinations of k numbers chosen from the range $[1, n]$* .

You may return the answer in **any order**.

Example 1:

Input: $n = 4, k = 2$

Output: `[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]`

Explanation: There are 4 choose 2 = 6 total combinations.

Note that combinations are unordered, i.e., `[1,2]` and `[2,1]` are considered to be the same combination.

Example 2:

Input: $n = 1, k = 1$

Output: `[[1]]`

Explanation: There is 1 choose 1 = 1 total combination.

Constraints:

- $1 \leq n \leq 20$
- $1 \leq k \leq n$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    void backtrack(int n, int k, int start, LinkedList<Integer> stack, List<List<Integer>> result) {
        if(stack.size() == k)
            result.add(new ArrayList<>(stack));
        for(int i=start; i <= n; ++i) {
            stack.addLast(i);
            backtrack(n, k, i+1, stack, result);
            stack.removeLast();
        }
    }

    public List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(n, k, 1, new LinkedList<>(), result);
        return result;
    }
}
```

22 Generate Parentheses (link)

Description

Given n pairs of parentheses, write a function to *generate all combinations of well-formed parentheses*.

Example 1:

```
Input: n = 3  
Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]
```

Example 2:

```
Input: n = 1  
Output: ["()"]
```

Constraints:

- $1 \leq n \leq 8$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Parantheses {
    StringBuilder value;
    int open;
    int close;

    Parantheses(StringBuilder value, int open, int close) {
        this.value = value;
        this.open = open;
        this.close = close;
    }
}

class Solution {
    public List<String> generateParenthesis(int n) {
        if(n < 2) return List.of("(");
        List<String> result = new ArrayList<>();
        Queue<Parantheses> qu = new LinkedList<>();
        qu.offer(new Parantheses(new StringBuilder(), 0, 0));

        while(!qu.isEmpty()) {
            int size = qu.size();
            Parantheses temp = qu.poll();

            if(temp.open == n && temp.close == n) {
                result.add(String.valueOf(temp.value));
                continue;
            }

            if(temp.open < n) {
                StringBuilder value = new StringBuilder(temp.value);
                value.append('(');
                qu.offer(new Parantheses(value , temp.open+1, temp.close));
            }

            if(temp.open > temp.close) {
                StringBuilder value = new StringBuilder(temp.value);
                value.append(')');
                qu.offer(new Parantheses(value , temp.open, temp.close+1));
            }
            temp.value.setLength(0);
        }

        return result;
    }
}
```


56 Merge Intervals (link)

Description

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

Constraints:

- $1 \leq \text{intervals.length} \leq 10^4$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] merge(int[][] intervals) {
        if(intervals.length < 2)
            return intervals;

        Arrays.sort(intervals, (a,b) -> (a[0]-b[0]));
        LinkedList<int[]> mergedIntervals = new LinkedList<>();
        mergedIntervals.addLast(intervals[0]);
        for(int i=0; i< intervals.length; i++) {
            int[] last = mergedIntervals.getLast();
            if(last[1] < intervals[i][0]) {
                mergedIntervals.addLast(intervals[i]);
            } else {
                last[1]=Math.max(last[1], intervals[i][1]);
            }
        }
        return mergedIntervals.toArray(new int[0][]);
    }
}
```

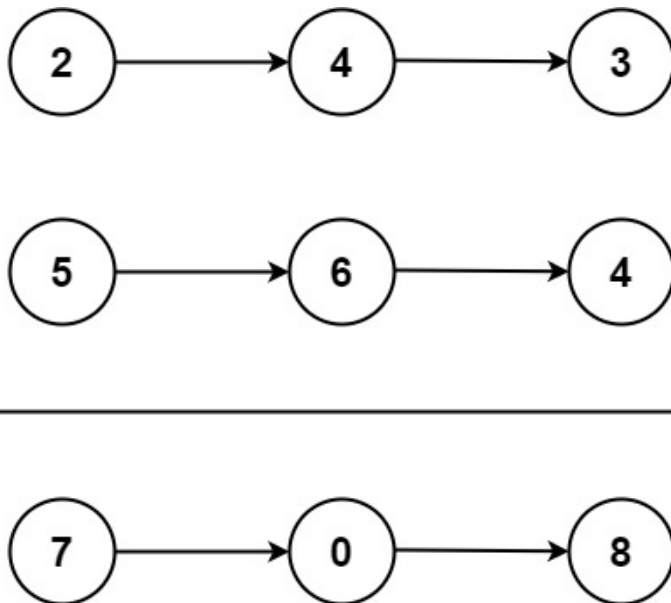
[2 Add Two Numbers \(link\)](#)

Description

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [7,0,8]
Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]
Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        int carry = 0;
        int sum = 0;
        ListNode sentinel = new ListNode(0);
        ListNode node = sentinel;
        while(!(l1 == null && l2 == null)) {
            if(l1 != null) {
                sum += l1.val;
                l1 = l1.next;
            }

            if(l2 != null) {
                sum += l2.val;
                l2 = l2.next;
            }

            sum += carry;
            carry = sum/10;
            node.next = new ListNode(sum%10);
            node = node.next;
            sum = 0;
        }
        if( carry != 0) {
            node.next = new ListNode(carry);
        }
        return sentinel.next;
    }
}
```

349 Intersection of Two Arrays (link)

Description

Given two integer arrays `nums1` and `nums2`, return *an array of their intersection*. Each element in the result must be **unique** and you may return the result in **any order**.

Example 1:

Input: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`
Output: `[2]`

Example 2:

Input: `nums1 = [4,9,5]`, `nums2 = [9,4,9,8,4]`
Output: `[9,4]`
Explanation: `[4,9]` is also accepted.

Constraints:

- `1 <= nums1.length, nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 1000`

(scroll down for solution)

Solution

Language: javascript

Status: Accepted

```
/**
 * @param {number[]} nums1
 * @param {number[]} nums2
 * @return {number[]}
 */
var intersection = function(nums1, nums2) {
  let map = {}
  let result = []
  nums1.map(i => {
    if(!map[i]) {
      map[i]=i
    }
  })

  nums2.map(i => {
    if(map[i] === i) {
      result.push(i)
      map[i]=-10000
    }
  })

  return result
};
```

1127 Last Stone Weight (link)

Description

You are given an array of integers `stones` where `stones[i]` is the weight of the i^{th} stone.

We are playing a game with the stones. On each turn, we choose the **heaviest two stones** and smash them together. Suppose the heaviest two stones have weights x and y with $x \leq y$. The result of this smash is:

- If $x == y$, both stones are destroyed, and
- If $x \neq y$, the stone of weight x is destroyed, and the stone of weight y has new weight $y - x$.

At the end of the game, there is **at most one** stone left.

Return *the weight of the last remaining stone*. If there are no stones left, return 0 .

Example 1:

Input: `stones = [2,7,4,1,8,1]`

Output: `1`

Explanation:

We combine 7 and 8 to get 1 so the array converts to `[2,4,1,1,1]` then,
we combine 2 and 4 to get 2 so the array converts to `[2,1,1,1]` then,
we combine 2 and 1 to get 1 so the array converts to `[1,1,1]` then,
we combine 1 and 1 to get 0 so the array converts to `[1]` then that's the value of the last stone.

Example 2:

Input: `stones = [1]`

Output: `1`

Constraints:

- $1 \leq \text{stones.length} \leq 30$
- $1 \leq \text{stones}[i] \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>((a,b) -> b-a);
        for(int stone: stones) {
            pq.offer(stone);
        }
        while(pq.size() > 1) {
            int n1 = pq.poll();
            int n2 = pq.poll();
            if(n1 != n2) {
                pq.offer(Math.abs(n1-n2));
            }
        }
        return pq.isEmpty() ? 0 : pq.poll();
    }
}
```


542 01 Matrix [\(link\)](#)

Description

Given an $m \times n$ binary matrix `mat`, return *the distance of the nearest 0 for each cell*.

The distance between two cells sharing a common edge is 1.

Example 1:

0	0	0
0	1	0
0	0	0

Input: `mat = [[0,0,0],[0,1,0],[0,0,0]]`
Output: `[[0,0,0],[0,1,0],[0,0,0]]`

Example 2:

0	0	0
0	1	0
1	1	1

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`
Output: `[[0,0,0],[0,1,0],[1,2,1]]`

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 10^4$
- $1 \leq m * n \leq 10^4$

- `mat[i][j]` is either 0 or 1.
- There is at least one 0 in `mat`.

Note: This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] updateMatrix(int[][] mat) {
        int rows = mat.length;
        int cols = mat[0].length;
        int[][] DIRS = {{1,0},{-1,0},{0,1},{0,-1}};
        int MAXV = Integer.MAX_VALUE;

        if(rows == 0)
            return mat;
        int[][] distances = new int[rows][cols];
        Queue<int[]> qu = new ArrayDeque();
        for(int i=0; i<rows;i++) {
            for(int j=0; j<cols;j++) {
                if(mat[i][j]==0) {
                    qu.offer(new int[]{i, j, 0});
                } else {
                    distances[i][j]=MAXV;
                }
            }
        }

        while(!qu.isEmpty()) {
            int[] coord = qu.poll();
            int r = coord[0];
            int c = coord[1];
            int steps = 1+coord[2];

            for(int[] dir:DIRS) {
                int rd = r+dir[0];
                int cd = c+dir[1];
                if(rd>-1 && cd >-1 && rd<rows && cd<cols && distances[rd][cd] > steps) {
                    distances[rd][cd] = steps;
                    qu.offer(new int[]{rd, cd, steps});
                }
            }
        }
        return distances;
    }
}
```

2759 Flatten Deeply Nested Array [\(link\)](#)

Description

Given a **multi-dimensional** array `arr` and a depth `n`, return a **flattened** version of that array.

A **multi-dimensional** array is a recursive data structure that contains integers or other **multi-dimensional** arrays.

A **flattened** array is a version of that array with some or all of the sub-arrays removed and replaced with the actual elements in that sub-array. This flattening operation should only be done if the current depth of nesting is less than `n`. The depth of the elements in the first array are considered to be 0.

Please solve it without the built-in `Array.flat` method.

Example 1:

Input

```
arr = [1, 2, 3, [4, 5, 6], [7, 8, [9, 10, 11], 12], [13, 14, 15]]  
n = 0
```

Output

```
[1, 2, 3, [4, 5, 6], [7, 8, [9, 10, 11], 12], [13, 14, 15]]
```

Explanation

Passing a depth of `n=0` will always result in the original array. This is because the smallest possible c

Example 2:

Input

```
arr = [1, 2, 3, [4, 5, 6], [7, 8, [9, 10, 11], 12], [13, 14, 15]]  
n = 1
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, [9, 10, 11], 12, 13, 14, 15]
```

Explanation

The subarrays starting with 4, 7, and 13 are all flattened. This is because their depth of 0 is less than

Example 3:

Input

```
arr = [[1, 2, 3], [4, 5, 6], [7, 8, [9, 10, 11], 12], [13, 14, 15]]  
n = 2
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

Explanation

The maximum depth of any subarray is 1. Thus, all of them are flattened.

Constraints:

- $0 \leq \text{count of numbers in arr} \leq 10^5$
- $0 \leq \text{count of subarrays in arr} \leq 10^5$
- $\text{maxDepth} \leq 1000$
- $-1000 \leq \text{each number} \leq 1000$
- $0 \leq n \leq 1000$

(scroll down for solution)

Solution

Language: javascript

Status: Accepted

```
/**
 * @param {Array} arr
 * @param {number} depth
 * @return {Array}
 */
var flat = function (arr, n) {
  if(n === 0) return arr;
  const result = [];
  //   const stack = [...arr.map(item => [item, n])]
  //   while(stack.length > 0) {
  //     const [item, n] = stack.pop()
  //     if(Array.isArray(item) && n > 0) {
  //       stack.push(...item.map(subItem => [subItem, n-1]))
  //     } else {
  //       result.unshift(item)
  //     }
  //   }
  for(let i=0; i < arr.length; i++) {
    if(n > 0 && Array.isArray(arr[i])) {
      result.push(...flat(arr[i], n-1))
    } else {
      result.push(arr[i])
    }
  }
  return result;
};
```

[503 Next Greater Element II \(link\)](#)

Description

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return *the next greater number for every element in* `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Example 1:

```
Input: nums = [1,2,1]
Output: [2,-1,2]
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.
```

Example 2:

```
Input: nums = [1,2,3,4,3]
Output: [2,3,4,-1,4]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int len = nums.length;
        int[] res = new int[len];
        Stack<Integer> stack = new Stack<>();

        for(int i=0; i< 2*len; i++) {
            while(!stack.isEmpty() && nums[stack.peek()] < nums[i%len]) {
                res[stack.pop()] = nums[i%len];
            }
            if(i<len) {
                stack.push(i);
            }
        }
        while(!stack.isEmpty()) {
            res[stack.pop()] = -1;
        }
        return res;
    }
}
```


[2858 Join Two Arrays by ID \(link\)](#)

Description

Given two arrays `arr1` and `arr2`, return a new array `joinedArray`. All the objects in each of the two inputs arrays will contain an `id` field that has an integer value.

`joinedArray` is an array formed by merging `arr1` and `arr2` based on their `id` key. The length of `joinedArray` should be the length of unique values of `id`. The returned array should be sorted in **ascending** order based on the `id` key.

If a given `id` exists in one array but not the other, the single object with that `id` should be included in the result array without modification.

If two objects share an `id`, their properties should be merged into a single object:

- If a key only exists in one object, that single key-value pair should be included in the object.
- If a key is included in both objects, the value in the object from `arr2` should override the value from `arr1`.

Example 1:

Input:

```
arr1 = [
  {"id": 1, "x": 1},
  {"id": 2, "x": 9}
],
arr2 = [
  {"id": 3, "x": 5}
]
```

Output:

```
[
  {"id": 1, "x": 1},
  {"id": 2, "x": 9},
  {"id": 3, "x": 5}
]
```

Explanation: There are no duplicate ids so `arr1` is simply concatenated with `arr2`.

Example 2:

Input:

```
arr1 = [
  {"id": 1, "x": 2, "y": 3},
  {"id": 2, "x": 3, "y": 6}
],
arr2 = [
  {"id": 2, "x": 10, "y": 20},
  {"id": 3, "x": 0, "y": 0}
]
```

Output:

```
[
  {"id": 1, "x": 2, "y": 3},
  {"id": 2, "x": 10, "y": 20},
  {"id": 3, "x": 0, "y": 0}
]
```

Explanation: The two objects with `id=1` and `id=3` are included in the result array without modification. The

Example 3:

Input:

```
arr1 = [  
  {"id": 1, "b": {"b": 94}, "v": [4, 3], "y": 48}  
]  
arr2 = [  
  {"id": 1, "b": {"c": 84}, "v": [1, 3]}  
]
```

Output:

```
[  
  {"id": 1, "b": {"c": 84}, "v": [1, 3], "y": 48}  
]
```

Explanation: The two objects with id=1 are merged together. For the keys "b" and "v" the values from arr

Constraints:

- arr1 and arr2 are valid JSON arrays
- Each object in arr1 and arr2 has a unique integer id key
- $2 \leq \text{JSON.stringify(arr1).length} \leq 10^6$
- $2 \leq \text{JSON.stringify(arr2).length} \leq 10^6$

(scroll down for solution)

Solution

Language: javascript

Status: Accepted

```
/**
 * @param {Array} arr1
 * @param {Array} arr2
 * @return {Array}
 */
var join = function(arr1, arr2) {
  let map = {};
  arr1.forEach(obj => {
    map[obj.id] = { ...obj }
  })

  arr2.forEach(obj => {
    if(map[obj.id]) {
      map[obj.id] = { ...map[obj.id], ...obj }
    } else {
      map[obj.id] = { ...obj }
    }
  })
  let result = new Array()
  for(let key in map) result.push(map[key])
  return result
};
```

[494 Target Sum \(link\)](#)

Description

You are given an integer array `nums` and an integer `target`.

You want to build an **expression** out of `nums` by adding one of the symbols '+' and '-' before each integer in `nums` and then concatenate all the integers.

- For example, if `nums = [2, 1]`, you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1".

Return the number of different **expressions** that you can build, which evaluates to `target`.

Example 1:

```
Input: nums = [1,1,1,1,1], target = 3
Output: 5
Explanation: There are 5 ways to assign symbols to make the sum of nums be target 3.
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3
```

Example 2:

```
Input: nums = [1], target = 1
Output: 1
```

Constraints:

- $1 \leq \text{nums.length} \leq 20$
- $0 \leq \text{nums}[i] \leq 1000$
- $0 \leq \text{sum}(\text{nums}[i]) \leq 1000$
- $-1000 \leq \text{target} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int count = 0;
    Map<String, Integer> memo = new HashMap<>();
    String fKey = "%s|%s";
    public int findTargetSumWays(int[] nums, int target) {
        return findWays(nums, 0, 0, target);
    }

    int findWays(int[] nums, int start, int sum, int target) {
        if(start == nums.length)
            return sum == target ? 1:0;
        String key = String.format(fKey, sum, start);
        if(memo.get(key) != null)
            return memo.get(key);

        int ways = findWays(nums, start+1, sum+nums[start], target)+
            findWays(nums, start+1, sum-nums[start], target);
        memo.put(key, ways);
        return ways;
    }
}
```

[463 Island Perimeter \(link\)](#)

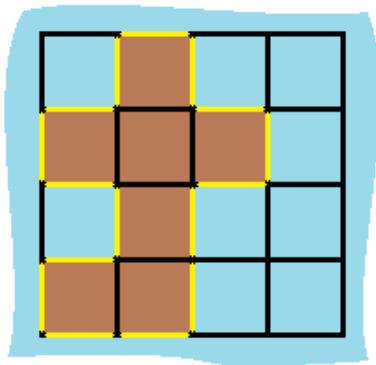
Description

You are given `row` x `col` grid representing a map where `grid[i][j] = 1` represents land and `grid[i][j] = 0` represents water.

Grid cells are connected **horizontally/vertically** (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



Input: `grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`

Output: 16

Explanation: The perimeter is the 16 yellow stripes in the image above.

Example 2:

Input: `grid = [[1]]`

Output: 4

Example 3:

Input: `grid = [[1,0]]`

Output: 4

Constraints:

- `row == grid.length`
- `col == grid[i].length`
- `1 <= row, col <= 100`
- `grid[i][j]` is 0 or 1.
- There is exactly one island in `grid`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int islandPerimeter(int[][] grid) {
        int perimeter = 0;
        for(int i=0; i < grid.length; i++) {
            for(int j=0; j < grid[0].length; j++) {
                if(grid[i][j] == 0) continue;
                perimeter+=4;
                if(i > 0 && grid[i-1][j] == 1) perimeter -=2;
                if(j > 0 && grid[i][j-1] == 1) perimeter -=2;
            }
        }
        return perimeter;
    }
}
```

448 Find All Numbers Disappeared in an Array (link)

Description

Given an array `nums` of n integers where `nums[i]` is in the range $[1, n]$, return *an array of all the integers in the range $[1, n]$ that do not appear in `nums`.*

Example 1:

Input: `nums = [4,3,2,7,8,2,3,1]`
Output: `[5,6]`

Example 2:

Input: `nums = [1,1]`
Output: `[2]`

Constraints:

- `n == nums.length`
- `1 <= n <= 105`
- `1 <= nums[i] <= n`

Follow up: Could you do it without extra space and in $O(n)$ runtime? You may assume the returned list does not count as extra space.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findDisappearedNumbers(int[] nums) {
        for(int i=0; i < nums.length; i++) {
            int index = Math.abs(nums[i])-1;
            if(nums[index] > 0) {
                nums[index] *=-1;
            }
        }
        System.out.println(Arrays.toString(nums));

        List<Integer> missing = new ArrayList<>();
        for(int i=0;i<nums.length;i++)
            if(nums[i] > 0)
                missing.add(i+1);
        return missing;
    }
}
```

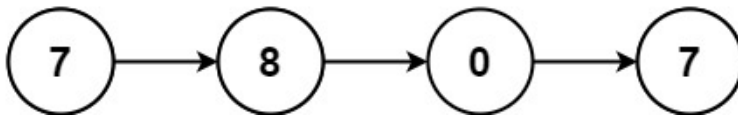
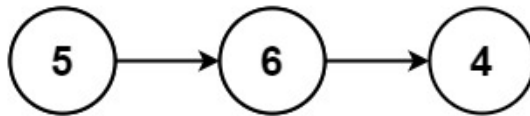
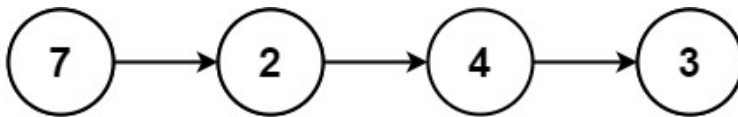
445 Add Two Numbers II (link)

Description

You are given two **non-empty** linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [7,2,4,3], l2 = [5,6,4]
Output: [7,8,0,7]

Example 2:

Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [8,0,7]

Example 3:

Input: l1 = [0], l2 = [0]
Output: [0]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

Follow up: Could you solve it without reversing the input lists?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {

        if(l1 == null && l2 == null) return null;
        else if(l1 != null && l2 == null) return l1;
        else if(l2 != null && l1 == null) return l2;

        int n1=0, n2=0;
        for(ListNode node = l1; node != null; n1++, node = node.next);
        for(ListNode node = l2; node != null; n2++, node = node.next);

        ListNode head = null, curr;
        int t=0;
        //making l1 bigger always
        if(n1 < n2) {
            curr = l1; l1 = l2; l2 = curr; t=n1; n1=n2;n2=t;
        }
        head = l1;
        while(n1 > n2) {
            n1--;
            l1 = l1.next;
        }
        int carry = add(l1, l2);
        if(carry > 0) {
            carry = addCarry(head, l1);
            return carry == 0 ? head: new ListNode(carry, head);
        }
        return head;
    }

    int addCarry(ListNode a, ListNode b) {
        if(a == null) return 0;
        if(a == b) return 1;
        int carry = addCarry(a.next, b);
        a.val += carry;
        carry = a.val/10;
        a.val %= 10;
        return carry;
    }

    int add(ListNode a, ListNode b) {
        if(a == null && b == null) return 0;
        int carry = add(a.next, b.next);
        a.val += b.val+carry;
        carry = a.val/10;
        a.val %= 10;
        return carry;
    }
}
```

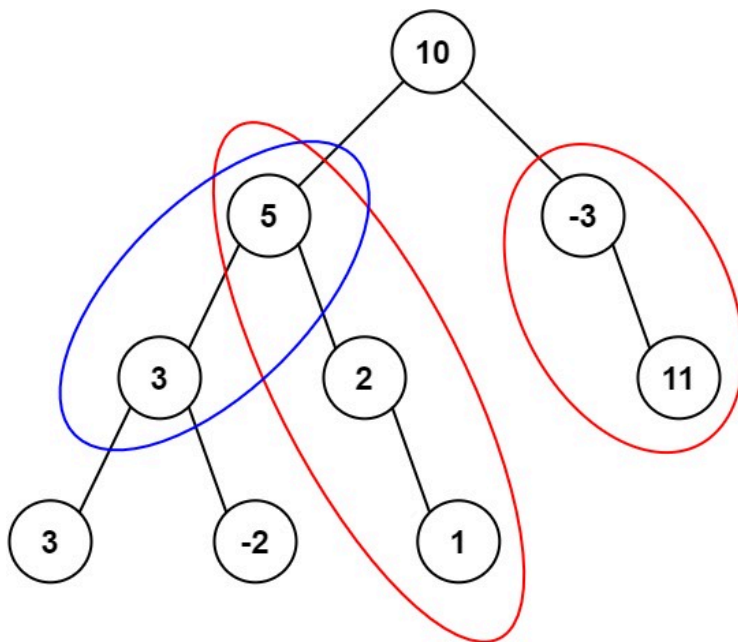
437 Path Sum III (link)

Description

Given the root of a binary tree and an integer `targetSum`, return *the number of paths where the sum of the values along the path equals targetSum*.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:



Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8

Output: 3

Explanation: The paths that sum to 8 are shown.

Example 2:

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

Output: 3

Constraints:

- The number of nodes in the tree is in the range $[0, 1000]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$
- $-1000 \leq \text{targetSum} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int pathSum(TreeNode root, int targetSum) {
        Map<Long, Integer> prefixSum = new HashMap<>();
        prefixSum.put(0L, 1);
        return count(root, targetSum, 0L, prefixSum);
    }

    int count(TreeNode root, int targetSum, long currentSum, Map<Long,Integer> prefixSum) {
        if(root == null) return 0;
        currentSum += root.val;
        int count = prefixSum.getOrDefault(currentSum - targetSum, 0);

        prefixSum.put(currentSum, prefixSum.getOrDefault(currentSum, 0)+1);
        count += count(root.left, targetSum, currentSum, prefixSum);
        count += count(root.right, targetSum, currentSum, prefixSum);
        prefixSum.put(currentSum, prefixSum.get(currentSum)-1);

        return count;
    }
}
```

[414 Third Maximum Number \(link\)](#)

Description

Given an integer array `nums`, return *the **third distinct maximum** number in this array. If the third maximum does not exist, return the **maximum** number.*

Example 1:

```
Input: nums = [3,2,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2.
The third distinct maximum is 1.
```

Example 2:

```
Input: nums = [1,2]
Output: 2
Explanation:
The first distinct maximum is 2.
The second distinct maximum is 1.
The third distinct maximum does not exist, so the maximum (2) is returned instead.
```

Example 3:

```
Input: nums = [2,2,3,1]
Output: 1
Explanation:
The first distinct maximum is 3.
The second distinct maximum is 2 (both 2's are counted together since they have the same value).
The third distinct maximum is 1.
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Can you find an $O(n)$ solution?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int thirdMax(int[] nums) {
        long first = Long.MIN_VALUE, second = first, third = first;
        for(int i=0; i < nums.length; i++) {
            if(nums[i] > first) {
                third = second;
                second = first;
                first = nums[i];
            } else if(nums[i] > second && nums[i] < first) {
                third = second;
                second = nums[i];
            } else if(nums[i] > third && nums[i] < second) {
                third = nums[i];
            }
        }
        return third == Long.MIN_VALUE ? (int) first: (int) third;
    }
}
```


871 Keys and Rooms (link)

Description

There are n rooms labeled from 0 to $n - 1$ and all the rooms are locked except for room 0 . Your goal is to visit all the rooms. However, you cannot enter a locked room without having its key.

When you visit a room, you may find a set of **distinct keys** in it. Each key has a number on it, denoting which room it unlocks, and you can take all of them with you to unlock the other rooms.

Given an array `rooms` where `rooms[i]` is the set of keys that you can obtain if you visited room i , return `true` if you can visit **all the rooms**, or `false` otherwise.

Example 1:

```
Input: rooms = [[1],[2],[3],[]]
Output: true
Explanation:
We visit room 0 and pick up key 1.
We then visit room 1 and pick up key 2.
We then visit room 2 and pick up key 3.
We then visit room 3.
Since we were able to visit every room, we return true.
```

Example 2:

```
Input: rooms = [[1,3],[3,0,1],[2],[0]]
Output: false
Explanation: We can not enter room number 2 since the only key that unlocks it is in that room.
```

Constraints:

- $n == \text{rooms.length}$
- $2 \leq n \leq 1000$
- $0 \leq \text{rooms}[i].\text{length} \leq 1000$
- $1 \leq \sum(\text{rooms}[i].\text{length}) \leq 3000$
- $0 \leq \text{rooms}[i][j] < n$
- All the values of `rooms[i]` are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int enteredRooms = 0;
    boolean[] visited;
    public boolean canVisitAllRooms(List<List<Integer>> rooms) {
        int N = rooms.size();
        visited = new boolean[N];
        visit(rooms, 0);
        return N==enteredRooms;
    }

    void visit(List<List<Integer>> rooms, int roomIdx) {
        enteredRooms++;
        visited[roomIdx] = true;
        if(rooms.get(roomIdx) == null) return;
        for(int newRoom: rooms.get(roomIdx)) {
            if(!visited[newRoom]) {
                visit(rooms, newRoom);
            }
        }
    }
}
```

416 Partition Equal Subset Sum (link)

Description

Given an integer array `nums`, return `true` *if you can partition the array into two subsets such that the sum of the elements in both subsets is equal* or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`

Output: `true`

Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`

Output: `false`

Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    Boolean[][] memo;
    int sum=0;
    public boolean canPartition(int[] nums) {
        int subSetSum = 0;
        for(int num: nums)
            subSetSum += num;

        if( (subSetSum&1) > 0)
            return false;

        sum = subSetSum >> 1;
        memo = new Boolean[nums.length+1][sum+1];
        return dp(nums, 0, 0);
    }

    boolean dp(int[] nums, int start, int subsetSum) {
        if(subsetSum == sum) return true;
        if(start+1 >= nums.length || subsetSum > sum) return false;
        if(memo[start][subsetSum] != null) return memo[start][subsetSum];

        memo[start][subsetSum] = dp(nums, start+1, subsetSum+nums[start+1]) || dp(nums, start+1, subsetSum);
        return memo[start][subsetSum];
    }
}
```

882 Peak Index in a Mountain Array (link)

Description

You are given an integer **mountain** array `arr` of length `n` where the values increase to a **peak element** and then decrease.

Return the index of the peak element.

Your task is to solve it in $O(\log(n))$ time complexity.

Example 1:

Input: `arr = [0,1,0]`

Output: 1

Example 2:

Input: `arr = [0,2,1,0]`

Output: 1

Example 3:

Input: `arr = [0,10,5,2]`

Output: 1

Constraints:

- $3 \leq \text{arr.length} \leq 10^5$
- $0 \leq \text{arr}[i] \leq 10^6$
- `arr` is **guaranteed** to be a mountain array.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int peakIndexInMountainArray(int[] arr) {
        int left = 0, right = arr.length-1;
        while(left < right) {
            int mid = left+(right-left)/2;
            if(arr[mid] < arr[mid+1]) {
                left = mid+1;
            } else {
                right = mid;
            }
        }
        return left;
    }
}
```

[941 Sort Array By Parity \(link\)](#)

Description

Given an integer array `nums`, move all the even integers at the beginning of the array followed by all the odd integers.

Return *any array that satisfies this condition*.

Example 1:

Input: `nums = [3,1,2,4]`

Output: `[2,4,3,1]`

Explanation: The outputs `[4,2,3,1]`, `[2,4,1,3]`, and `[4,2,1,3]` would also be accepted.

Example 2:

Input: `nums = [0]`

Output: `[0]`

Constraints:

- `1 <= nums.length <= 5000`
- `0 <= nums[i] <= 5000`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] sortByParity(int[] nums) {
        int left=0, right = nums.length-1;
        while(left < right) {
            while(left < nums.length && (nums[left]&1) == 0) left++;
            while(right > -1 && (nums[right]&1) != 0) right--;
            if( left < right) {
                int tmp = nums[left];
                nums[left]=nums[right];
                nums[right]=tmp;
            }
            left++;
            right--;
        }
        return nums;
    }
}
```


948 Sort an Array [\(link\)](#)

Description

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in** functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: `nums = [5,2,3,1]`

Output: `[1,2,3,5]`

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and



Example 2:

Input: `nums = [5,1,1,2,0,0]`

Output: `[0,0,1,1,2,5]`

Explanation: Note that the values of `nums` are not necessarily unique.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-5 * 10^4 \leq \text{nums}[i] \leq 5 * 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] sortArray(int[] nums) {
        quicksort(nums, 0, nums.length - 1);
        return nums;
    }

    void quicksort(int[] nums, int lo, int hi) {
        if(lo >= hi) return;
        int pivot = partition(nums, lo, hi);
        quicksort(nums, lo, pivot-1);
        quicksort(nums, pivot+1, hi);
    }

    int partition(int[] nums, int lo, int hi) {
        int l = lo;
        int r = hi-1;
        int pivot = lo+(int) (Math.random()*(hi-lo));

        swap(nums, pivot, hi);

        while (l <= r) {
            while(l <= r && nums[l] < nums[hi]) l++;
            while(l <= r && nums[r] > nums[hi]) r--;
            if(l <= r) swap(nums, l++, r--);
        }
        swap(nums, l, hi);
        return l;
    }

    void swap(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }
}
```

950 X of a Kind in a Deck of Cards (link)

Description

You are given an integer array `deck` where `deck[i]` represents the number written on the i^{th} card.

Partition the cards into **one or more groups** such that:

- Each group has **exactly** x cards where $x > 1$, and
- All the cards in one group have the same integer written on them.

Return `true` *if such partition is possible*, or `false` *otherwise*.

Example 1:

```
Input: deck = [1,2,3,4,4,3,2,1]
Output: true
Explanation: Possible partition [1,1],[2,2],[3,3],[4,4].
```

Example 2:

```
Input: deck = [1,1,1,2,2,2,3,3]
Output: false
Explanation: No possible partition.
```

Constraints:

- $1 \leq \text{deck.length} \leq 10^4$
- $0 \leq \text{deck}[i] < 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean hasGroupsSizeX(int[] deck) {
        int n = deck.length;
        int[] counts = new int[10001];
        for(int num: deck)
            counts[num]++;
        int equalPartitionSize = 0;
        int i = 0;
        while(i < n && equalPartitionSize == 0) {
            equalPartitionSize = counts[deck[i++]];
        }

        while(i < n) {
            if(counts[deck[i]] == 0) continue;
            equalPartitionSize = gcd(equalPartitionSize, counts[deck[i++]]);
        }
        return equalPartitionSize > 1;
    }

    public int gcd(int x, int y) {
        return x == 0 ? y : gcd(y%x, x);
    }
}
```

1029 Vertical Order Traversal of a Binary Tree (link)

Description

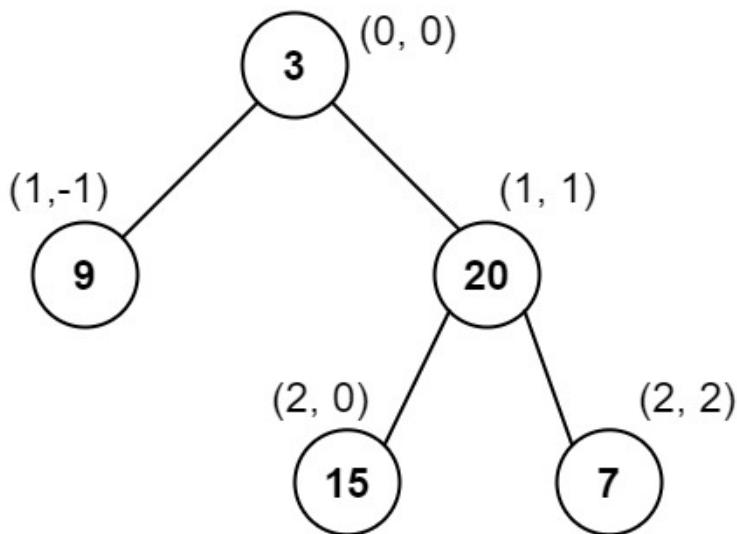
Given the root of a binary tree, calculate the **vertical order traversal** of the binary tree.

For each node at position (row, col), its left and right children will be at positions (row + 1, col - 1) and (row + 1, col + 1) respectively. The root of the tree is at (0, 0).

The **vertical order traversal** of a binary tree is a list of top-to-bottom orderings for each column index starting from the leftmost column and ending on the rightmost column. There may be multiple nodes in the same row and same column. In such a case, sort these nodes by their values.

Return *the vertical order traversal of the binary tree*.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: [[9],[3,15],[20],[7]]

Explanation:

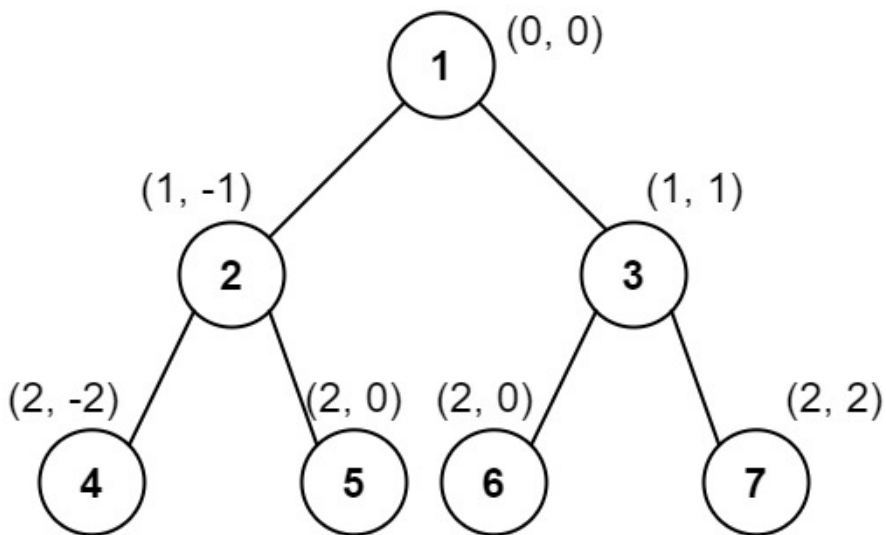
Column -1: Only node 9 is in this column.

Column 0: Nodes 3 and 15 are in this column in that order from top to bottom.

Column 1: Only node 20 is in this column.

Column 2: Only node 7 is in this column.

Example 2:



Input: root = [1,2,3,4,5,6,7]

Output: [[4],[2],[1,5,6],[3],[7]]

Explanation:

Column -2: Only node 4 is in this column.

Column -1: Only node 2 is in this column.

Column 0: Nodes 1, 5, and 6 are in this column.

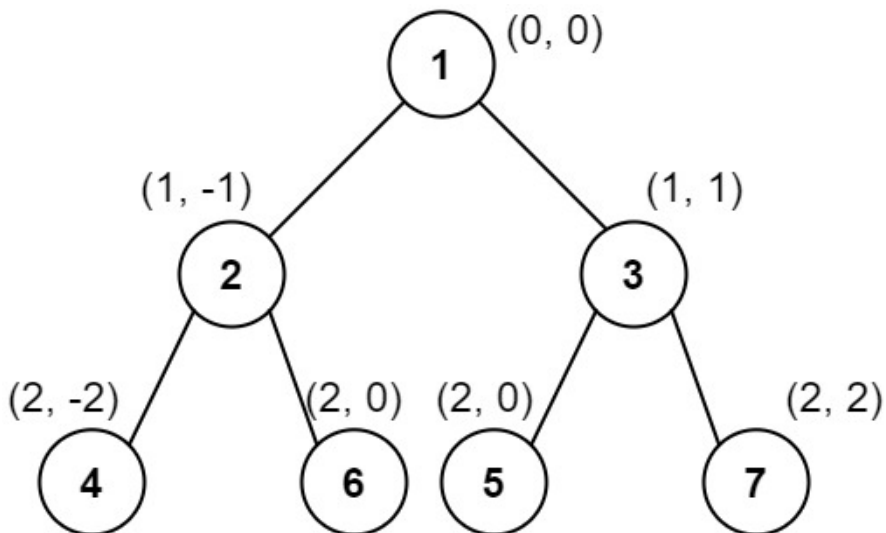
1 is at the top, so it comes first.

5 and 6 are at the same position (2, 0), so we order them by their value, 5 before 6.

Column 1: Only node 3 is in this column.

Column 2: Only node 7 is in this column.

Example 3:



Input: root = [1,2,3,4,6,5,7]

Output: [[4],[2],[1,5,6],[3],[7]]

Explanation:

This case is the exact same as example 2, but with nodes 5 and 6 swapped.

Note that the solution remains the same since 5 and 6 are in the same location and should be ordered by

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $0 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    Map<Integer, List<int[]>> map = new HashMap<>();
    int ltCol=Integer.MAX_VALUE, rtCol=Integer.MIN_VALUE;

    Comparator<int[]> comparator = (a, b) -> {
        if(a[0]==b[0]) {
            return Integer.compare(a[1],b[1]);
        }
        return Integer.compare(a[0], b[0]);
    };

    public List<List<Integer>> verticalTraversal(TreeNode root) {
        dfs(0,0,root);
        List<List<Integer>> result = new ArrayList<>();
        for(int i=ltCol; i <= rtCol; i++) {
            List<Integer> list = new ArrayList<>();
            Collections.sort(map.get(i), comparator);
            for(int[] num: map.get(i)) {
                list.add(num[1]);
            }

            result.add(list);
        }
        return result;
    }

    void dfs(int row, int col, TreeNode node) {
        if(node == null) return;
        ltCol = Math.min(ltCol, col);
        rtCol = Math.max(rtCol, col);
        dfs(row+1, col-1, node.left);
        map.putIfAbsent(col, new ArrayList<>());
        map.get(col).add(new int[]{row, node.val});
        dfs(row+1, col+1, node.right);
    }
}
```


[1019 Squares of a Sorted Array \(link\)](#)

Description

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of the squares of each number sorted in non-decreasing order*.

Example 1:

Input: `nums = [-4,-1,0,3,10]`
Output: `[0,1,9,16,100]`
Explanation: After squaring, the array becomes `[16,1,0,9,100]`.
After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`
Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int[] squares = new int[nums.length];
        int left=0, right = nums.length-1;
        int i=nums.length;
        while(left <= right) {
            if(Math.abs(nums[left]) < Math.abs(nums[right])) {
                squares[--i] = nums[right]*nums[right];
                right--;
            } else {
                squares[--i] = nums[left]*nums[left];
                left++;
            }
        }
        return squares;
    }
}
```

1371 Minimum Remove to Make Valid Parentheses (link)

Description

Given a string s of '(' , ')' and lowercase English characters.

Your task is to remove the minimum number of parentheses ('(' or ')' , in any positions) so that the resulting *parentheses string* is valid and return **any** valid string.

Formally, a *parentheses string* is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A) , where A is a valid string.

Example 1:

```
Input: s = "lee(t(c)o)de)"
Output: "lee(t(c)o)de"
Explanation: "lee(t(co)de)" , "lee(t(c)ode)" would also be accepted.
```

Example 2:

```
Input: s = "a)b(c)d"
Output: "ab(c)d"
```

Example 3:

```
Input: s = "))(("
Output: ""
Explanation: An empty string is also valid.
```

Constraints:

- $1 \leq s.length \leq 10^5$
- $s[i]$ is either '(' , ')' , or lowercase English letter.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String minRemoveToMakeValid(String s) {
        int openCount = 0;
        char[] arr = s.toCharArray();
        for(int i=0; i< arr.length; i++) {
            if(arr[i] == '(') {
                openCount++;
            } else if(arr[i] == ')') {
                if(openCount == 0) {
                    arr[i]='.';
                } else {
                    openCount--;
                }
            }
        }

        for(int i=arr.length-1; openCount > 0; i--) {
            if(arr[i] == '(') {
                arr[i]='.';
                openCount--;
            }
        }

        int p=0;
        for(int i=0; i < arr.length; i++) {
            if(arr[i]!='.') {
                arr[p++]=arr[i];
            }
        }

        return new String(arr,0, p);
    }
}
```

1298 Reverse Substrings Between Each Pair of Parentheses (link)

Description

You are given a string s that consists of lower case English letters and brackets.

Reverse the strings in each pair of matching parentheses, starting from the innermost one.

Your result should **not** contain any brackets.

Example 1:

```
Input: s = "(abcd)"
Output: "dcba"
```

Example 2:

```
Input: s = "(u(love)i)"
Output: "iloveu"
Explanation: The substring "love" is reversed first, then the whole string is reversed.
```

Example 3:

```
Input: s = "(ed(et(oc))el)"
Output: "leetcode"
Explanation: First, we reverse the substring "oc", then "etco", and finally, the whole string.
```

Constraints:

- $1 \leq s.length \leq 2000$
- s only contains lower case English characters and parentheses.
- It is guaranteed that all parentheses are balanced.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reverseParentheses(String s) {
        Stack<Integer> stack = new Stack();
        char[] arr = s.toCharArray();
        int bracketCount = 0;

        for(int i=0; i < arr.length; i++) {
            if(arr[i]=='(') {
                stack.push(i);
                bracketCount++;
            } else if(arr[i]==')') {
                bracketCount--;
                int l = stack.pop()+1;
                int r = i-1;
                while(l<r) {
                    char t = arr[l];
                    arr[l++]=arr[r];
                    arr[r--]=t;
                }
            }
        }

        char[] result = new char[arr.length-bracketCount];
        int i=0;
        for(char ch: arr)
            if(ch!='(' && ch!=')')
                result[i++]=ch;

        return new String(result);
    }
}
```

[1056 Capacity To Ship Packages Within D Days \(link\)](#)

Description

A conveyor belt has packages that must be shipped from one port to another within `days` days.

The i^{th} package on the conveyor belt has a weight of `weights[i]`. Each day, we load the ship with packages on the conveyor belt (in the order given by `weights`). We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within `days` days.

Example 1:

Input: `weights = [1,2,3,4,5,6,7,8,9,10]`, `days = 5`

Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:

1st day: 1, 2, 3, 4, 5

2nd day: 6, 7

3rd day: 8

4th day: 9

5th day: 10

Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the

Example 2:

Input: `weights = [3,2,2,4,1,4]`, `days = 3`

Output: 6

Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:

1st day: 3, 2

2nd day: 2, 4

3rd day: 1, 4

Example 3:

Input: `weights = [1,2,3,1,1]`, `days = 4`

Output: 3

Explanation:

1st day: 1

2nd day: 2

3rd day: 3

4th day: 1, 1

Constraints:

- $1 \leq \text{days} \leq \text{weights.length} \leq 5 * 10^4$
- $1 \leq \text{weights}[i] \leq 500$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        Map<Integer, Boolean> map = new HashMap<>();
        int left=0, right = 0;
        for(int weight: weights) {
            left = Math.max(weight, left);
            right += weight;
        }

        while(left < right) {
            int mid = left+(right-left)/2;
            map.putIfAbsent(mid, isShippable(weights, mid, days));
            if(map.get(mid)) {
                right = mid;
            } else {
                left = mid+1;
            }
        }
        return left;
    }

    boolean isShippable(int[] weights, int capacity, int days) {
        int countDays = 1, totalWeight = 0;
        for(int weight: weights) {
            totalWeight += weight;
            if(totalWeight > capacity) {
                countDays++;
                totalWeight = weight;
                if(countDays > days) {
                    return false;
                }
            }
        }
        return true;
    }
}
```


[225 Implement Stack using Queues \(link\)](#)

Description

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- void push(int x) Pushes element x to the top of the stack.
- int pop() Removes the element on the top of the stack and returns it.
- int top() Returns the element on the top of the stack.
- boolean empty() Returns true if the stack is empty, false otherwise.

Notes:

- You must use **only** standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

Input

```
["MyStack", "push", "push", "top", "pop", "empty"]  
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 2, 2, false]
```

Explanation

```
MyStack myStack = new MyStack();  
myStack.push(1);  
myStack.push(2);  
myStack.top(); // return 2  
myStack.pop(); // return 2  
myStack.empty(); // return False
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, top, and empty.
- All the calls to pop and top are valid.

Follow-up: Can you implement the stack using only one queue?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MyStack {
    private LinkedList<Integer> qu;
    public MyStack() {
        qu = new LinkedList<>();
    }

    public void push(int x) {
        qu.addLast(x);
    }

    public int pop() {
        return qu.removeLast();
    }

    public int top() {
        return qu.peekLast();
    }

    public boolean empty() {
        return qu.isEmpty();
    }
}

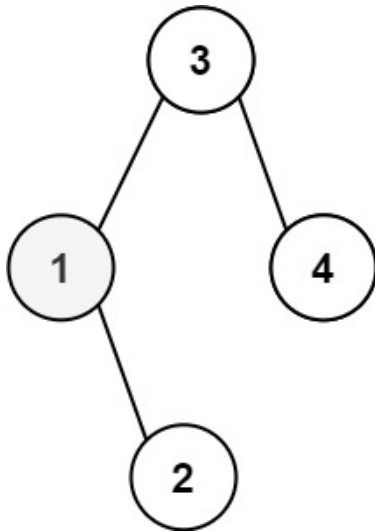
/**
 * Your MyStack object will be instantiated and called as such:
 * MyStack obj = new MyStack();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.top();
 * boolean param_4 = obj.empty();
 */
```

230 Kth Smallest Element in a BST (link)

Description

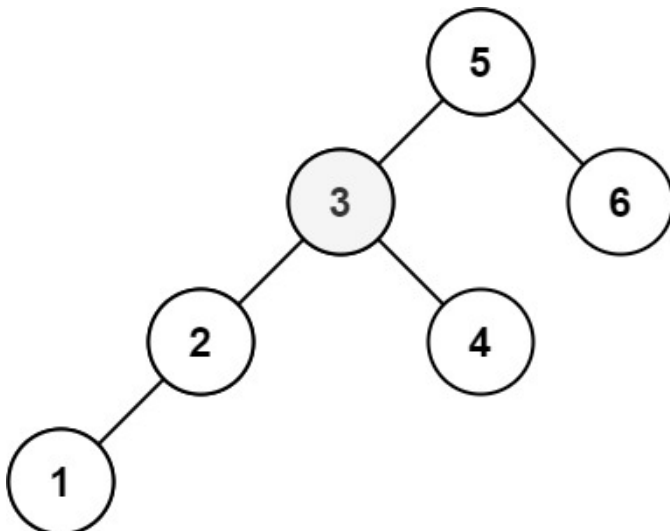
Given the root of a binary search tree, and an integer k , return the k^{th} smallest value (**1-indexed**) of all the values of the nodes in the tree.

Example 1:



Input: root = [3,1,4,null,2], k = 1
Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Constraints:

- The number of nodes in the tree is n .
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the k th smallest frequently, how would you optimize?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    Map<TreeNode, Integer> counts = new HashMap<>();

    public int kthSmallest(TreeNode root, int k) {
        int count = count(root.left);
        if (k <= count) {
            return kthSmallest(root.left, k);
        } else if (k > count+1) {
            return kthSmallest(root.right, k - 1 - count);
        }
        return root.val;
    }

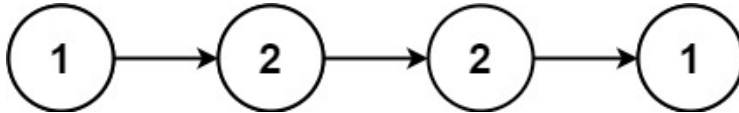
    private int count(TreeNode root) {
        if(root == null)
            return 0;
        if(counts.get(root) == null) {
            counts.put(root, 1 + count(root.left) + count(root.right));
        }
        return counts.get(root);
    }
}
```

234 Palindrome Linked List (link)

Description

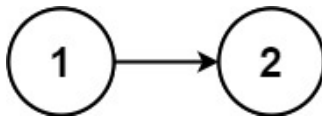
Given the head of a singly linked list, return `true` *if it is a palindrome* or `false` *otherwise*.

Example 1:



Input: head = [1,2,2,1]
Output: true

Example 2:



Input: head = [1,2]
Output: false

Constraints:

- The number of nodes in the list is in the range $[1, 10^5]$.
- $0 \leq \text{Node.val} \leq 9$

Follow up: Could you do it in $O(n)$ time and $O(1)$ space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    ListNode front;
    public boolean isPalindrome(ListNode head) {
        front = head;
        return recurse(head);
    }

    boolean recurse(ListNode back) {
        if(back == null) return true;
        boolean result = recurse(back.next) && back.val == front.val;
        front = front.next;
        return result;
    }
}
```

205 Isomorphic Strings (link)

Description

Given two strings s and t , *determine if they are isomorphic*.

Two strings s and t are isomorphic if the characters in s can be replaced to get t .

All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: $s = \text{"egg"}, t = \text{"add"}$

Output: true

Explanation:

The strings s and t can be made identical by:

- Mapping 'e' to 'a'.
- Mapping 'g' to 'd'.

Example 2:

Input: $s = \text{"foo"}, t = \text{"bar"}$

Output: false

Explanation:

The strings s and t can not be made identical as 'o' needs to be mapped to both 'a' and 'r'.

Example 3:

Input: $s = \text{"paper"}, t = \text{"title"}$

Output: true

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $t.length == s.length$
- s and t consist of any valid ascii character.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        int ls = s.length();
        int lt = t.length();
        if(ls != lt)
            return false;

        int[] smap = new int[256];
        int[] tmap = new int[256];
        for(int i=0; i < ls; i++) {
            char sch = s.charAt(i);
            char tch = t.charAt(i);

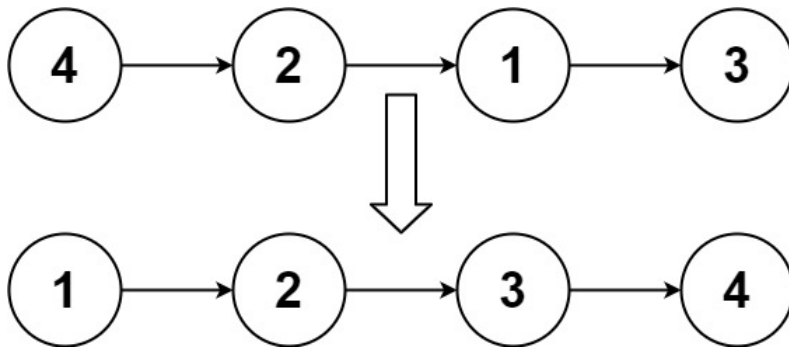
            if(smap[sch] != tmap[tch]) return false;
            smap[sch] = i+1;
            tmap[tch] = i+1;
        }
        return true;
    }
}
```

148 Sort List (link)

Description

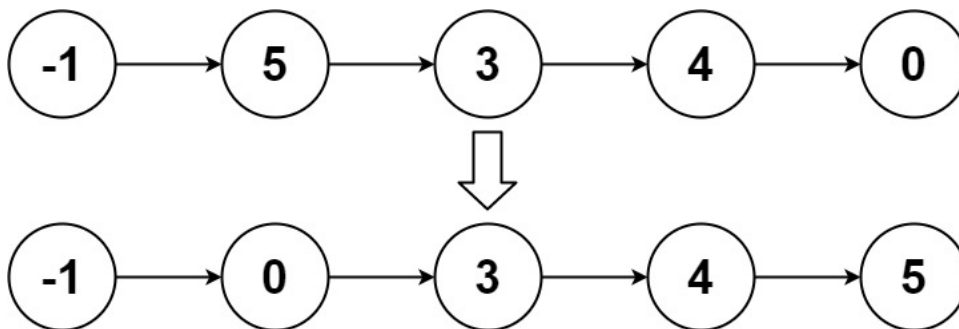
Given the head of a linked list, return *the list after sorting it in **ascending order***.

Example 1:



Input: head = [4,2,1,3]
Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$

Follow up: Can you sort the linked list in $O(n \log n)$ time and $O(1)$ memory (i.e. constant space)?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode sortList(ListNode head) {
        if(head == null || head.next == null)
            return head;

        ListNode sentinel = new ListNode();
        ListNode mid = mid(head);
        ListNode left = sortList(head);
        ListNode right = sortList(mid);
        return merge(sentinel, left, right);
    }

    ListNode merge(ListNode sentinel, ListNode n1, ListNode n2) {
        ListNode end = sentinel;
        while(n1 != null && n2 != null) {
            if(n1.val < n2.val) {
                end.next = n1;
                n1 = n1.next;
            } else {
                end.next = n2;
                n2 = n2.next;
            }
            end = end.next;
        }
        end.next = n2 == null ? n1:n2;
        return sentinel.next;
    }

    ListNode mid(ListNode start) {
        ListNode prev = new ListNode(0, start);
        while(start != null && start.next != null) {
            prev = prev.next;
            start = start.next.next;
        }
        ListNode mid = prev.next;
        prev.next = null;
        return mid;
    }
}
```

[40 Combination Sum II \(link\)](#)

Description

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sum to `target`.

Each number in `candidates` may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

Input: `candidates = [10,1,2,7,6,1,5]`, `target = 8`

Output:

```
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

Example 2:

Input: `candidates = [2,5,2,1,2]`, `target = 5`

Output:

```
[
  [1,2,2],
  [5]
]
```

Constraints:

- `1 <= candidates.length <= 100`
- `1 <= candidates[i] <= 50`
- `1 <= target <= 30`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> combinationSum2(int[] candidates, int target) {
        Arrays.sort(candidates);
        List<List<Integer>> result = new ArrayList<>();
        backtrack(candidates, target, new LinkedList<>(), 0, result);
        return result;
    }

    private void backtrack(int[] candidates, int target, LinkedList<Integer> combination, int start, List<List<Integer>> result) {
        if(target == 0) {
            result.add(new ArrayList<>(combination));
            return;
        }

        if(start >= candidates.length || target < 0) {
            return;
        }

        combination.addLast(candidates[start]);
        backtrack(candidates, target-candidates[start], combination, start+1, result);
        combination.removeLast();
        int nextStart = start+1;
        while(nextStart < candidates.length && candidates[start] == candidates[nextStart]) {
            nextStart++;
        }
        backtrack(candidates, target, combination, nextStart, result);
    }
}
```

[12 Integer to Roman \(link\)](#)

Description

Seven different symbols represent Roman numerals with the following values:

Symbol Value

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Roman numerals are formed by appending the conversions of decimal place values from highest to lowest. Converting a decimal place value into a Roman numeral has the following rules:

- If the value does not start with 4 or 9, select the symbol of the maximal value that can be subtracted from the input, append that symbol to the result, subtract its value, and convert the remainder to a Roman numeral.
- If the value starts with 4 or 9 use the **subtractive form** representing one symbol subtracted from the following symbol, for example, 4 is 1 (I) less than 5 (V): IV and 9 is 1 (I) less than 10 (X): IX. Only the following subtractive forms are used: 4 (IV), 9 (IX), 40 (XL), 90 (XC), 400 (CD) and 900 (CM).
- Only powers of 10 (I, X, C, M) can be appended consecutively at most 3 times to represent multiples of 10. You cannot append 5 (V), 50 (L), or 500 (D) multiple times. If you need to append a symbol 4 times use the **subtractive form**.

Given an integer, convert it to a Roman numeral.

Example 1:

Input: num = 3749

Output: "MMMDCCLXIX"

Explanation:

3000 = MMM as 1000 (M) + 1000 (M) + 1000 (M)
700 = DCC as 500 (D) + 100 (C) + 100 (C)
40 = XL as 10 (X) less of 50 (L)
9 = IX as 1 (I) less of 10 (X)
Note: 49 is not 1 (I) less of 50 (L) because the conversion is based on decimal places

Example 2:

Input: num = 58

Output: "LVIII"

Explanation:

50 = L
8 = VIII

Example 3:

Input: num = 1994

Output: "MCMXCIV"

Explanation:

```
1000 = M
 900 = CM
  90 = XC
   4 = IV
```

Constraints:

- $1 \leq \text{num} \leq 3999$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Tuple {
    int key;
    String value;
    public Tuple(int key, String value) {
        this.key = key;
        this.value = value;
    }
    public int key() {
        return this.key;
    }
    public String value() {
        return this.value;
    }
}

class Solution {
    static final List<Tuple> map = List.of(
        new Tuple(1,"I"),
        new Tuple(4,"IV"),
        new Tuple(5,"V"),
        new Tuple(9,"IX"),
        new Tuple(10,"X"),
        new Tuple(40,"XL"),
        new Tuple(50,"L"),
        new Tuple(90,"XC"),
        new Tuple(100,"C"),
        new Tuple(400,"CD"),
        new Tuple(500,"D"),
        new Tuple(900,"CM"),
        new Tuple(1000,"M")
    );

    public String intToRoman(int num) {
        StringBuilder sb = new StringBuilder();
        Tuple tmp;
        for(int i=map.size()-1; i > -1; i--) {
            tmp = map.get(i);
            while(num >= tmp.key()) {
                sb.append(tmp.value());
                num -= tmp.key();
            }
        }
        return sb.toString();
    }
}
```

[39 Combination Sum \(link\)](#)

Description

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all **unique combinations** of `candidates` where the chosen numbers sum to `target`*. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to `target` is less than 150 combinations for the given input.

Example 1:

Input: `candidates = [2,3,6,7], target = 7`

Output: `[[2,2,3],[7]]`

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: `candidates = [2,3,5], target = 8`

Output: `[[2,2,2,2],[2,3,3],[3,5]]`

Example 3:

Input: `candidates = [2], target = 1`

Output: `[]`

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of `candidates` are **distinct**.
- $1 \leq \text{target} \leq 40$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(candidates, target, new ArrayDeque<>(), 0, result);
        return result;
    }

    private void backtrack(int[] candidates, int target, ArrayDeque<Integer> combination, int start, List<List<Integer>> result) {
        if(target == 0) {
            result.add(new ArrayList<>(combination));
            return;
        }

        if(start == candidates.length || target < 0){
            return;
        }

        combination.addLast(candidates[start]);
        backtrack(candidates, target-candidates[start], combination, start, result);
        combination.removeLast();
        backtrack(candidates, target, combination, start+1, result);
    }
}
```

647 Palindromic Substrings [\(link\)](#)

Description

Given a string s , return *the number of **palindromic substrings** in it*.

A string is a **palindrome** when it reads the same backward as forward.

A **substring** is a contiguous sequence of characters within the string.

Example 1:

```
Input: s = "abc"
Output: 3
Explanation: Three palindromic strings: "a", "b", "c".
```

Example 2:

```
Input: s = "aaa"
Output: 6
Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".
```

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int countSubstrings(String s) {
        int n = s.length();
        boolean[][] dp = new boolean[n][n];

        int counter = 0;
        for (int dist = 1; dist <= n; dist++) {
            for (int j = 0; j + dist - 1 < n; j++) {
                if (s.charAt(j) == s.charAt(j + dist - 1) && (dist <= 2 || dp[j + 1][j + dist - 2])) {
                    dp[j][j + dist - 1] = true;
                    counter++;
                }
            }
        }
        return counter;
    }
}
```

[133 Clone Graph \(link\)](#)

Description

Given a reference of a node in a [connected](#) undirected graph.

Return a [deep copy](#) (clone) of the graph.

Each node in the graph contains a value (`int`) and a list (`List[Node]`) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

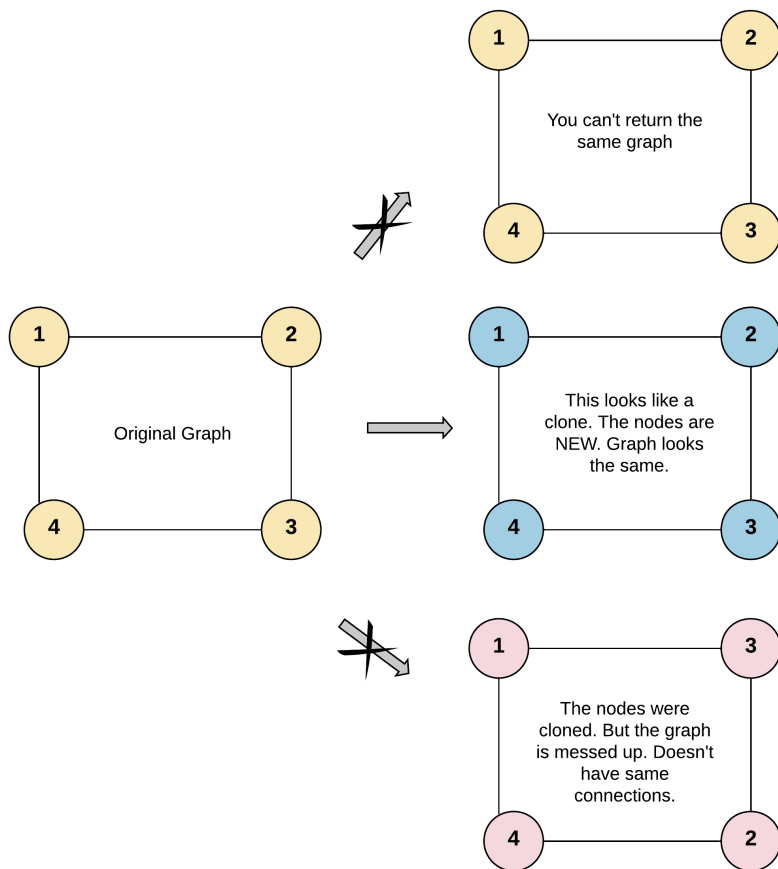
Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered **lists** used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with `val = 1`. You must return the **copy of the given node** as a reference to the cloned graph.

Example 1:



Input: `adjList = [[2,4],[1,3],[2,4],[1,3]]`

Output: `[[2,4],[1,3],[2,4],[1,3]]`

Explanation: There are 4 nodes in the graph.

1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).

4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).

Example 2:



Input: `adjList = [[]]`

Output: `[[]]`

Explanation: Note that the input contains one empty list. The graph consists of only one node with val =

Example 3:

Input: `adjList = []`

Output: `[]`

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range $[0, 100]$.
- $1 \leq \text{Node.val} \leq 100$
- `Node.val` is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> neighbors;
    public Node() {
        val = 0;
        neighbors = new ArrayList<Node>();
    }
    public Node(int _val) {
        val = _val;
        neighbors = new ArrayList<Node>();
    }
    public Node(int _val, ArrayList<Node> _neighbors) {
        val = _val;
        neighbors = _neighbors;
    }
}
*/

class Solution {
    private Map<Node, Node> clones = new HashMap<>();
    public Node cloneGraph(Node root) {
        try {
            Callable<Node> func = () -> solve(root);
            FutureTask<Node> task = new FutureTask(func);
            new Thread(task).start();
            return task.get();
        } catch (Exception ex) {
            return null;
        }
    }

    public Node solve(Node root) {
        if (root == null) return root;
        if (clones.containsKey(root)) return clones.get(root);

        Node clone = new Node(root.val);
        clones.put(root, clone);
        for (Node neighbor: root.neighbors) {
            clone.neighbors.add(cloneGraph(neighbor));
        }

        return clone;
    }
}
```

[**347 Top K Frequent Elements \(link\)**](#)

Description

Given an integer array `nums` and an integer `k`, return *the k most frequent elements*. You may return the answer in **any order**.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`
Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`
Output: `[1]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where `n` is the array's size.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        int max = Integer.MIN_VALUE, min = Integer.MAX_VALUE;
        for(int num: nums) {
            max = Math.max(max, num);
            min = Math.min(min, num);
        }
        int[] hash = new int[max-min+1];
        for(int num: nums) {
            hash[num-min]++;
        }

        int[][] pairs = new int[hash.length][2];
        for(int i=0; i < hash.length; i++) {
            pairs[i][0]=i+min;
            pairs[i][1]=hash[i];
        }

        Arrays.sort(pairs, (a,b) -> b[1]-a[1]);
        int[] kLargest= new int[k];
        for(int i=0; i < k; i++)
            kLargest[i]=pairs[i][0];
        return kLargest;
    }
}
```

[98 Validate Binary Search Tree \(link\)](#)

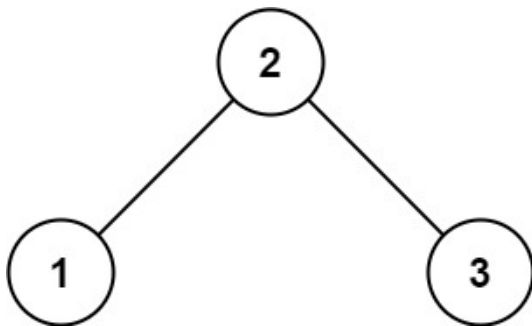
Description

Given the root of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

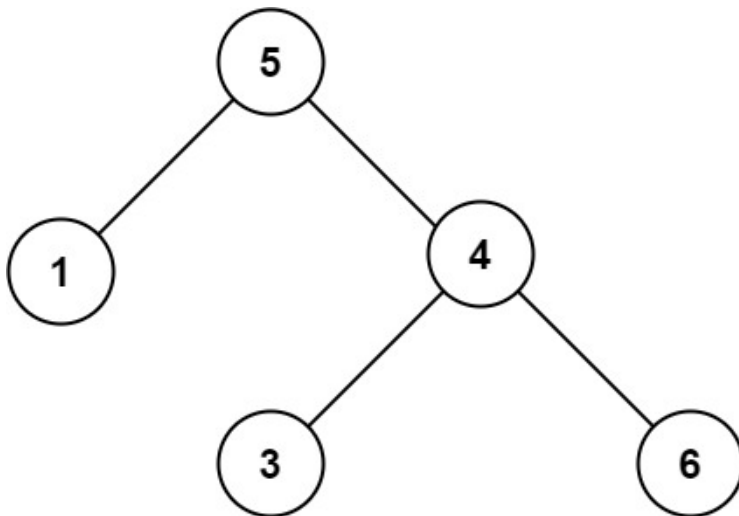
- The left subtree of a node contains only nodes with keys **less than** the node's key.
- The right subtree of a node contains only nodes with keys **greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



Input: root = [2,1,3]
Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.

- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isValidBST(TreeNode root) {
        return isValidBST(Long.MIN_VALUE, root, Long.MAX_VALUE);
    }

    boolean isValidBST(long left, TreeNode root, long right) {
        if(root == null)
            return true;
        if(root.val <= left || root.val >= right)
            return false;
        return isValidBST(left, root.left, root.val) && isValidBST(root.val, root.right, right);
    }
}
```

[152 Maximum Product Subarray \(link\)](#)

Description

Given an integer array `nums`, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Example 1:

```
Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.
```

Example 2:

```
Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.
```

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of `nums` is **guaranteed** to fit in a **32-bit** integer.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    // public int maxProduct(int[] nums) {
    //     int result = nums[0], max = result, min = result;
    //     for(int i=1; i < nums.length; i++) {
    //         int temp = min;
    //         min = Math.min(nums[i], Math.min(min*nums[i], max*nums[i]));
    //         max = Math.max(nums[i], Math.max(temp*nums[i], max*nums[i]));
    //         result = Math.max(result, max);
    //     }
    //     return result;
    // }

    public int maxProduct(int[] nums) {
        int prefix=1, suffix = 1, maxSoFar = Integer.MIN_VALUE;
        for(int i=0, n=nums.length; i < n; i++) {
            prefix = nums[i]*(prefix == 0 ? 1: prefix);
            suffix = nums[n-i-1]*(suffix == 0 ? 1 : suffix);
            maxSoFar = Math.max(maxSoFar, Math.max(prefix, suffix));
        }
        return maxSoFar;
    }
}
```


[424 Longest Repeating Character Replacement \(link\)](#)

Description

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return *the length of the longest substring containing the same letter you can get after performing the above operations*.

Example 1:

Input: $s = \text{"ABAB"}, k = 2$
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: $s = \text{"AABABBA"}, k = 1$
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA". The substring "BBBB" has the longest repeating letters, which is 4. There may exists other ways to achieve this answer too.

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists of only uppercase English letters.
- $0 \leq k \leq s.length$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int characterReplacement(String s, int k) {
        int[] cmap = new int[100];
        byte[] bytes = s.getBytes();
        int left=0, maxRepeatLetterCount=0, maxLength=0;
        for(int right=0; right < bytes.length; right++) {
            maxRepeatLetterCount = Math.max(maxRepeatLetterCount, ++cmap[bytes[right]]);
            if(right-left+1-maxRepeatLetterCount > k) {
                cmap[bytes[left++]]--;
            }
            maxLength = Math.max(maxLength, right-left+1);
        }
        return maxLength;
    }
}
```

[449 Serialize and Deserialize BST \(link\)](#)

Description

Serialization is converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a **binary search tree**. There is no restriction on how your serialization/deserialization algorithm should work. You need to ensure that a binary search tree can be serialized to a string, and this string can be deserialized to the original tree structure.

The encoded string should be as compact as possible.

Example 1:

```
Input: root = [2,1,3]
Output: [2,1,3]
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $0 \leq \text{Node.val} \leq 10^4$
- The input tree is **guaranteed** to be a binary search tree.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
public class Codec {
    int pos;
    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        serialize(root, sb);
        return sb.toString();
    }

    private StringBuilder serialize(TreeNode root, StringBuilder sb) {
        if(root == null) return sb;
        sb.append((char) root.val);
        serialize(root.left, sb);
        serialize(root.right, sb);
        return sb;
    }

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        if(data.isEmpty()) return null;
        pos = 0;
        return deserialize(Integer.MIN_VALUE, Integer.MAX_VALUE, data.toCharArray());
    }

    private TreeNode deserialize(Integer lower, Integer upper, char[] data) {
        if(data.length == 0 || pos == data.length) return null;
        int val = (int) data[pos];
        if(val < lower || val > upper) return null;

        TreeNode root = new TreeNode(data[pos++]);
        root.left = deserialize(lower, val, data);
        root.right = deserialize(val, upper, data);
        return root;
    }
}

// Your Codec object will be instantiated and called as such:
// Codec ser = new Codec();
// Codec deser = new Codec();
// String tree = ser.serialize(root);
// TreeNode ans = deser.deserialize(tree);
// return ans;
```

[211 Design Add and Search Words Data Structure \(link\)](#)

Description

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

Input

```
["WordDictionary", "addWord", "addWord", "addWord", "search", "search", "search", "search"]  
[[], ["bad"], ["dad"], ["mad"], ["pad"], ["bad"], [".ad"], ["b.."]]
```

Output

```
[null, null, null, null, false, true, true, true]
```

Explanation

```
WordDictionary wordDictionary = new WordDictionary();  
wordDictionary.addWord("bad");  
wordDictionary.addWord("dad");  
wordDictionary.addWord("mad");  
wordDictionary.search("pad"); // return False  
wordDictionary.search("bad"); // return True  
wordDictionary.search(".ad"); // return True  
wordDictionary.search("b.."); // return True
```

Constraints:

- $1 \leq \text{word.length} \leq 25$
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 10^4 calls will be made to addWord and search.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class WordDictionary {
    TrieNode root;
    public WordDictionary() {
        this.root = new TrieNode();
    }

    public void addWord(String word) {
        TrieNode node = root;
        for(char ch: word.toCharArray()) {
            if(node.children[ch-'a'] == null) {
                node.children[ch-'a'] = new TrieNode();
            }
            node = node.children[ch-'a'];
        }
        node.children[26] = new TrieNode();
    }

    public boolean search(char[] word, int pos, TrieNode trie) {
        TrieNode child;
        boolean returnVal = false;
        if(word[pos] == '.') {
            for(int i=0; i < 26; i++) {
                child = trie.children[i];
                if( child != null) {
                    if(pos == word.length - 1) {
                        returnVal = child.children[26] != null;
                    } else {
                        returnVal = search(word, pos+1, child);
                    }

                    if(returnVal)
                        return true;
                }
            }
            return false;
        } else {
            child = trie.children[word[pos]-'a'];
            if(child != null) {
                if(pos == word.length - 1) {
                    return child.children[26] != null;
                } else {
                    return search(word, pos+1, child);
                }
            }
            return false;
        }
    }

    public boolean search(String word) {
        return search(word.toCharArray(), 0, root);
    }
}

class TrieNode {
    TrieNode[] children = new TrieNode[27];
    public TrieNode() {}
}

/**
 * Your WordDictionary object will be instantiated and called as such:
 * WordDictionary obj = new WordDictionary();
 * obj.addWord(word);
 */
```

```
* boolean param_2 = obj.search(word);  
*/
```

[295 Find Median from Data Stream \(link\)](#)

Description

The **median** is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for `arr = [2,3,4]`, the median is 3.
- For example, for `arr = [2,3]`, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- `MedianFinder()` initializes the MedianFinder object.
- `void addNum(int num)` adds the integer `num` from the data stream to the data structure.
- `double findMedian()` returns the median of all elements so far. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]  
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();  
medianFinder.addNum(1);    // arr = [1]  
medianFinder.addNum(2);    // arr = [1, 2]  
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)  
medianFinder.addNum(3);    // arr[1, 2, 3]  
medianFinder.findMedian(); // return 2.0
```

Constraints:

- $-10^5 \leq \text{num} \leq 10^5$
- There will be at least one element in the data structure before calling `findMedian`.
- At most $5 * 10^4$ calls will be made to `addNum` and `findMedian`.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MedianFinder {

    PriorityQueue<Integer> minHeap;
    PriorityQueue<Integer> maxHeap;

    /** initialize your data structure here. */
    public MedianFinder() {
        minHeap = new PriorityQueue<>();
        maxHeap = new PriorityQueue<>((a,b)->(b-a));
    }

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());
        if(maxHeap.size() < minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if(maxHeap.size() == minHeap.size()) {
            return (double) (maxHeap.peek()+minHeap.peek())/2;
        } else {
            return maxHeap.peek();
        }
    }
}

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
```

49 Group Anagrams (link)

Description

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        int[] chars = new int[26];
        StringBuilder sb = new StringBuilder();
        for(String str: strs) {
            for(char c: str.toCharArray())
                chars[c-'a']++;
            for(int i=0; i<chars.length; i++) {
                sb.append("#").append(chars[i]);
                chars[i]=0;
            }

            map.computeIfAbsent(sb.toString(), k -> new ArrayList<>()).add(str);
            sb.setLength(0);
        }
        return new ArrayList<>(map.values());
    }
}
```

[242 Valid Anagram \(link\)](#)

Description

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

Example 1:

Input: `s = "anagram", t = "nagaram"`

Output: `true`

Example 2:

Input: `s = "rat", t = "car"`

Output: `false`

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- `s` and `t` consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if(s.length() != t.length()) {
            return false;
        }

        int[] charMap = new int[26];
        for(int i=0; i < s.length(); i++) {
            charMap[s.charAt(i) - 'a']++;
            charMap[t.charAt(i) - 'a']--;
        }

        for(int i=0; i < 26; i++) {
            if(charMap[i] != 0) {
                return false;
            }
        }
        return true;
    }
}
```

191 Number of 1 Bits (link)

Description

Given a positive integer n , write a function that returns the number of set bits in its binary representation (also known as the [Hamming weight](#)).

Example 1:

Input: $n = 11$

Output: 3

Explanation:

The input binary string **1011** has a total of three set bits.

Example 2:

Input: $n = 128$

Output: 1

Explanation:

The input binary string **10000000** has a total of one set bit.

Example 3:

Input: $n = 2147483645$

Output: 30

Explanation:

The input binary string **111111111111111111111111111101** has a total of thirty set bits.

Constraints:

- $1 \leq n \leq 2^{31} - 1$

Follow up: If this function is called many times, how would you optimize it?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int hammingWeight(int n) {
        int count=0;
        for(int i=0; i < 32; i++) {
            count += (n&(1<<i)) == 0 ? 0:1;
        }
        return count;
    }
}
```

268 Missing Number (link)

Description

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return *the only number in the range that is missing from the array*.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

$n = 3$ since there are 3 numbers, so all numbers are in the range $[0, 3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0, 2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0, 9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$

- $0 \leq \text{nums}[i] \leq n$
- All the numbers of `nums` are **unique**.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int missingNumber(int[] nums) {  
        int ret = 0, n = nums.length;  
        for(int i=0; i < n ; i++) {  
            ret ^= (i+1)^(nums[i]);  
        }  
        return ret;  
    }  
}
```

8 String to Integer (atoi) (link)

Description

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: `s = "42"`

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

Step 2: "42" (no characters read because there is neither a '-' nor '+')

Step 3: "42" ("42" is read in)

Example 2:

Input: `s = "-042"`

Output: -42

Explanation:

Step 1: " _-042" (leading whitespace is read and ignored)

Step 2: " _-042" ('-' is read, so the result should be negative)

Step 3: " _-042" ("042" is read in, leading zeros ignored in the result)

Example 3:

Input: `s = "1337c0d3"`

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)
 ^
Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')
 ^
Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)
 ^

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)
 ^
Step 2: "0-1" (no characters read because there is neither a '-' nor '+')
 ^
Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)
 ^

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

- $0 \leq s.length \leq 200$
- s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int myAtoi(String s) {
        int MAX_BY_10=Integer.MAX_VALUE/10;
        int sign = 1;
        int result = 0;
        int pos=-1;
        int length = s.length();

        while(++pos < length && s.charAt(pos) == ' ');

        if(pos < length && s.charAt(pos) == '+') {
            pos++;
        } else if(pos < length && s.charAt(pos) == '-') {
            pos++;
            sign = -1;
        }

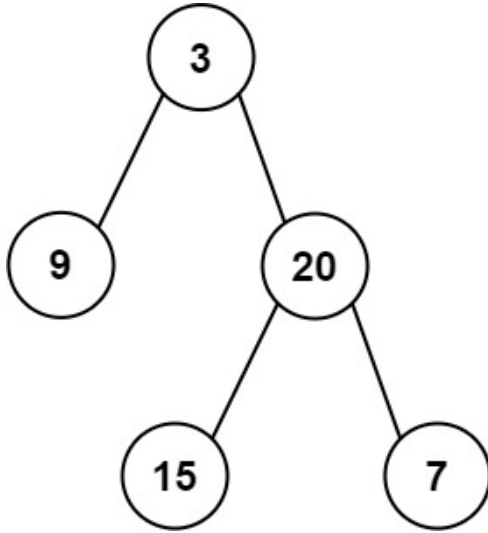
        while(pos < length && s.charAt(pos) < 58 && s.charAt(pos) > 47) {
            int digit = s.charAt(pos) - '0';
            if(result > MAX_BY_10 || (result == MAX_BY_10 && digit > (Integer.MAX_VALUE%10)))
                return sign == 1?Integer.MAX_VALUE:Integer.MIN_VALUE;
            result = result*10+digit;
            pos++;
        }
        return sign*result;
    }
}
```

105 Construct Binary Tree from Preorder and Inorder Traversal (link)

Description

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: `preorder = [3,9,20,15,7]`, `inorder = [9,3,15,20,7]`
Output: `[3,9,20,null,null,15,7]`

Example 2:

Input: `preorder = [-1]`, `inorder = [-1]`
Output: `[-1]`

Constraints:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` and `inorder` consist of **unique** values.
- Each value of `inorder` also appears in `preorder`.
- `preorder` is **guaranteed** to be the preorder traversal of the tree.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int preOrderIndex=0;
    Map<Integer, Integer> inorderIndexMap = new HashMap<>();
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        for(int i=0; i< inorder.length; i++)
            inorderIndexMap.put(inorder[i],i);

        return buildTree(preorder, 0, preorder.length-1);
    }

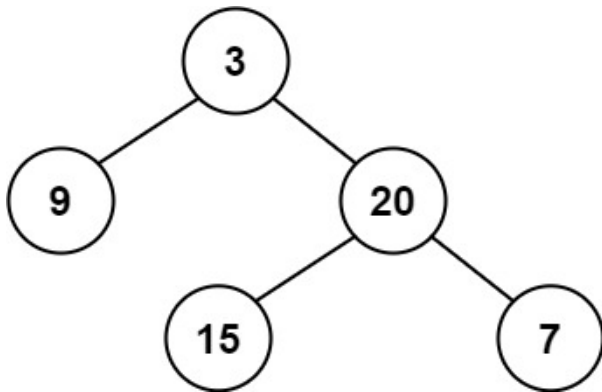
    TreeNode buildTree(int[] preorder, int start, int end) {
        if(start>end) return null;
        int rootValue = preorder[preOrderIndex++];
        return new TreeNode(rootValue,
                            buildTree(preorder, start, inorderIndexMap.get(rootValue)-1),
                            buildTree(preorder, inorderIndexMap.get(rootValue)+1, end));
    }
}
```

110 Balanced Binary Tree (link)

Description

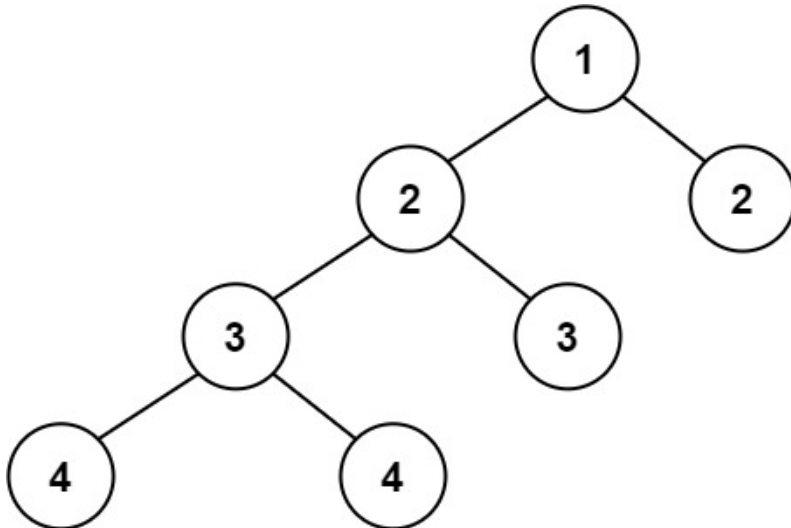
Given a binary tree, determine if it is **height-balanced**.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]
Output: false

Example 3:

Input: root = []
Output: true

Constraints:

- The number of nodes in the tree is in the range $[0, 5000]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isBalanced(TreeNode root) {
        return getBalance(root) != -1;
    }

    int getBalance(TreeNode root) {
        if(root == null) return 0;

        int rightBal = getBalance(root.right);
        if(rightBal == -1) return rightBal;

        int leftBal = getBalance(root.left);
        if(leftBal == -1) return leftBal;

        if(Math.abs(rightBal-leftBal) > 1) return -1;
        else return 1+Math.max(leftBal, rightBal);
    }
}
```

[81 Search in Rotated Sorted Array II \(link\)](#)

Description

There is an integer array `nums` sorted in non-decreasing order (not necessarily with **distinct** values).

Before being passed to your function, `nums` is **rotated** at an unknown pivot index k ($0 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,4,4,5,6,6,7]` might be rotated at pivot index 5 and become `[4,5,6,6,7,0,1,2,4,4]`.

Given the array `nums` **after** the rotation and an integer `target`, return `true` *if target is in nums*, *or false if it is not in nums*.

You must decrease the overall operation steps as much as possible.

Example 1:

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0`
Output: `true`

Example 2:

Input: `nums = [2,5,6,0,0,1,2]`, `target = 3`
Output: `false`

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is guaranteed to be rotated at some pivot.
- $-10^4 \leq \text{target} \leq 10^4$

Follow up: This problem is similar to [Search in Rotated Sorted Array](#), but `nums` may contain **duplicates**. Would this affect the runtime complexity? How and why?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean search(int[] nums, int target) {
        int lo=0, hi=nums.length-1;
        while(lo<=hi) {
            //skipping duplicates
            while(lo < hi && nums[lo] == nums[lo+1]) lo++;
            while(lo < hi && nums[hi-1] == nums[hi]) hi--;

            int mid = (lo+hi)/2;
            if(nums[mid] == target) {
                return true;
            } else if(nums[lo] <= nums[mid]) {
                if(target < nums[mid] && target >= nums[lo]) hi = mid-1;
                else lo = mid+1;
            } else{
                if(target > nums[mid] && target <= nums[hi]) lo = mid+1;
                else hi = mid-1;
            }
        }
        return false;
    }
}
```

[17 Letter Combinations of a Phone Number \(link\)](#)

Description

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

```
Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Example 2:

```
Input: digits = ""
Output: []
```

Example 3:

```
Input: digits = "2"
Output: ["a","b","c"]
```

Constraints:

- `0 <= digits.length <= 4`
- `digits[i]` is a digit in the range `['2', '9']`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    private char[][] cmap = {{'a','b','c'},
                             {'d','e','f'},
                             {'g','h','i'},
                             {'j','k','l'},
                             {'m','n','o'},
                             {'p','q','r','s'},
                             {'t','u','v'},
                             {'w','x','y','z'}};

    public List<String> letterCombinations(String digits) {
        List<String> result = new ArrayList<>();
        if(digits == null || digits.length()==0)
            return result;

        backtrack(digits, 0, new char[digits.length()], result);
        return result;
    }

    private void backtrack(String digits, int start, char[] bldr, List<String> result) {
        if(start == digits.length()) {
            result.add(String.valueOf(bldr));
            return;
        }

        char[] charr = cmap[digits.charAt(start) - '2'];
        for(char ch: charr) {
            bldr[start]=ch;
            backtrack(digits, start+1, bldr, result);
        }
    }
}
```

[14 Longest Common Prefix \(link\)](#)

Description

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

```
Input: strs = ["flower","flow","flight"]  
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]  
Output: ""  
Explanation: There is no common prefix among the input strings.
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ consists of only lowercase English letters if it is non-empty.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        int i=0, numStrings = strs.length;
        if(numStrings < 2)
            return strs[0];

        outer:
        while(true) {
            for(int j=1; j < numStrings; j++)
                if(i>=strs[j-1].length() || i >= strs[j].length() || (strs[j-1].charAt(i) != strs[j].charAt(i)))
                    break outer;
            i++;
        }
        return strs[0].substring(0,i);
    }
}
```



786 Search in a Sorted Array of Unknown Size (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * // This is ArrayReader's API interface.
 * // You should not implement it, or speculate about its implementation
 * interface ArrayReader {
 *     public int get(int index) {}
 * }
 */

class Solution {
    public int search(ArrayReader reader, int target) {
        int lo=0,hi=1;

        while(lo <= hi) {
            int mid = (lo+hi)/2;
            int val = reader.get(mid);
            if(val == target) {
                return mid;
            } else if(val < target) {
                lo = mid+1;
                hi <=&=1;
            } else {
                hi = mid-1;
            }
        }

        return -1;
    }
}
```

1143 Find Smallest Common Element in All Rows (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int smallestCommonElement(int[][] mat) {
        int m = mat.length;
        int n = mat[0].length;
        int[] counts = new int[10001];
        for(int i=0; i < n; i++) {
            for(int j=0; j < m; j++) {
                if(++counts[mat[j][i]] == m)
                    return mat[j][i];
            }
        }
        return -1;
    }
}
```

2416 Evaluate Boolean Binary Tree (link)

Description

You are given the `root` of a **full binary tree** with the following properties:

- **Leaf nodes** have either the value 0 or 1, where 0 represents `False` and 1 represents `True`.
- **Non-leaf nodes** have either the value 2 or 3, where 2 represents the boolean `OR` and 3 represents the boolean `AND`.

The **evaluation** of a node is as follows:

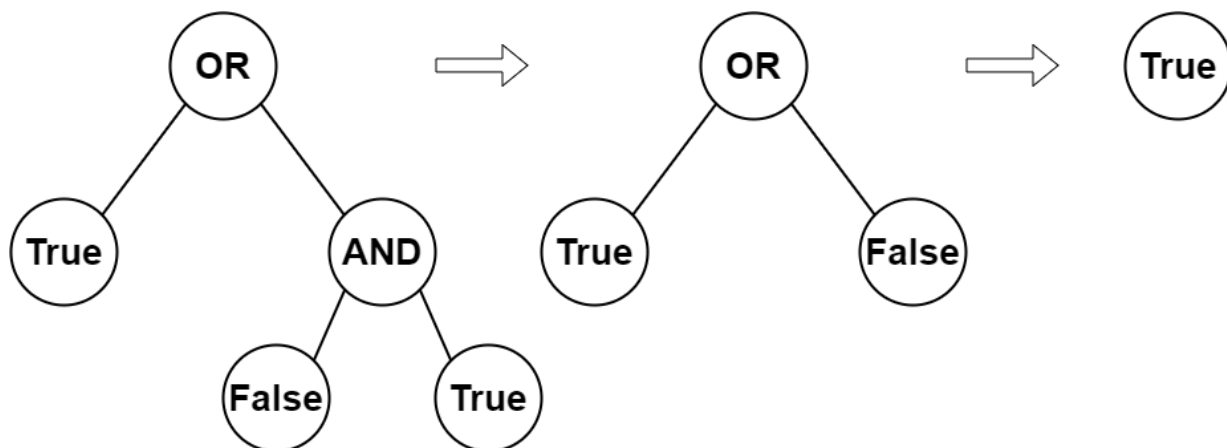
- If the node is a leaf node, the evaluation is the **value** of the node, i.e. `True` or `False`.
- Otherwise, **evaluate** the node's two children and **apply** the boolean operation of its value with the children's evaluations.

Return the boolean result of **evaluating** the `root` node.

A **full binary tree** is a binary tree where each node has either 0 or 2 children.

A **leaf node** is a node that has zero children.

Example 1:



Input: `root = [2,1,3,null,null,0,1]`

Output: `true`

Explanation: The above diagram illustrates the evaluation process.

The `AND` node evaluates to `False AND True = False`.

The `OR` node evaluates to `True OR False = True`.

The root node evaluates to `True`, so we return `true`.

Example 2:

Input: `root = [0]`

Output: `false`

Explanation: The root node is a leaf node and it evaluates to `false`, so we return `false`.

Constraints:

- The number of nodes in the tree is in the range `[1, 1000]`.
- `0 <= Node.val <= 3`

- Every node has either 0 or 2 children.
- Leaf nodes have a value of 0 or 1.
- Non-leaf nodes have a value of 2 or 3.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean evaluateTree(TreeNode root) {
        switch(root.val) {
            case 0:
                return false;
            case 1:
                return true;
            case 2:
                return evaluateTree(root.left) || evaluateTree(root.right);
            case 3:
                return evaluateTree(root.left) && evaluateTree(root.right);
        }
        return false;
    }
}
```

1149 Intersection of Three Sorted Arrays (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> arraysIntersection(int[] arr1, int[] arr2, int[] arr3) {
        List<Integer> result = new ArrayList<>();
        int l1 = arr1.length, l2 = arr2.length, l3 = arr3.length;
        int[] smallest = l1 > l2 ? (l2 > l3 ? arr3:arr2):(l1>l3 ? arr3:arr1);

        int[] middle = null;
        int[] largest = null;
        if(smallest == arr1) {
            middle = l2 > l3 ? arr3: arr2;
            largest = l2 > l3 ? arr2: arr3;
        } else if(smallest == arr2) {
            middle = l1 > l3 ? arr3: arr1;
            largest = l1 > l3 ? arr1: arr3;
        } else if(smallest == arr3) {
            middle = l1 > l2 ? arr2: arr1;
            largest = l1 > l2 ? arr1: arr2;
        }

        for(int num: smallest) {
            int match = binarySearch(middle, num);
            if(match == -1)
                continue;
            match = binarySearch(largest, num);
            if(match == num)
                result.add(num);
        }

        return result;
    }

    private int binarySearch(int[] arr, int num) {
        if(arr != null) {
            int lo = 0, hi = arr.length -1;
            while(lo <= hi) {
                int mid = (lo+hi)/2;
                if(arr[mid] == num) return num;
                else if(arr[mid] < num) lo = mid+1;
                else hi = mid-1;
            }
        }
        return -1;
    }
}
```

270 Closest Binary Search Tree Value ([link](#)).

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int closestValue = 0;
    double minDiff = Double.MAX_VALUE;
    public int closestValue(TreeNode root, double target) {
        closest(root, target);
        return closestValue;
    }

    private void closest(TreeNode root, double target) {
        if(root == null) return;
        double diff = Math.abs(root.val - target);
        if(diff < minDiff) {
            minDiff = diff;
            closestValue = root.val;
            if(minDiff == 0) return;
        }

        if(target < root.val) closest(root.left, target);
        else closest(root.right, target);
    }
}
```

[777 Toeplitz Matrix \(link\)](#)

Description

Given an $m \times n$ matrix, return *true* if the matrix is Toeplitz. Otherwise, return *false*.

A matrix is **Toeplitz** if every diagonal from top-left to bottom-right has the same elements.

Example 1:

1	2	3	4
5	1	2	3
9	5	1	2

Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]

Output: true

Explanation:

In the above grid, the diagonals are:

"[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]".

In each diagonal all elements are the same, so the answer is True.

Example 2:

1	2
2	2

Input: matrix = [[1,2],[2,2]]

Output: false

Explanation:

The diagonal "[1, 2]" has different elements.

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $0 \leq \text{matrix}[i][j] \leq 99$

Follow up:

- What if the `matrix` is stored on disk, and the memory is limited such that you can only load at most one row of the `matrix` into the memory at once?
- What if the `matrix` is so large that you can only load up a partial row into the memory at once?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isToeplitzMatrix(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        for(int i=1; i< m;i++)
            for(int j=1; j < n; j++)
                if(matrix[i][j] != matrix[i-1][j-1])
                    return false;
        return true;
    }
}
```

34 Find First and Last Position of Element in Sorted Array(link)

Description

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Example 3:

Input: `nums = []`, `target = 0`
Output: `[-1,-1]`

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- `nums` is a non-decreasing array.
- $-10^9 \leq \text{target} \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int first = find(nums, target, -1, true);
        if(first == -1) return new int[]{-1,-1};
        return new int[]{first, find(nums, target, first, false)};
    }

    int find(int[] nums, int target, int start, boolean isFirst) {
        int index=-1;
        int lo = start == -1? 0:start;
        int hi = nums.length-1;
        while(lo <= hi) {
            int mid = (lo+hi)/2;
            if(nums[mid] == target) {
                index = mid;
                if(isFirst) {
                    if(mid == lo || nums[mid-1] != target)
                        break;
                    hi=mid-1;
                } else {
                    if(mid == hi || nums[mid+1] != target)
                        break;
                    lo = mid+1;
                }
            } else if(nums[mid] > target) {
                hi = mid-1;
            } else {
                lo = mid+1;
            }
        }
        return index == -1 ? start:index;
    }
}
```


[238 Product of Array Except Self \(link\)](#)

Description

Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`
Output: `[0,0,9,0,0]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^5$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that `answer[i]` is **guaranteed** to fit in a **32-bit** integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array **does not** count as extra space for space complexity analysis.)

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int length = nums.length;
        int[] result = new int[length];

        //      result[length-1]=1;
        //      for(int i=length-2; i > -1;i--)
        //          result[i] = result[i+1]*nums[i+1];

        //      int ltProduct=1;
        //      for(int i=0;i<length;i++){
        //          result[i] *= ltProduct;
        //          ltProduct *= nums[i];
        //      }

        result[0] = 1;
        for(int i=1; i< length; i++)
            result[i]=result[i-1]*nums[i-1];

        int rtProduct = 1;
        for(int i=length-1; i>-1; i--) {
            result[i] *= rtProduct;
            rtProduct *= nums[i];
        }

        return result;
    }
}
```

[169 Majority Element \(link\)](#)

Description

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-109 <= nums[i] <= 109`

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int majorityElementII(int[] nums) {
        int idx = 0;
        int count = 1;
        for(int i=1;i<nums.length;i++){
            count += nums[idx]==nums[i] ? 1:-1;
            if(count==0){
                idx=i;
                count=1;
            }
        }
        return nums[idx];
    }

    public int majorityElement(int[] nums) {
        Map<Integer,Integer> map = new HashMap<>();
        int majorityCount=0, majorityNumber=0;
        for(int num: nums) {
            //map.merge(num,1,Integer::sum);
            map.put(num, map.getOrDefault(num,0)+1);
            int count = map.get(num);
            if(majorityCount < count) {
                majorityNumber = num;
                majorityCount = count;
            }
        }
        return majorityNumber;
    }
}
```

1072 Next Greater Node In Linked List (link)

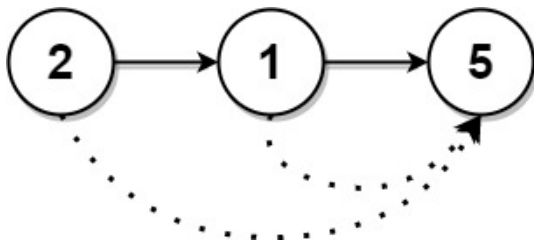
Description

You are given the head of a linked list with n nodes.

For each node in the list, find the value of the **next greater node**. That is, for each node, find the value of the first node that is next to it and has a **strictly larger** value than it.

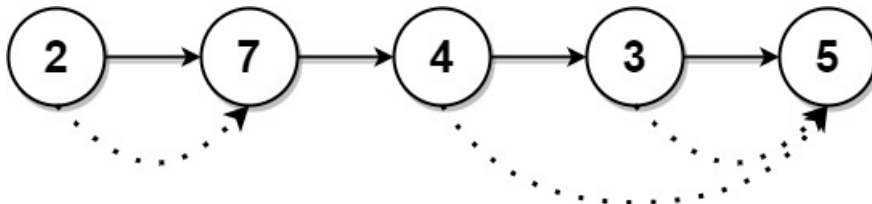
Return an integer array `answer` where `answer[i]` is the value of the next greater node of the i^{th} node (**1-indexed**). If the i^{th} node does not have a next greater node, set `answer[i] = 0`.

Example 1:



Input: head = [2,1,5]
Output: [5,5,0]

Example 2:



Input: head = [2,7,4,3,5]
Output: [7,0,5,5,0]

Constraints:

- The number of nodes in the list is n .
- $1 \leq n \leq 10^4$
- $1 \leq \text{Node.val} \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    int[] result = null, stack = null;
    int top = -1;
    int size = 0;

    public int[] nextLargerNodes(ListNode head) {
        nextLargerNodes(head, 0);
        return result;
    }

    private void nextLargerNodes(ListNode node, int pos) {
        if(node == null) {
            size = pos;
            result = new int[pos];
            stack = new int[pos];
            return;
        }

        nextLargerNodes(node.next, pos+1);
        while(top != -1 && stack[top] <= node.val)
            top--;

        result[pos] = top == -1 ? 0 : stack[top];
        if(top < size)
            stack[++top] = node.val;
    }
}
```

1951 Find the Winner of the Circular Game (link)

Description

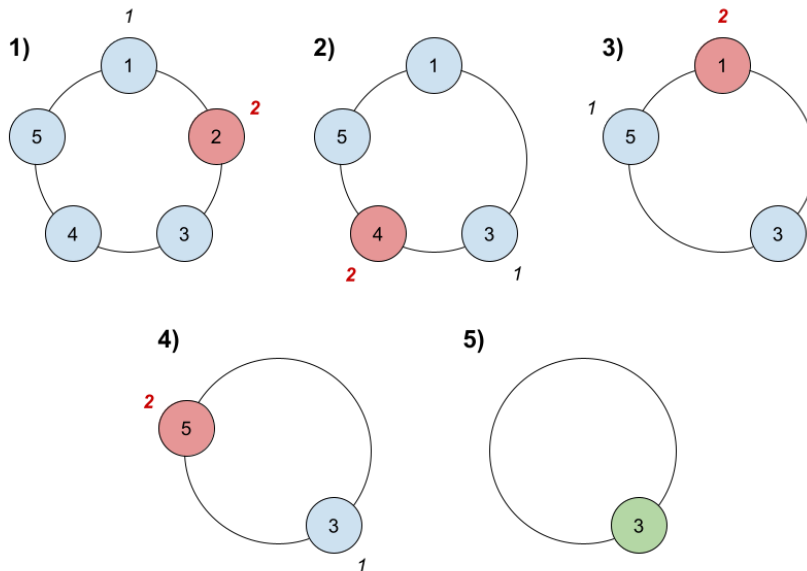
There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in **clockwise order**. More formally, moving clockwise from the i^{th} friend brings you to the $(i+1)^{\text{th}}$ friend for $1 \leq i < n$, and moving clockwise from the n^{th} friend brings you to the 1^{st} friend.

The rules of the game are as follows:

1. **Start** at the 1^{st} friend.
2. Count the next k friends in the clockwise direction **including** the friend you started at. The counting wraps around the circle and may count some friends more than once.
3. The last friend you counted leaves the circle and loses the game.
4. If there is still more than one friend in the circle, go back to step 2 **starting** from the friend **immediately clockwise** of the friend who just lost and repeat.
5. Else, the last friend in the circle wins the game.

Given the number of friends, n , and an integer k , return *the winner of the game*.

Example 1:



Input: $n = 5, k = 2$

Output: 3

Explanation: Here are the steps of the game:

- 1) Start at friend 1.
- 2) Count 2 friends clockwise, which are friends 1 and 2.
- 3) Friend 2 leaves the circle. Next start is friend 3.
- 4) Count 2 friends clockwise, which are friends 3 and 4.
- 5) Friend 4 leaves the circle. Next start is friend 5.
- 6) Count 2 friends clockwise, which are friends 5 and 1.
- 7) Friend 1 leaves the circle. Next start is friend 3.
- 8) Count 2 friends clockwise, which are friends 3 and 5.
- 9) Friend 5 leaves the circle. Only friend 3 is left, so they are the winner.

Example 2:

Input: $n = 6$, $k = 5$

Output: 1

Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.

Constraints:

- $1 \leq k \leq n \leq 500$

Follow up:

Could you solve this problem in linear time with constant space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findTheWinner(int n, int k) {
        return recurse(n,k)+1;
    }

    int recurse(int n, int k) {
        if(n==1) return 0;
        int winner = recurse(n-1,k);
        return (winner+k)%n;
    }
}
```

2058 Concatenation of Array [\(link\)](#)

Description

Given an integer array `nums` of length n , you want to create an array `ans` of length $2n$ where `ans[i] == nums[i]` and `ans[i + n] == nums[i]` for $0 \leq i < n$ (**0-indexed**).

Specifically, `ans` is the **concatenation** of two `nums` arrays.

Return *the array* `ans`.

Example 1:

```
Input: nums = [1,2,1]
Output: [1,2,1,1,2,1]
Explanation: The array ans is formed as follows:
- ans = [nums[0],nums[1],nums[2],nums[0],nums[1],nums[2]]
- ans = [1,2,1,1,2,1]
```

Example 2:

```
Input: nums = [1,3,2,1]
Output: [1,3,2,1,1,3,2,1]
Explanation: The array ans is formed as follows:
- ans = [nums[0],nums[1],nums[2],nums[3],nums[0],nums[1],nums[2],nums[3]]
- ans = [1,3,2,1,1,3,2,1]
```

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 1000$
- $1 \leq \text{nums}[i] \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] getConcatenation(int[] nums) {
        int length = nums.length;
        int[] ans = new int[2*length];
        for(int i=0; i < length; i++) {
            ans[i] = ans[length+i]= nums[i];
        }
        return ans;
    }
}
```

[1950 Sign of the Product of an Array \(link\)](#)

Description

Implement a function `signFunc(x)` that returns:

- 1 if x is positive.
- -1 if x is negative.
- 0 if x is equal to 0.

You are given an integer array `nums`. Let `product` be the product of all values in the array `nums`.

Return `signFunc(product)`.

Example 1:

Input: `nums = [-1,-2,-3,-4,3,2,1]`

Output: 1

Explanation: The product of all values in the array is 144, and `signFunc(144) = 1`

Example 2:

Input: `nums = [1,5,0,2,-3]`

Output: 0

Explanation: The product of all values in the array is 0, and `signFunc(0) = 0`

Example 3:

Input: `nums = [-1,1,-1,1,-1]`

Output: -1

Explanation: The product of all values in the array is -1, and `signFunc(-1) = -1`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-100 \leq \text{nums}[i] \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int arraySign(int[] nums) {
        int negSignCount=0;
        for(int num:nums) {
            if(num == 0) return 0;
            if(num < 0) negSignCount++;
        }
        return (negSignCount&1) == 0 ? 1:-1;
    }
}
```

[501 Find Mode in Binary Search Tree \(link\)](#)

Description

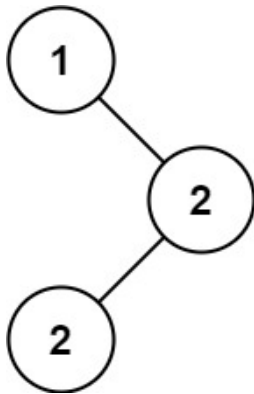
Given the `root` of a binary search tree (BST) with duplicates, return *all the [mode\(s\)](#)* (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in **any order**.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys **less than or equal to** the node's key.
- The right subtree of a node contains only nodes with keys **greater than or equal to** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



Input: `root = [1,null,2,2]`
Output: `[2]`

Example 2:

Input: `root = [0]`
Output: `[0]`

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$

Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int[] findMode(TreeNode root) {
        List<Integer> modes = new ArrayList<>();
        Integer[] counts = new Integer[3];
        counts[0] = counts[1] = 0;
        dfs(root, modes, counts);
        int[] result = new int[modes.size()];
        for(int i=0; i < result.length;i++)
            result[i]=modes.get(i);
        return result;
    }

    //counts[0] counter for current element
    //counts[1] max counter
    //counts[2] prev value
    void dfs(TreeNode root, List<Integer> modes, Integer[] counts) {
        if(root == null) return;

        dfs(root.left, modes, counts);
        if(counts[2] != null && counts[2] == root.val) {
            counts[0]++;
        } else {
            counts[0]=1;
        }

        if(counts[0] == counts[1]){
            modes.add(root.val);
        } else if(counts[0] > counts[1]) {
            counts[1] = counts[0];
            modes.clear();
            modes.add(root.val);
        }

        counts[2] = root.val;
        dfs(root.right, modes, counts);
    }
}
```

320 Generalized Abbreviation (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<String> generateAbbreviations(String word) {
        List<String> result = new ArrayList<>();
        generateAbbreviations(word, new StringBuilder(), 0, 0, result);
        return result;
    }

    public void generateAbbreviations(String word, StringBuilder bldr, int start, int count, List<String> result) {
        int length = bldr.length();
        if(start == word.length()) {
            if (count != 0)
                bldr.append(count);
            result.add(bldr.toString());
        } else {
            //abbreviate part
            generateAbbreviations(word, bldr, start+1, count+1, result);
            if (count != 0)
                bldr.append(count);

            //skip abbreviate part
            generateAbbreviations(word, bldr.append(word.charAt(start)), start+1, 0, result);
        }
        bldr.setLength(length);
    }
}
```

378 Kth Smallest Element in a Sorted Matrix (link)

Description

Given an $n \times n$ matrix where each of the rows and columns is sorted in ascending order, return *the* k^{th} *smallest element in the matrix*.

Note that it is the k^{th} smallest element **in the sorted order**, not the k^{th} **distinct** element.

You must find a solution with a memory complexity better than $O(n^2)$.

Example 1:

Input: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8

Output: 13

Explanation: The elements in the matrix are [1,5,9,10,11,12,13,13,15], and the 8th smallest number is 13

Example 2:

Input: matrix = [[-5]], k = 1

Output: -5

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 300$
- $-10^9 \leq \text{matrix}[i][j] \leq 10^9$
- All the rows and columns of matrix are **guaranteed** to be sorted in **non-decreasing order**.
- $1 \leq k \leq n^2$

Follow up:

- Could you solve the problem with a constant memory (i.e., $O(1)$ memory complexity)?
- Could you solve the problem in $O(n)$ time complexity? The solution may be too advanced for an interview but you may find reading [this paper](#) fun.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;
        int lo = matrix[0][0], hi = matrix[n-1][n-1];
        while(lo < hi) {
            int mid = lo+(hi-lo)/2;
            int count = count(matrix, mid);
            if(count < k)
                lo = mid+1;
            else
                hi = mid;
        }
        return lo;
    }

    private int count(int[][] matrix, int target) {
        int count=0;
        int n = matrix.length, row = n-1, col=0;
        while(row > -1 && col < n) {
            if(matrix[row][col] > target) {
                row--;
            } else {
                count += row+1;
                col++;
            }
        }
        return count;
    }
}
```

766 Flatten a Multilevel Doubly Linked List (link)

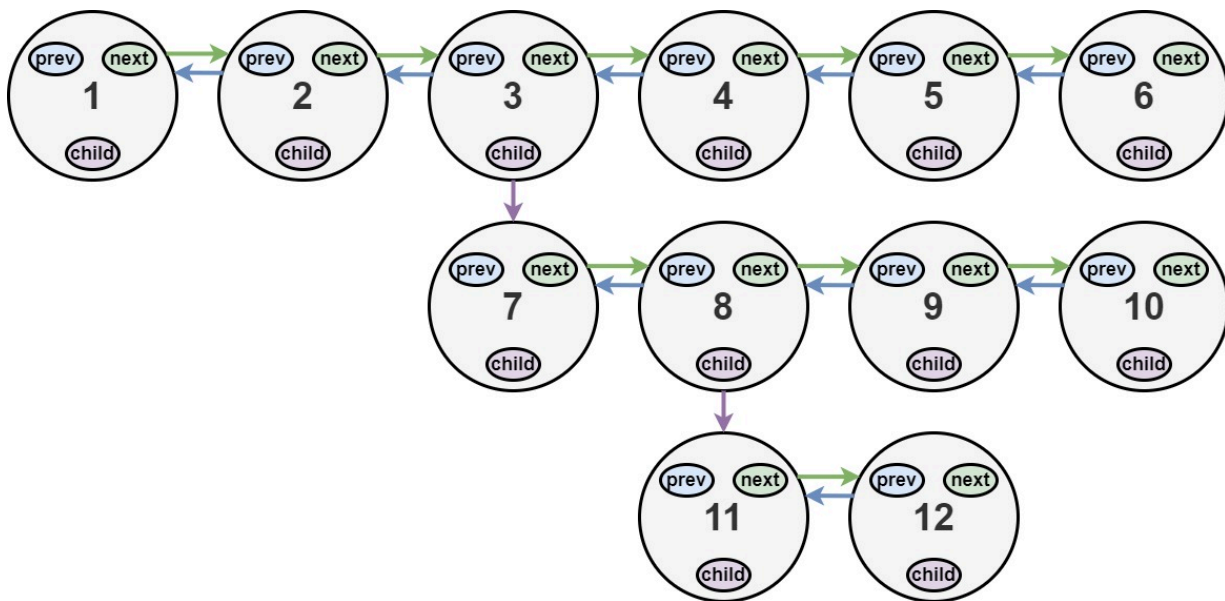
Description

You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional **child pointer**. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a **multilevel data structure** as shown in the example below.

Given the head of the first level of the list, **flatten** the list so that all the nodes appear in a single-level, doubly linked list. Let *curr* be a node with a child list. The nodes in the child list should appear **after** *curr* and **before** *curr.next* in the flattened list.

Return *the head of the flattened list*. The nodes in the list must have **all** of their child pointers set to null.

Example 1:

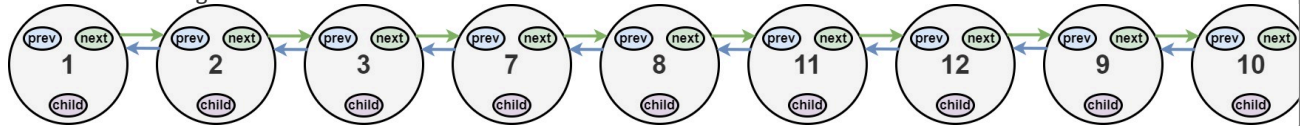


Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

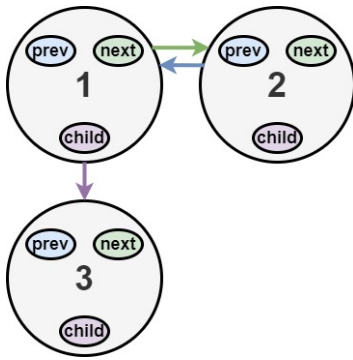
Output: [1,2,3,7,8,11,12,9,10,4,5,6]

Explanation: The multilevel linked list in the input is shown.

After flattening the multilevel linked list it becomes:



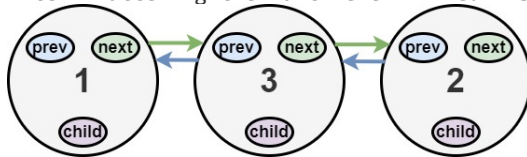
Example 2:



Input: head = [1,2,null,3]

Output: [1,3,2]

Explanation: The multilevel linked list in the input is shown. After flattening the multilevel linked list it becomes:



Example 3:

Input: head = []

Output: []

Explanation: There could be empty list in the input.

Constraints:

- The number of Nodes will not exceed 1000.
- $1 \leq \text{Node.val} \leq 10^5$

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from **Example 1** above:

```

1---2---3---4---5---6--NULL
      |
      7---8---9---10--NULL
          |
          11--12--NULL
  
```

The serialization of each level is as follows:

```

[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
  
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:

```

[1,    2,    3, 4, 5, 6, null]
   |
[null, null, 7,    8, 9, 10, null]
  
```

```
[
    null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public Node prev;
    public Node next;
    public Node child;
};
*/

class Solution {
    public Node flatten(Node head) {
        Node node = head, child, next;
        if (node == null)
            return node;

        child = node.child;
        next = node.next;
        if (child != null) {
            node.next = flatten(child);
            node.child = null;
            if (node.next != null)
                node.next.prev = node;
            while (node.next != null)
                node = node.next;
        }

        if (next != null) {
            node.next = flatten(next);
            if (node.next != null)
                node.next.prev = node;
        }
        return head;
    }
}
```

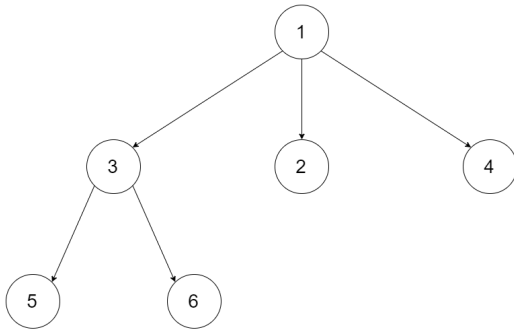
764 N-ary Tree Level Order Traversal (link)

Description

Given an n-ary tree, return the *level order* traversal of its nodes' values.

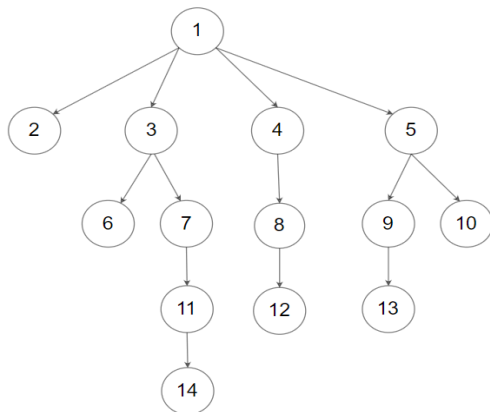
N-ary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

Example 1:



Input: root = [1,null,3,2,4,null,5,6]
Output: [[1],[3,2,4],[5,6]]

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [[1],[2,3,4,5],[6,7,8,9,10],[11,12,13],[14]]

Constraints:

- The height of the n-ary tree is less than or equal to 1000
- The total number of nodes is between [0, 10⁴]

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, List<Node> _children) {
        val = _val;
        children = _children;
    }
};
*/

class Solution {
    public List<List<Integer>> levelOrder(Node root) {
        List<List<Integer>> result = new ArrayList<>();
        traverse(root, 0, result);
        return result;
    }

    void traverse(Node root, int depth, List<List<Integer>> list) {
        if(root == null) return;

        if(list.size() == depth)
            list.add(depth, new ArrayList<>());
        list.get(depth).add(root.val);

        for(Node node: root.children)
            traverse(node, depth+1, list);
    }
}
```

277 Find the Celebrity [\(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/* The knows API is defined in the parent class Relation.
   boolean knows(int a, int b); */

public class Solution extends Relation {
    public int findCelebrity(int n) {
        int celebrity = 0;
        for(int i=0; i < n; i++) {
            if(knows(celebrity, i))
                celebrity = i;
        }

        for(int i=0; i < n; i++) {
            if (i == celebrity)
                continue;
            if (knows(celebrity, i) || !knows(i, celebrity))
                return -1;
        }

        return celebrity;
    }
}
```

[271 Encode and Decode Strings \(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
public class Codec {

    private String intToString(String str) {
        int length = str.length();
        char[] bytes = new char[4];
        for(int i=3; i > -1; i--) {
            bytes[3-i]= (char)((length >> (i*8)) & 0xff);
        }
        return new String(bytes);
    }

    // Encodes a list of strings to a single string.
    public String encode(List<String> strs) {
        StringBuilder sb = new StringBuilder();
        for(String str: strs)
            sb.append(intToString(str)).append(str);
        return sb.toString();
    }

    private int stringToInt(String length) {
        int result=0;
        for(char b: length.toCharArray())
            result = (result<<8) + (int)b;
        return result;
    }

    // Decodes a single string to a list of strings.
    public List<String> decode(String s) {
        List<String> result = new ArrayList<>();
        int i=0, length=s.length();
        while(i < length) {
            int chunkLength = stringToInt(s.substring(i,i+4));
            i+=4;
            result.add(s.substring(i, i+chunkLength));
            i+=chunkLength;
        }
        return result;
    }
}

// Your Codec object will be instantiated and called as such:
// Codec codec = new Codec();
// codec.decode(codec.encode(strs));
```

[287 Find the Duplicate Number \(link\)](#)

Description

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

Constraints:

- $1 \leq n \leq 10^5$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in `nums` appear only **once** except for **precisely one integer** which appears **two or more** times.

Follow up:

- How can we prove that at least one duplicate number must exist in `nums`?
- Can you solve the problem in linear runtime complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findDuplicate(int[] arr) {
        int i=0;
        while(i < arr.length) {
            if(arr[i] != i+1) {
                if(arr[i] == arr[arr[i]-1])
                    return arr[i];
                else
                    swap(arr, arr[i]-1,i);
            } else {
                i++;
            }
        }
        return -1;
    }

    void swap(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }
}
```

163 Missing Ranges (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<String> findMissingRanges(int[] nums, int lower, int upper) {
        // declare prev value
        //for index < nums.length
        //increase array to lower-1 to upper+1
        //if current - prev > 1 then missing range.
        List<String> resultArray = new ArrayList<>();
        int prev = lower-1;
        for( int i = 0; i <= nums.length; i++) {
            int curr = (i < nums.length) ? nums[i] : upper + 1;
            if(curr - prev > 1) { // missing range
                resultArray.add(formatRange(prev+1, curr-1));
            }
            prev = curr;
        }

        return resultArray;
    }

    // formats range in the requested format
    private String formatRange(int lower, int upper) {
        if (lower == upper) {
            return String.valueOf(lower);
        }
        return lower + "->" + upper;
    }
}
```

[228 Summary Ranges \(link\)](#)

Description

You are given a **sorted unique** integer array `nums`.

A **range** `[a,b]` is the set of all integers from `a` to `b` (inclusive).

Return *the **smallest sorted** list of ranges that **cover all the numbers in the array exactly***. That is, each element of `nums` is covered by exactly one of the ranges, and there is no integer `x` such that `x` is in one of the ranges but not in `nums`.

Each range `[a,b]` in the list should be output as:

- `"a->b"` if `a != b`
- `"a"` if `a == b`

Example 1:

```
Input: nums = [0,1,2,4,5,7]
Output: ["0->2","4->5","7"]
Explanation: The ranges are:
[0,2] --> "0->2"
[4,5] --> "4->5"
[7,7] --> "7"
```

Example 2:

```
Input: nums = [0,2,3,4,6,8,9]
Output: ["0","2->4","6","8->9"]
Explanation: The ranges are:
[0,0] --> "0"
[2,4] --> "2->4"
[6,6] --> "6"
[8,9] --> "8->9"
```

Constraints:

- $0 \leq \text{nums.length} \leq 20$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- All the values of `nums` are **unique**.
- `nums` is sorted in ascending order.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<String> summaryRanges(int[] nums) {
        List<String> result = new ArrayList<>();
        int lo=0, hi=0, length = nums.length;
        StringBuilder sb = new StringBuilder();
        while(hi < length) {
            if(sb.length()==0)
                sb.append(nums[hi]);
            if(hi < length-1 && nums[hi]+1 == nums[hi+1]) {
                hi++;
                continue;
            } else if(hi != lo) {
                sb.append("->").append(nums[hi]);
            }
            result.add(sb.toString());
            lo = ++hi;
            sb.setLength(0);
        }
        return result;
    }
}
```

346 Moving Average from Data Stream (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MovingAverage {
    int[] qu;
    int size, counter;
    double sum;
    int head;

    public MovingAverage(int size) {
        this.size = size;
        this.qu = new int[size];

        this.counter = 0;
        this.sum = 0.0;
        this.head = 0;
    }

    public double next(int val) {
        ++counter;
        head %= size;
        sum += -qu[head]+val;
        qu[head++] = val;
        return (sum)/Math.min(counter, size);
    }
}

/**
 * Your MovingAverage object will be instantiated and called as such:
 * MovingAverage obj = new MovingAverage(size);
 * double param_1 = obj.next(val);
 */
```

253 Meeting Rooms II (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        int length = intervals.length;
        int[] startTimes = new int[length];
        int[] endTimes = new int[length];

        int i=0;
        for(int[] interval: intervals) {
            startTimes[i] = interval[0];
            endTimes[i++] = interval[1];
        }

        Arrays.sort(startTimes);
        Arrays.sort(endTimes);

        int start=0, end = 0, requiredRooms=0;
        while(start < length) {
            if(startTimes[start] >= endTimes[end] ) {
                requiredRooms--;
                end++;
            }

            requiredRooms++;
            start++;
        }
        return requiredRooms;
    }
}
```

170 Two Sum III - Data structure design (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class TwoSum {
    private HashMap<Integer, Integer> map;
    public TwoSum() {
        map = new HashMap<>();
    }

    public void add(int number) {
        map.put(number, map.getOrDefault(number,0)+1);
    }

    public boolean find(int value) {
        for(int num: map.keySet()) {
            int complement = value - num;
            if(complement != num) {
                if(map.containsKey(complement)) {
                    return true;
                }
            } else if(map.get(num) > 1) {
                return true;
            }
        }
        return false;
    }
}

/**
 * Your TwoSum object will be instantiated and called as such:
 * TwoSum obj = new TwoSum();
 * obj.add(number);
 * boolean param_2 = obj.find(value);
 */
```

27 Remove Element (link)

Description

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
                             // It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [3,2,2,3]`, `val = 3`
Output: `2`, `nums = [2,2,_,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being `2`. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,1,2,2,3,0,4,2]`, `val = 2`
Output: `5`, `nums = [0,1,4,0,3,_,_,_]`
Explanation: Your function should return `k = 5`, with the first five elements of `nums` containing `0`, `0`, `1`, `4`, `3`. Note that the five elements can be returned in any order. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Constraints:

- $0 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 50$
- $0 \leq \text{val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int removeElement(int[] nums, int val) {
        //two index, if value of array == val, then replace left with non val
        int left = 0;
        for(int right = 0; right < nums.length; right++) {
            if(nums[right] != val) {
                nums[left] = nums[right];
                left++;
            }
        }
        return left;
    }
}
```

450 Delete Node in a BST (link)

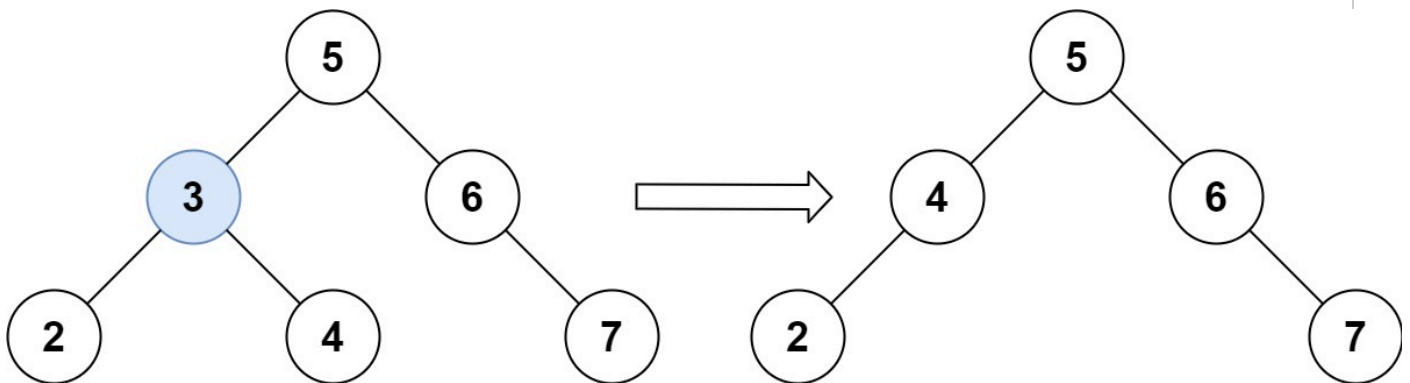
Description

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return *the root node reference (possibly updated) of the BST*.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:



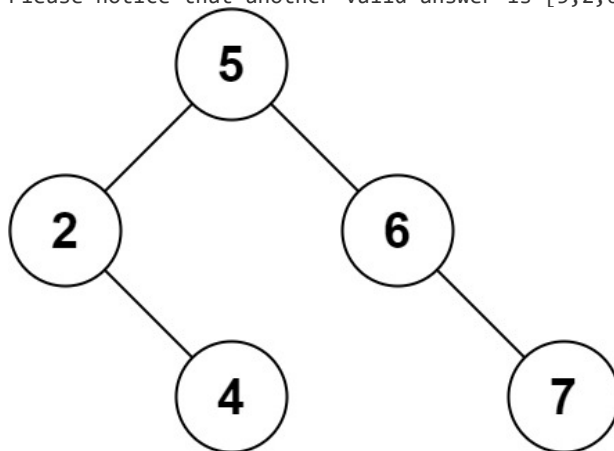
Input: root = [5,3,6,2,4,null,7], key = 3

Output: [5,4,6,2,null,null,7]

Explanation: Given key to delete is 3. So we find the node with value 3 and delete it.

One valid answer is [5,4,6,2,null,null,7], shown in the above BST.

Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.



Example 2:

Input: root = [5,3,6,2,4,null,7], key = 0

Output: [5,3,6,2,4,null,7]

Explanation: The tree does not contain a node with value = 0.

Example 3:

Input: root = [], key = 0

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- Each node has a **unique** value.
- root is a valid binary search tree.
- $-10^5 \leq \text{key} \leq 10^5$

Follow up: Could you solve it with time complexity $O(\text{height of tree})$?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode deleteNode(TreeNode root, int key) {
        if(root == null) return null;
        if(key > root.val) root.right = deleteNode(root.right, key);
        else if(key < root.val) root.left = deleteNode(root.left, key);
        else {
            if(root.left == null && root.right == null) root = null;
            else if(root.right != null) {
                root.val = successor(root);
                root.right = deleteNode(root.right, root.val);
            } else {
                root.val = predecessor(root);
                root.left = deleteNode(root.left, root.val);
            }
        }
        return root;
    }

    private int successor(TreeNode root) {
        root = root.right;
        while(root.left != null) root = root.left;
        return root.val;
    }

    private int predecessor(TreeNode root) {
        root = root.left;
        while(root.right != null) root = root.right;
        return root.val;
    }
}
```

26 Remove Duplicates from Sorted Array [\(link\)](#)

Description

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**. Then return *the number of unique elements in `nums`*.

Consider the number of unique elements of `nums` to be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the unique elements in the order they were present in `nums` initially. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`

Output: `2, nums = [1,2,_]`

Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`

Output: `5, nums = [0,1,2,3,4,_,_,_,_,_]`

Explanation: Your function should return `k = 5`, with the first five elements of `nums` being 0, 1, 2, 3, and 4. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- `nums` is sorted in **non-decreasing order**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int left = 1;
        for(int right = 0; right < nums.length -1 ; right++) {
            if(nums[right] != nums[right + 1]) {
                nums[left] = nums[right + 1];
                left++;
            }
        }
        return left;
    }
}
```

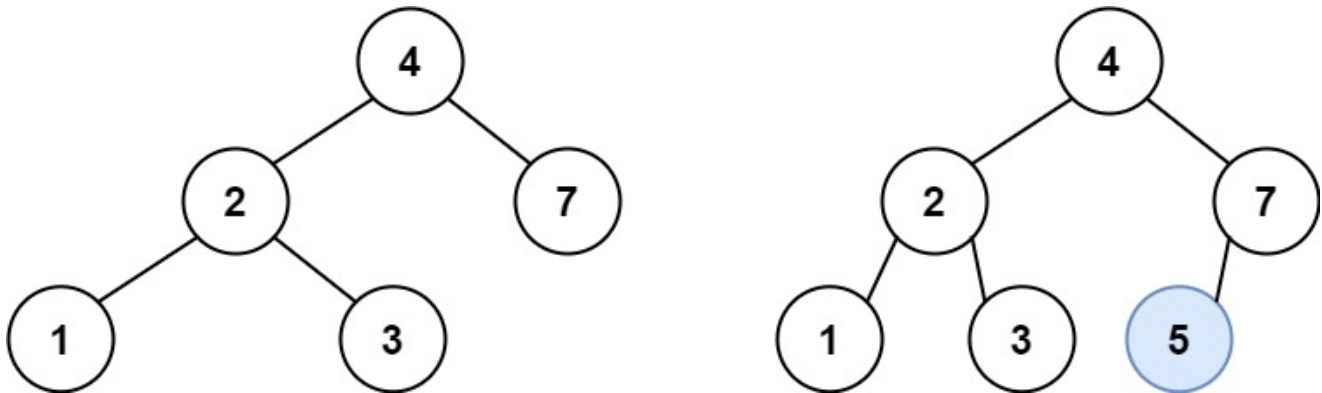

784 Insert into a Binary Search Tree (link)

Description

You are given the `root` node of a binary search tree (BST) and a `value` to insert into the tree. Return *the root node of the BST after the insertion*. It is **guaranteed** that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return **any of them**.

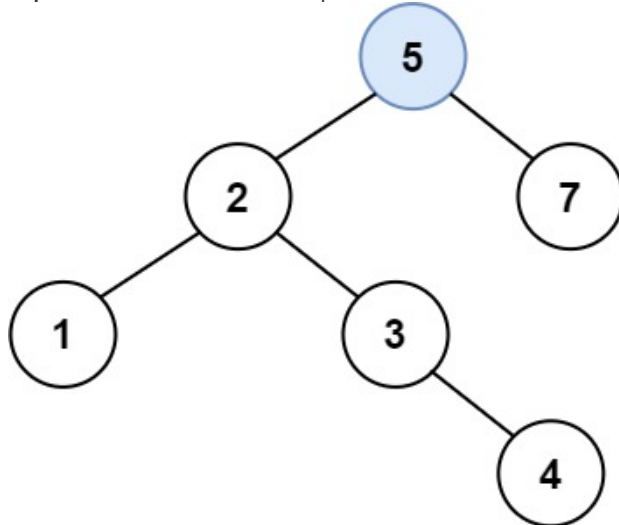
Example 1:



Input: root = [4,2,7,1,3], val = 5

Output: [4,2,7,1,3,5]

Explanation: Another accepted tree is:



Example 2:

Input: root = [40,20,60,10,30,50,70], val = 25

Output: [40,20,60,10,30,50,70,null,null,25]

Example 3:

Input: root = [4,2,7,1,3,null,null,null,null,null,null], val = 5

Output: [4,2,7,1,3,5]

Constraints:

- The number of nodes in the tree will be in the range $[0, 10^4]$.
- $-10^8 \leq \text{Node.val} \leq 10^8$
- All the values `Node.val` are **unique**.
- $-10^8 \leq \text{val} \leq 10^8$
- It's **guaranteed** that `val` does not exist in the original BST.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode insertIntoBST(TreeNode root, int val) {
        TreeNode node = root;
        while(node != null) {
            if(val > node.val) {
                if(node.right == null) {
                    node.right = new TreeNode(val);
                    return root;
                } else node = node.right;
            } else {
                if(node.left == null) {
                    node.left = new TreeNode(val);
                    return root;
                } else node = node.left;
            }
        }
        return new TreeNode(val);
    }
}
```

217 Contains Duplicate (link)

Description

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: `true`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Map<Integer, Boolean> containsMap = new HashMap<>(nums.length);

        for(int i = 0 ; i < nums.length; i++) {
            if(containsMap.containsKey(nums[i])) {
                return true;
            }
            containsMap.put(nums[i], true);
        }
        return false;
    }
}
```

215 Kth Largest Element in an Array (link)

Description

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        int lo = 0, length=nums.length, hi = length-1;
        int K = length-k;
        while(lo < hi) {
            int pivot=partition(nums, lo, hi);
            if(pivot < K) lo = pivot+1;
            else hi = pivot-1;
        }

        //kth index value is the kth smallest
        //Kth index value is the kth largest
        return nums[K];
    }

    private int partition(int[] S, int lo, int hi) {
        int left = lo;
        int right = hi-1;
        int pivot = (int) (Math.random()*(hi-lo))+lo;

        swap(S, pivot, hi);
        pivot = hi;

        while (left <= right) {
            while (left <= right && S[left] < S[pivot]) left++;
            while (left <= right && S[right] > S[pivot]) right--;
            if(left <= right) swap(S, left++, right--);
        }

        // put pivot into its final place (currently marked by left index)
        swap(S, left, hi);
        return left;
    }

    private void swap(int[] S, int a, int b) {
        int temp = S[a];
        S[a] = S[b];
        S[b] = temp;
    }
}
```

[199 Binary Tree Right Side View \(link\)](#)

Description

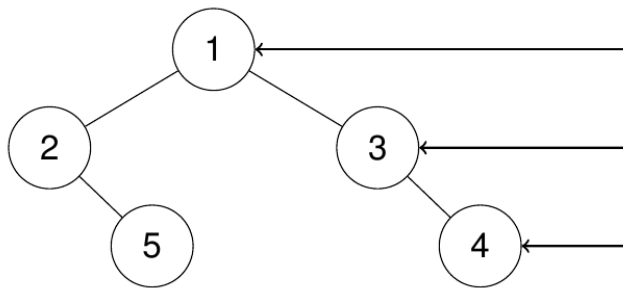
Given the `root` of a binary tree, imagine yourself standing on the **right side** of it, return *the values of the nodes you can see ordered from top to bottom*.

Example 1:

Input: `root = [1,2,3,null,5,null,4]`

Output: `[1,3,4]`

Explanation:

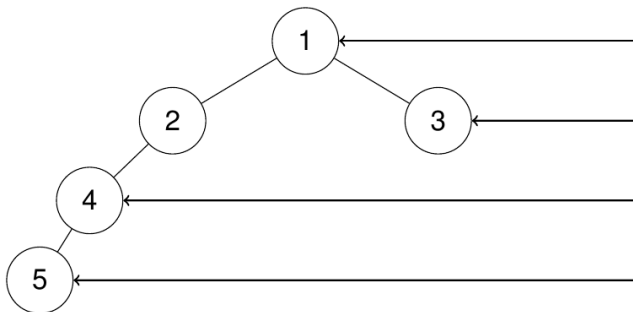


Example 2:

Input: `root = [1,2,3,4,null,null,null,5]`

Output: `[1,3,4,5]`

Explanation:



Example 3:

Input: `root = [1,null,3]`

Output: `[1,3]`

Example 4:

Input: `root = []`

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        traverse(root, 0, result);
        return result;
    }

    private void traverse(TreeNode root, int level, List<Integer> result) {
        if(root == null) return;

        if(level == result.size()) {
            result.add(root.val);
        }

        if(root.right != null) {
            traverse(root.right, level+1, result);
        }

        if(root.left != null) {
            traverse(root.left, level+1, result);
        }
    }
}
```

[125 Valid Palindrome \(link\)](#)

Description

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` *if it is a **palindrome***, or `false` *otherwise*.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

```
Input: s = " "
Output: true
Explanation: s is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.
```

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- `s` consists only of printable ASCII characters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isPalindrome(String s) {
        int length = s.length();
        int left=0, right=length-1;
        while(left < right) {
            while(left < length && !Character.isLetterOrDigit(s.charAt(left))) left++;
            while(right > -1 && !Character.isLetterOrDigit(s.charAt(right))) right--;

            if(left == length || right == -1 || left >= right) break;

            char lch = s.charAt(left++), rch = s.charAt(right--);
            if(lch >= '0' && lch <='9' && lch != rch) return false;
            if((lch|' ') != (rch|' ')) return false;
        }
        return true;
    }
}
```

[1 Two Sum \(link\)](#)

Description

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

Constraints:

- $2 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        //Create Hash of numbers with key = num, value = index
        //check if target - num in
        Map<Integer, Integer> numsMap = new HashMap<>();
        for(int i = 0 ; i < nums.length; i++) {
            int val = nums[i];
            int other = target - val;
            if(numsMap.containsKey(other) && numsMap.get(other) != i )
                return new int[]{i, numsMap.get(other)};
            numsMap.put(nums[i], i);
        }
        return new int[2];
    }
}
```

144 Binary Tree Preorder Traversal (link)

Description

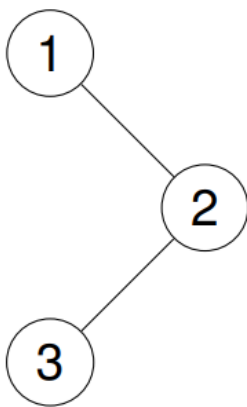
Given the root of a binary tree, return *the preorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

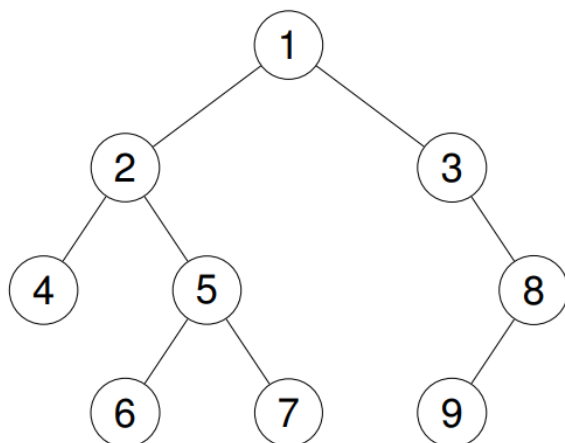


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        morrisTraversal(root, result::add);
        return result;
    }

    private void morrisTraversal(TreeNode node, Consumer<Integer> visitor) {
        while(node != null) {
            if(node.left == null) {
                visitor.accept(node.val);
                node = node.right;
            } else {
                TreeNode next = node.left;
                while(next.right != null && next.right != node)
                    next = next.right;

                if(next.right == null) {
                    visitor.accept(node.val);
                    next.right = node;
                    node = node.left;
                } else {
                    next.right = null;
                    node = node.right;
                }
            }
        }
    }
}
```

145 Binary Tree Postorder Traversal (link)

Description

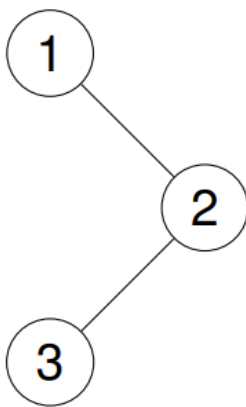
Given the root of a binary tree, return *the postorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

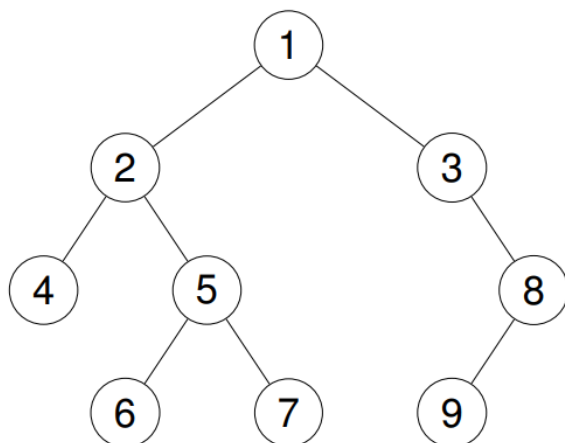


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of the nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        morrisTraversal(root, result::add);
        Collections.reverse(result);
        return result;
    }

    private void morrisTraversal(TreeNode node, Consumer<Integer> visitor) {
        while(node != null) {
            if(node.right == null) {
                visitor.accept(node.val);
                node = node.left;
            } else {
                TreeNode prev = node.right;
                while(prev.left != null && prev.left != node)
                    prev = prev.left;
                if(prev.left == null) {
                    visitor.accept(node.val);
                    prev.left = node;
                    node = node.right;
                } else {
                    prev.left = null;
                    node = node.left;
                }
            }
        }
    }
}
```

[66 Plus One \(link\)](#)

Description

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

Input: `digits = [1,2,3]`
Output: `[1,2,4]`
Explanation: The array represents the integer 123.
Incrementing by one gives $123 + 1 = 124$.
Thus, the result should be `[1,2,4]`.

Example 2:

Input: `digits = [4,3,2,1]`
Output: `[4,3,2,2]`
Explanation: The array represents the integer 4321.
Incrementing by one gives $4321 + 1 = 4322$.
Thus, the result should be `[4,3,2,2]`.

Example 3:

Input: `digits = [9]`
Output: `[1,0]`
Explanation: The array represents the integer 9.
Incrementing by one gives $9 + 1 = 10$.
Thus, the result should be `[1,0]`.

Constraints:

- $1 \leq \text{digits.length} \leq 100$
- $0 \leq \text{digits}[i] \leq 9$
- `digits` does not contain any leading 0's.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] plusOne(int[] digits) {
        int length = digits.length;
        for(int i=length-1; i > -1; i--) {
            if(digits[i] < 9) {
                digits[i]++;
                return digits;
            }
            digits[i]=0;
        }
        int[] result = new int[length+1];
        result[0]=1;
        return result;
    }
}
```

[213 House Robber II \(link\)](#)

Description

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are **arranged in a circle**. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have a security system connected, and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police***.

Example 1:

Input: `nums = [2,3,2]`

Output: 3

Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent.

Example 2:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 3:

Input: `nums = [1,2,3]`

Output: 3

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int rob(int[] nums) {
        int length = nums.length;
        if(length == 0)
            return 0;
        if(length == 1)
            return nums[0];
        return Math.max(rob(nums, 0, length-2), rob(nums, 1, length-1));
    }

    public int rob(int[] nums, int start, int end) {
        // int robFirst = nums[end], robSecond = 0;
        // for(int i=end-1; i >= start ; i--) {
        //     int current = Math.max(robFirst, robSecond+nums[i]);
        //     robSecond = robFirst;
        //     robFirst = current;
        // }
        // return robFirst;
        int robFirst=0, robSecond=0;
        for(int i=start; i <= end; i++) {
            int current = Math.max(robFirst+nums[i], robSecond);
            robFirst=robSecond;
            robSecond = current;
        }
        return robSecond;
    }
}
```


[198 House Robber \(link\)](#)

Description

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night.**

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police.***

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int rob(int[] nums) {
        int length = nums.length;
        if(length == 0)
            return 0;
        // int robFirst=nums[length-1], robSecond=0;
        // for(int i=length-2; i>=0;i--) {
        //     int current = Math.max(robFirst, nums[i]+robSecond);
        //     robSecond=robFirst;
        //     robFirst=current;
        // }
        //return robFirst;

        int first=0, second=0;
        for(int i=0; i < length; i++) {
            int current = Math.max(first+nums[i], second);
            first=second;
            second=current;
        }

        return second;
    }
}
```

167 Two Sum II - Input Array Is Sorted (link)

Description

Given a **1-indexed** array of integers `numbers` that is already ***sorted in non-decreasing order***, find two numbers such that they add up to a specific `target` number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where $1 \leq \text{index}_1 < \text{index}_2 \leq \text{numbers.length}$.

Return *the indices of the two numbers*, `index1` and `index2`, **added by one as an integer array** `[index1, index2]` of length 2.

The tests are generated such that there is **exactly one solution**. You **may not** use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: `numbers = [2,7,11,15]`, `target = 9`

Output: `[1,2]`

Explanation: The sum of 2 and 7 is 9. Therefore, `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

Example 2:

Input: `numbers = [2,3,4]`, `target = 6`

Output: `[1,3]`

Explanation: The sum of 2 and 4 is 6. Therefore `index1 = 1`, `index2 = 3`. We return `[1, 3]`.

Example 3:

Input: `numbers = [-1,0]`, `target = -1`

Output: `[1,2]`

Explanation: The sum of -1 and 0 is -1. Therefore `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

Constraints:

- $2 \leq \text{numbers.length} \leq 3 * 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- `numbers` is sorted in **non-decreasing order**.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is **exactly one solution**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int left=0, right = numbers.length-1;
        while(left < right) {
            int other = target - numbers[left];
            if(numbers[right] == other)
                return new int[]{left+1, right+1};
            else if(other < numbers[right])
                right--;
            else
                left++;
        }
        return new int[]{-1,-1};
    }
}
```

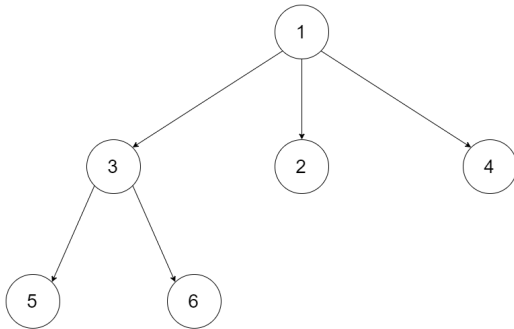
775 N-ary Tree Preorder Traversal (link)

Description

Given the root of an n-ary tree, return *the preorder traversal of its nodes' values*.

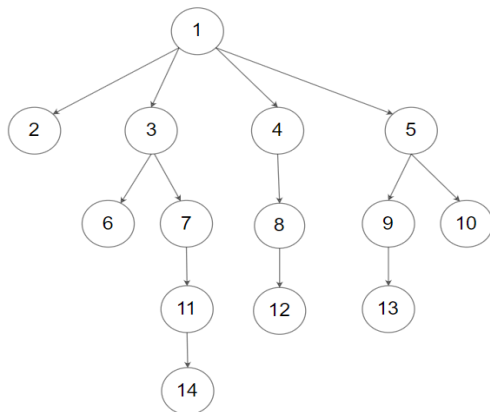
N-ary-Tree input serialization is represented in their level order traversal. Each group of children is separated by the null value (See examples)

Example 1:



Input: root = [1,null,3,2,4,null,5,6]
Output: [1,3,5,6,2,4]

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [1,2,3,6,7,11,14,4,8,12,5,9,13,10]

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $0 \leq \text{Node.val} \leq 10^4$
- The height of the n-ary tree is less than or equal to 1000.

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, List<Node> _children) {
        val = _val;
        children = _children;
    }
};
*/

class Solution {
    List<Integer> result = new ArrayList<>();
    public List<Integer> preorder(Node root) {
        if(root == null)
            return result;
        result.add(root.val);
        for(Node node: root.children)
            preorder(node);
        return result;
    }
}
```

[47 Permutations II \(link\)](#)

Description

Given a collection of numbers, `nums`, that might contain duplicates, return *all possible unique permutations in any order*.

Example 1:

```
Input: nums = [1,1,2]
Output:
[[1,1,2],
 [1,2,1],
 [2,1,1]]
```

Example 2:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Constraints:

- `1 <= nums.length <= 8`
- `-10 <= nums[i] <= 10`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> permuteUnique(int[] nums) {
        List<List<Integer>> permutations = new ArrayList<>();
        int length = nums.length;
        if(length < 2) {
            permutations.add(List.of(nums[0]));
        } else{
            Arrays.sort(nums);
            dfs(nums, length, new boolean[length], new LinkedList<>(), permutations);
        }
        return permutations;
    }

    void dfs(int[] arr, int k, boolean[] used, LinkedList<Integer> curr, List<List<Integer>> permutations) {
        if(curr.size() == k) {
            permutations.add(new ArrayList<>(curr));
            return;
        }

        for(int i=0; i < arr.length; i++) {
            if(used[i] || (i>0 && arr[i-1] == arr[i] && !used[i-1]))
                continue;

            curr.addLast(arr[i]);
            used[i] = true;

            dfs(arr, k, used, curr, permutations);

            curr.removeLast();
            used[i] = false;
        }
    }
}
```

[1444 Number of Steps to Reduce a Number to Zero \(link\)](#)

Description

Given an integer `num`, return *the number of steps to reduce it to zero*.

In one step, if the current number is even, you have to divide it by 2, otherwise, you have to subtract 1 from it.

Example 1:

```
Input: num = 14
Output: 6
Explanation:
Step 1) 14 is even; divide by 2 and obtain 7.
Step 2) 7 is odd; subtract 1 and obtain 6.
Step 3) 6 is even; divide by 2 and obtain 3.
Step 4) 3 is odd; subtract 1 and obtain 2.
Step 5) 2 is even; divide by 2 and obtain 1.
Step 6) 1 is odd; subtract 1 and obtain 0.
```

Example 2:

```
Input: num = 8
Output: 4
Explanation:
Step 1) 8 is even; divide by 2 and obtain 4.
Step 2) 4 is even; divide by 2 and obtain 2.
Step 3) 2 is even; divide by 2 and obtain 1.
Step 4) 1 is odd; subtract 1 and obtain 0.
```

Example 3:

```
Input: num = 123
Output: 12
```

Constraints:

- $0 \leq \text{num} \leq 10^6$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int numberOfSteps(int num) {  
        int count=0;  
        while(num != 0) {  
            if((num&1)==1) num--;  
            else num >>= 1;  
            count++;  
        }  
        return count;  
    }  
}
```

[90 Subsets II \(link\)](#)

Description

Given an integer array `nums` that may contain duplicates, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new LinkedList<>(), result);
        return result;
    }

    void backtrack(int[] nums, int start, LinkedList<Integer> subset, List<List<Integer>> result) {
        result.add(new ArrayList<>(subset));
        for(int i=start; i < nums.length; i++) {
            if(i > start && nums[i] == nums[i-1]) continue;

            subset.add(nums[i]);
            backtrack(nums, i+1, subset, result);
            subset.removeLast();
        }
    }
}
```

78 Subsets (link)

Description

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- All the numbers of `nums` are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, 0, new LinkedList<>(), result);
        return result;
    }

    void backtrack(int[] nums, int start, LinkedList<Integer> subset, List<List<Integer>> result) {
        result.add(new ArrayList<>(subset));
        for(int i=start; i < nums.length; i++) {
            subset.add(nums[i]);
            backtrack(nums, i+1, subset, result);
            subset.removeLast();
        }
    }
}
```

59 Spiral Matrix II [\(link\)](#)

Description

Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.

Example 1:

1	→	2	→	3
8	→	9		↓
↑				↓
7	←	6	←	5

Input: $n = 3$

Output: `[[1,2,3],[8,9,4],[7,6,5]]`

Example 2:

Input: $n = 1$

Output: `[[1]]`

Constraints:

- $1 \leq n \leq 20$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] generateMatrix(int n) {
        int[][] mat = new int[n][n];
        int N = n*n+1, counter=1, top=0, left=0, bottom=n-1, right=n-1;
        while(counter < N) {
            for(int i=left; i<=right; i++)
                mat[top][i]=counter++;
            for(int i=top+1; i<=bottom; i++)
                mat[i][right]=counter++;

            if(top != bottom)
                for(int i=right-1; i>=left; i--)
                    mat[bottom][i]=counter++;

            if(left != right)
                for(int i=bottom-1; i>top; i--)
                    mat[i][left]=counter++;

            top++;
            left++;
            right--;
            bottom--;
        }
        return mat;
    }
}
```

21 Merge Two Sorted Lists (link)

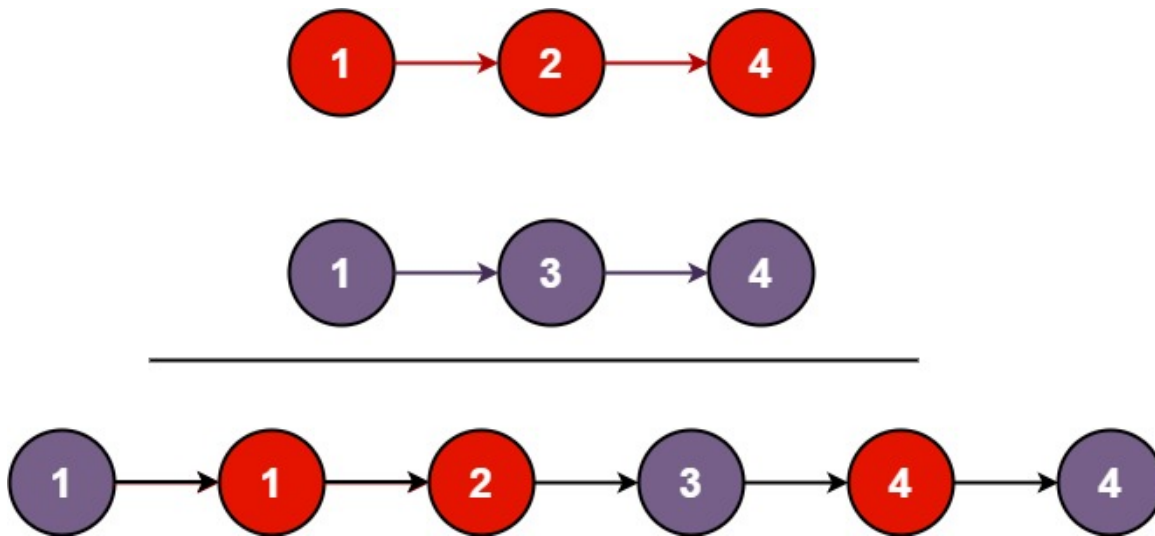
Description

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:



Input: `list1 = [1,2,4]`, `list2 = [1,3,4]`
Output: `[1,1,2,3,4,4]`

Example 2:

Input: `list1 = []`, `list2 = []`
Output: `[]`

Example 3:

Input: `list1 = []`, `list2 = [0]`
Output: `[0]`

Constraints:

- The number of nodes in both lists is in the range $[0, 50]$.
- $-100 \leq \text{Node.val} \leq 100$
- Both `list1` and `list2` are sorted in **non-decreasing** order.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode mergeTwoLists(ListNode n1, ListNode n2) {
        ListNode temp, dummy = new ListNode(-1), node = dummy;
        while (n1 != null && n2 != null) {
            if (n1.val < n2.val) {
                node.next = n1;
                n1 = n1.next;
            } else if (n1.val > n2.val) {
                node.next = n2;
                n2 = n2.next;
            } else {
                node.next = n1;
                node = node.next;
                n1 = n1.next;

                node.next = n2;
                n2 = n2.next;
            }

            node = node.next;
        }

        if (n1 != null) {
            node.next = n1;
        }

        if (n2 != null) {
            node.next = n2;
        }

        return dummy.next;
    }
}
```

[13 Roman to Integer \(link\)](#)

Description

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

```
Input: s = "III"
Output: 3
Explanation: III = 3.
```

Example 2:

```
Input: s = "LVIII"
Output: 58
Explanation: L = 50, V = 5, III = 3.
```

Example 3:

```
Input: s = "MCMXCIV"
Output: 1994
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.
```

Constraints:

- $1 \leq s.length \leq 15$
- s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is **guaranteed** that s is a valid roman numeral in the range [1, 3999].

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    static final Map<String, Integer> map = new HashMap() {{
        put("I", 1);
        put("IV", 4);
        put("V", 5);
        put("IX", 9);
        put("X", 10);
        put("XL", 40);
        put("L", 50);
        put("XC", 90);
        put("C", 100);
        put("CD", 400);
        put("D", 500);
        put("CM", 900);
        put("M", 1000);
    }};

    public int romanToInt(String s) {
        int result=0, curr = 0, prev = 0, length = s.length();
        int i = length-1;
        while(i > -1) {
            switch(s.charAt(i--)) {
                case 'I':
                    curr = 1; break;
                case 'V':
                    curr = 5; break;
                case 'X':
                    curr = 10; break;
                case 'L':
                    curr = 50; break;
                case 'C':
                    curr = 100; break;
                case 'D':
                    curr = 500; break;
                case 'M':
                    curr = 1000; break;
            }

            if(curr < prev)
                result-=curr;
            else
                result += curr;
            prev = curr;
        }
        return result;
    }
}
```

410 Split Array Largest Sum (link)

Description

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is **minimized**.

Return *the minimized largest sum of the split*.

A **subarray** is a contiguous part of the array.

Example 1:

Input: `nums = [7,2,5,10,8]`, `k = 2`

Output: 18

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is 18.

Example 2:

Input: `nums = [1,2,3,4,5]`, `k = 2`

Output: 9

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[1,2,3]` and `[4,5]`, where the largest sum among the two subarrays is 9.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 10^6$
- $1 \leq k \leq \min(50, \text{nums.length})$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int splitArray(int[] nums, int m) {
        int left=0, right = 0;
        for(int num:nums) {
            left = Math.max(left, num);
            right += num;
        }

        while(left < right) {
            int mid = left+(right-left)/2;
            if(feasible(nums, m, mid)) {
                right = mid;
            } else {
                left = mid+1;
            }
        }
        return left;
    }

    boolean feasible(int[] nums, int m, int largestSum) {
        int ways=1;
        int totalSum=0;
        for(int num: nums) {
            totalSum += num;
            if(totalSum > largestSum) {
                ways++;
                totalSum = num;
                if(ways > m)
                    return false;
            }
        }
        return true;
    }
}
```

366 Find Leaves of Binary Tree [\(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<List<Integer>> findLeaves(TreeNode root) {
        Map<Integer, List<Integer>> map = new HashMap<>();
        dfs(root, map, 0);
        return new ArrayList<>(map.values());
    }

    int dfs(TreeNode node, Map<Integer, List<Integer>> map, int level) {
        if(node == null)
            return level;

        int left = dfs(node.left, map, level);
        int right = dfs(node.right, map, level);
        level = Math.max(left, right);
        map.computeIfAbsent(level, ArrayList::new).add(node.val);
        return level+1;
    }
}
```

9 Palindrome Number [\(link\)](#)

Description

Given an integer x , return `true` if x is a **palindrome**, and `false` otherwise.

Example 1:

```
Input: x = 121
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.
```

Example 2:

```
Input: x = -121
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not
```



Example 3:

```
Input: x = 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.
```

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

Follow up: Could you solve it without converting the integer to a string?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isPalindrome(int x) {
        if(x < 0) {
            return false;
        } else if(x < 10) {
            return true;
        } else if(x%10==0 && x!=0) {
            return false;
        } else {
            int reverse = 0;
            while(x > reverse) {
                reverse = reverse*10+x%10;
                x/=10;
            }
            return x == reverse || x == reverse/10;
        }
    }
}
```

[4 Median of Two Sorted Arrays \(link\)](#)

Description

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return **the median** of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`
Output: `2.00000`
Explanation: merged array = `[1,2,3]` and median is `2`.

Example 2:

Input: `nums1 = [1,2]`, `nums2 = [3,4]`
Output: `2.50000`
Explanation: merged array = `[1,2,3,4]` and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$
- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-10^6 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^6$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        int m = nums1.length;
        int n = nums2.length;
        if(m > n)
            return findMedianSortedArrays(nums2, nums1);

        int total = m+n;
        int lo=0, hi = m;
        while(lo <= hi) {
            int cut1 = (lo+hi)/2;
            int cut2 = (total+1)/2 - cut1;

            int l1 = cut1 == 0 ? Integer.MIN_VALUE: nums1[cut1-1];
            int r1 = cut1 == m ? Integer.MAX_VALUE: nums1[cut1];

            int l2 = cut2 == 0 ? Integer.MIN_VALUE: nums2[cut2-1];
            int r2 = cut2 == n ? Integer.MAX_VALUE: nums2[cut2];

            if(l1 > r2) {
                hi=cut1-1;
            } else if(l2 > r1) {
                lo=cut1+1;
            } else {
                return (total&1)==0 ? (double) (Math.min(r1,r2)+Math.max(l1,l2))/2 : (double)Math.max(l1,r2);
            }
        }
        return -1;
    }
}
```

1274 Number of Days Between Two Dates (link)

Description

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is YYYY-MM-DD as shown in the examples.

Example 1:

```
Input: date1 = "2019-06-29", date2 = "2019-06-30"  
Output: 1
```

Example 2:

```
Input: date1 = "2020-01-15", date2 = "2019-12-31"  
Output: 15
```

Constraints:

- The given dates are valid dates between the years 1971 and 2100.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int[] months = {31,28,31,30,31,30,31,31,30,31,30,31};
    public int daysBetweenDates(String date1, String date2) {
        return Math.abs(days(date1) - days(date2));
    }

    boolean isLeapYear(int year) {
        return (year%400 == 0) || (year%100 != 0 && year%4==0);
    }

    int days(String date) {
        String[] split = date.split("-");
        int year = Integer.parseInt(split[0]);
        int month = Integer.parseInt(split[1]);
        int days = Integer.parseInt(split[2]);

        for(int y=1971; y < year; y++) {
            if(isLeapYear(y)) days+=1;
            days += 365;
        }

        boolean isLeapYear = isLeapYear(year);
        for(int m=0; m < month-1; m++) {
            if(m == 1 && isLeapYear) days+=1;
            days += months[m];
        }
        return days;
    }
}
```

168 Excel Sheet Column Title (link)

Description

Given an integer `columnNumber`, return *its corresponding column title as it appears in an Excel sheet*.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnNumber = 1
Output: "A"
```

Example 2:

```
Input: columnNumber = 28
Output: "AB"
```

Example 3:

```
Input: columnNumber = 701
Output: "ZY"
```

Constraints:

- $1 \leq \text{columnNumber} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String convertToTitle(int columnNumber) {
        StringBuilder sb = new StringBuilder();
        int n = columnNumber;
        while( n > 0 ) {
            n--;
            sb.insert(0, (char) ('A'+n%26));
            n/=26;
        }
        return sb.toString();
    }
}
```

1713 Dot Product of Two Sparse Vectors (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class SparseVector {
    public int[] nums;
    SparseVector(int[] nums) {
        this.nums = nums;
    }

    // Return the dotProduct of two sparse vectors
    public int dotProduct(SparseVector vec) {
        int result = 0;
        for(int i=0; i < this.nums.length; i++) {
            result += this.nums[i]*vec.nums[i];
        }
        return result;
    }
}

// Your SparseVector object will be instantiated and called as such:
// SparseVector v1 = new SparseVector(nums1);
// SparseVector v2 = new SparseVector(nums2);
// int ans = v1.dotProduct(v2);
```

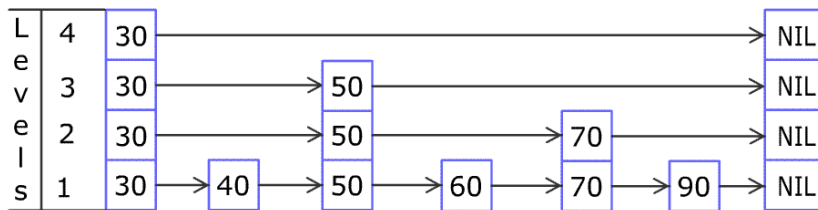
[1337 Design Skiplist \(link\)](#)

Description

Design a **Skiplist** without using any built-in libraries.

A **skiplist** is a data structure that takes $O(\log(n))$ time to add, erase and search. Comparing with treap and red-black tree which has the same function and performance, the code length of Skiplist can be comparatively short and the idea behind Skiplists is just simple linked lists.

For example, we have a Skiplist containing $[30, 40, 50, 60, 70, 90]$ and we want to add 80 and 45 into it. The Skiplist works this way:



Artyom Kalinin [CC BY-SA 3.0], via [Wikimedia Commons](#)

You can see there are many layers in the Skiplist. Each layer is a sorted linked list. With the help of the top layers, add, erase and search can be faster than $O(n)$. It can be proven that the average time complexity for each operation is $O(\log(n))$ and space complexity is $O(n)$.

See more about Skiplist: https://en.wikipedia.org/wiki/Skip_list

Implement the `Skiplist` class:

- `Skiplist()` Initializes the object of the skiplist.
- `bool search(int target)` Returns `true` if the integer `target` exists in the Skiplist or `false` otherwise.
- `void add(int num)` Inserts the value `num` into the Skiplist.
- `bool erase(int num)` Removes the value `num` from the Skiplist and returns `true`. If `num` does not exist in the Skiplist, do nothing and return `false`. If there exist multiple `num` values, removing any one of them is fine.

Note that duplicates may exist in the Skiplist, your code needs to handle this situation.

Example 1:

```
Input
["Skiplist", "add", "add", "add", "search", "add", "search", "erase", "erase", "search"]
[[], [1], [2], [3], [0], [4], [1], [0], [1], [1]]
Output
[null, null, null, null, false, null, true, false, true, false]
```

```
Explanation
Skiplist skiplist = new Skiplist();
skiplist.add(1);
skiplist.add(2);
skiplist.add(3);
skiplist.search(0); // return False
skiplist.add(4);
skiplist.search(1); // return True
skiplist.erase(0); // return False, 0 is not in skiplist.
```

```
skiplist.erase(1); // return True
skiplist.search(1); // return False, 1 has already been erased.
```

Constraints:

- $0 \leq \text{num}, \text{target} \leq 2 * 10^4$
- At most $5 * 10^4$ calls will be made to search, add, and erase.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Skiplist {
    Node head;
    Random rand = new Random();

    public Skiplist() {
        this.head = new Node(-1);
    }

    public boolean search(int target) {
        Node current = head;
        while(current != null) {
            while(current.next != null && current.next.val < target) {
                current = current.next;
            }
            if(current.next != null && current.next.val == target)
                return true;
            current = current.down;
        }
        return false;
    }

    public void add(int num) {
        Stack<Node> stack = new Stack<>();
        Node cur = head;
        while (cur != null) {
            while (cur.next != null && cur.next.val < num) {
                cur = cur.next;
            }
            stack.push(cur);
            cur = cur.down;
        }
        boolean insert = true;
        Node down = null;
        while (insert && !stack.isEmpty()) {
            cur = stack.pop();
            cur.next = new Node(num, cur.next, down);
            down = cur.next;
            insert = rand.nextDouble() < 0.5;
        }
        if (insert) head = new Node(-1, null, head);
    }

    public boolean erase(int num) {
        Node current = head;
        boolean found = false;
        while(current != null) {
            while(current.next != null && current.next.val < num) {
                current = current.next;
            }
            if(current.next != null && current.next.val == num) {
                found = true;
                current.next = current.next.next;
            }
            current = current.down;
        }
        return found;
    }
}

class Node {
    public int val;
    public Node next;
    public Node down;

    public Node(int val) {
```

```
        this(val, null, null);
    }

    public Node(int val, Node next, Node down) {
        this.val = val;
        this.next = next;
        this.down = down;
    }
}

/**
 * Your Skiplist object will be instantiated and called as such:
 * Skiplist obj = new Skiplist();
 * boolean param_1 = obj.search(target);
 * obj.add(num);
 * boolean param_3 = obj.erase(num);
 */
```

816 Design HashSet (link)

Description

Design a HashSet without using any built-in hash table libraries.

Implement MyHashSet class:

- void add(key) Inserts the value key into the HashSet.
- bool contains(key) Returns whether the value key exists in the HashSet or not.
- void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.

Example 1:

Input

```
["MyHashSet", "add", "add", "contains", "contains", "add", "contains", "remove", "contains"]  
[[], [1], [2], [1], [3], [2], [2], [2], [2]]
```

Output

```
[null, null, null, true, false, null, true, null, false]
```

Explanation

```
MyHashSet myHashSet = new MyHashSet();  
myHashSet.add(1);      // set = [1]  
myHashSet.add(2);      // set = [1, 2]  
myHashSet.contains(1); // return True  
myHashSet.contains(3); // return False, (not found)  
myHashSet.add(2);      // set = [1, 2]  
myHashSet.contains(2); // return True  
myHashSet.remove(2);   // set = [1]  
myHashSet.contains(2); // return False, (already removed)
```

Constraints:

- $0 \leq \text{key} \leq 10^6$
- At most 10^4 calls will be made to add, remove, and contains.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MyHashSet {

    boolean[] hashSet;

    public MyHashSet() {
        hashSet = new boolean[1000001];
    }

    public void add(int key) {
        hashSet[key] = true;
    }

    public void remove(int key) {
        hashSet[key] = false;
    }

    public boolean contains(int key) {
        return hashSet[key];
    }

    /*
    boolean[] hashSet;

    public MyHashSet() {
        hashSet = new boolean[1000001];
    }

    public void add(int key) {
        hashSet[key] = true;
    }

    public void remove(int key) {
        hashSet[key] = false;
    }

    public boolean contains(int key) {
        return hashSet[key];
    }
    */
}

/**
 * Your MyHashSet object will be instantiated and called as such:
 * MyHashSet obj = new MyHashSet();
 * obj.add(key);
 * obj.remove(key);
 * boolean param_3 = obj.contains(key);
 */
```

408 Valid Word Abbreviation (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean validWordAbbreviation(String word, String abbr) {
        int wordLength = word.length();
        int abbrLength = abbr.length();
        if(abbrLength > wordLength) return false;

        int i=0, numChars=0;
        for(int j=0; j < abbrLength; j++) {
            char ch = abbr.charAt(j);
            if(ch >= 48 && ch <=57) { //is ch numeric
                if(numChars == 0 && ch == 48)
                    return false;
                numChars = 10*numChars+(ch-'0');
            } else {
                i += numChars;
                if(i >= wordLength) return false;
                numChars = 0;

                if(ch != word.charAt(i))
                    return false;
                i++;
            }
        }
        return i+numChars == wordLength;
    }
}
```

1714 Rearrange Spaces Between Words (link)

Description

You are given a string `text` of words that are placed among some number of spaces. Each word consists of one or more lowercase English letters and are separated by at least one space. It's guaranteed that `text` **contains at least one word**.

Rearrange the spaces so that there is an **equal** number of spaces between every pair of adjacent words and that number is **maximized**. If you cannot redistribute all the spaces equally, place the **extra spaces at the end**, meaning the returned string should be the same length as `text`.

Return *the string after rearranging the spaces*.

Example 1:

Input: `text = " this is a sentence "`

Output: `"this is a sentence"`

Explanation: There are a total of 9 spaces and 4 words. We can evenly divide the 9 spaces between the words.

Example 2:

Input: `text = " practice makes perfect"`

Output: `"practice makes perfect "`

Explanation: There are a total of 7 spaces and 3 words. $7 / (3-1) = 3$ spaces plus 1 extra space. We place the extra space at the end.

Constraints:

- $1 \leq \text{text.length} \leq 100$
- `text` consists of lowercase English letters and ' '.
- `text` contains at least one word.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reorderSpaces(String text) {
        int wordCount=0, spaceCount=0, textLength = text.length();
        for(int i=0; i < textLength; i) {
            if(text.charAt(i) == ' ')
                for(++spaceCount, ++i; i < textLength && text.charAt(i) == ' '; ++i,++spaceCount);
            else
                for(++wordCount, ++i; i < textLength && text.charAt(i) != ' '; ++i);
        }

        int spaceForWords = wordCount == 1? 0: spaceCount/(wordCount-1);
        int spaceRemaining = wordCount == 1? spaceCount: spaceCount%(wordCount-1);
        char[] chars = new char[textLength];

        int i=0, k=0;
        while(wordCount-- > 0) {
            int spaces = 0;
            for(;i < textLength && text.charAt(i) == ' '; ++i);
            for(;i < textLength && text.charAt(i) != ' '; chars[k++]=text.charAt(i++));
            for(;k < textLength && spaces++ != spaceForWords; chars[k++]=' ');
        }

        for(;k < textLength && spaceRemaining-->0; chars[k++]=' ');
        return String.valueOf(chars);
    }
}
```

[367 Valid Perfect Square \(link\)](#)

Description

Given a positive integer num, return `true` *if num is a perfect square* or `false` *otherwise*.

A **perfect square** is an integer that is the square of an integer. In other words, it is the product of some integer with itself.

You must not use any built-in library function, such as `sqrt`.

Example 1:

```
Input: num = 16
Output: true
Explanation: We return true because 4 * 4 = 16 and 4 is an integer.
```

Example 2:

```
Input: num = 14
Output: false
Explanation: We return false because 3.742 * 3.742 = 14 and 3.742 is not an integer.
```

Constraints:

- $1 \leq \text{num} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isPerfectSquare(int num) {
        long left = 1, right = num;
        while (left <= right) {
            long mid = left + (right - left) / 2, t = mid * mid;
            if (t == num) return true;
            else if (t < num) left = mid + 1;
            else right = mid - 1;
        }
        return false;
    }
}
```

[551 Student Attendance Record I \(link\)](#)

Description

You are given a string s representing an attendance record for a student where each character signifies whether the student was absent, late, or present on that day. The record only contains the following three characters:

- 'A': Absent.
- 'L': Late.
- 'P': Present.

The student is eligible for an attendance award if they meet **both** of the following criteria:

- The student was absent ('A') for **strictly** fewer than 2 days **total**.
- The student was **never** late ('L') for 3 or more **consecutive** days.

Return `true` *if the student is eligible for an attendance award*, or `false` *otherwise*.

Example 1:

Input: `s = "PPALLP"`

Output: `true`

Explanation: The student has fewer than 2 absences and was never late 3 or more consecutive days.

Example 2:

Input: `s = "PPALLL"`

Output: `false`

Explanation: The student was late 3 consecutive days in the last 3 days, so is not eligible for the award.

Constraints:

- $1 \leq s.length \leq 1000$
- $s[i]$ is either 'A', 'L', or 'P'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean checkRecord(String s) {
        int absentCount=0;
        int lateCount=0, prevRecord='0';
        for(char record: s.toCharArray()) {
            if(record == 'A') {
                absentCount++;
                if(absentCount > 1) return false;
            } else if(record == 'L'){
                if(prevRecord == record) {
                    lateCount++;
                    if(lateCount == 3) return false;
                } else {
                    lateCount = 1;
                }
            }
            prevRecord = record;
        }
        return true;
    }
}
```

[405 Convert a Number to Hexadecimal \(link\)](#)

Description

Given a 32-bit integer `num`, return *a string representing its hexadecimal representation*. For negative integers, [two's complement](#) method is used.

All the letters in the answer string should be lowercase characters, and there should not be any leading zeros in the answer except for the zero itself.

Note: You are not allowed to use any built-in library method to directly solve this problem.

Example 1:

```
Input: num = 26
Output: "1a"
```

Example 2:

```
Input: num = -1
Output: "ffffffff"
```

Constraints:

- $-2^{31} \leq \text{num} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String toHex(int num) {
        if(num == 0) return "0";
        char[] hex = {'0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f'};
        StringBuilder sb = new StringBuilder();
        while(num != 0 ) {
            sb.insert(0,hex[num&15]);
            num >>=4;
        }
        return sb.toString();
    }
}
```

[1400 Find Winner on a Tic Tac Toe Game \(link\)](#)

Description

Tic-tac-toe is played by two players A and B on a 3 x 3 grid. The rules of Tic-Tac-Toe are:

- Players take turns placing characters into empty squares ' '.
- The first player A always places 'x' characters, while the second player B always places 'o' characters.
- 'x' and 'o' characters are always placed into empty squares, never on filled ones.
- The game ends when there are **three** of the same (non-empty) character filling any row, column, or diagonal.
- The game also ends if all squares are non-empty.
- No more moves can be played if the game is over.

Given a 2D integer array `moves` where `moves[i] = [rowi, coli]` indicates that the i^{th} move will be played on `grid[rowi][coli]`. return *the winner of the game if it exists* (A or B). In case the game ends in a draw return "Draw". If there are still movements to play return "Pending".

You can assume that `moves` is valid (i.e., it follows the rules of **Tic-Tac-Toe**), the grid is initially empty, and A will play first.

Example 1:

X		
	X	
O	O	X

Input: `moves = [[0,0],[2,0],[1,1],[2,1],[2,2]]`

Output: "A"

Explanation: A wins, they always play first.

Example 2:

X	X	O
X	O	
O		

Input: moves = `[[0,0],[1,1],[0,1],[0,2],[1,0],[2,0]]`

Output: "B"

Explanation: B wins.

Example 3:

X	X	O
O	O	X
X	O	X

Input: moves = `[[0,0],[1,1],[2,0],[1,0],[1,2],[2,1],[0,1],[0,2],[2,2]]`

Output: "Draw"

Explanation: The game ends in a draw since there are no moves to make.

Constraints:

- `1 <= moves.length <= 9`
- `moves[i].length == 2`
- `0 <= rowi, coli <= 2`
- There are no repeated elements on moves.
- moves follow the rules of tic tac toe.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String tictactoe(int[][] moves) {
        int length = moves.length;
        int n = 3;
        int[][] movesWithId = new int[n][n];
        for(int i=0; i<length;i+=2) {
            movesWithId[moves[i][0]][moves[i][1]]=1;
            if(i+1 < length)
                movesWithId[moves[i+1][0]][moves[i+1][1]]=-1;
        }

        int pendingCounter=0;
        int movesWithIdLength = movesWithId.length;
        for(int i=0; i < movesWithIdLength; i++) {
            int rowCounter=0, colCounter=0;
            for(int j=0; j < movesWithId[0].length; j++) {
                rowCounter += movesWithId[i][j];
                colCounter += movesWithId[j][i];
                if(movesWithId[i][j] == 0 || movesWithId[j][i] == 0)
                    pendingCounter++;
            }
            if(Math.abs(rowCounter) == n)
                return rowCounter == n ? "A":"B";
            if(Math.abs(colCounter) == n)
                return colCounter == n ? "A":"B";
        }

        int diag1Counter=0,diag2Counter=0;
        for(int i=0,j=0,k=0,l=movesWithIdLength-1; i < movesWithId.length; i++,j++,k++,l--) {
            diag1Counter += movesWithId[i][j];
            diag2Counter += movesWithId[k][l];
        }

        if(Math.abs(diag1Counter) == n)
            return diag1Counter == n ? "A":"B";
        if(Math.abs(diag2Counter) == n)
            return diag2Counter == n ? "A":"B";

        return pendingCounter != 0 ? "Pending": "Draw";
    }
}
```

2121 Find if Path Exists in Graph (link)

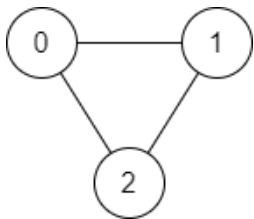
Description

There is a **bi-directional** graph with n vertices, where each vertex is labeled from 0 to $n - 1$ (**inclusive**). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [ui, vi]` denotes a bi-directional edge between vertex u_i and vertex v_i . Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

Given `edges` and the integers `n`, `source`, and `destination`, return `true` *if there is a valid path from source to destination*, *or false otherwise*.

Example 1:



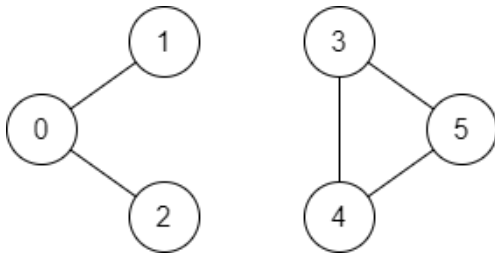
Input: `n = 3`, `edges = [[0,1],[1,2],[2,0]]`, `source = 0`, `destination = 2`

Output: `true`

Explanation: There are two paths from vertex 0 to vertex 2:

- `0 → 1 → 2`
- `0 → 2`

Example 2:



Input: `n = 6`, `edges = [[0,1],[0,2],[3,5],[5,4],[4,3]]`, `source = 0`, `destination = 5`

Output: `false`

Explanation: There is no path from vertex 0 to vertex 5.

Constraints:

- $1 \leq n \leq 2 * 10^5$
- $0 \leq \text{edges.length} \leq 2 * 10^5$
- `edges[i].length == 2`
- $0 \leq u_i, v_i \leq n - 1$
- $u_i \neq v_i$
- $0 \leq \text{source}, \text{destination} \leq n - 1$
- There are no duplicate edges.
- There are no self edges.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean validPath(int n, int[][] edges, int source, int destination) {
        if(source == destination)
            return true;
        boolean[] visited = new boolean[n];
        HashSet<Integer>[] graph = new HashSet[n];
        for(int i=0; i < n; i++)
            graph[i]=new HashSet<>();

        for(int[] edge: edges) {
            graph[edge[0]].add(edge[1]);
            graph[edge[1]].add(edge[0]);
        }

        if(graph[source].contains(destination))
            return true;

        Stack<Integer> stack = new Stack<>();
        stack.push(source);
        visited[source] = true;

        while(!stack.isEmpty()) {
            int from = stack.pop();
            if(from == destination)
                return true;
            for(int neighbor: graph[from]) {
                if(!visited[neighbor]) {
                    visited[neighbor] = true;
                    stack.push(neighbor);
                }
            }
        }

        return false;
    }
}
```

1476 Count Negative Numbers in a Sorted Matrix (link)

Description

Given a $m \times n$ matrix `grid` which is sorted in non-increasing order both row-wise and column-wise, return the number of **negative numbers** in `grid`.

Example 1:

```
Input: grid = [[4,3,2,-1],[3,2,1,-1],[1,1,-1,-2],[-1,-1,-2,-3]]
Output: 8
Explanation: There are 8 negatives number in the matrix.
```

Example 2:

```
Input: grid = [[3,2],[1,0]]
Output: 0
```

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-100 \leq \text{grid}[i][j] \leq 100$

Follow up: Could you find an $O(n + m)$ solution?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int countNegatives(int[][] grid) {
        int cols=grid[0].length, result=0;
        for(int[] row: grid) {
            int lo=0, hi = cols-1;
            if(row[lo] < 0){
                result += cols;
                continue;
            }

            int mid=0;
            while(lo < hi) {
                mid = (lo+hi)/2;
                if(row[mid] < 0)
                    hi = mid;
                else
                    lo = mid+1;
            }

            result += row[lo] < 0 ? cols-lo:0;
        }
        return result;
    }
}
```

1635 Number of Good Pairs (link)

Description

Given an array of integers `nums`, return *the number of good pairs*.

A pair (i, j) is called *good* if `nums[i] == nums[j]` and $i < j$.

Example 1:

Input: `nums = [1,2,3,1,1,3]`

Output: 4

Explanation: There are 4 good pairs $(0,3)$, $(0,4)$, $(3,4)$, $(2,5)$ 0-indexed.

Example 2:

Input: `nums = [1,1,1,1]`

Output: 6

Explanation: Each pair in the array are *good*.

Example 3:

Input: `nums = [1,2,3]`

Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $1 \leq \text{nums}[i] \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int numIdenticalPairs(int[] nums) {  
        int result=0;  
        int[] counts = new int[101];  
        for(int num:nums)  
            counts[num]++;  
  
        for(int count:counts)  
            if(count > 1)  
                result+=count*(count-1)/2;  
        return result;  
    }  
}
```

175 Combine Two Tables (link)

Description

Table: Person

Column Name	Type
personId	int
lastName	varchar
firstName	varchar

personId is the primary key (column with unique values) for this table.
This table contains information about the ID of some persons and their first and last names.

Table: Address

Column Name	Type
addressId	int
personId	int
city	varchar
state	varchar

addressId is the primary key (column with unique values) for this table.
Each row of this table contains information about the city and state of one person with ID = PersonId.

Write a solution to report the first name, last name, city, and state of each person in the `Person` table. If the address of a `personId` is not present in the `Address` table, report `null` instead.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Person table:

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

Address table:

addressId	personId	city	state
1	2	New York City	New York
2	3	Leetcode	California

Output:

firstName	lastName	city	state
Allen	Wang	Null	Null

	Bob		Alice		New York City		New York	
+	-	-	-	+	-	-	-	+

Explanation:

There is no address in the address table for the `personId = 1` so we return null in their city and state. `addressId = 1` contains information about the address of `personId = 2`.

(scroll down for solution)



Solution

Language: mysql

Status: Accepted

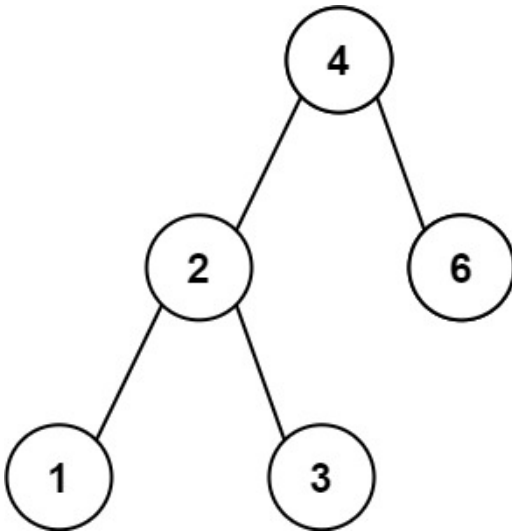
```
# Write your MySQL query statement below
select p.firstName, p.lastName, a.city, a.state
from Person p left outer join address a  using (personId);
```


530 Minimum Absolute Difference in BST (link)

Description

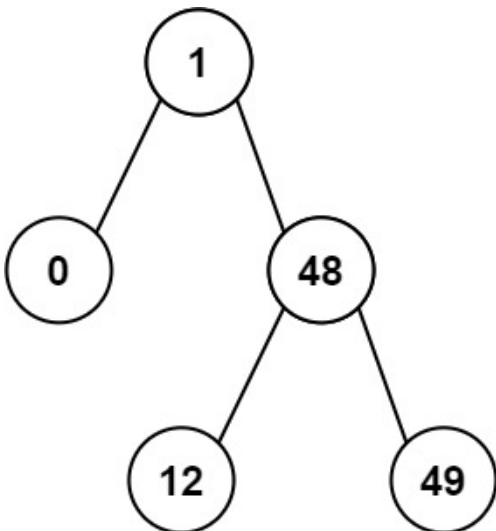
Given the root of a Binary Search Tree (BST), return *the minimum absolute difference between the values of any two different nodes in the tree.*

Example 1:



Input: root = [4,2,6,1,3]
Output: 1

Example 2:



Input: root = [1,0,48,null,null,12,49]
Output: 1

Constraints:

- The number of nodes in the tree is in the range $[2, 10^4]$.
- $0 \leq \text{Node.val} \leq 10^5$

Note: This question is the same as 783: <https://leetcode.com/problems/minimum-distance-between-bst-nodes/>

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int getMinimumDifference(TreeNode root) {
        int[] result={-1, Integer.MAX_VALUE};
        dfs(root, result);
        return result[1];
    }

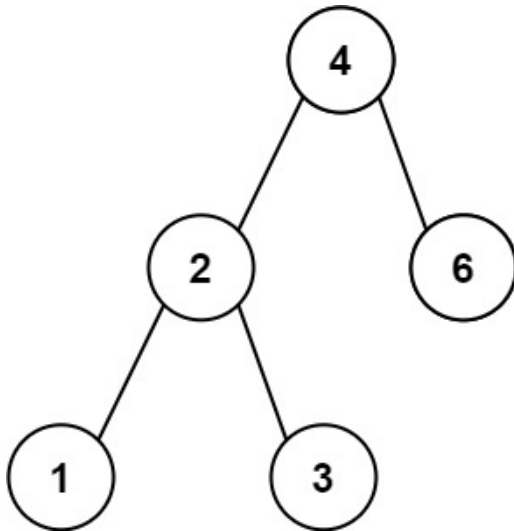
    void dfs(TreeNode root, int[] result) {
        if(root == null)
            return;
        dfs(root.left, result);
        if(result[0] != -1) {
            result[1] = Math.min(result[1], root.val-result[0]);
        }
        result[0]=root.val;
        dfs(root.right, result);
    }
}
```

799 Minimum Distance Between BST Nodes (link)

Description

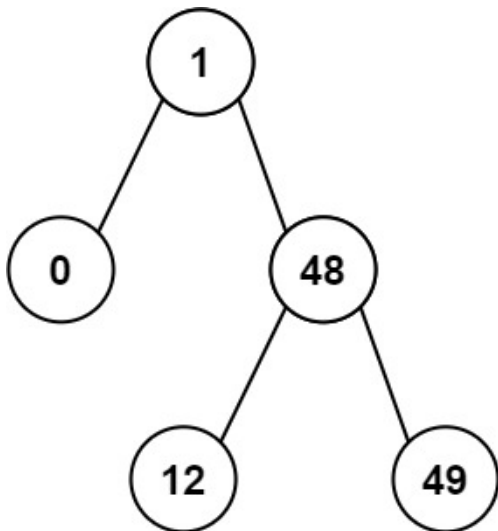
Given the root of a Binary Search Tree (BST), return *the minimum difference between the values of any two different nodes in the tree*.

Example 1:



Input: root = [4,2,6,1,3]
Output: 1

Example 2:



Input: root = [1,0,48,null,null,12,49]
Output: 1

Constraints:

- The number of nodes in the tree is in the range $[2, 100]$.
- $0 \leq \text{Node.val} \leq 10^5$

Note: This question is the same as 530: <https://leetcode.com/problems/minimum-absolute-difference-in-bst/>

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int minDiff = Integer.MAX_VALUE;
    public int minDiffInBST(TreeNode root) {
        dfs(root, null, null);
        return minDiff;
    }

    void dfs(TreeNode root, TreeNode left, TreeNode right) {
        if(root == null)
            return;
        if(left != null)
            minDiff = Math.min(minDiff, root.val-left.val);
        if(right != null)
            minDiff = Math.min(minDiff, right.val - root.val);
        dfs(root.left, left, root);
        dfs(root.right, root, right);
    }
}
```

[1646 Kth Missing Positive Number \(link\)](#)

Description

Given an array `arr` of positive integers sorted in a **strictly increasing order**, and an integer `k`.

Return *the k^{th} **positive** integer that is **missing** from this array.*

Example 1:

Input: `arr = [2,3,4,7,11]`, `k = 5`

Output: 9

Explanation: The missing positive integers are `[1,5,6,8,9,10,12,13,...]`. The 5th missing positive integer is 9.

Example 2:

Input: `arr = [1,2,3,4]`, `k = 2`

Output: 6

Explanation: The missing positive integers are `[5,6,7,...]`. The 2nd missing positive integer is 6.

Constraints:

- `1 <= arr.length <= 1000`
- `1 <= arr[i] <= 1000`
- `1 <= k <= 1000`
- `arr[i] < arr[j]` for `1 <= i < j <= arr.length`

Follow up:

Could you solve this problem in less than $O(n)$ complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findKthPositive(int[] arr, int k) {
        int counter=0, missing=0, i=0;

        while(i < arr.length) {
            if(++missing < arr[i]) {
                counter++;
            } else if(missing == arr[i]) {
                i++;
            }

            if(counter == k)
                break;
        }

        return counter == k ? missing: missing+k-counter;
    }
}
```


747 Min Cost Climbing Stairs (link)

Description

You are given an integer array `cost` where `cost[i]` is the cost of i^{th} step on a staircase. Once you pay the cost, you can either climb one or two steps.

You can either start from the step with index 0, or the step with index 1.

Return *the minimum cost to reach the top of the floor*.

Example 1:

```
Input: cost = [10,15,20]
Output: 15
Explanation: You will start at index 1.
- Pay 15 and climb two steps to reach the top.
The total cost is 15.
```

Example 2:

```
Input: cost = [1,100,1,1,1,100,1,1,100,1]
Output: 6
Explanation: You will start at index 0.
- Pay 1 and climb two steps to reach index 2.
- Pay 1 and climb two steps to reach index 4.
- Pay 1 and climb two steps to reach index 6.
- Pay 1 and climb one step to reach index 7.
- Pay 1 and climb two steps to reach index 9.
- Pay 1 and climb one step to reach the top.
The total cost is 6.
```

Constraints:

- $2 \leq \text{cost.length} \leq 1000$
- $0 \leq \text{cost}[i] \leq 999$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minCostClimbingStairs(int[] cost) {
        int firstCost = 0;
        int secondCost = 0;
        for(int i=2; i <= cost.length; i++) {
            int current = Math.min(firstCost+cost[i-1], secondCost+cost[i-2]);
            secondCost = firstCost;
            firstCost = current;
        }
        return firstCost;
    }
}
```

[171 Excel Sheet Column Number \(link\)](#)

Description

Given a string `columnTitle` that represents the column title as appears in an Excel sheet, return *its corresponding column number*.

For example:

```
A -> 1
B -> 2
C -> 3
...
Z -> 26
AA -> 27
AB -> 28
...
```

Example 1:

```
Input: columnTitle = "A"
Output: 1
```

Example 2:

```
Input: columnTitle = "AB"
Output: 28
```

Example 3:

```
Input: columnTitle = "ZY"
Output: 701
```

Constraints:

- $1 \leq \text{columnTitle.length} \leq 7$
- `columnTitle` consists only of uppercase English letters.
- `columnTitle` is in the range ["A", "FXSHRXW"].

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int titleToNumber(String columnTitle) {
        int result=0;
        for(char ch: columnTitle.toCharArray()) {
            result = 26*result+(ch-'A'+1);
        }
        return result;
    }
}
```

[545 Boundary of Binary Tree \(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/* Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> boundaryOfBinaryTree(TreeNode root) {
        List<Integer> boundaries = new ArrayList<>();
        if(root == null)
            return boundaries;

        if(!isLeaf(root)) {
            boundaries.add(root.val);
        }

        addLeft(root, boundaries);
        addLeaves(root, boundaries);
        addRight(root, boundaries);

        return boundaries;
    }

    private void addLeft(TreeNode root, List<Integer> list) {
        TreeNode node = root.left;
        while(node != null) {
            if(!isLeaf(node))
                list.add(node.val);
            if(node.left != null)
                node = node.left;
            else
                node = node.right;
        }
    }

    private void addLeaves(TreeNode root, List<Integer> list) {
        if(isLeaf(root)) {
            list.add(root.val);
        } else {
            if(root.left != null)
                addLeaves(root.left, list);
            if(root.right != null)
                addLeaves(root.right, list);
        }
    }

    private void addRight(TreeNode root, List<Integer> list) {
        TreeNode node = root.right;
        Stack<Integer> stack = new Stack<>();
        while(node != null) {
            if(!isLeaf(node))
                stack.push(node.val);
            if(node.right != null)
                node = node.right;
            else
                node = node.left;
        }
    }
}
```

```
    }  
    while(!stack.isEmpty())  
        list.add(stack.pop());  
}  
boolean isLeaf(TreeNode root) {  
    return root.left == null && root.right == null;  
}  
}
```

1256 Rank Transform of an Array (link)

Description

Given an array of integers `arr`, replace each element with its rank.

The rank represents how large the element is. The rank has the following rules:

- Rank is an integer starting from 1.
- The larger the element, the larger the rank. If two elements are equal, their rank must be the same.
- Rank should be as small as possible.

Example 1:

Input: `arr = [40,10,20,30]`

Output: `[4,1,2,3]`

Explanation: 40 is the largest element. 10 is the smallest. 20 is the second smallest. 30 is the third s

Example 2:

Input: `arr = [100,100,100]`

Output: `[1,1,1]`

Explanation: Same elements share the same rank.

Example 3:

Input: `arr = [37,12,28,9,100,56,80,5,12]`

Output: `[5,3,4,2,8,6,7,1,3]`

Constraints:

- $0 \leq \text{arr.length} \leq 10^5$
- $-10^9 \leq \text{arr}[i] \leq 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] arrayRankTransform(int[] arr) {
        if(arr == null || arr.length == 0) return new int[0];
        int[] result = new int[arr.length];
        System.arraycopy(arr, 0, result, 0, arr.length);
        Arrays.sort(result);

        int k = 1;
        Map<Integer, Integer> map = new HashMap<>();
        map.put(result[0], k++);

        for(int i=1; i < arr.length; i++)
            if(result[i-1] != result[i])
                map.put(result[i], k++);

        for(int i=0; i < arr.length; i++)
            result[i] = map.get(arr[i]);

        return result;
    }
}
```

733 Flood Fill ([link](#))

Description

You are given an image represented by an $m \times n$ grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a **flood fill** on the image starting from the pixel `image[sr][sc]`.

To perform a **flood fill**:

1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is **directly adjacent** (pixels that share a side with the original pixel, either horizontally or vertically) and shares the **same color** as the starting pixel.
3. Keep **repeating** this process by checking neighboring pixels of the *updated* pixels and modifying their color if it matches the original color of the starting pixel.
4. The process **stops** when there are **no more** adjacent pixels of the original color to update.

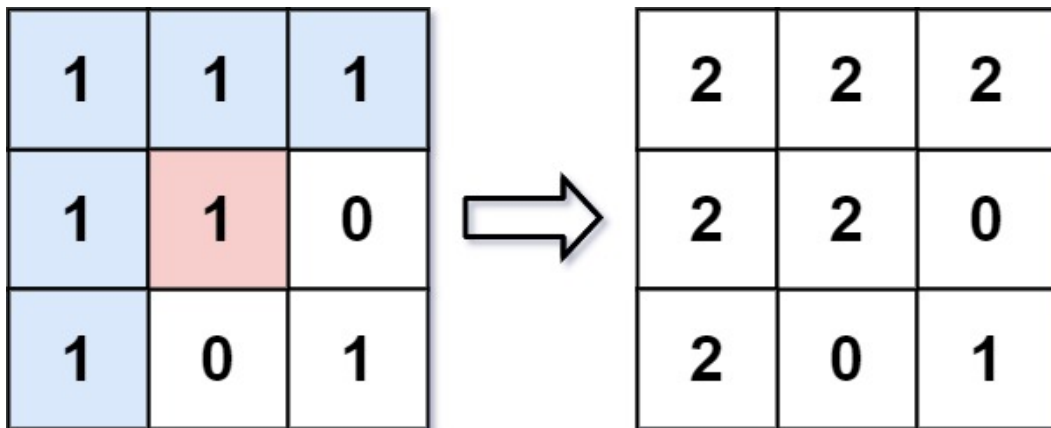
Return the **modified** image after performing the flood fill.

Example 1:

Input: `image = [[1,1,1],[1,1,0],[1,0,1]]`, `sr = 1`, `sc = 1`, `color = 2`

Output: `[[2,2,2],[2,2,0],[2,0,1]]`

Explanation:



From the center of the image with position $(sr, sc) = (1, 1)$ (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is **not** colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: `image = [[0,0,0],[0,0,0]]`, `sr = 0`, `sc = 0`, `color = 0`

Output: `[[0,0,0],[0,0,0]]`

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- `m == image.length`
- `n == image[i].length`
- `1 <= m, n <= 50`
- `0 <= image[i][j], color < 216`
- `0 <= sr < m`
- `0 <= sc < n`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
        int color = image[sr][sc];
        int m = image.length;
        int n = image[0].length;
        if(color != newColor) {
            Queue<Integer> queue = new ArrayDeque<>();
            queue.add(sr*n+sc);

            while(!queue.isEmpty()) {
                int coords = queue.poll();
                int r = coords/n;
                int c = coords%n;
                image[r][c] = newColor;
                if(r>0 && image[r-1][c] == color)
                    queue.add((r-1)*n+c);
                if(c>0 && image[r][c-1] == color)
                    queue.add(r*n+(c-1));
                if(r+1 < m && image[r+1][c] == color)
                    queue.add((r+1)*n+c);
                if(c+1 < n && image[r][c+1] == color)
                    queue.add(r*n+c+1);
            }
        }
        return image;
    }
}
```

1983 Maximum Population Year (link)

Description

You are given a 2D integer array `logs` where each `logs[i] = [birthi, deathi]` indicates the birth and death years of the i^{th} person.

The **population** of some year x is the number of people alive during that year. The i^{th} person is counted in year x 's population if x is in the **inclusive** range `[birthi, deathi - 1]`. Note that the person is **not** counted in the year that they die.

Return *the earliest year with the maximum population*.

Example 1:

Input: `logs = [[1993,1999],[2000,2010]]`

Output: 1993

Explanation: The maximum population is 1, and 1993 is the earliest year with this population.

Example 2:

Input: `logs = [[1950,1961],[1960,1971],[1970,1981]]`

Output: 1960

Explanation:

The maximum population is 2, and it had happened in years 1960 and 1970.

The earlier year between them is 1960.

Constraints:

- `1 <= logs.length <= 100`
- `1950 <= birthi < deathi <= 2050`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maximumPopulation(int[][] logs) {
        int BEGIN = 1950;
        int[] populationCounts = new int[101];
        for(int[] log: logs) {
            for(int year=log[0]; year < log[1]; year++)
                populationCounts[year-BEGIN]++;
        }

        int maxPopulation = Integer.MIN_VALUE;
        int index=0;
        for(int i=0; i < 101; i++) {
            if(maxPopulation < populationCounts[i]) {
                maxPopulation = populationCounts[i];
                index=i;
            }
        }
        return BEGIN+index;
    }
}
```

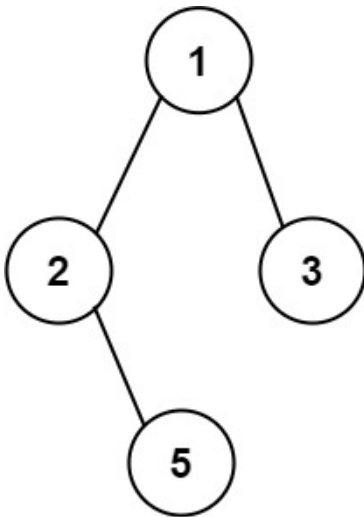
[257 Binary Tree Paths \(link\)](#)

Description

Given the root of a binary tree, return *all root-to-leaf paths in any order*.

A **leaf** is a node with no children.

Example 1:



Input: root = [1,2,3,null,5]
Output: ["1->2->5", "1->3"]

Example 2:

Input: root = [1]
Output: ["1"]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<String> binaryTreePaths(TreeNode root) {
        List<String> result = new ArrayList<>();
        traverse(root, new StringBuilder(), result);
        return result;
    }

    private void traverse(TreeNode root, StringBuilder sb, List<String> list) {
        if(root == null) {
            return;
        }

        int length = sb.length();
        sb.append(root.val).append("->");
        if(root.left == null && root.right == null) {
            list.add(sb.substring(0,sb.length()-2).toString());
            sb.setLength(length);
            return;
        }
        traverse(root.left, sb, list);
        traverse(root.right, sb, list);
        sb.setLength(length);
    }
}
```


389 Find the Difference (link)

Description

You are given two strings s and t .

String t is generated by random shuffling string s and then add one more letter at a random position.

Return the letter that was added to t .

Example 1:

```
Input: s = "abcd", t = "abcde"  
Output: "e"  
Explanation: 'e' is the letter that was added.
```

Example 2:

```
Input: s = "", t = "y"  
Output: "y"
```

Constraints:

- $0 \leq s.length \leq 1000$
- $t.length == s.length + 1$
- s and t consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public char findTheDifference(String s, String t) {
        char[] cmap = new char[26];
        for(char ch: t.toCharArray())
            cmap[ch-'a']++;
        for(char ch: s.toCharArray())
            cmap[ch-'a']--;
        for(int i=0; i < 26; i++)
            if(cmap[i] == 1)
                return (char)(i+'a');
        return '\0';
    }
}
```

[412 Fizz Buzz \(link\)](#)

Description

Given an integer n , return a *string array* `answer` (**1-indexed**) where:

- `answer[i] == "FizzBuzz"` if i is divisible by 3 and 5.
- `answer[i] == "Fizz"` if i is divisible by 3.
- `answer[i] == "Buzz"` if i is divisible by 5.
- `answer[i] == i` (as a string) if none of the above conditions are true.

Example 1:

```
Input: n = 3
Output: ["1","2","Fizz"]
```

Example 2:

```
Input: n = 5
Output: ["1","2","Fizz","4","Buzz"]
```

Example 3:

```
Input: n = 15
Output: ["1","2","Fizz","4","Buzz","Fizz","7","8","Fizz","Buzz","11","Fizz","13","14","FizzBuzz"]
```

Constraints:

- $1 \leq n \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<String> fizzBuzz(int n) {
        String FIZZ="Fizz";
        String BUZZ="Buzz";
        String FBUZZ="FizzBuzz";
        List<String> result = new ArrayList<>(n);
        for(int i=1; i <= n; i++) {
            if(i%3 > 0 && i%5 > 0)
                result.add(i+"");
            else if(i%3 == 0 && i%5 == 0)
                result.add(FBUZZ);
            else if(i%3 == 0)
                result.add(FIZZ);
            else
                result.add(BUZZ);
        }
        return result;
    }
}
```

[1217 Relative Sort Array \(link\)](#)

Description

Given two arrays `arr1` and `arr2`, the elements of `arr2` are distinct, and all elements in `arr2` are also in `arr1`.

Sort the elements of `arr1` such that the relative ordering of items in `arr1` are the same as in `arr2`. Elements that do not appear in `arr2` should be placed at the end of `arr1` in **ascending** order.

Example 1:

```
Input: arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
Output: [2,2,2,1,4,3,3,9,6,7,19]
```

Example 2:

```
Input: arr1 = [28,6,22,8,44,17], arr2 = [22,28,8,6]
Output: [22,28,8,6,17,44]
```

Constraints:

- $1 \leq \text{arr1.length}, \text{arr2.length} \leq 1000$
- $0 \leq \text{arr1}[i], \text{arr2}[i] \leq 1000$
- All the elements of `arr2` are **distinct**.
- Each `arr2[i]` is in `arr1`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] relativeSortArray(int[] arr1, int[] arr2) {
        int[] counts = new int[1001];
        for(int num: arr1)
            counts[num]++;

        int k=0;
        for(int num:arr2)
            while(counts[num]-- > 0)
                arr1[k++]=num;

        for(int i=0; i<1001; i++)
            while(counts[i]-- > 0)
                arr1[k++]=i;
        return arr1;
    }
}
```

[1112 Find Words That Can Be Formed by Characters](#) ([link](#)).

Description

You are given an array of strings `words` and a string `chars`.

A string is **good** if it can be formed by characters from `chars` (each character can only be used once for **each** word in `words`).

Return *the sum of lengths of all good strings in words*.

Example 1:

Input: `words = ["cat","bt","hat","tree"], chars = "atach"`

Output: 6

Explanation: The strings that can be formed are "cat" and "hat" so the answer is $3 + 3 = 6$.

Example 2:

Input: `words = ["hello","world","leetcode"], chars = "welldonehoneyr"`

Output: 10

Explanation: The strings that can be formed are "hello" and "world" so the answer is $5 + 5 = 10$.

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length}, \text{chars.length} \leq 100$
- `words[i]` and `chars` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int countCharacters(String[] words, String chars) {
        int counter=0;
        int[] charr = new int[26];
        for(char ch: chars.toCharArray()) {
            charr[ch-'a']++;
        }

        int[] wordArr = new int[26];
        for(String word: words) {
            int wordCount = 0;
            for(char ch: word.toCharArray())
                wordArr[ch-'a']++;

            for(int i=0; i < 26; i++) {
                if(charr[i] < wordArr[i]) {
                    wordCount = 0;
                    break;
                } else {
                    wordCount += wordArr[i];
                }
            }
            counter+=wordCount;
            Arrays.fill(wordArr,0);
        }
        return counter;
    }
}
```


[1128 Remove All Adjacent Duplicates In String](#)

Description

You are given a string s consisting of lowercase English letters. A **duplicate removal** consists of choosing two **adjacent** and **equal** letters and removing them.

We repeatedly make **duplicate removals** on s until we no longer can.

Return *the final string after all such duplicate removals have been made*. It can be proven that the answer is **unique**.

Example 1:

Input: $s = \text{"abbaca"}$

Output: "ca"

Explanation:

For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the



Example 2:

Input: $s = \text{"azxxzy"}$

Output: "ay"

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String removeDuplicates(String S) {
        Stack<Character> stack = new Stack<>();
        int length = 0;
        for(char ch: S.toCharArray()) {
            if(!stack.isEmpty() && stack.peek() == ch) {
                stack.pop();
            } else {
                stack.push(ch);
            }
        }
        if(stack.isEmpty())
            return "";

        char[] charr = new char[stack.size()];
        int i=0;
        for(char c:stack)
            charr[i++]=c;
        return String.valueOf(charr);
    }
}
```

[2345 Minimum Number of Operations to Convert Time \(link\)](#)

Description

You are given two strings `current` and `correct` representing two **24-hour times**.

24-hour times are formatted as "HH:MM", where HH is between 00 and 23, and MM is between 00 and 59. The earliest 24-hour time is 00:00, and the latest is 23:59.

In one operation you can increase the time `current` by 1, 5, 15, or 60 minutes. You can perform this operation **any** number of times.

Return *the minimum number of operations needed to convert* `current` *to* `correct`.

Example 1:

Input: `current = "02:30", correct = "04:35"`

Output: 3

Explanation:

We can convert `current` to `correct` in 3 operations as follows:

- Add 60 minutes to `current`. `current` becomes "03:30".
- Add 60 minutes to `current`. `current` becomes "04:30".
- Add 5 minutes to `current`. `current` becomes "04:35".

It can be proven that it is not possible to convert `current` to `correct` in fewer than 3 operations.

Example 2:

Input: `current = "11:00", correct = "11:01"`

Output: 1

Explanation: We only have to add one minute to `current`, so the minimum number of operations needed is 1.

Constraints:

- `current` and `correct` are in the format "HH:MM"
- `current` <= `correct`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    private char ZERO='0';
    private int[] timeUnits = new int[]{60,15,5,1};

    public int convertTime(String current, String correct) {
        int minuteDiff = getMinutes(correct)-getMinutes(current);
        int count = 0;

        int i=0;
        while(minuteDiff > 0 && i < 4) {
            if(minuteDiff >= timeUnits[i]) {
                count += minuteDiff/timeUnits[i];
                minuteDiff %= timeUnits[i];
            }
            i++;
        }

        return count;
    }

    int getMinutes(String time) {
        return 600*(time.charAt(0)-ZERO) + 60*(time.charAt(1)-ZERO)+10*(time.charAt(3)-ZERO)+(time.charAt(4)-ZERO);
    }
}
```

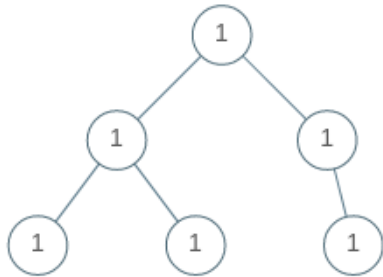
[1005 Univalued Binary Tree \(link\)](#)

Description

A binary tree is **uni-valued** if every node in the tree has the same value.

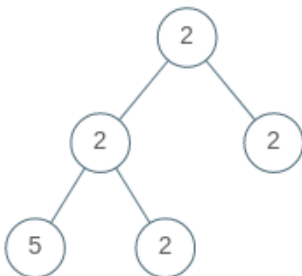
Given the root of a binary tree, return `true` *if the given tree is **uni-valued***, or `false` *otherwise*.

Example 1:



Input: root = [1,1,1,1,1,null,1]
Output: true

Example 2:



Input: root = [2,2,2,5,2]
Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $0 \leq \text{Node.val} < 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isUnivalTree(TreeNode root) {
        if(root == null) return true;
        return isSame(root, root.val);
    }

    boolean isSame(TreeNode root, int val) {
        if(root == null) return true;
        return root.val == val && isSame(root.left, val) && isSame(root.right, val);
    }
}
```

359 Logger Rate Limiter (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import static java.util.AbstractMap.SimpleEntry;

class Logger {
    private LinkedList<SimpleEntry<String, Integer>> messageQueue;
    private HashSet<String> messageSet;

    public Logger() {
        messageQueue = new LinkedList<>();
        messageSet = new HashSet<>();
    }

    public boolean shouldPrintMessage(int timestamp, String message) {
        while(!messageQueue.isEmpty()) {
            SimpleEntry<String, Integer> head = messageQueue.peek();
            if(timestamp - head.getValue() >=10 ) {
                messageQueue.poll();
                messageSet.remove(head.getKey());
            } else {
                break;
            }
        }

        if(messageSet.contains(message)) {
            return false;
        } else {
            messageQueue.offer(new SimpleEntry<>(message, timestamp));
            messageSet.add(message);
        }
        return true;
    }
}

/**
 * Your Logger object will be instantiated and called as such:
 * Logger obj = new Logger();
 * boolean param_1 = obj.shouldPrintMessage(timestamp,message);
 */
```


2226 Rings and Rods (link)

Description

There are n rings and each ring is either red, green, or blue. The rings are distributed **across ten rods** labeled from 0 to 9.

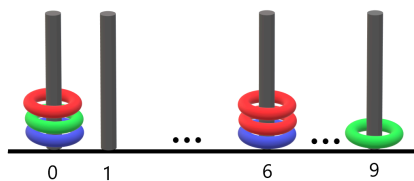
You are given a string `rings` of length $2n$ that describes the n rings that are placed onto the rods. Every two characters in `rings` forms a **color-position pair** that is used to describe each ring where:

- The **first** character of the i^{th} pair denotes the i^{th} ring's **color** ('R', 'G', 'B').
- The **second** character of the i^{th} pair denotes the **rod** that the i^{th} ring is placed on ('0' to '9').

For example, "R3G2B1" describes $n == 3$ rings: a red ring placed onto the rod labeled 3, a green ring placed onto the rod labeled 2, and a blue ring placed onto the rod labeled 1.

Return *the number of rods that have **all three colors** of rings on them.*

Example 1:



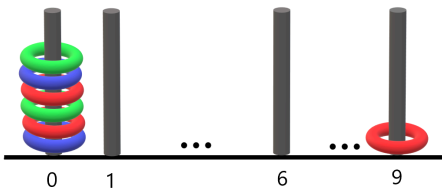
Input: `rings = "B0B6G0R6R0R6G9"`

Output: 1

Explanation:

- The rod labeled 0 holds 3 rings with all colors: red, green, and blue.
 - The rod labeled 6 holds 3 rings, but it only has red and blue.
 - The rod labeled 9 holds only a green ring.
- Thus, the number of rods with all three colors is 1.

Example 2:



Input: `rings = "B0R0G0R9R0B0G0"`

Output: 1

Explanation:

- The rod labeled 0 holds 6 rings with all colors: red, green, and blue.
 - The rod labeled 9 holds only a red ring.
- Thus, the number of rods with all three colors is 1.

Example 3:

Input: `rings = "G4"`

Output: 0

Explanation:

Only one ring is given. Thus, no rods have all three colors.

Constraints:

- `rings.length == 2 * n`
- `1 <= n <= 100`
- `rings[i]` where `i` is **even** is either 'R', 'G', or 'B' (**0-indexed**).
- `rings[i]` where `i` is **odd** is a digit from '0' to '9' (**0-indexed**).

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int countPoints(String rings) {
        char Z = '0';
        int len = rings.length();
        Set<Character>[] rods = new HashSet[10];

        int counter=0;
        for(int i=0; i < len; i+=2) {
            int idx = rings.charAt(i+1)-Z;
            if(rods[idx] == null)
                rods[idx]=new HashSet<>();
            if(rods[idx].size() == 3)
                continue;
            rods[idx].add(rings.charAt(i));
            if(rods[idx].size() == 3) {
                counter++;
            }
        }

        return counter;
    }
}
```

[586 Customer Placing the Largest Number of Orders \(link\)](#)

Description

Table: Orders

Column Name	Type
order_number	int
customer_number	int

order_number is the primary key (column with unique values) for this table.
This table contains information about the order ID and the customer ID.

Write a solution to find the customer_number for the customer who has placed **the largest number of orders**.

The test cases are generated so that **exactly one customer** will have placed more orders than any other customer.

The result format is in the following example.

Example 1:

Input:

Orders table:

order_number	customer_number
1	1
2	2
3	3
4	3

Output:

customer_number
3

Explanation:

The customer with number 3 has two orders, which is greater than either customer 1 or 2 because each of them has only one order. So the result is customer_number 3.

Follow up: What if more than one customer has the largest number of orders, can you find all the customer_number in this case?

(scroll down for solution)

Solution

Language: mysql

Status: Accepted

```
# Write your MySQL query statement below
select customer_number from Orders group by 1 order by count(1) desc limit 1
```

1013 Fibonacci Number (link)

Description

The **Fibonacci numbers**, commonly denoted $F(n)$ form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$\begin{aligned} F(0) &= 0, F(1) = 1 \\ F(n) &= F(n-1) + F(n-2), \text{ for } n > 1. \end{aligned}$$

Given n , calculate $F(n)$.

Example 1:

Input: $n = 2$
Output: 1
Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1$.

Example 2:

Input: $n = 3$
Output: 2
Explanation: $F(3) = F(2) + F(1) = 1 + 1 = 2$.

Example 3:

Input: $n = 4$
Output: 3
Explanation: $F(4) = F(3) + F(2) = 2 + 1 = 3$.

Constraints:

- $0 \leq n \leq 30$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int fib(int n) {  
        if(n == 0) return 0;  
        if(n == 1) return 1;  
        int first = 0;  
        int second = 1;  
        int third;  
        for(int i=2; i <= n; i++) {  
            third = first+second;  
            first = second;  
            second = third;  
        }  
        return second;  
    }  
}
```

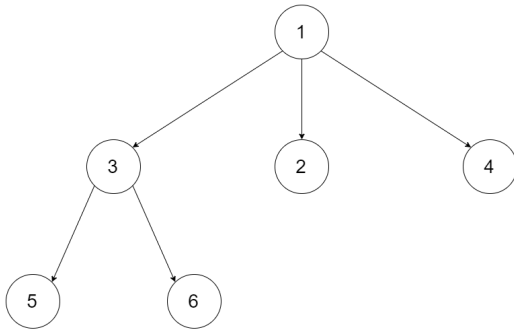
776 N-ary Tree Postorder Traversal [\(link\)](#)

Description

Given the root of an n-ary tree, return *the postorder traversal of its nodes' values*.

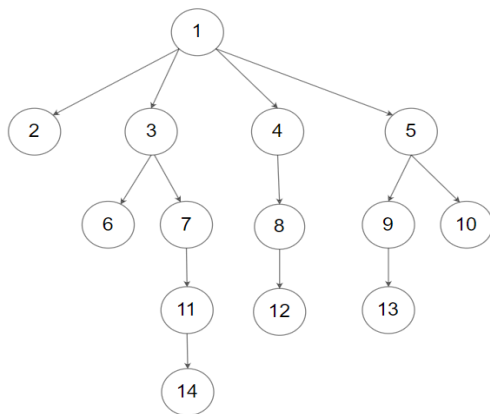
Nary-Tree input serialization is represented in their level order traversal. Each group of children is separated by the null value (See examples)

Example 1:



Input: root = [1,null,3,2,4,null,5,6]
Output: [5,6,3,2,4,1]

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [2,6,14,11,7,3,12,8,4,13,9,10,5,1]

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $0 \leq \text{Node.val} \leq 10^4$
- The height of the n-ary tree is less than or equal to 1000.

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, List<Node> _children) {
        val = _val;
        children = _children;
    }
};
*/

class Solution {
    private List<Integer> result = new ArrayList<>();
    public List<Integer> postorder(Node root) {
        if(root == null) return result;
        for(Node node: root.children)
            postorder(node);
        result.add(root.val);
        return result;
    }
}
```

[595 Big Countries \(link\)](#)

Description

Table: World

Column Name	Type
name	varchar
continent	varchar
area	int
population	int
gdp	bigint

name is the primary key (column with unique values) for this table.

Each row of this table gives information about the name of a country, the continent to which it belongs,

A country is **big** if:

- it has an area of at least three million (i.e., 3000000 km²), or
- it has a population of at least twenty-five million (i.e., 25000000).

Write a solution to find the name, population, and area of the **big countries**.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

World table:

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000

Output:

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

(scroll down for solution)

Solution

Language: mysql

Status: Accepted

```
select name, population, area from World  
where area >= 3000000 || population >=25000000;
```

2292 Counting Words With a Given Prefix (link)

Description

You are given an array of strings `words` and a string `pref`.

Return *the number of strings in* `words` *that contain* `pref` *as a* **prefix**.

A **prefix** of a string `s` is any leading contiguous substring of `s`.

Example 1:

Input: `words = ["pay", "attention", "practice", "attend"]`, `pref = "at"`

Output: 2

Explanation: The 2 strings that contain "at" as a prefix are: "attention" and "attend".

Example 2:

Input: `words = ["leetcode", "win", "loops", "success"]`, `pref = "code"`

Output: 0

Explanation: There are no strings that contain "code" as a prefix.

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length}, \text{pref.length} \leq 100$
- `words[i]` and `pref` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int prefixCount(String[] words, String pref) {
        int prefixCount=0;
        int prefixLength=pref.length();

        for(String word: words) {
            if(word.length() < prefixLength)
                continue;
            int index=0;
            for(char ch: word.toCharArray()) {
                if(index == prefixLength || ch != pref.charAt(index))
                    break;
                index++;
            }
            if(index == prefixLength)
                prefixCount++;
        }

        return prefixCount;
    }
}
```

[2210 Find Target Indices After Sorting Array \(link\)](#)

Description

You are given a **0-indexed** integer array `nums` and a target element `target`.

A **target index** is an index `i` such that `nums[i] == target`.

Return *a list of the target indices of `nums` after sorting `nums` in **non-decreasing** order*. If there are no target indices, return *an **empty** list*. The returned list must be sorted in **increasing** order.

Example 1:

```
Input: nums = [1,2,5,2,3], target = 2
Output: [1,2]
Explanation: After sorting, nums is [1,2,2,3,5].
The indices where nums[i] == 2 are 1 and 2.
```

Example 2:

```
Input: nums = [1,2,5,2,3], target = 3
Output: [3]
Explanation: After sorting, nums is [1,2,2,3,5].
The index where nums[i] == 3 is 3.
```

Example 3:

```
Input: nums = [1,2,5,2,3], target = 5
Output: [4]
Explanation: After sorting, nums is [1,2,2,3,5].
The index where nums[i] == 5 is 4.
```

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $1 \leq \text{nums}[i], \text{target} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> targetIndices(int[] nums, int target) {
        List<Integer> list = new ArrayList<>();
        int equalCount=0, lessCount=0;
        for(int num:nums) {
            if(num == target) equalCount++;
            else if(num < target) lessCount++;
        }

        if(equalCount == 0) return list;
        for(int i=lessCount; i<lessCount+equalCount; i++)
            list.add(i);

        return list;
    }
}
```


[861 Flipping an Image \(link\)](#)

Description

Given an $n \times n$ binary matrix `image`, flip the image **horizontally**, then invert it, and return *the resulting image*.

To flip an image horizontally means that each row of the image is reversed.

- For example, flipping `[1,1,0]` horizontally results in `[0,1,1]`.

To invert an image means that each `0` is replaced by `1`, and each `1` is replaced by `0`.

- For example, inverting `[0,1,1]` results in `[1,0,0]`.

Example 1:

```
Input: image = [[1,1,0],[1,0,1],[0,0,0]]
Output: [[1,0,0],[0,1,0],[1,1,1]]
Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
Then, invert the image: [[1,0,0],[0,1,0],[1,1,1]]
```

Example 2:

```
Input: image = [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]
Output: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
Explanation: First reverse each row: [[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]].
Then invert the image: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
```

Constraints:

- `n == image.length`
- `n == image[i].length`
- `1 <= n <= 20`
- `images[i][j]` is either `0` or `1`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] flipAndInvertImage(int[][] image) {
        int m = image.length;
        int n = image[0].length;

        for(int k=0; k < m; k++) {
            for(int i=0,j=n-1; i <= j; i++,j--) {
                int t1 = image[k][i];
                int t2 = image[k][j];

                image[k][i]=t2 == 0 ? 1:0;
                image[k][j]=t1 == 0 ? 1:0;
            }
        }
        return image;
    }
}
```

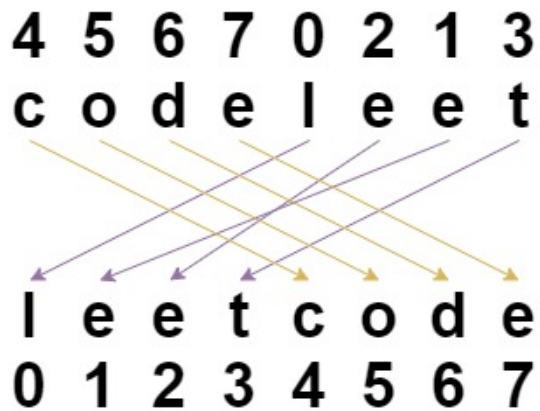
1651 Shuffle String [\(link\)](#)

Description

You are given a string `s` and an integer array `indices` of the **same length**. The string `s` will be shuffled such that the character at the i^{th} position moves to `indices[i]` in the shuffled string.

Return *the shuffled string*.

Example 1:



Input: `s = "codeleet", indices = [4,5,6,7,0,2,1,3]`
Output: `"leetcode"`
Explanation: As shown, "codeleet" becomes "leetcode" after shuffling.

Example 2:

Input: `s = "abc", indices = [0,1,2]`
Output: `"abc"`
Explanation: After shuffling, each character remains in its position.

Constraints:

- `s.length == indices.length == n`
- `1 <= n <= 100`
- `s` consists of only lowercase English letters.
- `0 <= indices[i] < n`
- All values of `indices` are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public String restoreString(String s, int[] indices) {  
        char[] chars = new char[indices.length];  
        for(int i=0; i < indices.length; i++)  
            chars[indices[i]]=s.charAt(i);  
        return String.valueOf(chars);  
    }  
}
```

[1241 Decompress Run-Length Encoded List \(link\)](#)

Description

We are given a list `nums` of integers representing a list compressed with run-length encoding.

Consider each adjacent pair of elements `[freq, val] = [nums[2*i], nums[2*i+1]]` (with $i \geq 0$). For each such pair, there are `freq` elements with value `val` concatenated in a sublist. Concatenate all the sublists from left to right to generate the decompressed list.

Return the decompressed list.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[2,4,4,4]`

Explanation: The first pair `[1,2]` means we have `freq = 1` and `val = 2` so we generate the array `[2]`. The second pair `[3,4]` means we have `freq = 3` and `val = 4` so we generate `[4,4,4]`. At the end the concatenation `[2] + [4,4,4]` is `[2,4,4,4]`.

Example 2:

Input: `nums = [1,1,2,3]`

Output: `[1,3,3]`

Constraints:

- `2 <= nums.length <= 100`
- `nums.length % 2 == 0`
- `1 <= nums[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] decompressRLElist(int[] nums) {
        int length = 0;
        for(int i=0; i < nums.length; i+=2)
            length += nums[i];

        int[] result = new int[length];
        int start=0;
        for(int i=0; i < nums.length; i+=2) {
            Arrays.fill(result, start, start+nums[i], nums[i+1]);
            start += nums[i];
        }
        return result;
    }
}
```

[1482 How Many Numbers Are Smaller Than the Current Number \(link\)](#)

Description

Given the array `nums`, for each `nums[i]` find out how many numbers in the array are smaller than it. That is, for each `nums[i]` you have to count the number of valid `j`'s such that `j != i` **and** `nums[j] < nums[i]`.

Return the answer in an array.

Example 1:

```
Input: nums = [8,1,2,2,3]
Output: [4,0,1,1,3]
Explanation:
For nums[0]=8 there exist four smaller numbers than it (1, 2, 2 and 3).
For nums[1]=1 does not exist any smaller number than it.
For nums[2]=2 there exist one smaller number than it (1).
For nums[3]=2 there exist one smaller number than it (1).
For nums[4]=3 there exist three smaller numbers than it (1, 2 and 2).
```

Example 2:

```
Input: nums = [6,5,4,8]
Output: [2,1,0,3]
```

Example 3:

```
Input: nums = [7,7,7,7]
Output: [0,0,0,0]
```

Constraints:

- `2 <= nums.length <= 500`
- `0 <= nums[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        int length = nums.length;
        int[] ans = new int[length];
        System.arraycopy(nums, 0, ans, 0, length);
        Arrays.sort(ans);

        int prev=ans[0];
        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(prev, 0);

        for(int i=1; i < length; i++) {
            if(prev != ans[i])
                map.put(ans[i], i);
            prev=ans[i];
        }

        for(int i=0; i < length; i++) {
            ans[i]=map.get(nums[i]);
        }

        return ans;
    }
}
```


[1205 Defanging an IP Address \(link\)](#)

Description

Given a valid (IPv4) IP address, return a defanged version of that IP address.

A *defanged IP address* replaces every period "." with "[.]".

Example 1:

```
Input: address = "1.1.1.1"
Output: "1[.]1[.]1[.]1"
```

Example 2:

```
Input: address = "255.100.50.0"
Output: "255[.]100[.]50[.]0"
```

Constraints:

- The given address is a valid IPv4 address.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String defangIPaddr(String address) {
        int length = address.length();
        char[] chars = new char[length+2*3];
        int i=0;
        for(char ch: address.toCharArray()) {
            if(ch == '.') {
                chars[i++]='[';
                chars[i++]='.';
                chars[i++]=']';
            } else {
                chars[i++]=ch;
            }
        }
        return String.valueOf(chars);
    }
}
```

2383 Add Two Integers (link)

Description

Given two integers num1 and num2, return *the **sum** of the two integers*.

Example 1:

Input: num1 = 12, num2 = 5

Output: 17

Explanation: num1 is 12, num2 is 5, and their sum is $12 + 5 = 17$, so 17 is returned.

Example 2:

Input: num1 = -10, num2 = 4

Output: -6

Explanation: $\text{num1} + \text{num2} = -6$, so -6 is returned.

Constraints:

- $-100 \leq \text{num1}, \text{num2} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int sum(int num1, int num2) {
        if(num1 == 0) return num2;
        else if(num2 == 0) return num1;
        else {
            int carry = 0;
            while(num2 != 0) {
                carry = num1&num2;
                num1 ^= num2;
                num2 = carry << 1;
            }
            return num1;
        }
    }
}
```

[2048 Build Array from Permutation \(link\)](#)

Description

Given a **zero-based permutation** `nums` (**0-indexed**), build an array `ans` of the **same length** where `ans[i] = nums[nums[i]]` for each $0 \leq i < \text{nums.length}$ and return it.

A **zero-based permutation** `nums` is an array of **distinct** integers from 0 to `nums.length - 1` (**inclusive**).

Example 1:

Input: `nums = [0,2,1,5,3,4]`

Output: `[0,1,2,4,5,3]`

Explanation: The array `ans` is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]], nums[nums[5]]]
      = [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]
      = [0,1,2,4,5,3]
```

Example 2:

Input: `nums = [5,0,1,2,3,4]`

Output: `[4,5,0,1,2,3]`

Explanation: The array `ans` is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]], nums[nums[5]]]
      = [nums[5], nums[0], nums[1], nums[2], nums[3], nums[4]]
      = [4,5,0,1,2,3]
```

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] < \text{nums.length}$
- The elements in `nums` are **distinct**.

Follow-up: Can you solve it without using an extra space (i.e., $O(1)$ memory)?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int[] buildArray(int[] nums) {  
        int length = nums.length;  
        for(int i=0; i < length; i++) {  
            int temp = nums[i];  
            nums[i] += length*(nums[temp]%length);  
        }  
        for(int i=0; i < length; i++) {  
            nums[i] /=length;  
        }  
        return nums;  
    }  
}
```

[43 Multiply Strings \(link\)](#)

Description

Given two non-negative integers `num1` and `num2` represented as strings, return the product of `num1` and `num2`, also represented as a string.

Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.

Example 1:

```
Input: num1 = "2", num2 = "3"
Output: "6"
```

Example 2:

```
Input: num1 = "123", num2 = "456"
Output: "56088"
```

Constraints:

- `1 <= num1.length, num2.length <= 200`
- `num1` and `num2` consist of digits only.
- Both `num1` and `num2` do not contain any leading zero, except the number `0` itself.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    static final char ZERO='0';
    public String multiply(String s1, String s2) {
        int l1 = s1.length(), l2 = s2.length();
        if(s1.equals("0") || s2.equals("0")) return "0";
        if(s1.equals("1")) return s2;
        if(s2.equals("1")) return s1;

        int[] sn1 = toInt(s1);
        int[] sn2 = toInt(s2);

        int length=l1+l2;
        int[] result = new int[length];
        int l=length-1,k=0;
        //while(j>-1) {
        for(int j=l2-1; j>-1; j--) {
            k = l--;
            for(int i=l1-1; i>-1; i--) {
                int dig = sn1[i]*sn2[j]+result[k];
                result[k-1] += dig/10;
                result[k--] = dig%10;
            }
        }

        if(result[k]==0)
            k++;

        StringBuilder bldr = new StringBuilder();
        while(k < length)
            bldr.append(result[k++]);
        return bldr.toString();
    }

    private int[] toInt(String num) {
        int len = num.length();
        int[] res = new int[len];
        for(int i=0; i < len; i++)
            res[i] = num.charAt(i) - ZERO;
        return res;
    }
}
```


[724 Find Pivot Index \(link\)](#)

Description

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return *the leftmost pivot index*. If no such index exists, return -1.

Example 1:

```
Input: nums = [1,7,3,6,5,6]
Output: 3
Explanation:
The pivot index is 3.
Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11
Right sum = nums[4] + nums[5] = 5 + 6 = 11
```

Example 2:

```
Input: nums = [1,2,3]
Output: -1
Explanation:
There is no index that satisfies the conditions in the problem statement.
```

Example 3:

```
Input: nums = [2,1,-1]
Output: 0
Explanation:
The pivot index is 0.
Left sum = 0 (no elements to the left of index 0)
Right sum = nums[1] + nums[2] = 1 + -1 = 0
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$

Note: This question is the same as 1991: <https://leetcode.com/problems/find-the-middle-index-in-array/>

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int pivotIndex(int[] nums) {
        int leftSum = 0;
        int rightSum = 0;
        for(int num: nums)
            rightSum += num;

        for(int i=0; i < nums.length; i++) {
            rightSum -= nums[i];
            if(rightSum == leftSum)
                return i;
            leftSum += nums[i];
        }
        return -1;
    }
}
```

[392 Is Subsequence \(link\)](#)

Description

Given two strings s and t , return `true` if s is a **subsequence** of t , or `false` otherwise.

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1:

```
Input: s = "abc", t = "ahbgdc"
Output: true
```

Example 2:

```
Input: s = "axc", t = "ahbgdc"
Output: false
```

Constraints:

- $0 \leq s.length \leq 100$
- $0 \leq t.length \leq 10^4$
- s and t consist only of lowercase English letters.

Follow up: Suppose there are lots of incoming s , say $s_1, s_2, \dots, s_k$ where $k \geq 10^9$, and you want to check one by one to see if t has its subsequence. In this scenario, how would you change your code?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isSubsequence(String s, String t) {
        int subSequenceCount=0;
        int ls = s.length();
        int lt = t.length();
        int si=0;
        for(int ti=0; ti < lt && si < ls; ti++ ) {
            if(t.charAt(ti) == s.charAt(si)) {
                si++;
                subSequenceCount++;
            }
        }
        return subSequenceCount == s.length();
    }
}
```

658 Find K Closest Elements (link)

Description

Given a **sorted** integer array `arr`, two integers `k` and `x`, return the `k` closest integers to `x` in the array. The result should also be sorted in ascending order.

An integer `a` is closer to `x` than an integer `b` if:

- $|a - x| < |b - x|$, or
- $|a - x| == |b - x|$ and $a < b$

Example 1:

Input: `arr = [1,2,3,4,5]`, `k = 4`, `x = 3`

Output: `[1,2,3,4]`

Example 2:

Input: `arr = [1,1,2,3,4,5]`, `k = 4`, `x = -1`

Output: `[1,1,2,3]`

Constraints:

- $1 \leq k \leq \text{arr.length}$
- $1 \leq \text{arr.length} \leq 10^4$
- `arr` is sorted in **ascending** order.
- $-10^4 \leq \text{arr}[i], x \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findClosestElements(int[] arr, int k, int target) {
        //get the index of element that is closest to target
        int left=0, right = arr.length,mid;
        while(left < right) {
            mid = (left+right)/2;
            if(arr[mid]>=target) {
                right = mid;
            } else {
                left = mid+1;
            }
        }
        left -= 1;

        //slide the window outward from left, right positions
        //after exiting while loop all elements inside
        //left and right(both exclusive) form the result
        while(right-left-1<k) {
            if(left == -1) {
                right++;
                continue;
            }

            if(right == arr.length || (target-arr[left] <= arr[right]-target))
                left--;
            else
                right++;
        }

        List<Integer> list = new ArrayList<>();
        for(int i=left+1; i < right; i++)
            list.add(arr[i]);
        return list;
    }
}
```

[1693 Sum of All Odd Length Subarrays \(link\)](#)

Description

Given an array of positive integers `arr`, return *the sum of all possible **odd-length subarrays** of `arr`.*

A **subarray** is a contiguous subsequence of the array.

Example 1:

Input: `arr = [1,4,2,5,3]`

Output: 58

Explanation: The odd-length subarrays of `arr` and their sums are:

`[1]` = 1

`[4]` = 4

`[2]` = 2

`[5]` = 5

`[3]` = 3

`[1,4,2]` = 7

`[4,2,5]` = 11

`[2,5,3]` = 10

`[1,4,2,5,3]` = 15

If we add all these together we get $1 + 4 + 2 + 5 + 3 + 7 + 11 + 10 + 15 = 58$

Example 2:

Input: `arr = [1,2]`

Output: 3

Explanation: There are only 2 subarrays of odd length, `[1]` and `[2]`. Their sum is 3.

Example 3:

Input: `arr = [10,11,12]`

Output: 66

Constraints:

- $1 \leq \text{arr.length} \leq 100$
- $1 \leq \text{arr}[i] \leq 1000$

Follow up:

Could you solve this problem in $O(n)$ time complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int sumOddLengthSubarrays(int[] arr) {
        int sumOddLengthSubarrays=0;
        for(int i=1; i <= arr.length; i+=2) {
            sumOddLengthSubarrays += getPrefixSum(arr, i);
        }

        return sumOddLengthSubarrays;
    }

    int getPrefixSum(int[] arr, int k) {
        int prefixSum = 0;

        int left=0, right=0, slidingSum = 0;
        int length = arr.length, maxIdx = length-k+1;

        while(left < maxIdx && right < length) {
            slidingSum += arr[right];
            if(++right-left == k) {
                prefixSum += slidingSum;
                slidingSum -= arr[left++];
            }
        }
        return prefixSum;
    }
}
```


[452 Minimum Number of Arrows to Burst Balloons \(link\)](#)

Description

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [xstart, xend]` denotes a balloon whose **horizontal diameter** stretches between `xstart` and `xend`. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up **directly vertically** (in the positive y-direction) from different points along the x-axis. A balloon with `xstart` and `xend` is **burst** by an arrow shot at `x` if `xstart ≤ x ≤ xend`. There is **no limit** to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array `points`, return *the minimum number of arrows that must be shot to burst all balloons*.

Example 1:

```
Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 6, bursting the balloons [2,8] and [1,6].
- Shoot an arrow at x = 11, bursting the balloons [10,16] and [7,12].
```

Example 2:

```
Input: points = [[1,2],[3,4],[5,6],[7,8]]
Output: 4
Explanation: One arrow needs to be shot for each balloon for a total of 4 arrows.
```

Example 3:

```
Input: points = [[1,2],[2,3],[3,4],[4,5]]
Output: 2
Explanation: The balloons can be burst by 2 arrows:
- Shoot an arrow at x = 2, bursting the balloons [1,2] and [2,3].
- Shoot an arrow at x = 4, bursting the balloons [3,4] and [4,5].
```

Constraints:

- $1 \leq \text{points.length} \leq 10^5$
- $\text{points}[i].\text{length} == 2$
- $-2^{31} \leq x_{\text{start}} < x_{\text{end}} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findMinArrowShots(int[][] points) {
        Arrays.sort(points, (a,b) -> Integer.compare(a[1],b[1]));
        int end = points[0][1];
        int arrows = 1;
        for(int i=1; i < points.length; i++) {
            if(points[i][0] > end ) {
                arrows++;
                end = points[i][1];
            }
        }
        return arrows;
    }
}
```

[325 Maximum Size Subarray Sum Equals k \(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxSubArrayLen(int[] nums, int k) {
        int maxSize=0, prefixSum=0;
        int length = nums.length;

        Map<Integer, Integer> prefixMap = new HashMap<>();
        for(int i=0; i < length; i++) {
            prefixSum += nums[i];

            if(prefixSum == k)
                maxSize = i+1;

            if(prefixMap.containsKey(prefixSum - k))
                maxSize = Math.max(maxSize, i - prefixMap.get(prefixSum - k));

            if(!prefixMap.containsKey(prefixSum))
                prefixMap.put(prefixSum, i);
        }

        return maxSize;
    }
}
```

[42 Trapping Rain Water \(link\)](#)

Description

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1].

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

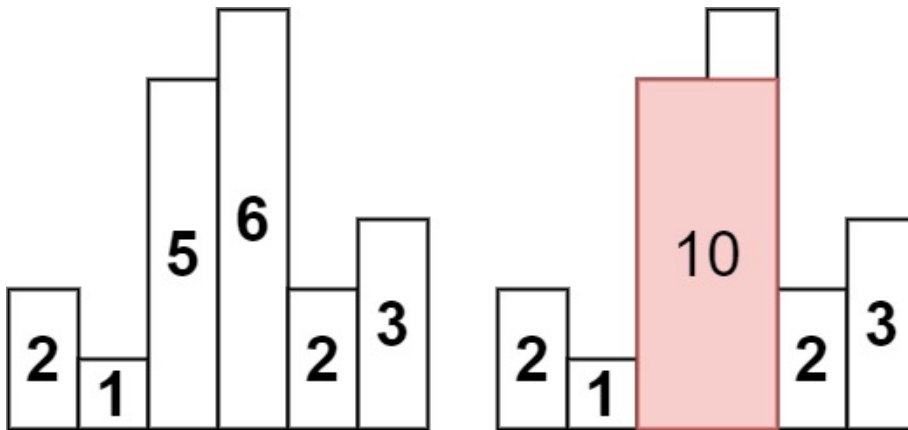
```
class Solution {
    public int trap(int[] heights) {
        Stack<Integer> stack = new Stack<>();
        int waterTrapped = 0, totalElements = heights.length;
        int popped,length,height;
        for(int i=0; i < totalElements; i++) {
            while(!stack.isEmpty() && heights[stack.peek()] < heights[i]) {
                popped = stack.pop();
                if(stack.isEmpty())
                    break;
                length = i - stack.peek()-1;
                height = Math.min(heights[stack.peek()], heights[i]) - heights[popped];
                waterTrapped += length*height;
            }
            stack.push(i);
        }
        return waterTrapped;
    }
}
```

84 Largest Rectangle in Histogram (link)

Description

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.

Example 1:

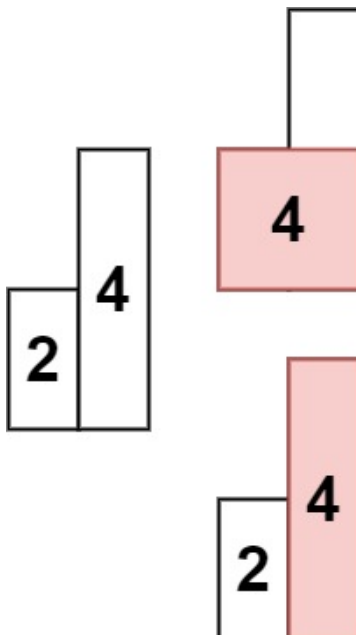


Input: `heights = [2,1,5,6,2,3]`

Output: 10

Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: `heights = [2,4]`

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 10^5$
- $0 \leq \text{heights}[i] \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        int maxArea = 0;
        Stack<Integer> stack = new Stack<>();
        int i=0, length=0, height, arrLength = heights.length;
        int currentValue;

        while(i <= arrLength) {
            currentValue = i < arrLength ? heights[i]:-1;
            if(currentValue == -1 && stack.isEmpty())
                break;

            while(!stack.isEmpty() && heights[stack.peek()] > currentValue) {
                height = heights[stack.pop()];
                if(stack.isEmpty())
                    length=i;
                else
                    length=i-stack.peek()-1;
                maxArea = Math.max(maxArea, length*height);
            }

            if(i < arrLength) {
                stack.push(i);
                i++;
            }
        }
        return maxArea;
    }
}
```

6 Zigzag Conversion (link)

Description

The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows like this: (you may want to display this pattern in a fixed font for better legibility)

```
P   A   H   N  
A P L S I I G  
Y   I   R
```

And then read line by line: "PAHNAPLSIIGYIR"

Write the code that will take a string and make this conversion given a number of rows:

```
string convert(string s, int numRows);
```

Example 1:

Input: s = "PAYPALISHIRING", numRows = 3
Output: "PAHNAPLSIIGYIR"

Example 2:

Input: s = "PAYPALISHIRING", numRows = 4
Output: "PINALSIGYAHRPI"
Explanation:
P I N
A L S I G
Y A H R
P I

Example 3:

Input: s = "A", numRows = 1
Output: "A"

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of English letters (lower-case and upper-case), ',', and '.'.
- $1 \leq numRows \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String convert(String s, int numRows) {
        int length = s.length();
        if(numRows < 2 || numRows >= length)
            return s;

        StringBuilder[] builders = new StringBuilder[numRows];
        for(int i=0; i < numRows; i++)
            builders[i] = new StringBuilder();

        int k=0, idx=0;
        boolean toRight = true;
        for(char c: s.toCharArray()) {
            if(toRight) {
                builders[idx++].append(c);
                if(idx == numRows) {
                    toRight = false;
                    idx = numRows-1;
                }
            } else {
                builders[--idx].append(c);
                if(idx == 0) {
                    toRight = true;
                    idx++;
                }
            }
            k++;
        }

        StringBuilder builder = new StringBuilder();
        for(int i=0; i < numRows; i++) {
            builder.append(builders[i].toString());
        }
        return builder.toString();
    }
}
```

1603 Running Sum of 1d Array (link)

Description

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`

Output: `[1,2,3,4,5]`

Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`

Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        int[] sum = new int[nums.length];  
        sum[0] = nums[0];  
        for(int i=1; i < nums.length; i++)  
            sum[i]=sum[i-1]+nums[i];  
        return sum;  
    }  
}
```

[202 Happy Number \(link\)](#)

Description

Write an algorithm to determine if a number n is happy.

A **happy number** is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it **loops endlessly in a cycle** which does not include 1.
- Those numbers for which this process **ends in 1** are happy.

Return `true` if n is a happy number, and `false` if not.

Example 1:

```
Input: n = 19
Output: true
Explanation:
 $1^2 + 9^2 = 82$ 
 $8^2 + 2^2 = 68$ 
 $6^2 + 8^2 = 100$ 
 $1^2 + 0^2 + 0^2 = 1$ 
```

Example 2:

```
Input: n = 2
Output: false
```

Constraints:

- $1 \leq n \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isHappy(int n) {
        int fast = n, slow = n;
        do {
            slow = squareOf(slow);
            fast = squareOf(squareOf(fast));
        } while(slow != fast);

        return slow == 1;
    }

    private int squareOf(int n) {
        int sum = 0, digit;
        while(n != 0) {
            digit = n%10;
            sum += digit*digit;
            n /= 10;
        }
        return sum;
    }
}
```

328 Odd Even Linked List (link)

Description

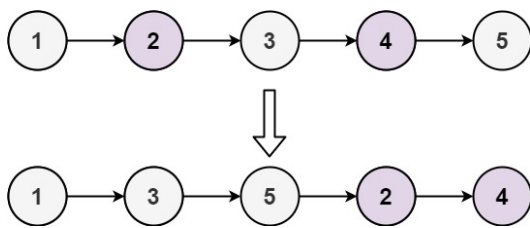
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return *the reordered list*.

The **first** node is considered **odd**, and the **second** node is **even**, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

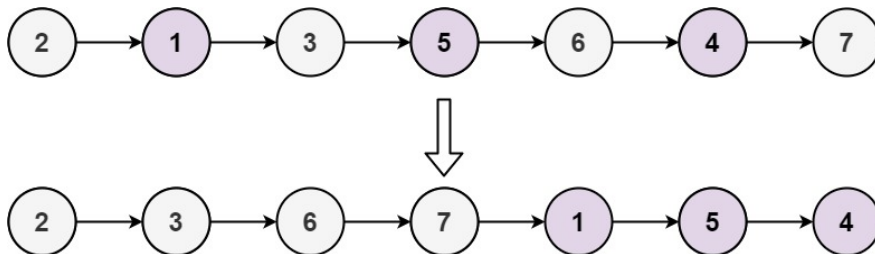
You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:



Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]

Example 2:



Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range $[0, 10^4]$.
- $-10^6 \leq \text{Node.val} \leq 10^6$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode oddEvenList(ListNode head) {
        boolean isOdd = true;
        ListNode odd = new ListNode(), node = head;
        ListNode even = new ListNode(), evenHead = even;

        while(node != null) {
            if(isOdd) {
                odd.next = node;
                odd = odd.next;
            } else {
                even.next = node;
                even = even.next;
            }
            isOdd = !isOdd;
            node = node.next;
        }
        odd.next = evenHead.next;
        even.next = null;
        return head;
    }
}
```

318 Maximum Product of Word Lengths (link)

Description

Given a string array `words`, return *the maximum value of* $\text{length}(\text{word}[i]) * \text{length}(\text{word}[j])$ *where the two words do not share common letters*. If no such two words exist, return 0.

Example 1:

Input: `words = ["abcw", "baz", "foo", "bar", "xtfn", "abcdef"]`
Output: 16
Explanation: The two words can be "abcw", "xtfn".

Example 2:

Input: `words = ["a", "ab", "abc", "d", "cd", "bcd", "abcd"]`
Output: 4
Explanation: The two words can be "ab", "cd".

Example 3:

Input: `words = ["a", "aa", "aaa", "aaaa"]`
Output: 0
Explanation: No such pair of words.

Constraints:

- $2 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 1000$
- `words[i]` consists only of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxProduct(String[] words) {
        int n = words.length;
        int[] masks = new int[n];
        for(int i=0; i < n; i++)
            for(char ch: words[i].toCharArray())
                masks[i] |= 1 << (ch - 'a');

        int maxProd = 0;
        for(int i=0; i < n; i++)
            for(int j=i+1; j < n; j++)
                if((masks[i]&masks[j]) == 0)
                    maxProd = Math.max(maxProd, words[i].length() * words[j].length());

        return maxProd;
    }
}
```

[151 Reverse Words in a String \(link\)](#)

Description

Given an input string s , reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in s will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space*.

Note that s may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

```
Input: s = "the sky is blue"
Output: "blue is sky the"
```

Example 2:

```
Input: s = "  hello world  "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.
```

Example 3:

```
Input: s = "a good  example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.
```

Constraints:

- $1 \leq s.length \leq 10^4$
- s contains English letters (upper-case and lower-case), digits, and spaces ' '.
- There is **at least one** word in s .

Follow-up: If the string data type is mutable in your language, can you solve it **in-place** with $O(1)$ extra space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reverseWords(String s) {
        int resultPosition=0, length = s.length(), idx=length-1;
        char[] revArr = new char[length+1];

        while(idx > -1) {
            //trimming right for empty chars
            while(idx > -1 && s.charAt(idx) == ' ')
                idx--;

            int right = idx;

            //getting to next empty space from last non empty space
            while(idx > -1 && s.charAt(idx) != ' ')
                idx--;

            for(int left=idx+1; left <= right; left++) {
                revArr[resultPosition++] = s.charAt(left);
                if(left == right)
                    revArr[resultPosition++] = ' ';
            }
        }

        return resultPosition == 0? " ": new String(revArr, 0, resultPosition-1);
    }
}
```

94 Binary Tree Inorder Traversal [\(link\)](#)

Description

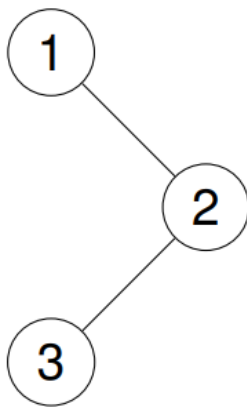
Given the root of a binary tree, return *the inorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

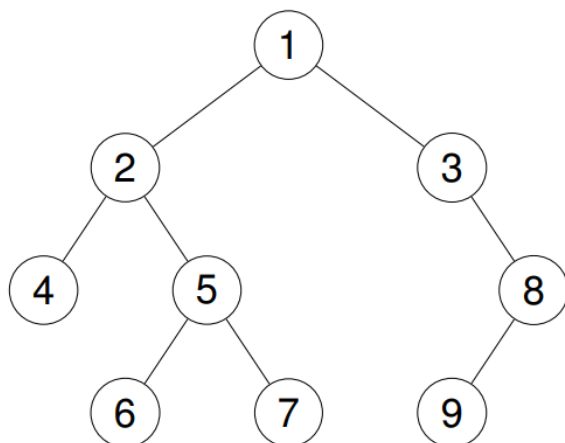


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        morrisTraversal(root, result::add);
        return result;
    }

    private void morrisTraversal(TreeNode node, Consumer<Integer> visitor) {
        while(node != null) {
            if(node.left == null) {
                visitor.accept(node.val);
                node = node.right;
            } else {
                TreeNode next = getPredecessor(node);
                if(next.right == null) {
                    next.right = node;
                    node = node.left;
                } else {
                    next.right = null;
                    visitor.accept(node.val);
                    node = node.right;
                }
            }
        }
    }

    private TreeNode getPredecessor(TreeNode node) {
        TreeNode curr = node.left;
        while(curr.right != null && curr.right != node) {
            curr = curr.right;
        }
        return curr;
    }
}
```


[232 Implement Queue using Stacks \(link\)](#)

Description

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input

```
["MyQueue", "push", "push", "peek", "pop", "empty"]  
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 1, 1, false]
```

Explanation

```
MyQueue myQueue = new MyQueue();  
myQueue.push(1); // queue is: [1]  
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)  
myQueue.peek(); // return 1  
myQueue.pop(); // return 1, queue is [2]  
myQueue.empty(); // return false
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is [amortized](#) $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MyQueue {
    Stack<Integer> stack;

    public MyQueue() {
        stack = new Stack<>();
    }

    public void push(int x) {
        stack.push(x);
    }

    public int pop() {
        int popped = stack.elementAt(0);
        stack.removeElementAt(0);
        return popped;
    }

    public int peek() {
        return stack.elementAt(0);
    }

    public boolean empty() {
        return stack.isEmpty();
    }
}

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = new MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * boolean param_4 = obj.empty();
 */
```

278 First Bad Version (link)

Description

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether `version` is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

```
Input: n = 5, bad = 4
Output: 4
Explanation:
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.
```

Example 2:

```
Input: n = 1, bad = 1
Output: 1
```

Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/* The isBadVersion API is defined in the parent class VersionControl.
   boolean isBadVersion(int version); */

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int lo=1;
        int hi=n;
        while(lo < hi) {
            int mid = lo+(-lo+hi)/2;
            if(isBadVersion(mid)) {
                hi=mid;
            } else {
                lo=mid+1;
            }
        }
        return lo;
    }
}
```

111 Minimum Depth of Binary Tree (link)

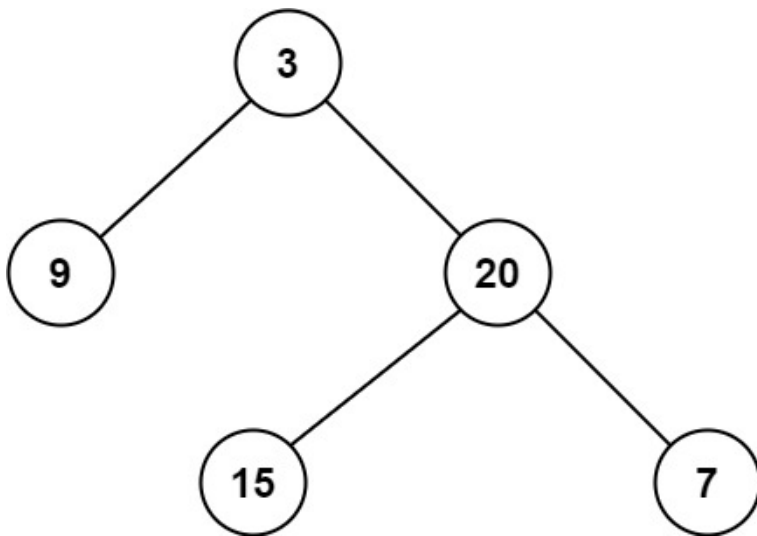
Description

Given a binary tree, find its minimum depth.

The minimum depth is the number of nodes along the shortest path from the root node down to the nearest leaf node.

Note: A leaf is a node with no children.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 2

Example 2:

Input: root = [2,null,3,null,4,null,5,null,6]
Output: 5

Constraints:

- The number of nodes in the tree is in the range $[0, 10^5]$.
- $-1000 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int minDepth(TreeNode root) {
        if(root==null) { return 0; }
        if(root.left==null) { return minDepth(root.right) +1; }
        if(root.right==null) { return minDepth(root.left)+1; }
        return Math.min(minDepth(root.left),minDepth(root.right))+1;
    }
}
```

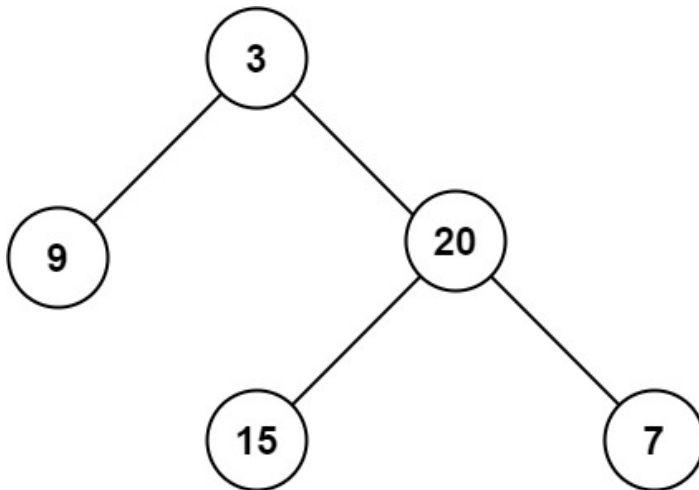
[104 Maximum Depth of Binary Tree \(link\)](#)

Description

Given the `root` of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: `root = [3,9,20,null,null,15,7]`
Output: 3

Example 2:

Input: `root = [1,null,2]`
Output: 2

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

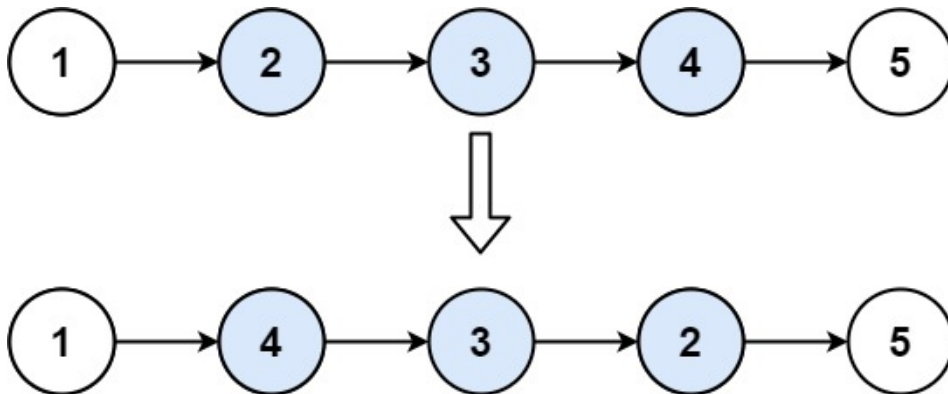
```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
    }
}
```


92 Reverse Linked List II (link)

Description

Given the head of a singly linked list and two integers `left` and `right` where $\text{left} \leq \text{right}$, reverse the nodes of the list from position `left` to position `right`, and return *the reversed list*.

Example 1:



Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]

Example 2:

Input: head = [5], left = 1, right = 1
Output: [5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq n \leq 500$
- $-500 \leq \text{Node.val} \leq 500$
- $1 \leq \text{left} \leq \text{right} \leq n$

Follow up: Could you do it in one pass?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        if(left == right)
            return head;
        int diff = right - left;

        ListNode dHead = new ListNode(0, head), start = dHead, end = dHead;

        //slow is to be positioned just before left
        for(int i=0; i<left-1; i++) {
            start = start.next;
            end = end.next;
        }

        //end is to be positioned just after right;
        for(int i=0; i <= diff+1; i++)
            end = end.next;

        head = start.next;
        ListNode prev = head, first = head, next;

        //reverse everything b/w start and end
        while(head != end) {
            next = head.next;
            head.next = prev;

            prev = head;
            head = next;
        }

        //first element of reversed list attached to start
        //first element before reversal attached to end
        start.next = prev;
        first.next = end;

        return dHead.next;
    }
}
```

70 Climbing Stairs (link)

Description

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int climbStairs(int n) {
        if (n == 1) {
            return 1;
        }

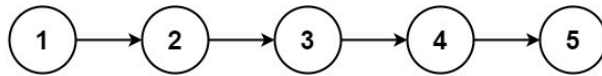
        int[] dp = new int[n+1];
        dp[0] = 1;
        dp[1] = 1;
        for(int i=2; i<=n;i++) {
            dp[i] = dp[i-1]+dp[i-2];
        }
        return dp[n];
    }
}
```

61 Rotate List (link)

Description

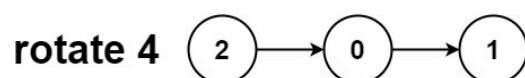
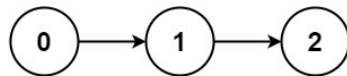
Given the head of a linked list, rotate the list to the right by k places.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4
Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range $[0, 500]$.
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 * 10^9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        // if k is not positive, or head is null or head is the only element
        // resultant list is same as head
        if (k < 1 || head == null || head.next == null)
            return head;

        ListNode tail = head;
        int n = 1;
        while (tail.next != null) {
            n++;
            tail = tail.next;
        }

        // to avoid unnecessary iterations if k >= n
        k %= n;
        if (k == 0) {
            return head;
        }

        //make it circle
        tail.next = head;
        k = n - k;
        while (k-- > 0)
            tail = tail.next;

        head = tail.next;
        tail.next = null;
        return head;
    }
}
```

54 Spiral Matrix (link)

Description

Given an $m \times n$ matrix, return *all elements of the matrix in spiral order*.

Example 1:

1 →	2 →	3
4 →	5	↓ 6
↑ 7	←	8 ←
		↓ 9

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]

Example 2:

1 →	2 →	3 →	4
5 →	6 →	7	↓ 8
↑ 9	←	10 ←	11 ←
			↓ 12

Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $-100 \leq \text{matrix}[i][j] \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> spiral = new ArrayList<>();
        int rows = matrix.length;
        int columns = matrix[0].length;
        int top=0, left=0, right=columns-1, bottom=rows-1;

        while(spiral.size() < rows*columns) {
            //left to right
            for(int col=left; col<=right; col++)
                spiral.add(matrix[top][col]);

            //go down
            for(int row=top+1; row <= bottom; row++)
                spiral.add(matrix[row][right]);

            //
            if(top != bottom) {
                //traverse from right to left
                for(int col=right-1; col>=left; col--)
                    spiral.add(matrix[bottom][col]);
            }

            if(left != right) {
                //traverse down to up
                for(int row=bottom-1; row > top; row--)
                    spiral.add(matrix[row][left]);
            }

            left++;
            right--;
            top++;
            bottom--;
        }
        return spiral;
    }
}
```

24 Swap Nodes in Pairs (link)

Description

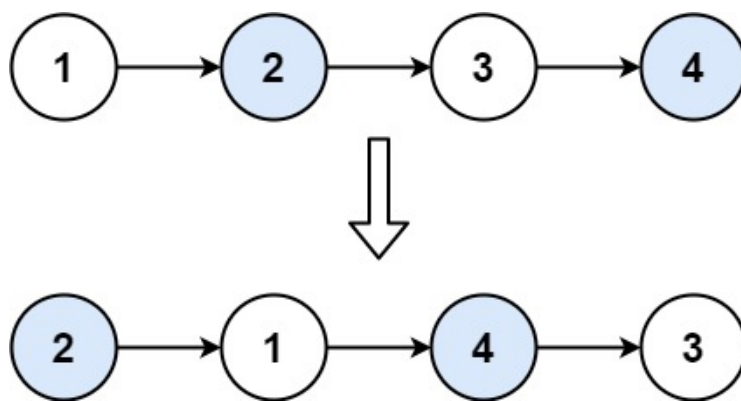
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range $[0, 100]$.
- $0 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode swapPairs(ListNode head) {
        if(head == null || head.next == null) {
            return head;
        }
        ListNode first = head, second = head.next;
        first.next = swapPairs(second.next);
        second.next = first;
        return second;
    }
}
```

[322 Coin Change \(link\)](#)

Description

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

```
Input: coins = [2], amount = 3
Output: -1
```

Example 3:

```
Input: coins = [1], amount = 0
Output: 0
```

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 2^{31} - 1$
- $0 \leq \text{amount} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        Queue<int[]> qu = new LinkedList<>();
        Set<Integer> seen = new HashSet<>();
        qu.offer(new int[]{amount, 0});
        while(!qu.isEmpty()) {
            //entry[0] => amount
            //entry[1] => number of coins for that amount
            int[] entry = qu.poll();
            if(entry[0] == 0)
                return entry[1];
            for(int coin:coins) {
                int newAmount = entry[0]-coin;
                if(newAmount>=0 && !seen.contains(newAmount)) {
                    qu.offer(new int[]{newAmount, entry[1]+1});
                    seen.add(newAmount);
                }
            }
        }
        return -1;
    }
}
```

[153 Find Minimum in Rotated Sorted Array \(link\)](#)

Description

Suppose an array of length n sorted in ascending order is **rotated** between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that **rotating** an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: 1

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: 0

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: 11

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5000$
- $-5000 \leq \text{nums}[i] \leq 5000$
- All the integers of `nums` are **unique**.
- `nums` is sorted and rotated between 1 and n times.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findMin(int[] nums) {
        int start=0, end = nums.length -1;
        while(start < end) {
            if(nums[start] < nums[end])
                break;

            int mid = (start+end)/2;
            if(nums[mid] < nums[end]) {
                end = mid;
            } else {
                start = mid+1;
            }
        }
        return nums[start];
    }
}
```

1648 Minimum Insertions to Balance a Parentheses String[\(link\)](#)

Description

Given a parentheses string s containing only the characters '(' and ')'. A parentheses string is **balanced** if:

- Any left parenthesis '(' must have a corresponding two consecutive right parenthesis '))'.
- Left parenthesis '(' must go before the corresponding two consecutive right parenthesis '))'.

In other words, we treat '(' as an opening parenthesis and '))' as a closing parenthesis.

- For example, "()", "()(())" and "(()())" are balanced, "())", "())" and "(()))" are not balanced.

You can insert the characters '(' and ')' at any position of the string to balance it if needed.

Return *the minimum number of insertions* needed to make s balanced.

Example 1:

Input: $s = "(()))"$

Output: 1

Explanation: The second '(' has two matching '))', but the first '(' has only ')' matching. We need to add one more ')' to match the first '('.

Example 2:

Input: $s = "()"$

Output: 0

Explanation: The string is already balanced.

Example 3:

Input: $s = "))(())"$

Output: 3

Explanation: Add '(' to match the first '))', Add '))' to match the last '('.

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists of '(' and ')' only.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minInsertions(String s) {
        char L = '(', R = ')';
        int leftCount=0, insertCount = 0;
        for(int i=0, length = s.length(); i < length; i++) {
            char ch = s.charAt(i);
            if(ch == L) {
                leftCount++;
            } else {
                //case of ))
                if(i+1 < length && s.charAt(i+1) == R) {
                    if(leftCount>0)
                        leftCount--;
                    else
                        insertCount++;
                    i++; //next char already processed
                } else { //case )(
                    insertCount++; //making )( )(
                    if(leftCount > 0)
                        leftCount--; // ()
                    else
                        insertCount++; // no left )( changed to ()
                }
            }
        }
        return insertCount+2*leftCount;
    }
}
```

791 Split BST (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode[] splitBST(TreeNode root, int target) {
        if(root == null) {
            return new TreeNode[2];
        } else if(root.val <= target) {
            TreeNode[] nodes = splitBST(root.right, target);
            root.right = nodes[0];
            nodes[0] = root;
            return nodes;
        } else {
            TreeNode[] nodes = splitBST(root.left, target);
            root.left = nodes[1];
            nodes[1] = root;
            return nodes;
        }
    }
}
```

698 Partition to K Equal Sum Subsets (link)

Description

Given an integer array `nums` and an integer `k`, return `true` if it is possible to divide this array into `k` non-empty subsets whose sums are all equal.

Example 1:

Input: `nums = [4,3,2,3,5,2,1]`, `k = 4`

Output: `true`

Explanation: It is possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with equal sums.

Example 2:

Input: `nums = [1,2,3,4]`, `k = 3`

Output: `false`

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 16$
- $1 \leq \text{nums}[i] \leq 10^4$
- The frequency of each element is in the range $[1, 4]$.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int n = nums.length;
        int total = 0;
        for(int num:nums)
            total += num;

        if(total % k != 0)
            return false;

        return backtrack(nums, 0, 0, 0, k, total/k, new boolean[n]);
    }

    boolean backtrack(int[] arr, int start, int count, int currentSum, int k, int target, boolean[] taken) {
        if(count == k-1)
            return true;

        if(currentSum > target)
            return false;

        if(currentSum == target)
            return backtrack(arr, 0, count+1, 0, k, target, taken);

        for(int j=start, limit = arr.length; j < limit; j++) {
            if(taken[j])
                continue;
            taken[j] = true;
            if(backtrack(arr, j+1, count, currentSum+arr[j], k, target, taken))
                return true;
            taken[j] = false;
        }
        return false;
    }
}
```

[269 Alien Dictionary \(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String alienOrder(String[] words) {
        Map<Character, List<Character>> adjList = new HashMap<>();
        Map<Character, Integer> incomingCounts = new HashMap<>();

        for(String word: words) {
            for(char c: word.toCharArray()) {
                incomingCounts.put(c, 0);
                adjList.put(c, new ArrayList<>());
            }
        }

        for(int i=0, length = words.length-1; i < length; i++) {
            String first = words[i];
            String second = words[i+1];

            if(first.length() > second.length() && first.startsWith(second))
                return "";

            int limit = Math.min(first.length(), second.length());
            for(int j=0; j < limit; j++) {
                if(first.charAt(j) != second.charAt(j)){
                    adjList.get(first.charAt(j)).add(second.charAt(j));
                    incomingCounts.put(second.charAt(j), incomingCounts.get(second.charAt(j))+1);
                    break;
                }
            }
        }

        StringBuilder sb = new StringBuilder();
        Queue<Character> qu = new LinkedList<>();
        for(Map.Entry<Character,Integer> entry: incomingCounts.entrySet())
            if(entry.getValue() != null && entry.getValue() == 0)
                qu.offer(entry.getKey());

        while(!qu.isEmpty()) {
            char ch = qu.poll();
            sb.append(ch);
            for(Character chr: adjList.get(ch)) {
                incomingCounts.put(chr, incomingCounts.get(chr)-1);
                if(incomingCounts.get(chr) == 0)
                    qu.offer(chr);
            }
        }

        if(sb.length() != incomingCounts.size())
            return "";
        return sb.toString();
    }
}
```

778 Reorganize String (link)

Description

Given a string s , rearrange the characters of s so that any two adjacent characters are not the same.

Return *any possible rearrangement of s* or return "" if not possible.

Example 1:

```
Input: s = "aab"  
Output: "aba"
```

Example 2:

```
Input: s = "aaab"  
Output: ""
```

Constraints:

- $1 \leq s.length \leq 500$
- s consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String reorganizeString(String s) {
        int max = (s.length()+1)/2;
        int[] cmap = new int[26];
        for(char ch: s.toCharArray()) {
            cmap[ch-'a']++;
            if(cmap[ch-'a'] > max)
                return "";
        }

        PriorityQueue<Character> qu = new PriorityQueue<>((a,b) -> (cmap[b-'a']-cmap[a-'a']));
        for(int i=0;i<cmap.length;i++) {
            if(cmap[i] != 0)
                qu.offer((char)(i+'a'));
        }

        StringBuilder sb = new StringBuilder();
        while(!qu.isEmpty()) {
            char first = qu.poll();
            if(sb.length() == 0 || first != sb.charAt(sb.length()-1)) {
                sb.append(first);
                if(--cmap[first-'a'] > 0)
                    qu.offer(first);
            } else {
                char second = qu.poll();
                sb.append(second);
                if(--cmap[second-'a'] > 0)
                    qu.offer(second);
                qu.offer(first);
            }
        }
        return sb.toString();
    }
}
```

621 Task Scheduler (link)

Description

You are given an array of CPU `tasks`, each labeled with a letter from A to Z, and a number `n`. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** `n` intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Example 1:

Input: `tasks = ["A","A","A","B","B","B"], n = 2`

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: `tasks = ["A","C","A","B","D","B"], n = 1`

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: `tasks = ["A","A","A", "B","B","B"], n = 3`

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- `1 <= tasks.length <= 104`
- `tasks[i]` is an uppercase English letter.
- `0 <= n <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] freqs = new int[26];
        for(char c: tasks)
            freqs[c-'A']++;

        Arrays.sort(freqs);
        int maxFreq = freqs[25];
        int idleTime = (maxFreq-1)*n;
        for(int i=freqs.length-2; i>-1 && idleTime > 0; --i) {
            idleTime -= Math.min(maxFreq-1, freqs[i]);
        }

        return tasks.length + Math.max(0, idleTime);
    }
}
```

131 Palindrome Partitioning (link)

Description

Given a string s , partition s such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of s* .

Example 1:

```
Input: s = "aab"  
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"  
Output: [["a"]]
```

Constraints:

- $1 \leq s.length \leq 16$
- s contains only lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<String>> partition(String s) {
        List<List<String>> result = new ArrayList<>();
        backtrack(s, 0, new Stack<>(), result);
        return result;
    }

    void backtrack(String s, int start, Stack<String> stack, List<List<String>> result) {
        if(start >= s.length())
            result.add(new ArrayList<>(stack));

        for(int end=start; end < s.length(); end++) {
            if(isPalindromic(s, start, end)) {
                stack.push(s.substring(start, end+1));
                backtrack(s, end+1, stack, result);
                stack.pop();
            }
        }
    }

    boolean isPalindromic(String s, int start, int end) {
        while(start < end) {
            if(s.charAt(start++) != s.charAt(end--))
                return false;
        }
        return true;
    }
}
```

654 Maximum Binary Tree (link)

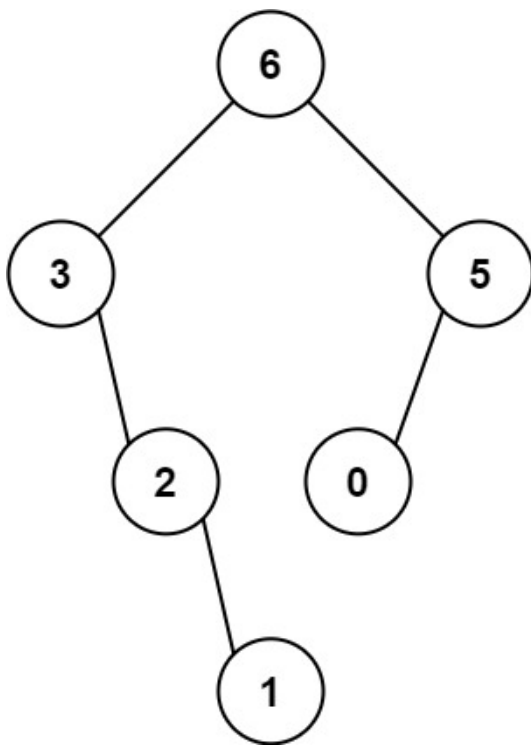
Description

You are given an integer array `nums` with no duplicates. A **maximum binary tree** can be built recursively from `nums` using the following algorithm:

1. Create a root node whose value is the maximum value in `nums`.
2. Recursively build the left subtree on the **subarray prefix** to the **left** of the maximum value.
3. Recursively build the right subtree on the **subarray suffix** to the **right** of the maximum value.

Return *the maximum binary tree built from* `nums`.

Example 1:



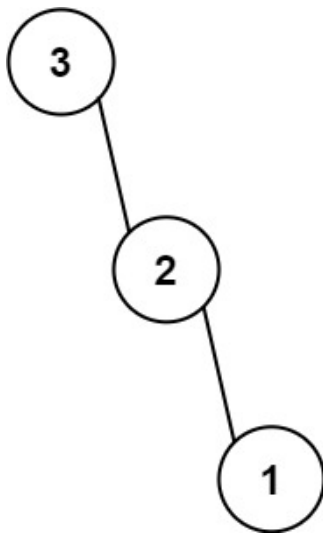
Input: `nums = [3,2,1,6,0,5]`

Output: `[6,3,5,null,2,0,null,null,1]`

Explanation: The recursive calls are as follow:

- The largest value in `[3,2,1,6,0,5]` is 6. Left prefix is `[3,2,1]` and right suffix is `[0,5]`.
 - The largest value in `[3,2,1]` is 3. Left prefix is `[]` and right suffix is `[2,1]`.
 - Empty array, so no child.
 - The largest value in `[2,1]` is 2. Left prefix is `[]` and right suffix is `[1]`.
 - Empty array, so no child.
 - Only one element, so child is a node with value 1.
 - The largest value in `[0,5]` is 5. Left prefix is `[0]` and right suffix is `[]`.
 - Only one element, so child is a node with value 0.
 - Empty array, so no child.

Example 2:



Input: nums = [3,2,1]

Output: [3,null,2,null,1]

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 1000$
- All integers in nums are **unique**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    Map<Integer, Integer> map = new HashMap<>();
    public TreeNode constructMaximumBinaryTree(int[] nums) {
        if(nums.length < 2)
            return new TreeNode(nums[0]);

        for(int i=0; i < nums.length; i++)
            map.put(nums[i], i);

        return buildMaxTree(nums, 0, nums.length);
    }

    TreeNode buildMaxTree(int[] nums, int start, int end) {
        if(start >= end)
            return null;

        int max=nums[start];
        for(int i=start+1; i < end; i++)
            max = Math.max(max, nums[i]);

        TreeNode root = new TreeNode(max);
        int maxIdx = map.get(max);
        map.remove(max);

        root.left = buildMaxTree(nums, start, maxIdx);
        root.right = buildMaxTree(nums, maxIdx+1, end);

        return root;
    }
}
```


662 Maximum Width of Binary Tree (link)

Description

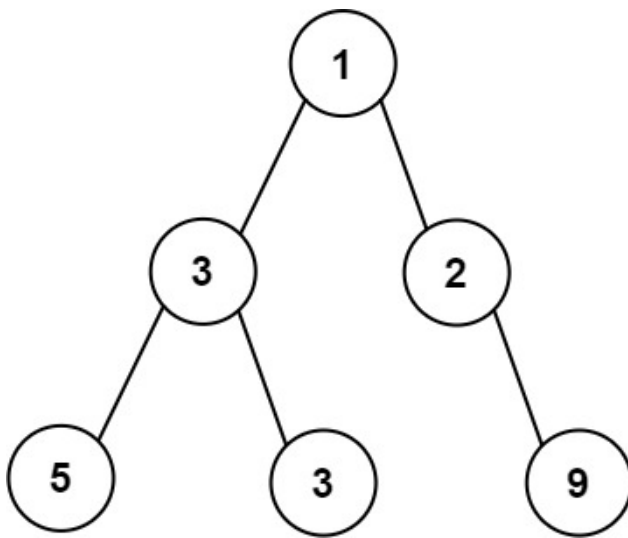
Given the root of a binary tree, return *the maximum width of the given tree*.

The **maximum width** of a tree is the maximum **width** among all levels.

The **width** of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is **guaranteed** that the answer will in the range of a **32-bit** signed integer.

Example 1:

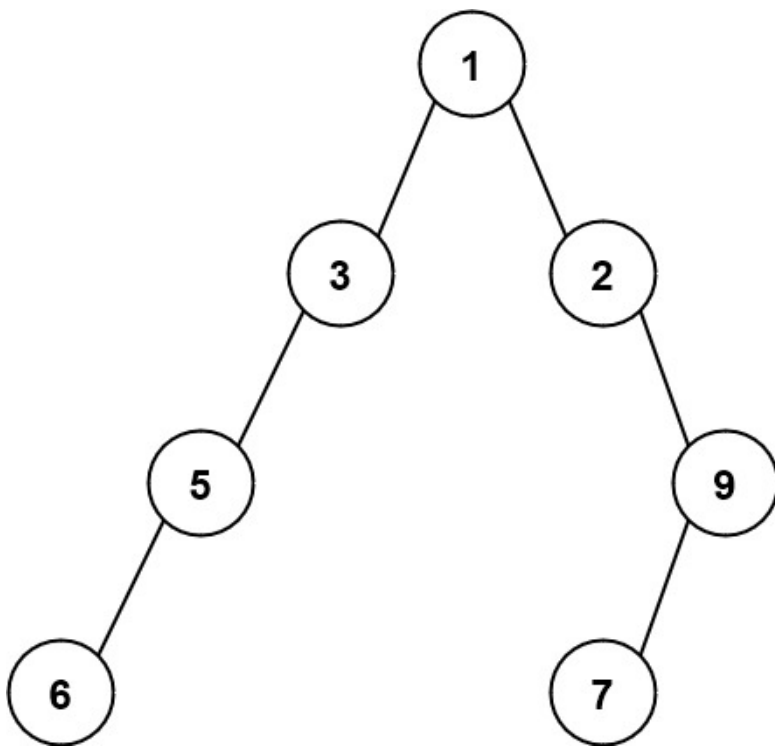


Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:

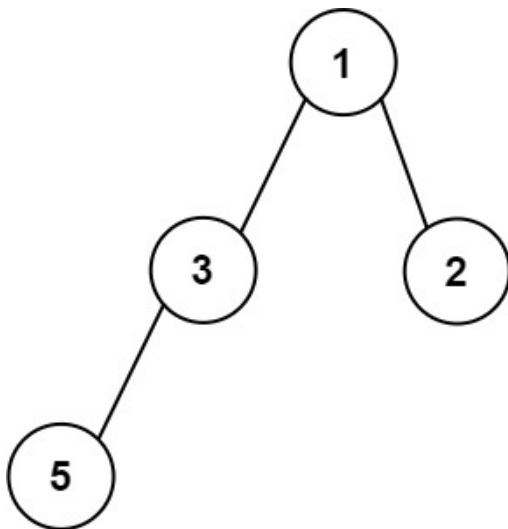


Input: root = [1,3,2,5,null,null,9,6,null,7]

Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).

Example 3:



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

Constraints:

- The number of nodes in the tree is in the range [1, 3000].
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int maxWidth = 0;
    public int widthOfBinaryTree(TreeNode root) {
        dfs(root, 0, 0, new HashMap<>());
        return this.maxWidth;
    }

    void dfs(TreeNode root, int depth, int endIndex, Map<Integer, Integer> map) {
        if(root == null)
            return;
        if(!map.containsKey(depth))
            map.put(depth, endIndex);
        int startIndex = map.get(depth);
        maxWidth = Math.max(maxWidth, endIndex - startIndex + 1);
        dfs(root.left, depth+1, 2*endIndex, map);
        dfs(root.right, depth+1, 2*endIndex+1, map);
    }
}
```

[421 Maximum XOR of Two Numbers in an Array \(link\)](#)

Description

Given an integer array `nums`, return *the maximum result of* `nums[i] XOR nums[j]`, where $0 \leq i \leq j < n$.

Example 1:

Input: `nums = [3,10,5,25,2,8]`

Output: 28

Explanation: The maximum result is 5 XOR 25 = 28.

Example 2:

Input: `nums = [14,70,53,83,49,91,36,80,92,51,66,70]`

Output: 127

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^5$
- $0 \leq \text{nums}[i] \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class TrieNode {
    public TrieNode[] next = new TrieNode[2];
    TrieNode() {}
}

class Solution {
    public int findMaximumXOR(int[] nums) {
        //get max bit length
        int n = nums.length;
        int maxNum = nums[0];
        for(int num : nums) maxNum = Math.max(maxNum, num);
        int maxLen = Integer.toBinaryString(maxNum).length();

        //get strings out of nums
        String[] strings = new String[n];
        Arrays.sort(nums);
        int bitmask = 1 << maxLen;
        for(int i=0; i < n; i++)
            strings[i] = Integer.toBinaryString(bitmask|nums[i]).substring(1);

        TrieNode root = new TrieNode();
        int result = 0;
        for(int i=0; i<n; i++) {
            TrieNode nd = root, xd = root;
            int currXOR = 0;
            for(char ch: strings[i].toCharArray()) {
                int bit = ch - '0';
                if(nd.next[bit] == null)
                    nd.next[bit] = new TrieNode();
                nd = nd.next[bit];

                if(xd.next[1-bit] == null) {
                    currXOR = currXOR << 1;
                    xd = xd.next[bit];
                } else {
                    currXOR = (currXOR << 1) | 1;
                    xd = xd.next[1-bit];
                }
                result = Math.max(result, currXOR);
            }
        }
        return result;
    }
}
```

673 Number of Longest Increasing Subsequence (link)

Description

Given an integer array `nums`, return *the number of longest increasing subsequences*.

Notice that the sequence has to be **strictly** increasing.

Example 1:

Input: `nums = [1,3,5,4,7]`

Output: 2

Explanation: The two longest increasing subsequences are `[1, 3, 4, 7]` and `[1, 3, 5, 7]`.

Example 2:

Input: `nums = [2,2,2,2,2]`

Output: 5

Explanation: The length of the longest increasing subsequence is 1, and there are 5 increasing subsequences of length 1.

Constraints:

- $1 \leq \text{nums.length} \leq 2000$
- $-10^6 \leq \text{nums}[i] \leq 10^6$
- The answer is guaranteed to fit inside a 32-bit integer.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findNumberOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        int[] counts = new int[n];
        int max = 0, result=0;

        for(int i=0; i < n; i++) {
            dp[i] = counts[i] = 1;
            for(int j=0; j < i; j++){
                if(nums[i] > nums[j]) {
                    if(dp[i] == 1+dp[j]) {
                        counts[i] += counts[j];
                    } else if(dp[i]<1+dp[j]) {
                        dp[i] = 1+dp[j];
                        counts[i] = counts[j];
                    }
                }
            }

            if (dp[i] > max) {
                result = counts[i];
                max = dp[i];
            } else if (dp[i] == max) {
                result += counts[i];
            }
        }
        return result;
    }
}
```


774 Maximum Depth of N-ary Tree (link)

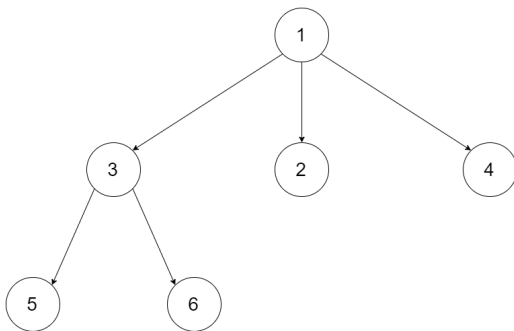
Description

Given a n-ary tree, find its maximum depth.

The maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

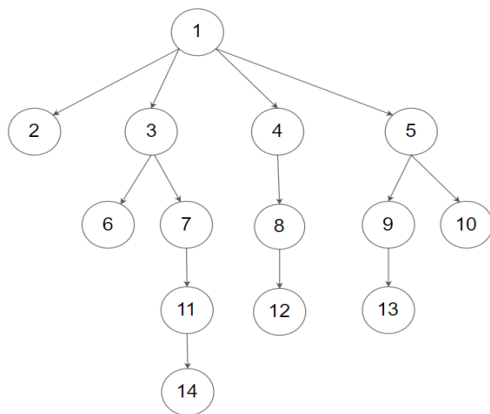
N-ary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

Example 1:



Input: root = [1,null,3,2,4,null,5,6]
Output: 3

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: 5

Constraints:

- The total number of nodes is in the range $[0, 10^4]$.
- The depth of the n-ary tree is less than or equal to 1000.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, List<Node> _children) {
        val = _val;
        children = _children;
    }
};
*/

class Solution {
    public int maxDepth(Node root) {
        if (root == null) return 0;
        int cur = 0;
        for (Node node : root.children) {
            cur = Math.max(cur, maxDepth(node));
        }
        return 1 + cur;
    }
}
```

666 Path Sum IV (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    int pathSum = 0;
    HashMap<Integer, Integer> map = new HashMap<>();
    public int pathSum(int[] nums) {
        //split dpth+pos and value in to key value pairs
        for(int num:nums)
            map.put(num/10,num%10);
        dfs(nums[0]/10, 0);
        return pathSum;
    }

    void dfs(int key, int currentSum) {
        if(!map.containsKey(key))
            return;

        currentSum += map.get(key);
        int depth=key/10, pos = key%10;
        int left = (depth+1)*10+2*pos-1;
        int right = left+1;

        if(!(map.containsKey(left) || map.containsKey(right))) {
            pathSum += currentSum;
        } else {
            dfs(left, currentSum);
            dfs(right, currentSum);
        }
    }
}
```

713 Subarray Product Less Than K (link)

Description

Given an array of integers `nums` and an integer `k`, return *the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than k*.

Example 1:

Input: `nums = [10,5,2,6]`, `k = 100`

Output: 8

Explanation: The 8 subarrays that have product less than 100 are:

[10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6]

Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.

Example 2:

Input: `nums = [1,2,3]`, `k = 0`

Output: 0

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $1 \leq \text{nums}[i] \leq 1000$
- $0 \leq k \leq 10^6$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int numSubarrayProductLessThanK(int[] nums, int k) {
        if(k<=1)
            return 0;
        int left=0, prod=1, count=0;
        for(int right=0; right<nums.length; right++) {
            prod *= nums[right];
            while(prod >= k)
                prod /= nums[left++];
            count += right-left+1;
        }
        return count;
    }
}
```

373 Find K Pairs with Smallest Sums (link)

Description

You are given two integer arrays `nums1` and `nums2` sorted in **non-decreasing order** and an integer `k`.

Define a pair (u, v) which consists of one element from the first array and one element from the second array.

Return *the k pairs $(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)$ with the smallest sums.*

Example 1:

Input: `nums1 = [1,7,11], nums2 = [2,4,6], k = 3`

Output: `[[1,2],[1,4],[1,6]]`

Explanation: The first 3 pairs are returned from the sequence: `[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6]`

Example 2:

Input: `nums1 = [1,1,2], nums2 = [1,2,3], k = 2`

Output: `[[1,1],[1,1]]`

Explanation: The first 2 pairs are returned from the sequence: `[1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3]`

Constraints:

- $1 \leq \text{nums1.length}, \text{nums2.length} \leq 10^5$
- $-10^9 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^9$
- `nums1` and `nums2` both are sorted in **non-decreasing order**.
- $1 \leq k \leq 10^4$
- $k \leq \text{nums1.length} * \text{nums2.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[0]+a[1]-b[0]-b[1]);
        List<List<Integer>> result = new ArrayList<>();
        if(nums1.length == 0 || nums2.length == 0 || k == 0)
            return result;

        int K = Math.min(nums1.length, k);
        for(int i=0; i < K; i++)
            pq.offer(new int[]{nums1[i], nums2[0],0});

        while(k-->0 && !pq.isEmpty()) {
            int[] minSum = pq.poll();
            result.add(List.of(minSum[0], minSum[1]));
            int num2Idx = minSum[2]+1;
            if(num2Idx == nums2.length)
                continue;
            pq.offer(new int[]{minSum[0], nums2[num2Idx], num2Idx});
        }
        return result;
    }
}
```

1075 Index Pairs of a String[\(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class TrieNode {
    public TrieNode[] next = new TrieNode[26];
    public boolean word;
}

class Solution {
    public int[][] indexPairs(String text, String[] words) {
        TrieNode root = new TrieNode();
        for(String word: words) {
            TrieNode child = root;
            for(char c: word.toCharArray()) {
                int i = c-'a';
                if(child.next[i] == null)
                    child.next[i] = new TrieNode();
                child = child.next[i];
            }
            child.word = true;
        }

        List<int[]> list = new ArrayList<>();
        int len = text.length();

        int start = 0, end=0;
        for (int i = 0; i < len; i++) {
            TrieNode cur = root;
            for (int j = i; j < len; j++) {
                cur = cur.next[text.charAt(j)-'a'];
                if (cur == null) break;
                if (cur.word) list.add(new int[] {i,j});
            }
        }
        return list.toArray(new int[0][]);
    }
}
```

[209 Minimum Size Subarray Sum \(link\)](#)

Description

Given an array of positive integers `nums` and a positive integer `target`, return *the minimal length of a subarray whose sum is greater than or equal to* `target`. If there is no such subarray, return `0` instead.

Example 1:

Input: `target = 7, nums = [2,3,1,2,4,3]`

Output: `2`

Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

Example 2:

Input: `target = 4, nums = [1,4,4]`

Output: `1`

Example 3:

Input: `target = 11, nums = [1,1,1,1,1,1,1,1]`

Output: `0`

Constraints:

- $1 \leq \text{target} \leq 10^9$
- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution of which the time complexity is $O(n \log(n))$.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minSubArrayLen(int target, int[] nums) {
        int minLength = Integer.MAX_VALUE;
        int lt=0, sum=0;
        for(int rt=0; rt<nums.length; rt++) {
            sum += nums[rt];
            while(sum >= target) {
                minLength = Math.min(minLength, rt-lt+1);
                sum -= nums[lt++];
            }
        }
        return minLength == Integer.MAX_VALUE?0:minLength;
    }
}
```

377 Combination Sum IV (link)

Description

Given an array of **distinct** integers `nums` and a target integer `target`, return *the number of possible combinations that add up to* `target`.

The test cases are generated so that the answer can fit in a **32-bit** integer.

Example 1:

```
Input: nums = [1,2,3], target = 4
Output: 7
Explanation:
The possible combination ways are:
(1, 1, 1, 1)
(1, 1, 2)
(1, 2, 1)
(1, 3)
(2, 1, 1)
(2, 2)
(3, 1)
Note that different sequences are counted as different combinations.
```

Example 2:

```
Input: nums = [9], target = 3
Output: 0
```

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 1000$
- All the elements of `nums` are **unique**.
- $1 \leq \text{target} \leq 1000$

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target+1];
        dp[0] = 1;

        for(int i=1; i <= target; i++) {
            for(int num: nums)
                if(i-num > -1)
                    dp[i] += dp[i-num];
        }
        return dp[target];
    }
}
```

1028 Interval List Intersections (link)

Description

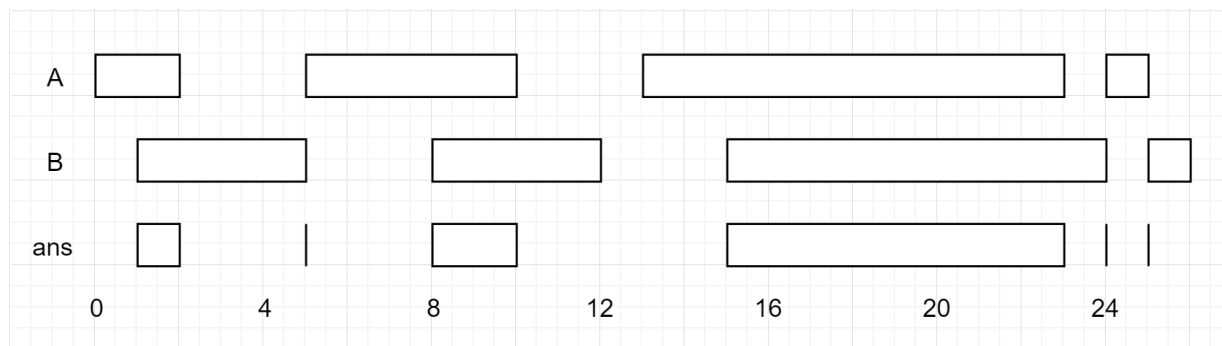
You are given two lists of closed intervals, `firstList` and `secondList`, where `firstList[i] = [starti, endi]` and `secondList[j] = [startj, endj]`. Each list of intervals is pairwise **disjoint** and in **sorted order**.

Return *the intersection of these two interval lists*.

A **closed interval** `[a, b]` (with $a \leq b$) denotes the set of real numbers x with $a \leq x \leq b$.

The **intersection** of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of `[1, 3]` and `[2, 4]` is `[2, 3]`.

Example 1:



Input: `firstList = [[0,2],[5,10],[13,23],[24,25]]`, `secondList = [[1,5],[8,12],[15,24],[25,26]]`
Output: `[[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]`

Example 2:

Input: `firstList = [[1,3],[5,9]]`, `secondList = []`
Output: `[]`

Constraints:

- $0 \leq \text{firstList.length}, \text{secondList.length} \leq 1000$
- $\text{firstList.length} + \text{secondList.length} \geq 1$
- $0 \leq \text{start}_i < \text{end}_i \leq 10^9$
- $\text{end}_i < \text{start}_{i+1}$
- $0 \leq \text{start}_j < \text{end}_j \leq 10^9$
- $\text{end}_j < \text{start}_{j+1}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] intervalIntersection(int[][] firstList, int[][] secondList) {
        int m=firstList.length,i=0;
        int n= secondList.length,j=0;

        //both are not empty per constraints
        if(m == 0 || n == 0)
            return new int[][]{};

        List<int[]> list = new ArrayList<>();
        while(i < m && j < n) {
            int lo = Math.max(firstList[i][0], secondList[j][0]);
            int hi = Math.min(firstList[i][1], secondList[j][1]);

            if(lo <= hi)
                list.add(new int[]{lo,hi});

            if(firstList[i][1] < secondList[j][1])
                i++;
            else
                j++;
        }

        return list.toArray(new int[0][]);
    }
}
```

940 Fruit Into Baskets (link)

Description

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the **type** of fruit the i^{th} tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

- You only have **two** baskets, and each basket can only hold a **single type** of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick **exactly one fruit** from **every** tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.
- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return *the **maximum** number of fruits you can pick*.

Example 1:

Input: `fruits = [1,2,1]`
Output: 3
Explanation: We can pick from all 3 trees.

Example 2:

Input: `fruits = [0,1,2,2]`
Output: 3
Explanation: We can pick from trees `[1,2,2]`.
If we had started at the first tree, we would only pick from trees `[0,1]`.

Example 3:

Input: `fruits = [1,2,3,2,2]`
Output: 4
Explanation: We can pick from trees `[2,3,2,2]`.
If we had started at the first tree, we would only pick from trees `[1,2]`.

Constraints:

- $1 \leq \text{fruits.length} \leq 10^5$
- $0 \leq \text{fruits}[i] < \text{fruits.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int totalFruit(int[] fruits) {
        int left = 0, maxLength = 0;
        Map<Integer, Integer> fMap = new HashMap<>(2);
        for (int right = 0; right < fruits.length; right++) {
            fMap.put(fruits[right], fMap.getOrDefault(fruits[right], 0) + 1);
            while (fMap.size() > 2) {
                fMap.put(fruits[left], fMap.get(fruits[left]) - 1);
                if (fMap.get(fruits[left]) == 0)
                    fMap.remove(fruits[left]);
                left++;
            }
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }
}
```

[162 Find Peak Element \(link\)](#)

Description

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2:

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- $\text{nums}[i] \neq \text{nums}[i + 1]$ for all valid i .

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findPeakElement(int[] nums) {
        int lo=0, hi=nums.length-1;
        while(lo < hi) {
            int mi = (lo+hi)>>1;
            if(nums[mi] > nums[mi+1])
                hi=mi;
            else
                lo=mi+1;
        }
        return hi;
    }
}
```

874 Backspace String Compare (link)

Description

Given two strings s and t , return `true` *if they are equal when both are typed into empty text editors*. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: $s = "ab\#c"$, $t = "ad\#c"$
Output: `true`
Explanation: Both s and t become `"ac"`.

Example 2:

Input: $s = "ab\#\#"$, $t = "c\#d\#"$
Output: `true`
Explanation: Both s and t become `""`.

Example 3:

Input: $s = "a\#c"$, $t = "b"$
Output: `false`
Explanation: s becomes `"c"` while t becomes `"b"`.

Constraints:

- $1 \leq s.length, t.length \leq 200$
- s and t only contain lowercase letters and '#' characters.

Follow up: Can you solve it in $O(n)$ time and $O(1)$ space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean backspaceCompare(String s, String t) {
        int i=s.length()-1;
        int j=t.length()-1;
        while(i>-1 || j > -1) {
            i = getNextChar(s, i);
            j = getNextChar(t, j);

            //i & j point to the last valid char
            if(i>-1 && j > -1 && s.charAt(i) != t.charAt(j))
                return false;
            //if one of them reaches the beginning while the other is not
            if((i>-1 && j<0) || (i<0 && j>-1))
                return false;

            i--;j--;
        }

        return true;
    }

    int getNextChar(String str, int k) {
        int backSpaces = 0;
        while(k>-1) {
            if(str.charAt(k) == '#') { backSpaces++; k--;}
            else if(backSpaces > 0) { backSpaces--; k--;}
            else break;
        }
        return k;
    }
}
```

2132 Convert 1D Array Into 2D Array (link)

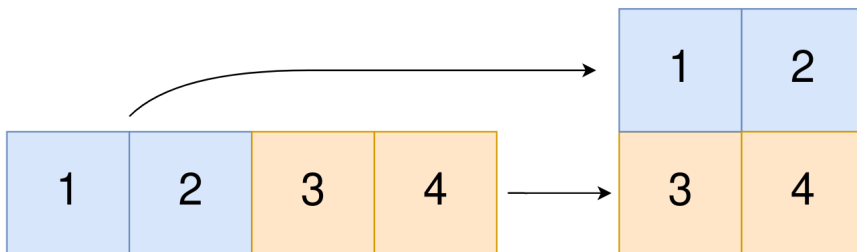
Description

You are given a **0-indexed** 1-dimensional (1D) integer array `original`, and two integers, `m` and `n`. You are tasked with creating a 2-dimensional (2D) array with `m` rows and `n` columns using **all** the elements from `original`.

The elements from indices `0` to `n - 1` (**inclusive**) of `original` should form the first row of the constructed 2D array, the elements from indices `n` to `2 * n - 1` (**inclusive**) should form the second row of the constructed 2D array, and so on.

Return an `m x n` 2D array constructed according to the above procedure, or an empty 2D array if it is impossible.

Example 1:



Input: `original = [1,2,3,4]`, `m = 2`, `n = 2`

Output: `[[1,2],[3,4]]`

Explanation: The constructed 2D array should contain 2 rows and 2 columns.

The first group of `n=2` elements in `original`, `[1,2]`, becomes the first row in the constructed 2D array.

The second group of `n=2` elements in `original`, `[3,4]`, becomes the second row in the constructed 2D array.

Example 2:

Input: `original = [1,2,3]`, `m = 1`, `n = 3`

Output: `[[1,2,3]]`

Explanation: The constructed 2D array should contain 1 row and 3 columns.

Put all three elements in `original` into the first row of the constructed 2D array.

Example 3:

Input: `original = [1,2]`, `m = 1`, `n = 1`

Output: `[]`

Explanation: There are 2 elements in `original`.

It is impossible to fit 2 elements in a `1x1` 2D array, so return an empty 2D array.

Constraints:

- `1 <= original.length <= 5 * 104`
- `1 <= original[i] <= 105`
- `1 <= m, n <= 4 * 104`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] construct2DArray(int[] original, int m, int n) {
        int length = original.length;
        if(length != m*n) {
            return new int[][]{};
        }
        int[][] result = new int[m][n];
        for(int i=0; i < length; i++) {
            result[i/n][i%n]=original[i];
        }
        return result;
    }
}
```

[442 Find All Duplicates in an Array \(link\)](#)

Description

Given an integer array `nums` of length `n` where all the integers of `nums` are in the range `[1, n]` and each integer appears **at most twice**, return *an array of all the integers that appears **twice***.

You must write an algorithm that runs in $O(n)$ time and uses only *constant* auxiliary space, excluding the space needed to store the output

Example 1:

```
Input: nums = [4,3,2,7,8,2,3,1]
Output: [2,3]
```

Example 2:

```
Input: nums = [1,1,2]
Output: [1]
```

Example 3:

```
Input: nums = [1]
Output: []
```

Constraints:

- `n == nums.length`
- `1 <= n <= 105`
- `1 <= nums[i] <= n`
- Each element in `nums` appears **once** or **twice**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findDuplicates(int[] nums) {
        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < nums.length; ++i) {
            int index = Math.abs(nums[i])-1;
            if (nums[index] < 0)
                res.add(Math.abs(index+1));
            nums[index] *= -1;
        }
        return res;
    }
}
```

[543 Diameter of Binary Tree \(link\)](#)

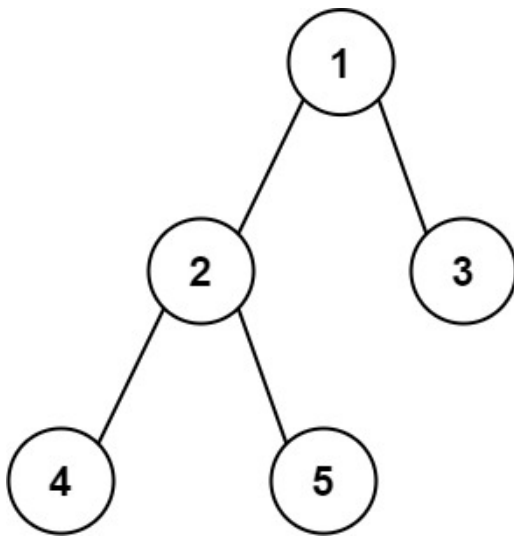
Description

Given the root of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    private int diameter;
    public int diameterOfBinaryTree(TreeNode root) {
        diameter=0;
        dfs(root);
        return diameter;
    }

    int dfs(TreeNode root) {
        if(root == null)
            return 0;
        int ltd = dfs(root.left);
        int rtd = dfs(root.right);

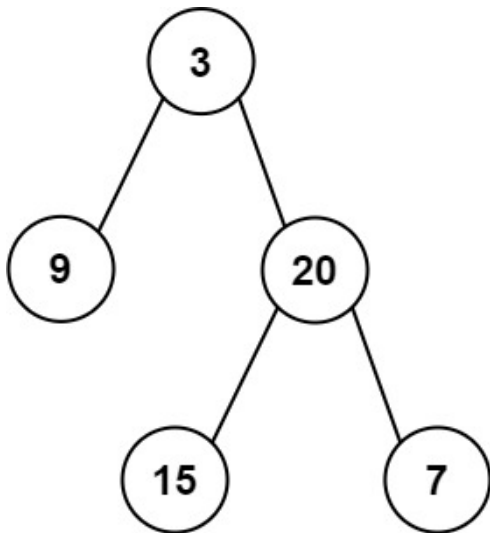
        diameter = Math.max(diameter, ltd+rtd);
        return Math.max(ltd, rtd)+1;
    }
}
```

637 Average of Levels in Binary Tree (link)

Description

Given the root of a binary tree, return *the average value of the nodes on each level in the form of an array*. Answers within 10^{-5} of the actual answer will be accepted.

Example 1:

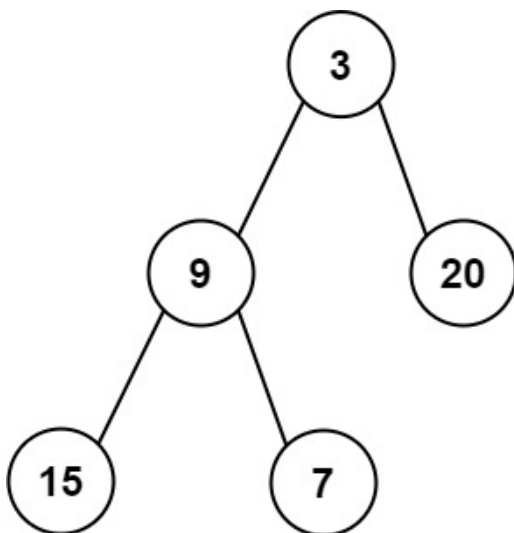


Input: root = [3,9,20,null,null,15,7]

Output: [3.00000,14.50000,11.00000]

Explanation: The average value of nodes on level 0 is 3, on level 1 is 14.5, and on level 2 is 11. Hence return [3, 14.5, 11].

Example 2:



Input: root = [3,9,20,15,7]

Output: [3.00000,14.50000,11.00000]

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public List<Double> averageOfLevels(TreeNode root) {
        List<Double> result = new ArrayList<>();
        if(root == null)
            return result;

        Queue<TreeNode> qu = new LinkedList<>();
        qu.add(root);

        while(!qu.isEmpty()) {
            int size = qu.size();
            int counter = size;
            long sum = 0;
            while(counter-- > 0) {
                TreeNode node = qu.poll();
                sum += node.val;
                if(node.left != null)
                    qu.offer(node.left);
                if(node.right != null)
                    qu.offer(node.right);
            }
            result.add((double)sum/size);
        }
        return result;
    }
}
```

[745 Find Smallest Letter Greater Than Target \(link\)](#)

Description

You are given an array of characters `letters` that is sorted in **non-decreasing order**, and a character `target`. There are **at least two different** characters in `letters`.

Return *the smallest character in `letters` that is lexicographically greater than `target`*. If such a character does not exist, return the first character in `letters`.

Example 1:

Input: `letters = ["c","f","j"], target = "a"`

Output: `"c"`

Explanation: The smallest character that is lexicographically greater than 'a' in letters is 'c'.

Example 2:

Input: `letters = ["c","f","j"], target = "c"`

Output: `"f"`

Explanation: The smallest character that is lexicographically greater than 'c' in letters is 'f'.

Example 3:

Input: `letters = ["x","x","y","y"], target = "z"`

Output: `"x"`

Explanation: There are no characters in letters that is lexicographically greater than 'z' so we return

Constraints:

- $2 \leq \text{letters.length} \leq 10^4$
- `letters[i]` is a lowercase English letter.
- `letters` is sorted in **non-decreasing** order.
- `letters` contains at least two different characters.
- `target` is a lowercase English letter.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public char nextGreatestLetter(char[] letters, char target) {  
        int lo = 0, hi = letters.length;  
        while (lo < hi) {  
            int mi = lo + (hi - lo) / 2;  
            if (letters[mi] <= target) lo = mi + 1;  
            else hi = mi;  
        }  
        return letters[lo % letters.length];  
    }  
}
```

212 Word Search II (link)

Description

Given an $m \times n$ board of characters and a list of strings `words`, return *all words on the board*.

Each word must be constructed from letters of sequentially adjacent cells, where **adjacent cells** are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board = [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words = ["oath","eat"]
Output: ["eat","oath"]

Example 2:

a	b
c	d

Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- `board[i][j]` is a lowercase English letter.

- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- `words[i]` consists of lowercase English letters.
- All the strings of `words` are unique.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class TrieNode {
    TrieNode[] next = new TrieNode[26];
    String word;
}
class Solution {
    private int rows;
    private int cols;
    private char[][] board;

    private static final int[][] DIRS = {{0,-1},{0,1}, {1,0}, {-1,0}};

    public List<String> findWords(char[][] board, String[] words) {
        this.rows = board.length;
        this.cols = board[0].length;
        this.board = board;
        TrieNode dict = buildTrie(words);

        List<String> list = new ArrayList<>();
        for(int i=0; i<rows; i++)
            for(int j=0; j<cols; j++)
                backtrack(i, j, dict, list);
        return list;
    }

    void backtrack(int row, int col, TrieNode root, List<String> result) {
        char c = board[row][col];
        if(c == '*' || root.next[c-'a'] == null)
            return;

        root = root.next[c-'a'];
        if(root.word != null) {
            result.add(root.word);
            root.word = null;
        }

        board[row][col] = '*';
        if(row > 0)
            backtrack(row-1, col, root, result);
        if(row < rows-1)
            backtrack(row+1, col, root, result);
        if(col > 0)
            backtrack(row, col-1, root, result);
        if(col < cols-1)
            backtrack(row, col+1, root, result);
        board[row][col] = c;
    }

    TrieNode buildTrie(String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) {
            TrieNode p = root;
            for (char c : w.toCharArray()) {
                int i = c - 'a';
                if (p.next[i] == null) p.next[i] = new TrieNode();
                p = p.next[i];
            }
            p.word=w;
        }
        return root;
    }
}
```

323 Number of Connected Components in an Undirected Graph (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class UnionFind {
    private int[] parent;

    public UnionFind(int n) {
        parent = new int[n];
        for(int i=0; i < n; i++) {
            parent[i] = i;
        }
    }

    public int find(int i) {
        while(parent[i] != i) {
            i = parent[i];
        }
        return i;
    }

    public boolean union(int i, int j) {
        int ri = find(i);
        int rj = find(j);

        if(ri == rj)
            return false;

        parent[rj]=ri;
        return true;
    }
}

class Solution {
    public int countComponents(int n, int[][] edges) {
        UnionFind unionFind = new UnionFind(n);
        for(int[] edge: edges) {
            unionFind.union(edge[0],edge[1]);
        }
        Set<Integer> set = new HashSet<>();
        for(int i=0; i<n;i++)
            set.add(unionFind.find(i));
        return set.size();
    }
}
```


261 Graph Valid Tree (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class UnionFind {
    private int[] parent;

    public UnionFind(int n) {
        parent = new int[n];
        for(int i=0; i < n; i++) {
            parent[i] = i;
        }
    }

    public int find(int i) {
        while(parent[i] != i) {
            i = parent[i];
        }
        return i;
    }

    public boolean union(int i, int j) {
        int ri = find(i);
        int rj = find(j);

        if(ri == rj)
            return false;

        parent[rj]=ri;
        return true;
    }
}

class Solution {
    public boolean validTree(int n, int[][] edges) {
        if(edges.length != n-1)
            return false;

        UnionFind unionFind = new UnionFind(n);
        for(int[] edge: edges) {
            if(!unionFind.union(edge[0], edge[1]))
                return false;
        }
        return true;
    }
}
```

73 Set Matrix Zeroes (link)

Description

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it [in place](#).

Example 1:

1	1	1
1	0	1
1	1	1

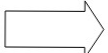


1	0	1
0	0	0
1	0	1

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0
3	4	5	2
1	3	1	5



0	0	0	0
0	4	5	0
0	3	1	0

Input: `matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]`
Output: `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`

Constraints:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-231 <= matrix[i][j] <= 231 - 1`

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public void setZeroes(int[][] matrix) {
        boolean firstRow = false;
        boolean firstCol = false;

        for(int i=0; i< matrix.length; i++) {
            if(matrix[i][0] == 0) {
                firstCol = true;
                break;
            }
        }

        for(int i=0; i< matrix[0].length; i++) {
            if(matrix[0][i] == 0) {
                firstRow = true;
                break;
            }
        }

        for(int i=1; i < matrix.length; i++) {
            for(int j=1; j < matrix[0].length; j++) {
                if(matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }

        for(int i=1; i < matrix.length; i++) {
            for(int j=1; j < matrix[0].length; j++) {
                if(matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }

        if(firstCol)
            for(int i=0; i < matrix.length; i++)
                matrix[i][0] = 0;

        if(firstRow)
            for(int i=0; i < matrix[0].length; i++)
                matrix[0][i] = 0;
    }
}
```

[55 Jump Game \(link\)](#)

Description

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return `true` *if you can reach the last index*, or `false` *otherwise*.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`

Output: `false`

Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $0 \leq \text{nums}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canJump(int[] nums) {
        int len = nums.length;
        int max = 0;
        for(int i=0; i<=max; i++){
            max = Math.max(max, i+nums[i]);
            if(max >= len-1) return true;
        }
        return false;
    }
}
```

[216 Combination Sum III \(link\)](#)

Description

Find all valid combinations of k numbers that sum up to n such that the following conditions are true:

- Only numbers 1 through 9 are used.
- Each number is used **at most once**.

Return *a list of all possible valid combinations*. The list must not contain the same combination twice, and the combinations may be returned in any order.

Example 1:

```
Input: k = 3, n = 7
Output: [[1,2,4]]
Explanation:
1 + 2 + 4 = 7
There are no other valid combinations.
```

Example 2:

```
Input: k = 3, n = 9
Output: [[1,2,6],[1,3,5],[2,3,4]]
Explanation:
1 + 2 + 6 = 9
1 + 3 + 5 = 9
2 + 3 + 4 = 9
There are no other valid combinations.
```

Example 3:

```
Input: k = 4, n = 1
Output: []
Explanation: There are no valid combinations.
Using 4 different numbers in the range [1,9], the smallest sum we can get is 1+2+3+4 = 10 and since 10 >
```

Constraints:

- $2 \leq k \leq 9$
- $1 \leq n \leq 60$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> combinationSum3(int k, int n) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(n, k, new LinkedList<>(), 0, result);
        return result;
    }

    private void backtrack(int n, int k, LinkedList<Integer> combination, int nextStart, List<List<Integer>> result) {
        if( n == 0 && combination.size() == k) {
            result.add(new ArrayList<>(combination));
        } else if(combination.size() == k || n < 0) {
            return;
        }

        for(int i=nextStart; i < 9; i++) {
            combination.add(i+1);
            backtrack(n-i-1, k, combination, i+1, result);
            combination.removeLast();
        }
    }
}
```


117 Populating Next Right Pointers in Each Node II (link)

Description

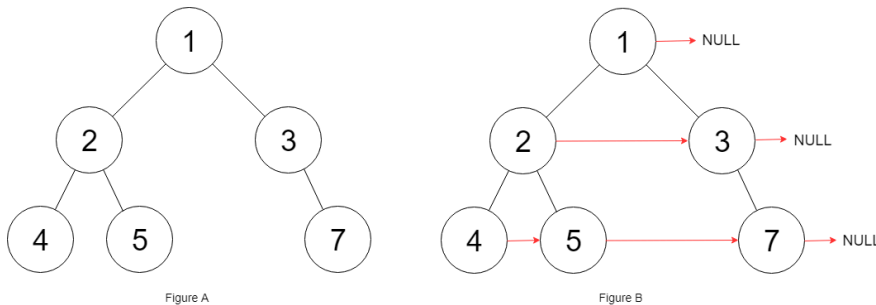
Given a binary tree

```
struct Node {  
    int val;  
    Node *left;  
    Node *right;  
    Node *next;  
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:



Input: root = [1,2,3,4,5,null,7]

Output: [1,#,2,3,#,4,5,7,#]

Explanation: Given the above binary tree (Figure A), your function should populate each next pointer to

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 6000]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node next;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, Node _left, Node _right, Node _next) {
        val = _val;
        left = _left;
        right = _right;
        next = _next;
    }
};
*/

class Solution {
    public Node connect(Node root) {
        if(root == null)
            return root;
        Queue<Node> queue = new ArrayDeque<>();
        queue.add(root);
        while(!queue.isEmpty()) {
            int size = queue.size();
            Node prev = null;
            while(size-- > 0) {
                Node temp = queue.poll();
                if(prev != null)
                    prev.next = temp;

                if(temp.left != null)
                    queue.add(temp.left);
                if(temp.right != null)
                    queue.add(temp.right);
                prev = temp;
            }
        }
        return root;
    }
}
```

[208 Implement Trie \(Prefix Tree\) \(link\)](#)

Description

A **trie** (pronounced as "try") or **prefix tree** is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- `Trie()` Initializes the trie object.
- `void insert(String word)` Inserts the string `word` into the trie.
- `boolean search(String word)` Returns `true` if the string `word` is in the trie (i.e., was inserted before), and `false` otherwise.
- `boolean startsWith(String prefix)` Returns `true` if there is a previously inserted string `word` that has the prefix `prefix`, and `false` otherwise.

Example 1:

Input

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]  
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

Output

```
[null, null, true, false, true, null, true]
```

Explanation

```
Trie trie = new Trie();  
trie.insert("apple");  
trie.search("apple"); // return True  
trie.search("app");   // return False  
trie.startsWith("app"); // return True  
trie.insert("app");  
trie.search("app");   // return True
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- `word` and `prefix` consist only of lowercase English letters.
- At most $3 \cdot 10^4$ calls **in total** will be made to `insert`, `search`, and `startsWith`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Trie {
    TrieNode root;
    public Trie() {
        this.root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode current = this.root;
        for(char chr: word.toCharArray()) {
            if(!current.containsKey(chr)) {
                current.put(chr, new TrieNode());
            }
            current = current.get(chr);
        }
        current.setWord();
    }

    public boolean search(String word) {
        TrieNode node = searchPrefix(word);
        return node!=null && node.isWord();
    }

    public boolean startsWith(String prefix) {
        TrieNode node = searchPrefix(prefix);
        return node!=null;
    }

    private TrieNode searchPrefix(String word) {
        TrieNode current = this.root;
        for(char chr: word.toCharArray()) {
            if(current.containsKey(chr))
                current = current.get(chr);
            else
                return null;
        }
        return current;
    }
}

class TrieNode {
    private TrieNode[] children;
    private boolean isWord;
    private final int SIZE=26;

    public TrieNode() {
        children = new TrieNode[SIZE];
    }

    public boolean containsKey(char ch) {
        return children[ch-'a'] != null;
    }

    public TrieNode get(char ch) {
        return children[ch-'a'];
    }

    public void put(char ch, TrieNode node) {
        children[ch-'a'] = node;
    }

    public void setWord() {
        isWord = true;
    }

    public boolean isWord() {
```

```
        return isWord;
    }
}

/**
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 */
```

768 Partition Labels (link)

Description

You are given a string *s*. We want to partition the string into as many parts as possible so that each letter appears in at most one part. For example, the string "ababcc" can be partitioned into ["abab", "cc"], but partitions such as ["aba", "bcc"] or ["ab", "ab", "cc"] are invalid.

Note that the partition is done so that after concatenating all the parts in order, the resultant string should be *s*.

Return *a list of integers representing the size of these parts*.

Example 1:

Input: *s* = "ababcbacdefegdehijklij"

Output: [9,7,8]

Explanation:

The partition is "ababcbaca", "defegde", "hijklij".

This is a partition so that each letter appears in at most one part.

A partition like "ababcbacdefegde", "hijklij" is incorrect, because it splits *s* into less parts.

Example 2:

Input: *s* = "eccbbbbdec"

Output: [10]

Constraints:

- $1 \leq s.length \leq 500$
- *s* consists of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> partitionLabels(String s) {
        int[] last = new int[26];
        for(int i=0; i < s.length();i++)
            last[s.charAt(i)-'a'] = i;

        int j=0, anchor=0;
        List<Integer> result = new ArrayList<>();
        for(int i=0; i < s.length();i++) {
            j = Math.max(j, last[s.charAt(i)-'a']);
            if(i == j) {
                result.add(i-anchor+1);
                anchor = i+1;
            }
        }
        return result;
    }
}
```

[451 Sort Characters By Frequency \(link\)](#)

Description

Given a string s , sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string.

Return *the sorted string*. If there are multiple answers, return *any of them*.

Example 1:

Input: $s = \text{"tree"}$

Output: "eert"

Explanation: 'e' appears twice while 'r' and 't' both appear once.

So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: $s = \text{"cccaaa"}$

Output: "aaaccc"

Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers.

Note that "cacaca" is incorrect, as the same characters must be together.

Example 3:

Input: $s = \text{"Aabb"}$

Output: "bbAa"

Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.

Note that 'A' and 'a' are treated as two different characters.

Constraints:

- $1 \leq s.length \leq 5 * 10^5$
- s consists of uppercase and lowercase English letters and digits.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String frequencySort(String s) {
        int[] cmap = new int[128];
        for(int i=0; i < s.length(); i++) {
            cmap[s.charAt(i)]++;
        }

        PriorityQueue<Integer> pq = new PriorityQueue<>((a,b)->cmap[b]-cmap[a]);
        for(int i=0; i < 128; i++)
            if(cmap[i] > 0)
                pq.offer(i);

        StringBuilder sb = new StringBuilder();
        while(!pq.isEmpty()) {
            int i = pq.poll();
            char ch = (char) i;
            while(cmap[i]-- > 0) {
                sb.append(ch);
            }
        }
        return sb.toString();
    }
}
```

1014 K Closest Points to Origin (link)

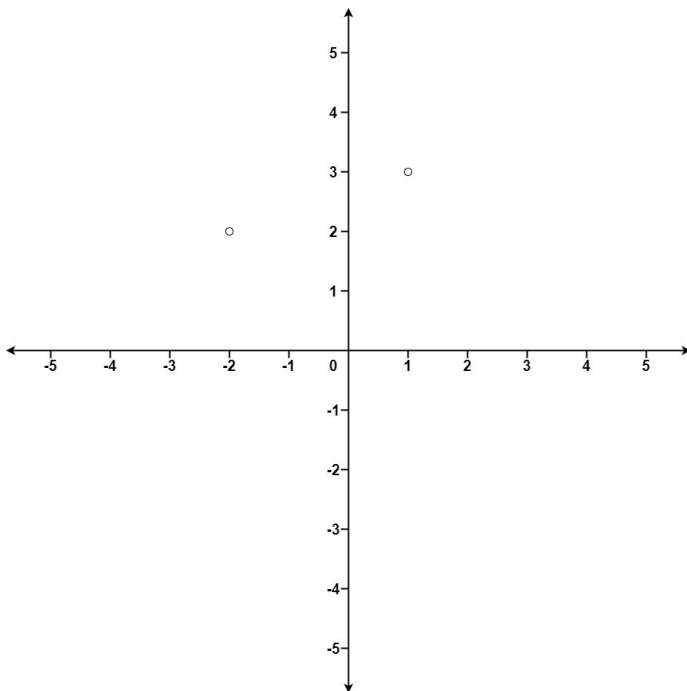
Description

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the **X-Y** plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the **X-Y** plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in **any order**. The answer is **guaranteed** to be **unique** (except for the order that it is in).

Example 1:



Input: `points = [[1,3],[-2,2]]`, `k = 1`

Output: `[[-2,2]]`

Explanation:

The distance between $(1, 3)$ and the origin is $\sqrt{10}$.

The distance between $(-2, 2)$ and the origin is $\sqrt{8}$.

Since $\sqrt{8} < \sqrt{10}$, $(-2, 2)$ is closer to the origin.

We only want the closest $k = 1$ points from the origin, so the answer is just `[[-2,2]]`.

Example 2:

Input: `points = [[3,3],[5,-1],[-2,4]]`, `k = 2`

Output: `[[3,3],[-2,4]]`

Explanation: The answer `[[-2,4],[3,3]]` would also be accepted.

Constraints:

- $1 \leq k \leq \text{points.length} \leq 10^4$

- $-10^4 \leq x_i, y_i \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
import java.util.AbstractMap.SimpleEntry;
class Solution {
    public int[][] kClosest(int[][] points, int k) {
        PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a,b)->(b[1]-a[1]));
        int n = points.length;
        for(int i=0; i<n; i++) {
            int[] entry = {i, getOriginDistance(points[i])};
            if(maxHeap.size()<k) {
                maxHeap.add(entry);
            } else if(entry[1] < maxHeap.peek()[1]) {
                maxHeap.poll();
                maxHeap.add(entry);
            }
        }
        int[][] result = new int[k][2];
        while(--k > -1)
            result[k]=points[maxHeap.poll()[0]];
        return result;
    }

    private int getOriginDistance(int[] arr) {
        return arr[0]*arr[0]+arr[1]*arr[1];
    }
}
```

[1039 Find the Town Judge \(link\)](#)

Description

In a town, there are n people labeled from 1 to n . There is a rumor that one of these people is secretly the town judge.

If the town judge exists, then:

1. The town judge trusts nobody.
2. Everybody (except for the town judge) trusts the town judge.
3. There is exactly one person that satisfies properties **1** and **2**.

You are given an array `trust` where `trust[i] = [ai, bi]` representing that the person labeled a_i trusts the person labeled b_i . If a trust relationship does not exist in `trust` array, then such a trust relationship does not exist.

Return *the label of the town judge if the town judge exists and can be identified, or return -1 otherwise.*

Example 1:

Input: $n = 2$, `trust = [[1,2]]`
Output: 2

Example 2:

Input: $n = 3$, `trust = [[1,3],[2,3]]`
Output: 3

Example 3:

Input: $n = 3$, `trust = [[1,3],[2,3],[3,1]]`
Output: -1

Constraints:

- $1 \leq n \leq 1000$
- $0 \leq \text{trust.length} \leq 10^4$
- `trust[i].length == 2`
- All the pairs of `trust` are **unique**.
- $a_i \neq b_i$
- $1 \leq a_i, b_i \leq n$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findJudge(int n, int[][] trusts) {
        if(trusts.length < n-1)
            return -1;

        int[] trustScores = new int[n+1];
        for(int[] trust: trusts) {
            trustScores[trust[0]]--;
            trustScores[trust[1]]++;
        }

        for(int i=1; i <=n; i++)
            if(trustScores[i]==n-1)
                return i;
        return -1;
    }
}
```

1661 Minimum Number of Vertices to Reach All Nodes

[\(link\)](#)

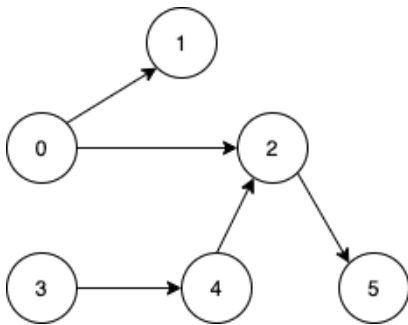
Description

Given a **directed acyclic graph**, with n vertices numbered from 0 to $n-1$, and an array `edges` where `edges[i] = [fromi, toi]` represents a directed edge from node `fromi` to node `toi`.

Find *the smallest set of vertices from which all nodes in the graph are reachable*. It's guaranteed that a unique solution exists.

Notice that you can return the vertices in any order.

Example 1:

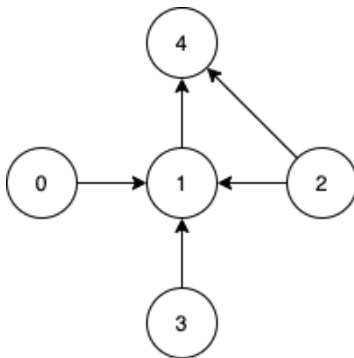


Input: $n = 6$, `edges = [[0,1],[0,2],[2,5],[3,4],[4,2]]`

Output: `[0,3]`

Explanation: It's not possible to reach all the nodes from a single vertex. From 0 we can reach [0,1,2,5]

Example 2:



Input: $n = 5$, `edges = [[0,1],[2,1],[3,1],[1,4],[2,4]]`

Output: `[0,2,3]`

Explanation: Notice that vertices 0, 3 and 2 are not reachable from any other node, so we must include them in the solution.

Constraints:

- $2 \leq n \leq 10^5$
- $1 \leq \text{edges.length} \leq \min(10^5, n * (n - 1) / 2)$

- `edges[i].length == 2`
- $0 \leq \text{from}_i, \text{to}_i < n$
- All pairs $(\text{from}_i, \text{to}_i)$ are distinct.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<Integer> findSmallestSetOfVertices(int n, List<List<Integer>> edges) {
        boolean[] visited = new boolean[n];
        List<Integer> vertices = new ArrayList<>();

        for(List<Integer> edge: edges)
            if(!visited[edge.get(1)])
                visited[edge.get(1)]=true;

        for(int i=0; i < n; i++)
            if(!visited[i])
                vertices.add(i);
        return vertices;
    }
}
```

11 Container With Most Water (link)

Description

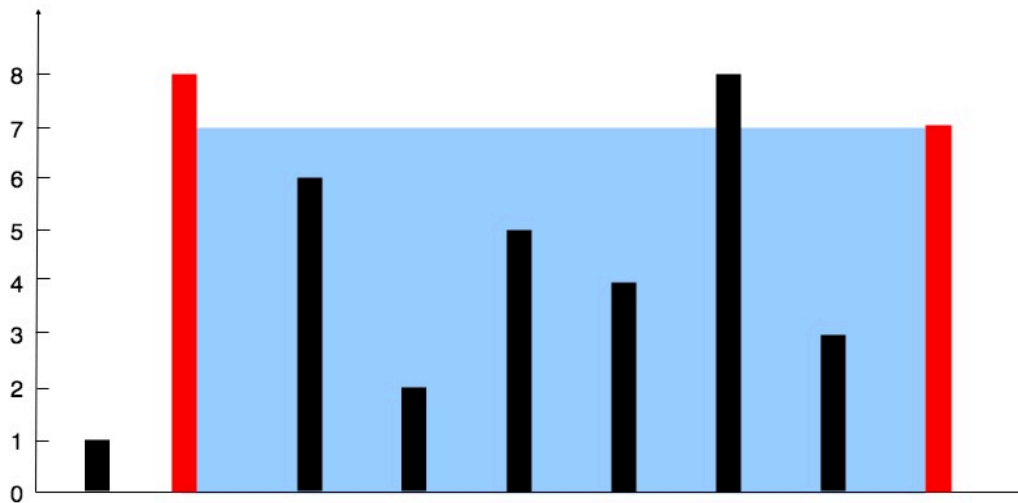
You are given an integer array `height` of length n . There are n vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

Notice that you may not slant the container.

Example 1:



Input: `height = [1,8,6,2,5,4,8,3,7]`

Output: 49

Explanation: The above vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. In this case, the maximum area is 49.

Example 2:

Input: `height = [1,1]`

Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 10^5$
- $0 \leq \text{height}[i] \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxArea(int[] height) {
        int maxArea = 0;
        int left = 0, right = height.length-1;
        while(left < right) {
            maxArea = Math.max(maxArea, (right-left)*(Math.min(height[right], height[left])));
            if(height[left] < height[right])
                left++;
            else
                right--;
        }
        return maxArea;
    }
}
```

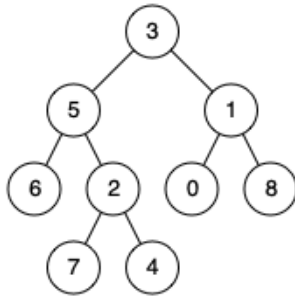
236 Lowest Common Ancestor of a Binary Tree (link)

Description

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in τ that has both p and q as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

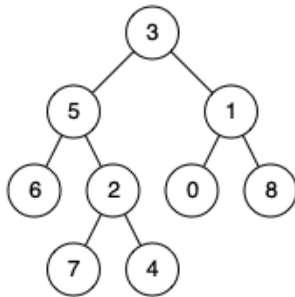


Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range $[2, 10^5]$.

- $-10^9 \leq \text{Node.val} \leq 10^9$
- All `Node.val` are **unique**.
- $p \neq q$
- p and q will exist in the tree.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root == null || root.val==p.val || root.val == q.val)
            return root;
        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);

        if(left != null && right != null)
            return root;
        else if(left == null)
            return right;
        else
            return left;
    }
}
```

334 Increasing Triplet Subsequence (link)

Description

Given an integer array `nums`, return `true` *if there exists a triple of indices* (i, j, k) *such that* $i < j < k$ *and* $nums[i] < nums[j] < nums[k]$. If no such indices exists, return `false`.

Example 1:

Input: `nums = [1,2,3,4,5]`
Output: `true`
Explanation: Any triplet where $i < j < k$ is valid.

Example 2:

Input: `nums = [5,4,3,2,1]`
Output: `false`
Explanation: No triplet exists.

Example 3:

Input: `nums = [2,1,5,0,4,6]`
Output: `true`
Explanation: The triplet $(3, 4, 5)$ is valid because $nums[3] == 0 < nums[4] == 4 < nums[5] == 6$.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^5$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Could you implement a solution that runs in $O(n)$ time complexity and $O(1)$ space complexity?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean increasingTriplet(int[] nums) {
        int N = nums.length;
        int first=Integer.MAX_VALUE, second=Integer.MAX_VALUE;
        for(int num: nums) {
            if(num < first)
                first = num;
            else if(first < num && num < second)
                second = num;
            else if(num > first && num > second)
                return true;
        }
        return false;
    }
}
```


[716 Max Stack \(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MaxStack {
    private Stack<Integer> stack;
    private Stack<int[]> maxStack;

    public MaxStack() {
        stack = new Stack<>();
        maxStack = new Stack<>();
    }

    public void push(int x) {
        stack.push(x);
        if(maxStack.isEmpty() || maxStack.peek()[0] < x )
            maxStack.push(new int[]{x, 1});
        else if(maxStack.peek()[0] == x)
            maxStack.peek()[1]++;
    }

    public int pop() {
        int x = stack.pop();
        if(maxStack.peek()[0] == x)
            maxStack.peek()[1]--;
        if(maxStack.peek()[1]==0)
            maxStack.pop();
        return x;
    }

    public int top() {
        return stack.peek();
    }

    public int peekMax() {
        return maxStack.peek()[0];
    }

    public int popMax() {
        int x = maxStack.peek()[0];
        Stack<Integer> tempStack = new Stack<>();
        while(top() != x) {
            tempStack.push(pop());
        }
        pop();
        while(!tempStack.isEmpty()) {
            push(tempStack.pop());
        }
        return x;
    }
}

/**
 * Your MaxStack object will be instantiated and called as such:
 * MaxStack obj = new MaxStack();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.top();
 * int param_4 = obj.peekMax();
 * int param_5 = obj.popMax();
 */
```

116 Populating Next Right Pointers in Each Node (link)

Description

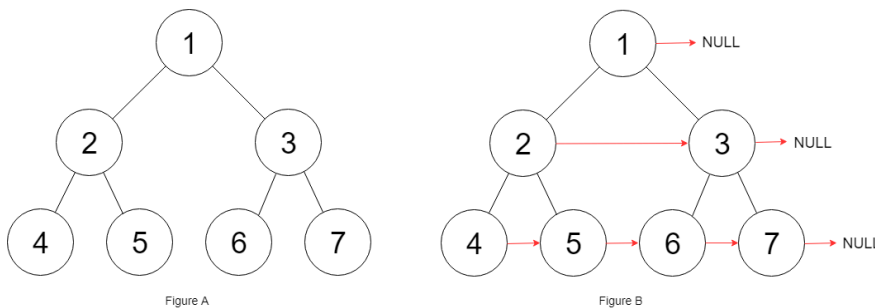
You are given a **perfect binary tree** where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {  
    int val;  
    Node *left;  
    Node *right;  
    Node *next;  
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to `NULL`.

Initially, all next pointers are set to `NULL`.

Example 1:



Input: root = [1,2,3,4,5,6,7]

Output: [1,#,2,3,#,4,5,6,7,#]

Explanation: Given the above perfect binary tree (Figure A), your function should populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to `NULL`.

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2^{12} - 1]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node next;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, Node _left, Node _right, Node _next) {
        val = _val;
        left = _left;
        right = _right;
        next = _next;
    }
};
*/

class Solution {
    public Node connect(Node root) {
        if(root == null)
            return root;
        Queue<Node> queue = new ArrayDeque<>();
        queue.add(root);
        while(!queue.isEmpty()) {
            int size = queue.size();
            Node prev = null;
            while(size-- > 0) {
                Node temp = queue.poll();
                if(prev != null)
                    prev.next = temp;

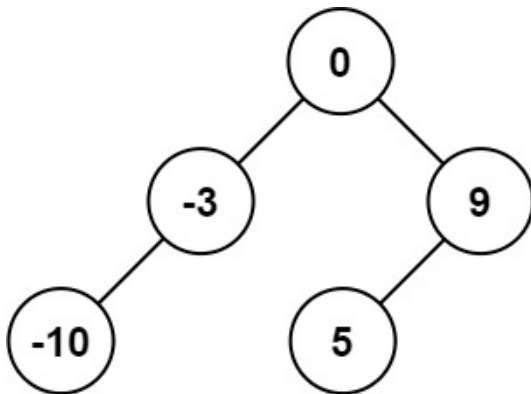
                if(temp.left != null)
                    queue.add(temp.left);
                if(temp.right != null)
                    queue.add(temp.right);
                prev = temp;
            }
        }
        return root;
    }
}
```

[108 Convert Sorted Array to Binary Search Tree \(link\)](#)

Description

Given an integer array `nums` where the elements are sorted in **ascending order**, convert it to a **height-balanced** binary search tree.

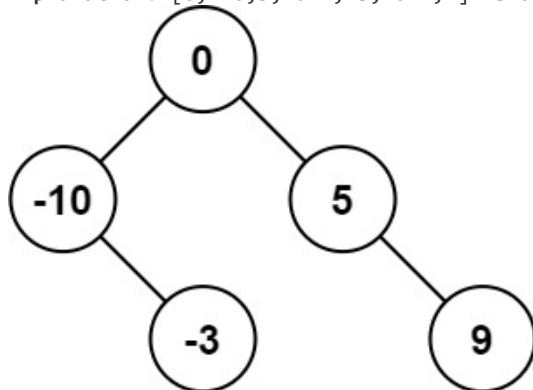
Example 1:



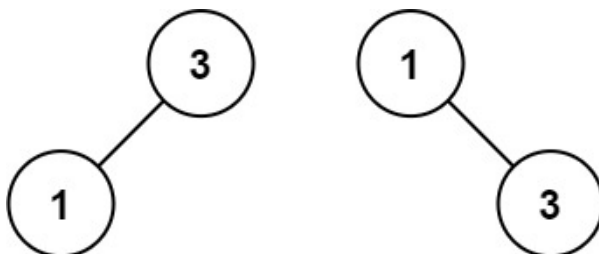
Input: `nums = [-10,-3,0,5,9]`

Output: `[0,-3,9,-10,null,5]`

Explanation: `[0,-10,5,null,-3,null,9]` is also accepted:



Example 2:



Input: `nums = [1,3]`

Output: `[3,1]`

Explanation: `[1,null,3]` and `[3,1]` are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in a **strictly increasing** order.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode sortedArrayToBST(int[] nums) {
        return sortedArrayToBST(nums, 0, nums.length - 1);
    }

    private TreeNode sortedArrayToBST(int[] nums, int start, int end) {
        if(start > end)
            return null;
        if(start == end)
            return new TreeNode(nums[start]);

        int mid = (start+end)>>1;
        TreeNode root = new TreeNode(nums[mid]);
        root.left = sortedArrayToBST(nums, start, mid-1);
        root.right = sortedArrayToBST(nums, mid+1, end);
        return root;
    }
}
```


[409 Longest Palindrome \(link\)](#)

Description

Given a string `s` which consists of lowercase or uppercase letters, return the length of the **longest palindrome** that can be built with those letters.

Letters are **case sensitive**, for example, "Aa" is not considered a palindrome.

Example 1:

Input: `s = "abcccd"`

Output: 7

Explanation: One longest palindrome that can be built is "dcccdd", whose length is 7.

Example 2:

Input: `s = "a"`

Output: 1

Explanation: The longest palindrome that can be built is "a", whose length is 1.

Constraints:

- `1 <= s.length <= 2000`
- `s` consists of lowercase **and/or** uppercase English letters only.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int longestPalindrome(String s) {
        int[] cmap = new int[128];
        int length=0;
        for(char c: s.toCharArray()) {
            if(++cmap[c] == 2) {
                cmap[c] = 0;
                length += 2;
            }
        }
        return length == s.length()?length:length+1;
    }
}
```

1780 Lowest Common Ancestor of a Binary Tree II (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root == null || p == null || q == null)
            return null;
        TreeNode result = lca(root, p, q);
        if(result == p) {
            return lca(p, q, q) != null? result:null;
        } else if(result == q) {
            return lca(q, p, p) != null? result:null;
        }
        return result;
    }

    private TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
        if(root == null || root.val==p.val || root.val== q.val)
            return root;
        TreeNode left = lca(root.left, p, q);
        TreeNode right = lca(root.right, p, q);
        return left == null? right: right == null? left:root;
    }
}
```

838 Design Linked List (link)

Description

Design your implementation of the linked list. You can choose to use a singly or doubly linked list.

A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node.

If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are **0-indexed**.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the index^{th} node in the linked list. If the index is invalid, return `-1`.
- `void addAtHead(int val)` Add a node of value `val` before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- `void addAtTail(int val)` Append a node of value `val` as the last element of the linked list.
- `void addAtIndex(int index, int val)` Add a node of value `val` before the index^{th} node in the linked list. If `index` equals the length of the linked list, the node will be appended to the end of the linked list. If `index` is greater than the length, the node **will not be inserted**.
- `void deleteAtIndex(int index)` Delete the index^{th} node in the linked list, if the index is valid.

Example 1:

Input

```
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get", "deleteAtIndex", "get"]  
[[], [1], [3], [1, 2], [1], [1], [1]]
```

Output

```
[null, null, null, null, 2, null, 3]
```

Explanation

```
MyLinkedList myLinkedList = new MyLinkedList();  
myLinkedList.addAtHead(1);  
myLinkedList.addAtTail(3);  
myLinkedList.addAtIndex(1, 2);    // linked list becomes 1->2->3  
myLinkedList.get(1);              // return 2  
myLinkedList.deleteAtIndex(1);    // now the linked list is 1->3  
myLinkedList.get(1);              // return 3
```

Constraints:

- $0 \leq \text{index}, \text{val} \leq 1000$
- Please do not use the built-in `LinkedList` library.
- At most 2000 calls will be made to `get`, `addAtHead`, `addAtTail`, `addAtIndex` and `deleteAtIndex`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MyLinkedList {
    Node head,tail;
    int size;
    public MyLinkedList() {
        head = null;
        tail = null;
        size = 0;
    }

    public int get(int index) {
        if(index < 0 || index >= size)
            return -1;
        Node node = head;
        for(int i=0; i < index; i++)
            node = node.next;
        return node.val;
    }

    public void addAtHead(int val) {
        this.head = new Node(val, head);
        size++;
    }

    public void addAtTail(int val) {
        if(head == null) {
            head = new Node(val);
        } else {
            Node node = head;
            for(int i=0; i < size-1; i++)
                node=node.next;
            node.next = new Node(val);
        }
        ++size;
    }

    public void addAtIndex(int index, int val) {
        if(index > size)
            return;

        if(index == 0) {
            addAtHead(val);
        } else if(index == size){
            addAtTail(val);
        } else {
            Node node = head;
            for(int i=0; i < index-1; i++)
                node = node.next;
            node.next = new Node(val, node.next);
            ++size;
        }
    }

    public void deleteAtIndex(int index) {
        if(index < 0 || size <= index)
            return;

        if(index == 0) {
            head = head.next;
            size--;
        } else {
            Node node = head;
            for(int i=0; i < index-1; i++)
                node=node.next;
            node.next = node.next.next;
            size--;
        }
    }
}
```

```

    }
}

class Node {
    public int val;
    public Node next;

    public Node() {
    }

    public Node(int d) {
        val = d;
    }
    public Node(int d, Node next) {
        val = d;
        this.next = next;
    }
}

/**
 * Your MyLinkedList object will be instantiated and called as such:
 * MyLinkedList obj = new MyLinkedList();
 * int param_1 = obj.get(index);
 * obj.addAtHead(val);
 * obj.addAtTail(val);
 * obj.addAtIndex(index,val);
 * obj.deleteAtIndex(index);
 */

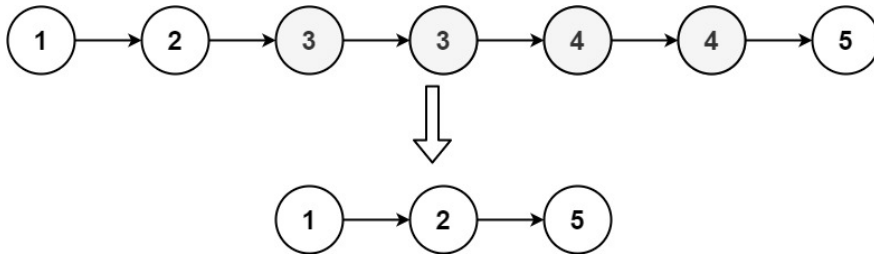
```

82 Remove Duplicates from Sorted List II (link)

Description

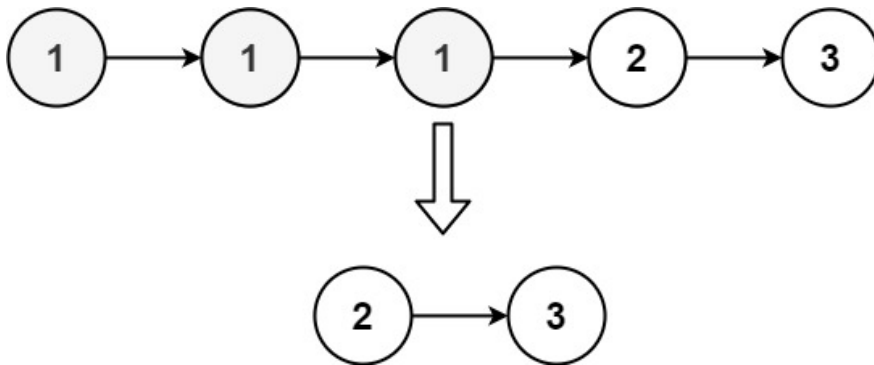
Given the head of a sorted linked list, *delete all nodes that have duplicate numbers, leaving only distinct numbers from the original list*. Return the linked list **sorted** as well.

Example 1:



Input: head = [1,2,3,3,4,4,5]
Output: [1,2,5]

Example 2:



Input: head = [1,1,1,2,3]
Output: [2,3]

Constraints:

- The number of nodes in the list is in the range $[0, 300]$.
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be **sorted** in ascending order.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        if(head == null) {
            return head;
        }

        ListNode dHead = new ListNode(1, head), prev = dHead;
        while(head != null) {
            if(head.next != null && head.val == head.next.val) {
                while(head.next != null && head.val == head.next.val)
                    head = head.next;
                prev.next = head.next;
            } else {
                prev = prev.next;
            }

            head = head.next;
        }

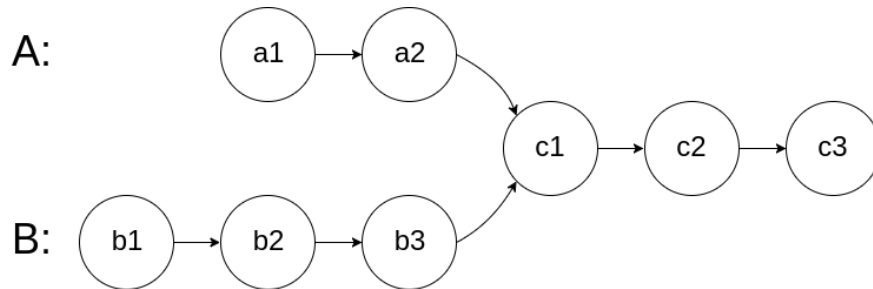
        return dHead.next;
    }
}
```

160 Intersection of Two Linked Lists (link)

Description

Given the heads of two singly linked-lists `headA` and `headB`, return *the node at which the two lists intersect*. If the two linked lists have no intersection at all, return `null`.

For example, the following two linked lists begin to intersect at node `c1`:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must **retain their original structure** after the function returns.

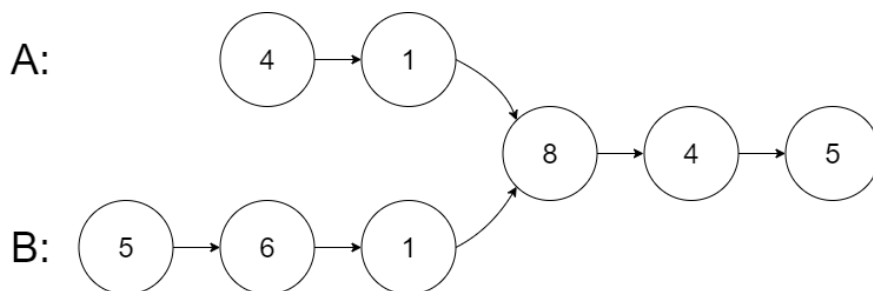
Custom Judge:

The inputs to the **judge** are given as follows (your program is **not** given these inputs):

- `intersectVal` - The value of the node where the intersection occurs. This is `0` if there is no intersected node.
- `listA` - The first linked list.
- `listB` - The second linked list.
- `skipA` - The number of nodes to skip ahead in `listA` (starting from the head) to get to the intersected node.
- `skipB` - The number of nodes to skip ahead in `listB` (starting from the head) to get to the intersected node.

The judge will then create the linked structure based on these inputs and pass the two heads, `headA` and `headB` to your program. If you correctly return the intersected node, then your solution will be **accepted**.

Example 1:



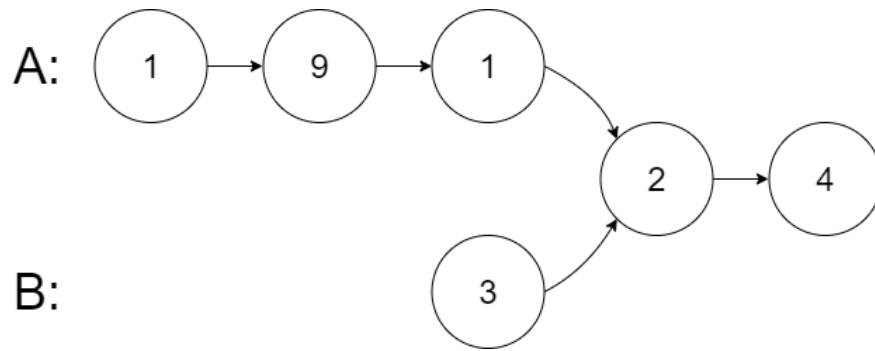
Input: `intersectVal = 8`, `listA = [4,1,8,4,5]`, `listB = [5,6,1,8,4,5]`, `skipA = 2`, `skipB = 3`

Output: Intersected at '8'

Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect). From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersection in A and 3 nodes before the intersection in B.

- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are not the same node.

Example 2:

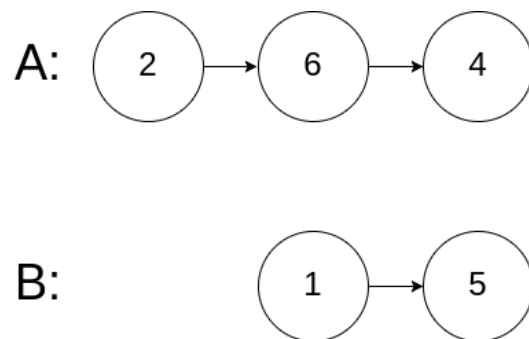


Input: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1

Output: Intersected at '2'

Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect). From the head of A, it reads as [1,9,1,2,4]. From the head of B, it reads as [3,2,4]. There are 3 nodes

Example 3:



Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2

Output: No intersection

Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5]. Since the two lists do not intersect, return null.

Constraints:

- The number of nodes of listA is in the m.
- The number of nodes of listB is in the n.
- $1 \leq m, n \leq 3 \times 10^4$
- $1 \leq \text{Node.val} \leq 10^5$
- $0 \leq \text{skipA} \leq m$
- $0 \leq \text{skipB} \leq n$
- intersectVal is 0 if listA and listB do not intersect.
- $\text{intersectVal} == \text{listA}[\text{skipA}] == \text{listB}[\text{skipB}]$ if listA and listB intersect.

Follow up: Could you write a solution that runs in $O(m + n)$ time and use only $O(1)$ memory?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {

        ListNode a = headA;
        ListNode b = headB;
        while(a!=b){
            a = a == null?headB:a.next;
            b = b == null?headA:b.next;
        }
        return a;
    }
}
```

567 Permutation in String[\(link\)](#)

Description

Given two strings `s1` and `s2`, return `true` if `s2` contains a permutation of `s1`, or `false` otherwise.

In other words, return `true` if one of `s1`'s permutations is the substring of `s2`.

Example 1:

```
Input: s1 = "ab", s2 = "eidbaooo"  
Output: true  
Explanation: s2 contains one permutation of s1 ("ba").
```

Example 2:

```
Input: s1 = "ab", s2 = "eidboao"  
Output: false
```

Constraints:

- $1 \leq s1.length, s2.length \leq 10^4$
- `s1` and `s2` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean checkInclusion(String s1, String s2) {
        int s1l = s1.length(), s2l = s2.length();
        if(s1l > s2l)
            return false;

        int[] s1map = new int[26];
        for(int i=0; i<s1l; i++)
            s1map[s1.charAt(i)-'a']++;

        for (int i = 0; i <= s2.length() - s1.length(); i++) {
            int[] s2map = new int[26];
            for (int j = 0; j < s1.length(); j++) {
                s2map[s2.charAt(i + j) - 'a']++;
            }

            if (matches(s1map, s2map))
                return true;
        }
        return false;
    }

    public boolean matches(int[] s1map, int[] s2map) {
        for (int i = 0; i < 26; i++) {
            if (s1map[i] != s2map[i])
                return false;
        }
        return true;
    }
}
```

[120 Triangle \(link\)](#)

Description

Given a triangle array, return *the minimum path sum from top to bottom*.

For each step, you may move to an adjacent number of the row below. More formally, if you are on index i on the current row, you may move to either index i or index $i + 1$ on the next row.

Example 1:

Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]

Output: 11

Explanation: The triangle looks like:

```
  2
 3 4
6 5 7
4 1 8 3
```

The minimum path sum from top to bottom is $2 + 3 + 5 + 1 = 11$ (underlined above).

Example 2:

Input: triangle = [[-10]]

Output: -10

Constraints:

- $1 \leq \text{triangle.length} \leq 200$
- $\text{triangle}[0].\text{length} == 1$
- $\text{triangle}[i].\text{length} == \text{triangle}[i - 1].\text{length} + 1$
- $-10^4 \leq \text{triangle}[i][j] \leq 10^4$

Follow up: Could you do this using only $O(n)$ extra space, where n is the total number of rows in the triangle?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int size = triangle.size();
        List<Integer> prev = triangle.get(0), curr = null;
        int minSum = prev.get(0);

        for(int row=1; row < size; row++) {
            int temp, currMinSum = Integer.MAX_VALUE;
            curr = triangle.get(row);
            for(int col=0; col < row+1; col++) {
                temp = curr.get(col);
                if(col==0) {
                    temp += prev.get(col);
                } else if(col==row) {
                    temp += prev.get(col-1);
                } else {
                    temp += Math.min(prev.get(col-1), prev.get(col));
                }
                curr.set(col, temp);
                currMinSum = Math.min(currMinSum, temp);
            }
            minSum = currMinSum;
            prev = curr;
        }
        return minSum;
    }
}
```


142 Linked List Cycle II (link)

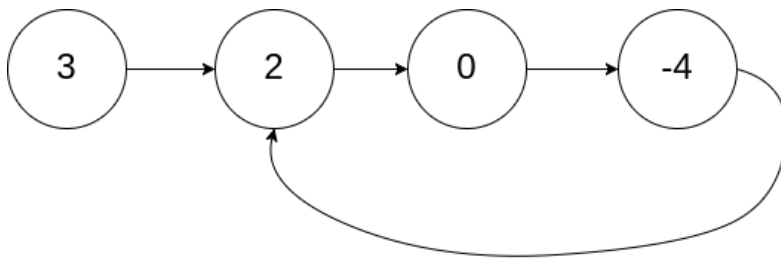
Description

Given the head of a linked list, return *the node where the cycle begins*. If there is no cycle, return `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that `pos` is not passed as a parameter**.

Do not modify the linked list.

Example 1:

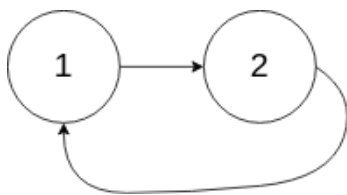


Input: `head = [3,2,0,-4]`, `pos = 1`

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.

Example 2:

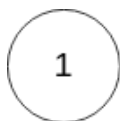


Input: `head = [1,2]`, `pos = 0`

Output: tail connects to node index 0

Explanation: There is a cycle in the linked list, where tail connects to the first node.

Example 3:



Input: `head = [1]`, `pos = -1`

Output: no cycle

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$

- pos is -1 or a **valid index** in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode s = head, f = head;
        do {
            if (f == null || f.next == null) {
                return null;
            } else {
                f = f.next.next;
                s = s.next;
            }
        } while (s != f);

        s = head;
        while (s != f) {
            s = s.next;
            f = f.next;
        }
        return s;
    }
}
```

136 Single Number (link)

Description

Given a **non-empty** array of integers `nums`, every element appears *twice* except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`

Output: 1

Example 2:

Input: `nums = [4,1,2,1,2]`

Output: 4

Example 3:

Input: `nums = [1]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int singleNumber(int[] nums) {  
        int result=0;  
        for(int num: nums)  
            result ^= num;  
        return result;  
    }  
}
```

[190 Reverse Bits \(link\)](#)

Description

Reverse bits of a given 32 bits unsigned integer.

Note:

- Note that in some languages, such as Java, there is no unsigned integer type. In this case, both input and output will be given as a signed integer type. They should not affect your implementation, as the integer's internal binary representation is the same, whether it is signed or unsigned.
- In Java, the compiler represents the signed integers using [2's complement notation](#). Therefore, in **Example 2** above, the input represents the signed integer -3 and the output represents the signed integer -1073741825.

Example 1:

```
Input: n = 00000010100101000001111010011100
Output:  964176192 (00111001011110000010100101000000)
Explanation: The input binary string 00000010100101000001111010011100 represents the unsigned integer 4315.
```

Example 2:

```
Input: n = 11111111111111111111111111111101
Output:  3221225471 (10111111111111111111111111111111)
Explanation: The input binary string 11111111111111111111111111111101 represents the unsigned integer 4294967295.
```

Constraints:

- The input must be a **binary string** of length 32

Follow up: If this function is called many times, how would you optimize it?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
public class Solution {
    // you need treat n as an unsigned value
    public int reverseBits(int n) {
        int result = 0, counter=0;
        for (int i = 0; i < 32; i++) {
            int bit = (n & (1 << i)) == 0 ? 0 : 1;

            bit <<= 31-i;
            result |= bit;
        }
        return result;
    }
}
```

231 Power of Two (link)

Description

Given an integer n , return *true* if it is a power of two. Otherwise, return *false*.

An integer n is a power of two, if there exists an integer x such that $n == 2^x$.

Example 1:

```
Input: n = 1
Output: true
Explanation:  $2^0 = 1$ 
```

Example 2:

```
Input: n = 16
Output: true
Explanation:  $2^4 = 16$ 
```

Example 3:

```
Input: n = 3
Output: false
```

Constraints:

- $-2^{31} \leq n \leq 2^{31} - 1$

Follow up: Could you solve it without loops/recursion?

(scroll down for solution)

Solution

Language: java

Status: Accepted

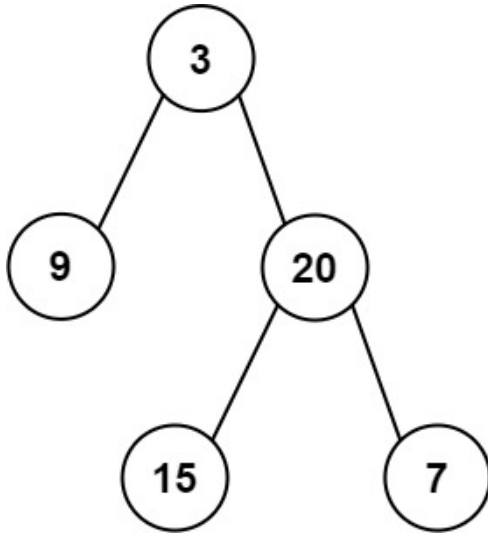
```
class Solution {  
    public boolean isPowerOfTwo(int n) {  
        return n<1?false: (n&(n-1)) == 0;  
    }  
}
```

106 Construct Binary Tree from Inorder and Postorder Traversal (link)

Description

Given two integer arrays `inorder` and `postorder` where `inorder` is the inorder traversal of a binary tree and `postorder` is the postorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: `inorder = [9,3,15,20,7]`, `postorder = [9,15,7,20,3]`
Output: `[3,9,20,null,null,15,7]`

Example 2:

Input: `inorder = [-1]`, `postorder = [-1]`
Output: `[-1]`

Constraints:

- `1 <= inorder.length <= 3000`
- `postorder.length == inorder.length`
- `-3000 <= inorder[i], postorder[i] <= 3000`
- `inorder` and `postorder` consist of **unique** values.
- Each value of `postorder` also appears in `inorder`.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.
- `postorder` is **guaranteed** to be the postorder traversal of the tree.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    int postOrderIndex;
    Map<Integer, Integer> inorderIndexMap;
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        postOrderIndex = postorder.length-1;
        inorderIndexMap = new HashMap<>();
        for(int i=0; i< inorder.length; i++)
            inorderIndexMap.put(inorder[i],i);

        return buildTree(postorder, 0, postorder.length-1);
    }

    TreeNode buildTree(int[] postorder, int start, int end) {
        if(start > end)
            return null;
        int rootValue = postorder[postOrderIndex--];
        TreeNode root = new TreeNode(rootValue);
        root.right = buildTree(postorder, inorderIndexMap.get(rootValue)+1, end);
        root.left = buildTree(postorder, start, inorderIndexMap.get(rootValue)-1);
        return root;
    }
}
```

[290 Word Pattern \(link\)](#)

Description

Given a `pattern` and a string `s`, find if `s` follows the same pattern.

Here **follow** means a full match, such that there is a bijection between a letter in `pattern` and a **non-empty** word in `s`. Specifically:

- Each letter in `pattern` maps to **exactly** one unique word in `s`.
- Each unique word in `s` maps to **exactly** one letter in `pattern`.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: `pattern = "abba"`, `s = "dog cat cat dog"`

Output: `true`

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: `pattern = "abba"`, `s = "dog cat cat fish"`

Output: `false`

Example 3:

Input: `pattern = "aaaa"`, `s = "dog cat cat dog"`

Output: `false`

Constraints:

- `1 <= pattern.length <= 300`
- `pattern` contains only lower-case English letters.
- `1 <= s.length <= 3000`
- `s` contains only lowercase English letters and spaces ' '.
- `s` **does not contain** any leading or trailing spaces.
- All the words in `s` are separated by a **single space**.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean wordPattern(String pattern, String s) {
        String[] splits = s.split(" ");
        if(splits.length != pattern.length())
            return false;

        Map<Character, String> cmap = new HashMap<>();
        Map<String, Character> smap = new HashMap<>();
        for(int i=0; i < splits.length;i++) {
            char c = pattern.charAt(i);
            String sValue = cmap.putIfAbsent(c, splits[i]);
            Character cValue = smap.putIfAbsent(splits[i], c);

            //sValue == splits[i] && cValue = c

            if(sValue == null && cValue == null)
                continue;
            if(!(splits[i].equals(sValue) && cValue.equals(c)))
                return false;
        }
        return true;
    }
}
```

[415 Add Strings \(link\)](#)

Description

Given two non-negative integers, `num1` and `num2` represented as string, return *the sum of* `num1` *and* `num2` *as a string*.

You must solve the problem without using any built-in library for handling large integers (such as `BigInteger`). You must also not convert the inputs to integers directly.

Example 1:

```
Input: num1 = "11", num2 = "123"  
Output: "134"
```

Example 2:

```
Input: num1 = "456", num2 = "77"  
Output: "533"
```

Example 3:

```
Input: num1 = "0", num2 = "0"  
Output: "0"
```

Constraints:

- $1 \leq \text{num1.length}, \text{num2.length} \leq 10^4$
- `num1` and `num2` consist of only digits.
- `num1` and `num2` don't have any leading zeros except for the zero itself.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    static char ZERO='0';
    public String addStrings(String num1, String num2) {
        int i=num1.length()-1, j=num2.length()-1;
        int carry=0, k=Math.max(i,j)+1;
        char[] result = new char[k];
        while(i>=0 || j>=0) {
            int ii = i>=0 ? num1.charAt(i--)-ZERO:0;
            int jj = j>=0 ? num2.charAt(j--)-ZERO:0;
            int sum = ii+jj+carry;
            carry = sum/10;
            sum %=10;
            result[--k]=(char) (ZERO+sum);
        }

        String resultStr = String.valueOf(result);
        return carry == 1? "1".concat(resultStr):resultStr;
    }
}
```

800 Letter Case Permutation (link)

Description

Given a string `s`, you can transform every letter individually to be lowercase or uppercase to create another string.

Return *a list of all possible strings we could create*. Return the output in **any order**.

Example 1:

```
Input: s = "a1b2"  
Output: ["a1b2", "a1B2", "A1b2", "A1B2"]
```

Example 2:

```
Input: s = "3z4"  
Output: ["3z4", "3Z4"]
```

Constraints:

- `1 <= s.length <= 12`
- `s` consists of lowercase English letters, uppercase English letters, and digits.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<String> letterCasePermutation(String s) {
        List<String> permutations = new ArrayList<>();

        permutations.add(s);
        for(int i=0; i < s.length(); i++) {
            if(Character.isDigit(s.charAt(i)))
                continue;
            int n = permutations.size();
            for(int j=0; j < n; j++) {
                char[] chrArray = permutations.get(j).toCharArray();
                chrArray[i] ^= ' ';
                permutations.add(String.valueOf(chrArray));
            }
        }
        return permutations;
    }
}
```

1036 Rotting Oranges (link)

Description

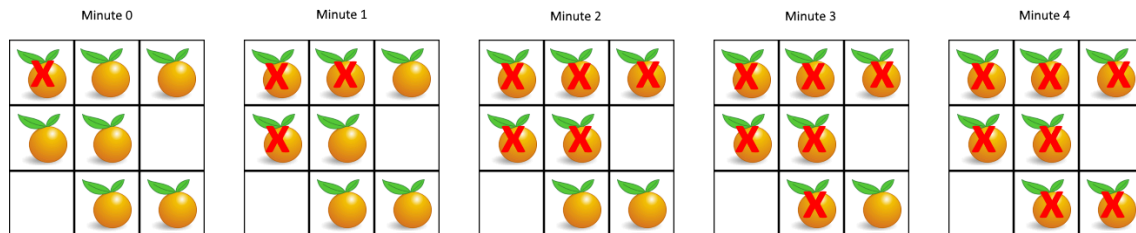
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only

Example 3:

Input: grid = [[0,2]]
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $\text{grid}[i][j]$ is 0, 1, or 2.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int orangesRotting(int[][] grid) {
        //freshCount to track count while some oranges rot
        int m = grid.length, n = grid[0].length, freshCount=0;
        Queue<int[]> qu = new ArrayDeque<>();
        for(int i=0; i < m; i++) {
            for(int j=0; j < n; j++) {
                if(grid[i][j] == 2)
                    qu.offer(new int[]{i,j});
                else if(grid[i][j] == 1)
                    freshCount++;
            }
        }

        if(qu.size() == 0)
            return freshCount == 0 ? 0:-1;

        //result initialized to -1, since incrementing for
        //oranges that were already rotten at start does not count
        //when popping them from Q
        int minutesElapsed=-1;
        int[][] directions = { {-1, 0}, {0, 1}, {1, 0}, {0, -1}};
        while(!qu.isEmpty()) {
            minutesElapsed++;
            int size = qu.size();
            while(size-- > 0 ) {
                int[] coords = qu.poll();
                for(int[] d: directions) {
                    int rd = coords[0]+d[0];
                    int cd = coords[1]+d[1];

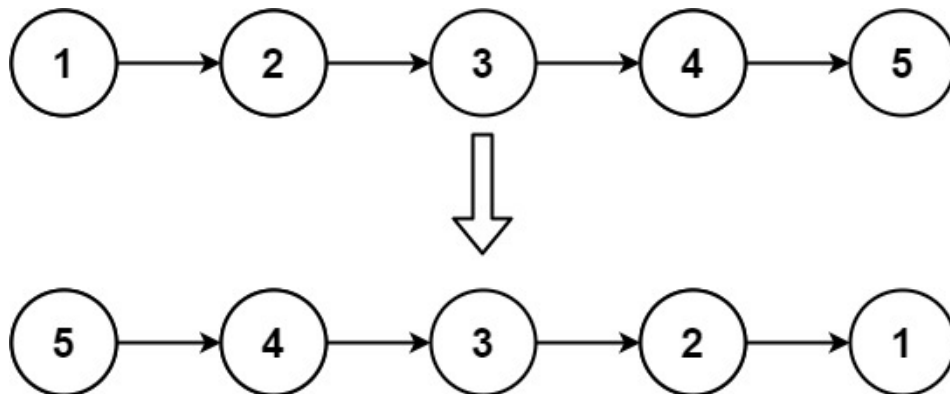
                    if(rd>-1 && rd < m && cd>-1 && cd < n) {
                        if(grid[rd][cd]==1) {
                            grid[rd][cd]=2;
                            freshCount--;
                            qu.offer(new int[]{rd,cd});
                        }
                    }
                }
            }
        }
        return freshCount==0 ?minutesElapsed:-1;
    }
}
```

206 Reverse Linked List ([link](#))

Description

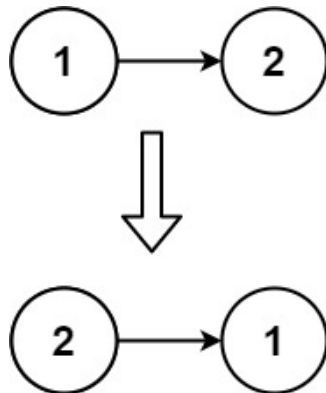
Given the head of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:



Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]
Output: [2,1]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode next, prev = null, current = head;
        while (current != null) {
            next = current.next;
            current.next = prev;
            prev = current;
            current = next;
        }
        head = prev;
        return head;
    }
}
```

[560 Subarray Sum Equals K \(link\)](#)

Description

Given an array of integers `nums` and an integer `k`, return *the total number of subarrays whose sum equals to k*.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [1,1,1], k = 2  
Output: 2
```

Example 2:

```
Input: nums = [1,2,3], k = 3  
Output: 2
```

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int count=0, sum=0;
        HashMap<Integer,Integer> map = new HashMap<>();
        for(int i=0; i < nums.length; i++) {
            if((sum += nums[i]) == k)
                count++;
            if(map.containsKey(sum-k))
                count += map.get(sum-k);
            map.put(sum, map.getOrDefault(sum, 0)+1);
        }
        return count;
    }
}
```


240 Search a 2D Matrix II (link)

Description

Write an efficient algorithm that searches for a value `target` in an $m \times n$ integer matrix `matrix`. This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: `matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]]`, `target =`
Output: `true`

Example 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: `matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]]`, `target =`
Output: `false`

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= n, m <= 300`
- `-109 <= matrix[i][j] <= 109`
- All the integers in each row are **sorted** in ascending order.
- All the integers in each column are **sorted** in ascending order.
- `-109 <= target <= 109`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int row = 0, col = n-1;
        while(row < m && col > -1) {
            if(matrix[row][col] < target)
                row++;
            else if(matrix[row][col] > target)
                col--;
            else
                return true;
        }
        return false;
    }
}
```

[435 Non-overlapping Intervals \(link\)](#)

Description

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return *the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping*.

Note that intervals which only touch at a point are **non-overlapping**. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`

Output: 2

Explanation: You need to remove two `[1,2]` to make the rest of the intervals non-overlapping.

Example 3:

Input: `intervals = [[1,2],[2,3]]`

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 10^5$
- `intervals[i].length == 2`
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        if(intervals.length < 2)
            return 0;

        Arrays.sort(intervals, (a,b)->(a[1]-b[1]));
        int intervalsToRemove = 0, left=0;
        int end=intervals[0][1];
        for(int i=1; i < intervals.length; i++) {
            if(end > intervals[i][0])
                intervalsToRemove++;
            else
                end = intervals[i][1];
        }
        return intervalsToRemove;
    }
}
```

617 Merge Two Binary Trees (link)

Description

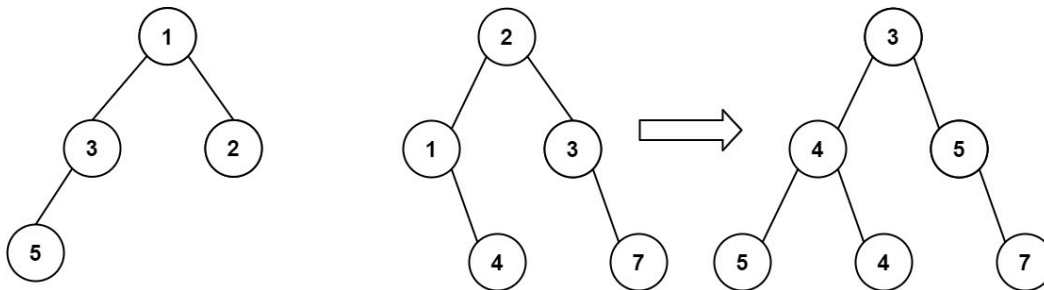
You are given two binary trees `root1` and `root2`.

Imagine that when you put one of them to cover the other, some nodes of the two trees are overlapped while the others are not. You need to merge the two trees into a new binary tree. The merge rule is that if two nodes overlap, then sum node values up as the new value of the merged node. Otherwise, the NOT null node will be used as the node of the new tree.

Return *the merged tree*.

Note: The merging process must start from the root nodes of both trees.

Example 1:



Input: `root1 = [1,3,2,5]`, `root2 = [2,1,3,null,4,null,7]`
Output: `[3,4,5,5,4,null,7]`

Example 2:

Input: `root1 = [1]`, `root2 = [1,2]`
Output: `[2,2]`

Constraints:

- The number of nodes in both trees is in the range $[0, 2000]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode mergeTrees(TreeNode root1, TreeNode root2) {
        if(root1 == null)
            return root2;
        if(root2 == null)
            return root1;

        root1.val += root2.val;
        root1.left = mergeTrees(root1.left, root2.left);
        root1.right = mergeTrees(root1.right, root2.right);

        return root1;
    }
}
```

48 Rotate Image (link)

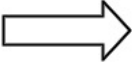
Description

You are given an $n \times n$ 2D matrix representing an image, rotate the image by **90** degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:

1	2	3
4	5	6
7	8	9

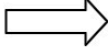


7	4	1
8	5	2
9	6	3

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        //do either column reverse and transpose
        //or transpose and row reverse

        //        //columnar reverse
        //        for(int col=0; col < n; col++)
        //            for(int tRow=0, bRow=n-1; tRow < bRow; tRow++, bRow--)
        //                swap(matrix, tRow, col, bRow, col);

        //transpose a matrix
        for(int row=0; row < n; row++) {
            for(int col=0; col < n; col++) {
                if(row <= col)
                    break;

                swap(matrix, row, col, col, row);
            }
        }

        //row reverse
        for(int row=0; row < n; row++)
            for(int fCol=0, lCol=n-1; fCol < lCol; fCol++, lCol--)
                swap(matrix, row, lCol, row, fCol);
    }

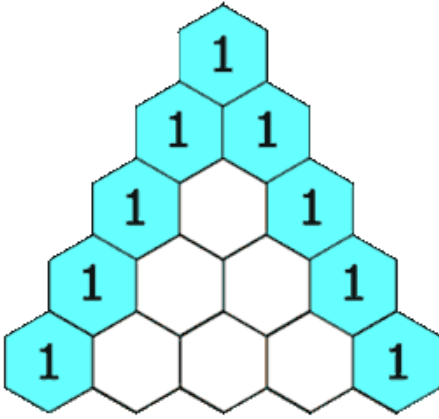
    void swap(int[][] matrix, int sr, int sc, int dr, int dc) {
        int t = matrix[dr][dc];
        matrix[dr][dc] = matrix[sr][sc];
        matrix[sr][sc] = t;
    }
}
```

[118 Pascal's Triangle \(link\)](#)

Description

Given an integer `numRows`, return the first `numRows` of **Pascal's triangle**.

In **Pascal's triangle**, each number is the sum of the two numbers directly above it as shown:



Example 1:

Input: `numRows = 5`
Output: `[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]`

Example 2:

Input: `numRows = 1`
Output: `[[1]]`

Constraints:

- $1 \leq \text{numRows} \leq 30$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> list = new ArrayList<>();
        for(int i=0; i < numRows; i++)
            list.add(getPascalRow(i));
        return list;
    }

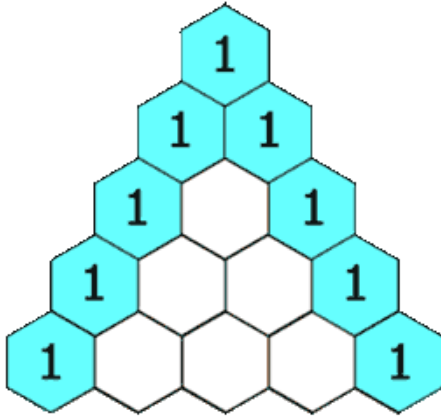
    public List<Integer> getPascalRow(int i) {
        ArrayList<Integer> ans = new ArrayList<>();
        long val = 1;
        for (int j = 0; j <= i; j++) {
            ans.add((int) val);
            val = val * (i - j) / (j + 1);
        }
        return ans;
    }
}
```

119 Pascal's Triangle II [\(link\)](#)

Description

Given an integer `rowIndex`, return the `rowIndexth` (**0-indexed**) row of the **Pascal's triangle**.

In **Pascal's triangle**, each number is the sum of the two numbers directly above it as shown:



Example 1:

Input: `rowIndex = 3`
Output: `[1,3,3,1]`

Example 2:

Input: `rowIndex = 0`
Output: `[1]`

Example 3:

Input: `rowIndex = 1`
Output: `[1,1]`

Constraints:

- $0 \leq \text{rowIndex} \leq 33$

Follow up: Could you optimize your algorithm to use only $O(\text{rowIndex})$ extra space?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public List<Integer> getRow(int i) {  
        ArrayList<Integer> ans = new ArrayList<>();  
        long val = 1;  
        for (int j = 0; j <= i; j++) {  
            ans.add((int) val);  
            val = val * (i - j) / (j + 1);  
        }  
        return ans;  
    }  
}
```

817 Design HashMap (link)

Description

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- void remove(key) removes the key and its corresponding value if the map contains the mapping for the key.

Example 1:

Input

```
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]  
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
```

Output

```
[null, null, null, 1, -1, null, 1, null, -1]
```

Explanation

```
MyHashMap myHashMap = new MyHashMap();  
myHashMap.put(1, 1); // The map is now [[1,1]]  
myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]  
myHashMap.get(1);    // return 1, The map is now [[1,1], [2,2]]  
myHashMap.get(3);    // return -1 (i.e., not found), The map is now [[1,1], [2,2]]  
myHashMap.put(2, 1); // The map is now [[1,1], [2,1]] (i.e., update the existing value)  
myHashMap.get(2);    // return 1, The map is now [[1,1], [2,1]]  
myHashMap.remove(2); // remove the mapping for 2, The map is now [[1,1]]  
myHashMap.get(2);    // return -1 (i.e., not found), The map is now [[1,1]]
```

Constraints:

- $0 \leq \text{key}, \text{value} \leq 10^6$
- At most 10^4 calls will be made to put, get, and remove.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class MyHashMap {
    private static int KEY_SPACE=2503;
    private List<Bucket> hashTable;

    public MyHashMap() {
        this.hashTable = new ArrayList<>();
        for(int i=0; i < KEY_SPACE; i++) {
            this.hashTable.add(new Bucket());
        }
    }

    public void put(int key, int value) {
        this.hashTable.get(key%KEY_SPACE).put(key, value);
    }

    public int get(int key) {
        return this.hashTable.get(key%KEY_SPACE).get(key);
    }

    public void remove(int key) {
        this.hashTable.get(key%KEY_SPACE).remove(key);
    }
}

class Pair<K,V> {
    public K key;
    public V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
}

class Bucket {
    private List<Pair<Integer, Integer>> bucket;

    public Bucket() {
        this.bucket = new LinkedList<>();
    }

    public void put(int key, int value) {
        boolean found = false;
        for(Pair<Integer, Integer> pair: this.bucket) {
            if(pair.key.equals(key)) {
                pair.value = value;
                found = true;
                break;
            }
        }
        if(!found)
            this.bucket.add(new Pair<>(key,value));
    }

    public int get(int key) {
        for(Pair<Integer, Integer> pair: this.bucket) {
            if(pair.key.equals(key)) {
                return pair.value;
            }
        }
        return -1;
    }

    public void remove(int key) {
        for(Pair<Integer, Integer> pair: this.bucket) {
```

```
        if(pair.key.equals(key)) {
            this.bucket.remove(pair);
            break;
        }
    }
}
```

```
/**
```

```
 * Your MyHashMap object will be instantiated and called as such:
```

```
 * MyHashMap obj = new MyHashMap();
```

```
 * obj.put(key,value);
```

```
 * int param_2 = obj.get(key);
```

```
 * obj.remove(key);
```

```
 */
```


57 Insert Interval (link)

Description

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the i^{th} interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` *after the insertion*.

Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Example 1:

Input: `intervals = [[1,3],[6,9]]`, `newInterval = [2,5]`
Output: `[[1,5],[6,9]]`

Example 2:

Input: `intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]]`, `newInterval = [4,8]`
Output: `[[1,2],[3,10],[12,16]]`
Explanation: Because the new interval `[4,8]` overlaps with `[3,5]`, `[6,7]`, `[8,10]`.

Constraints:

- $0 \leq \text{intervals.length} \leq 10^4$
- `intervals[i].length == 2`
- $0 \leq \text{start}_i \leq \text{end}_i \leq 10^5$
- `intervals` is sorted by `starti` in **ascending** order.
- `newInterval.length == 2`
- $0 \leq \text{start} \leq \text{end} \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> merged = new ArrayList<>();
        int i=0;
        while(i < intervals.length && intervals[i][1] < newInterval[0])
            merged.add(intervals[i++]);
        while(i < intervals.length && intervals[i][0] <= newInterval[1]) {
            newInterval[0]=Math.min(newInterval[0], intervals[i][0]);
            newInterval[1]=Math.max(newInterval[1], intervals[i][1]);
            i++;
        }
        merged.add(newInterval);

        while(i < intervals.length)
            merged.add(intervals[i++]);

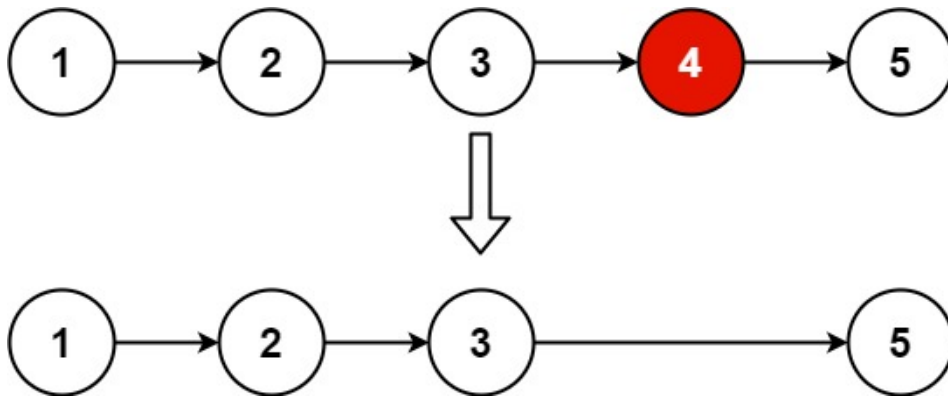
        return merged.toArray(new int[0][0]);
    }
}
```

19 Remove Nth Node From End of List ([link](#))

Description

Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz .
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0, head), fast = dummy, slow = dummy;
        while(n-- > -1 && fast != null) {
            fast = fast.next;
        }

        while(fast != null) {
            fast = fast.next;
            slow = slow.next;
        }

        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

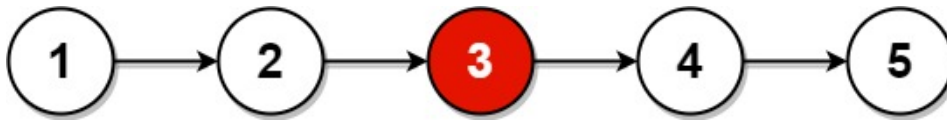
908 Middle of the Linked List (link)

Description

Given the head of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle** node.

Example 1:

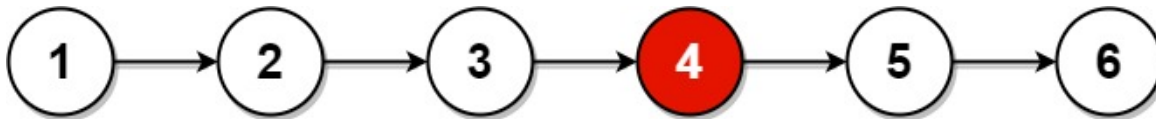


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode middleNode(ListNode head) {
        if(head == null || head.next == null)
            return head;
        ListNode slow = head, fast = head;
        while(fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

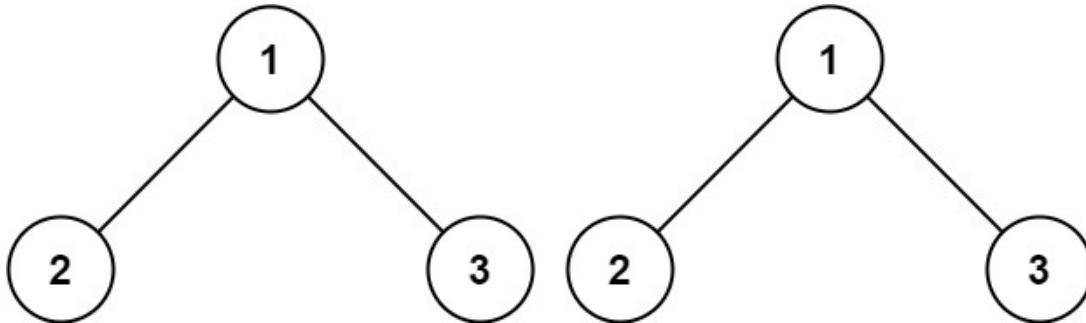
100 Same Tree (link)

Description

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

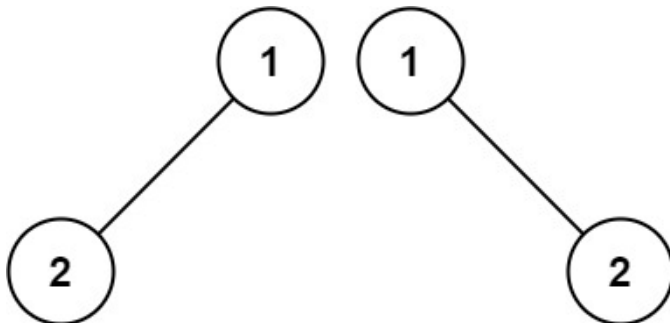
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



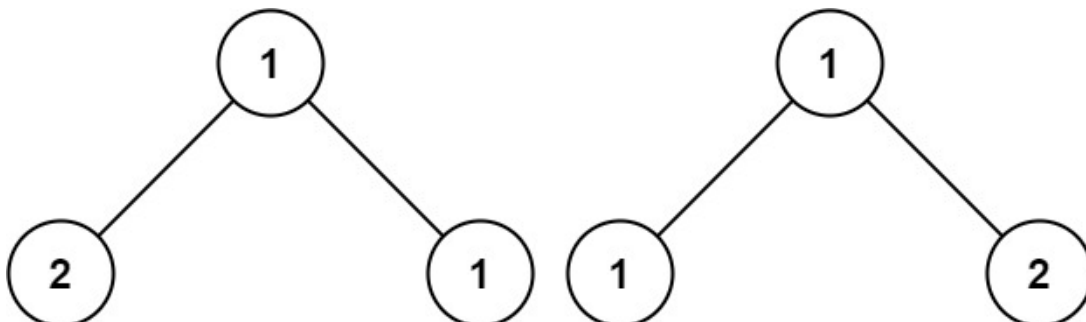
Input: $p = [1,2,3]$, $q = [1,2,3]$
Output: true

Example 2:



Input: $p = [1,2]$, $q = [1,null,2]$
Output: false

Example 3:



Input: $p = [1,2,1]$, $q = [1,1,2]$

Output: false

Constraints:

- The number of nodes in both trees is in the range $[0, 100]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isSameTree(TreeNode s, TreeNode t) {
        if (s == null && t == null) return true;
        if (s == null || t == null) return false;
        return s.val == t.val && isSameTree(s.left, t.left) && isSameTree(s.right, t.right);
    }
}
```

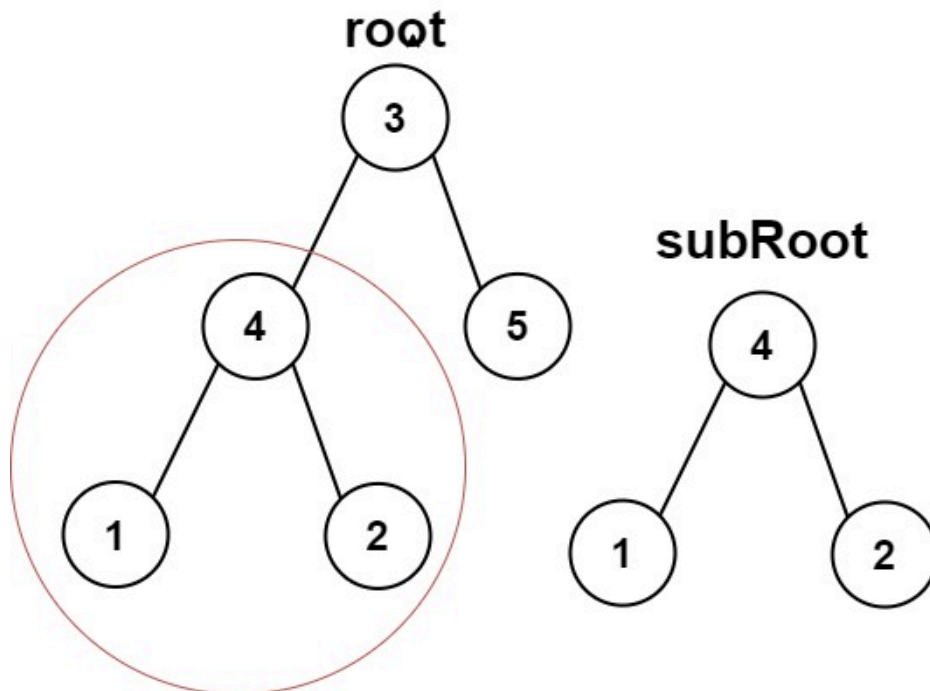
[572 Subtree of Another Tree \(link\)](#)

Description

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

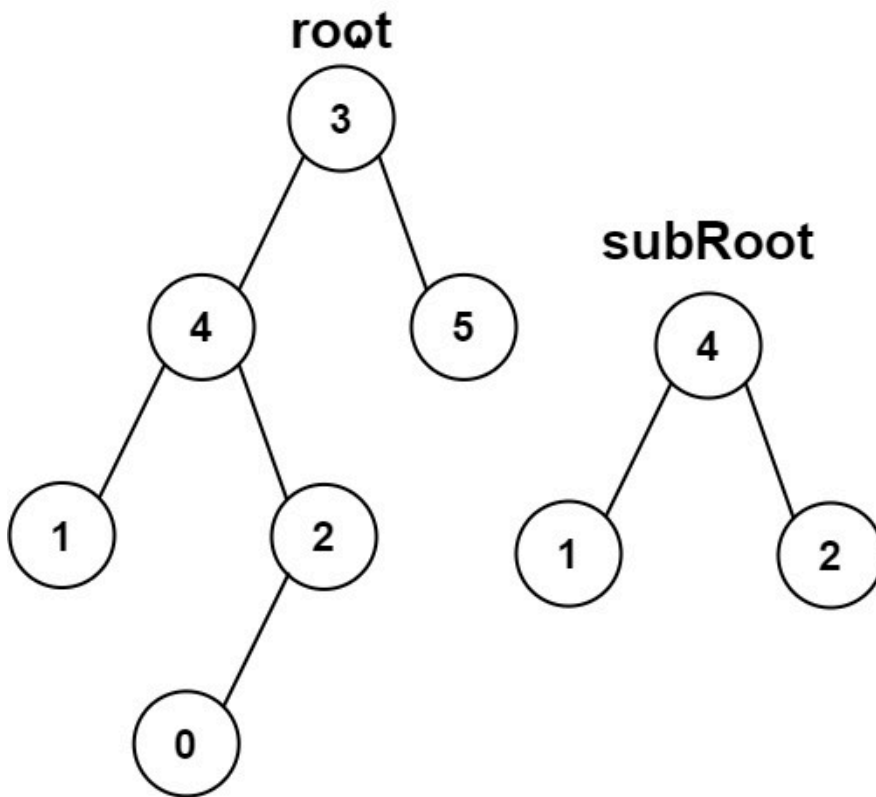
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: `root = [3,4,5,1,2]`, `subRoot = [4,1,2]`
Output: `true`

Example 2:



Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false

Constraints:

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].
- $-10^4 \leq \text{root.val} \leq 10^4$
- $-10^4 \leq \text{subRoot.val} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isSubtree(TreeNode s, TreeNode t) {
        if (s == null && t == null) return true;
        if (s == null || t == null) return false;
        return isSame(s, t) || isSubtree(s.left, t) || isSubtree(s.right, t);
    }

    boolean isSame(TreeNode s, TreeNode t) {
        if (s == null && t == null) return true;
        if (s == null || t == null) return false;
        return s.val == t.val && isSame(s.left, t.left) && isSame(s.right, t.right);
    }
}
```

33 Search in Rotated Sorted Array [\(link\)](#)

Description

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly rotated** at an unknown pivot index k ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index 3 and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of target if it is in nums, or -1 if it is not in nums*.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
Output: -1

Example 3:

Input: `nums = [1]`, `target = 0`
Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- $-10^4 \leq \text{target} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int search(int[] nums, int target) {
        int start=0, end=nums.length -1;
        while(start <= end) {
            int mid = (start+end)/2;
            if(nums[mid] == target)
                return mid;
            else if(nums[mid] >= nums[start]) {
                if(target >= nums[start] && target < nums[mid])
                    end = mid -1;
                else
                    start = mid+1;
            } else {
                if(target <= nums[end] && target > nums[mid])
                    start=mid+1;
                else
                    end=mid-1;
            }
        }
        return -1;
    }
}
```

338 Counting Bits (link)

Description

Given an integer n , return *an array* `ans` *of length* $n + 1$ *such that for each* i ($0 \leq i \leq n$), `ans[i]` *is the number of 1's in the binary representation of* i .

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
5 --> 101
```

Constraints:

- $0 \leq n \leq 10^5$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    static int[] bitmap = new int[256];
    static {
        for (int i = 0; i < 256; i++)
            bitmap[i] = (i & 1) + bitmap[i / 2];
    }

    public int[] countBits(int n) {
        int[] bitcounts = new int[n+1];
        for(int i=0; i < n+1;i++)
            bitcounts[i] = bitmap[i & 0xff] + bitmap[(i >> 8) & 0xff] + bitmap[(i >> 16) & 0xff] + bitmap[(i >> 24) & 0xff];
        return bitcounts;
    }

    public int[] countBitsTest(int n) {
        int[] bitcounts = new int[n+1];
        for(int i=0; i < n+1;i++)
            bitcounts[i] = count(i);
        return bitcounts;
    }

    private int count(int n) {
        return n == 0 ? 0: 1 + count(n&(n - 1));
    }
}
```

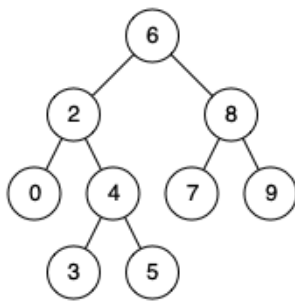

[235 Lowest Common Ancestor of a Binary Search Tree](#) ([link](#)).

Description

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in τ that has both p and q as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

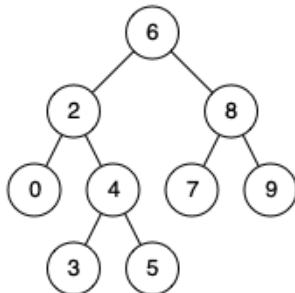


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the

Example 3:

Input: root = [2,1], p = 2, q = 1

Output: 2

Constraints:

- The number of nodes in the tree is in the range $[2, 10^5]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$
- All `Node.val` are **unique**.
- $p \neq q$
- p and q will exist in the BST.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */

class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if(root.val < p.val && root.val < q.val)
            return lowestCommonAncestor(root.right, p, q);
        else if(root.val > p.val && root.val > q.val)
            return lowestCommonAncestor(root.left, p, q);
        else
            return root;
    }
}
```

283 Move Zeroes [\(link\)](#)

Description

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Could you minimize the total number of operations done?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public void moveZeroes(int[] nums) {  
        int j=0, length=nums.length;  
        for(int i=0; i<length;i++)  
            if(nums[i] != 0)  
                nums[j++]=nums[i];  
        while(j < length)  
            nums[j++]=0;  
    }  
}
```

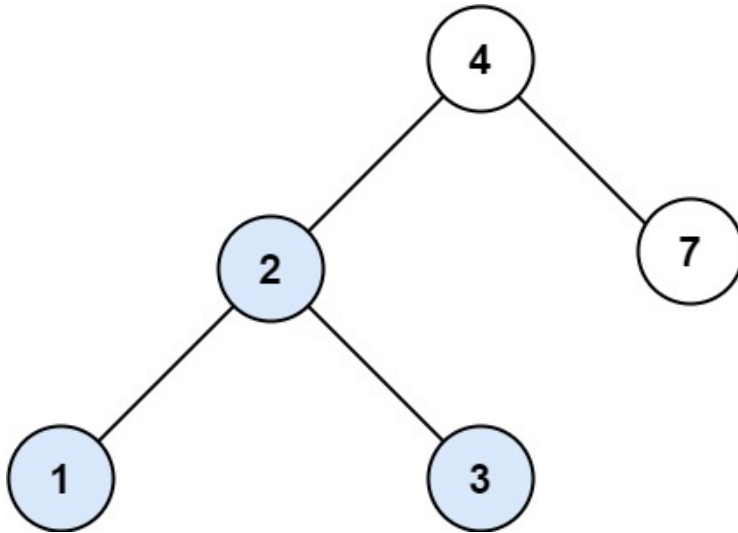
[783 Search in a Binary Search Tree \(link\)](#)

Description

You are given the `root` of a binary search tree (BST) and an integer `val`.

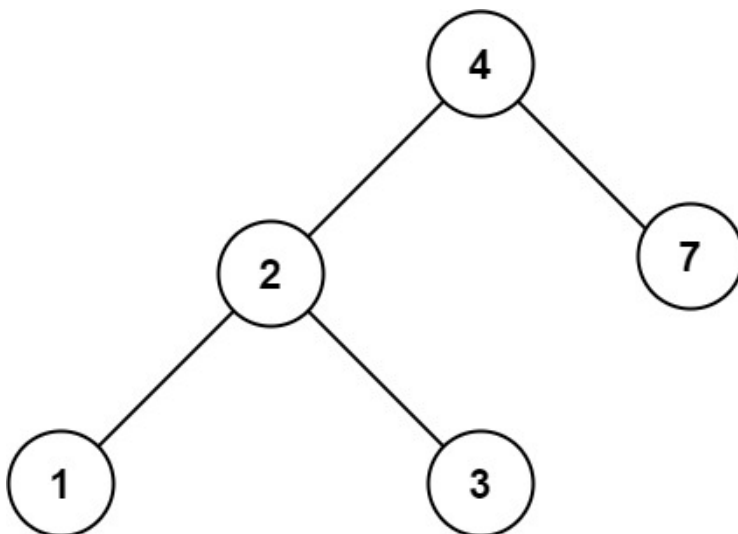
Find the node in the BST that the node's value equals `val` and return the subtree rooted with that node. If such a node does not exist, return `null`.

Example 1:



Input: `root = [4,2,7,1,3]`, `val = 2`
Output: `[2,1,3]`

Example 2:



Input: `root = [4,2,7,1,3]`, `val = 5`
Output: `[]`

Constraints:

- The number of nodes in the tree is in the range $[1, 5000]$.
- $1 \leq \text{Node.val} \leq 10^7$
- root is a binary search tree.
- $1 \leq \text{val} \leq 10^7$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        if(root == null)
            return null;
        if(root.val == val)
            return root;
        else
            return root.val > val? searchBST(root.left, val):searchBST(root.right, val);
    }
}
```


[792 Binary Search \(link\)](#)

Description

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are **unique**.
- `nums` is sorted in ascending order.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while(lo <= hi) {
            int mid = lo+((hi-lo) >> 1);
            if(nums[mid] == target)
                return mid;
            else if(nums[mid] < target)
                lo = mid+1;
            else
                hi = mid-1;
        }
        return -1;
    }
}
```

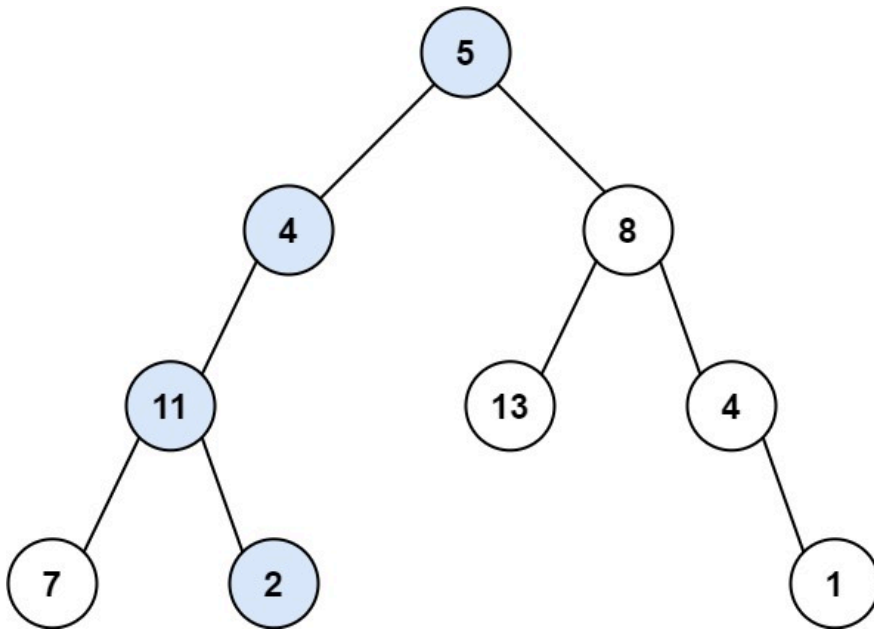
112 Path Sum (link)

Description

Given the root of a binary tree and an integer `targetSum`, return `true` if the tree has a **root-to-leaf** path such that adding up all the values along the path equals `targetSum`.

A **leaf** is a node with no children.

Example 1:

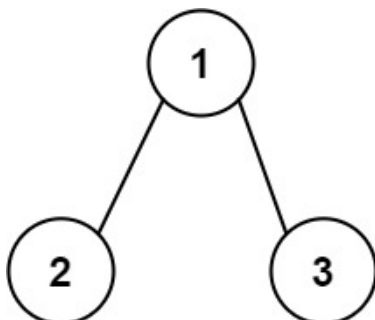


Input: `root = [5,4,8,11,null,13,4,7,2,null,null,null,1]`, `targetSum = 22`

Output: `true`

Explanation: The root-to-leaf path with the target sum is shown.

Example 2:



Input: `root = [1,2,3]`, `targetSum = 5`

Output: `false`

Explanation: There are two root-to-leaf paths in the tree:

(1 --> 2): The sum is 3.

(1 --> 3): The sum is 4.

There is no root-to-leaf path with sum = 5.

Example 3:

Input: root = [], targetSum = 0

Output: false

Explanation: Since the tree is empty, there are no root-to-leaf paths.

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

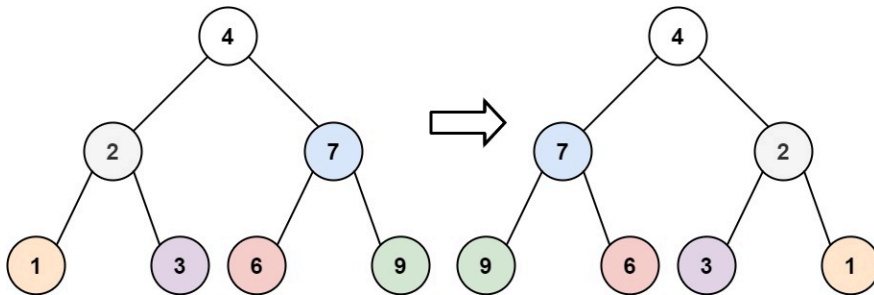
```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        if(root == null)
            return false;
        if(root.left == null && root.right == null && root.val == targetSum)
            return true;
        return hasPathSum(root.left, targetSum-root.val)||hasPathSum(root.right, targetSum-root.val);
    }
}
```

226 Invert Binary Tree (link)

Description

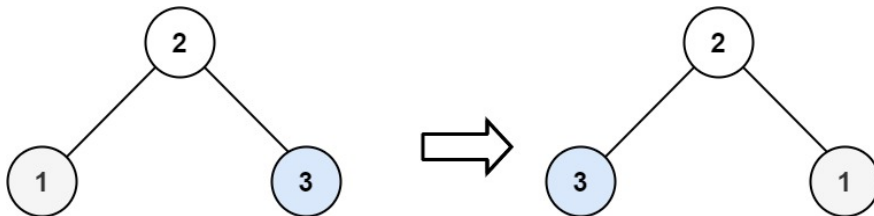
Given the root of a binary tree, invert the tree, and return *its* root.

Example 1:



Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]
Output: [2,3,1]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if(root == null)
            return null;
        TreeNode right = invertTree(root.right);
        TreeNode left = invertTree(root.left);
        root.left = right;
        root.right = left;
        return root;
    }
}
```

36 Valid Sudoku (link)

Description

Determine if a 9×9 Sudoku board is valid. Only the filled cells need to be validated **according to the following rules**:

1. Each row must contain the digits 1-9 without repetition.
2. Each column must contain the digits 1-9 without repetition.
3. Each of the nine 3×3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

```
Input: board =
[["5","3",".",".","7",".",".",".","."],
 ["6",".",".","1","9","5",".",".","."],
 [".","9","8",".",".",".","6","."],
 ["8",".",".","6",".",".","3"],
 ["4",".","8",".","3",".","1"],
 ["7",".","2",".","6"],
 [".","6",".","2","8","."],
 [".","4","1","9",".","5"],
 [".","8",".","7","9"]]
Output: true
```

Example 2:

```
Input: board =
[["8","3",".",".","7",".",".",".","."],
 ["6",".",".","1","9","5",".",".","."],
 [".","9","8",".",".",".","6","."],
 ["8",".",".","6",".",".","3"],
 ["4",".","8",".","3",".","1"],
 ["7",".","2",".","6"],
 [".","6",".","2","8","."],
 [".","4","1","9",".","5"],
 [".","8",".","7","9"]]
Output: false
Explanation: Same as Example 1, except with the 5 in the top left corner being modified to 8. Since the
```


Constraints:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` is a digit 1-9 or '.'.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean isValidSudoku(char[][] board) {
        int n = 9;
        int[] rows=new int[n];
        int[] columns = new int[n];
        int[] boxes = new int[n];

        for(int r=0; r<n;r++) {
            for(int c=0; c<n;c++) {
                if(board[r][c] == '.')
                    continue;

                int val = board[r][c]-'0';
                int pos = 1 << (val-1);
                int idx = (r/3)*3+(c/3);

                if((rows[r]&pos)>0 || (columns[c]&pos)>0 || (boxes[idx]&pos)>0)
                    return false;
                rows[r] |=pos;
                columns[c] |=pos;
                boxes[idx] |= pos;
            }
        }
        return true;
    }
}
```

74 Search a 2D Matrix (link)

Description

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if target is in matrix* or `false` *otherwise*.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`
Output: `true`

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 13`
Output: `false`

Constraints:

- `m == matrix.length`

- `n == matrix[i].length`
- `1 <= m, n <= 100`
- `-104 <= matrix[i][j], target <= 104`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length, n = matrix[0].length;
        int lo = 0, hi = m*n -1, midElement, mid;

        while(lo <= hi) {
            mid = (lo+hi)>>1;
            midElement = matrix[mid/n][mid%n];

            if(target == midElement)
                return true;
            else if(target < midElement)
                hi = mid-1;
            else
                lo = mid+1;
        }
        return false;
    }
}
```

383 Ransom Note (link)

Description

Given two strings `ransomNote` and `magazine`, return `true` *if* `ransomNote` *can be constructed by using the letters from* `magazine` *and* `false` *otherwise*.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

```
Input: ransomNote = "a", magazine = "b"  
Output: false
```

Example 2:

```
Input: ransomNote = "aa", magazine = "ab"  
Output: false
```

Example 3:

```
Input: ransomNote = "aa", magazine = "aab"  
Output: true
```

Constraints:

- `1 <= ransomNote.length, magazine.length <= 105`
- `ransomNote` and `magazine` consist of lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        if(ransomNote.length() > magazine.length())
            return false;
        int[] cmap = new int[26];
        for(int i=0; i < magazine.length(); i++)
            cmap[magazine.charAt(i)-'a']++;

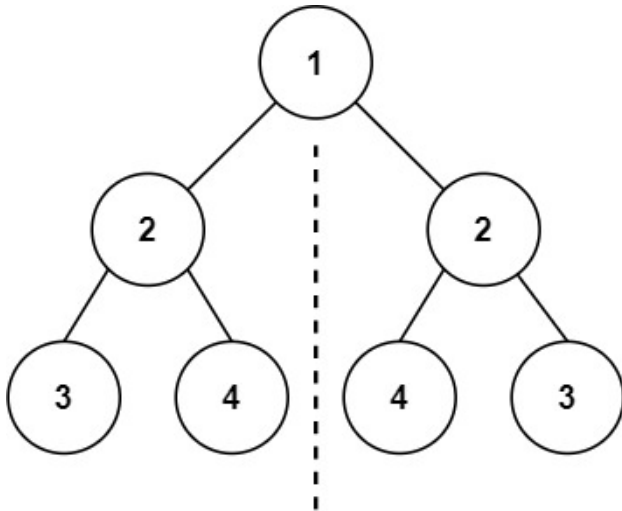
        for(int i=0; i < ransomNote.length();i++) {
            int idx = ransomNote.charAt(i) - 'a';
            if(cmap[idx] > 0) {
                cmap[idx]--;
            } else {
                return false;
            }
        }
        return true;
    }
}
```

[101 Symmetric Tree \(link\)](#)

Description

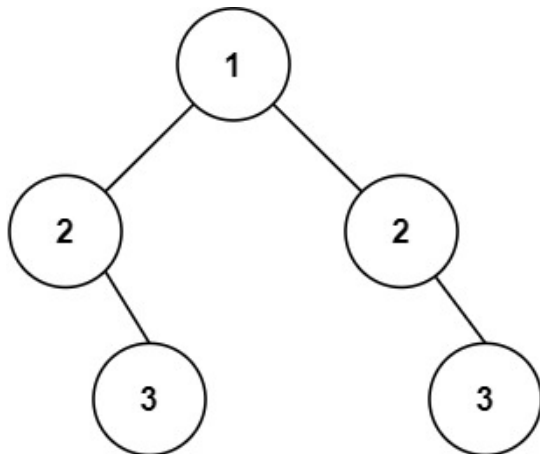
Given the root of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

Example 1:



Input: root = [1,2,2,3,4,4,3]
Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]
Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Could you solve it both recursively and iteratively?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public boolean isSymmetric(TreeNode root) {
        return isSymmetric(root.left, root.right);
    }

    private boolean isSymmetric(TreeNode left, TreeNode right) {
        if(left == null && right == null) {
            return true;
        } else if(left == null || right == null) {
            return false;
        } else if( left.val != right.val) {
            return false;
        } else {
            return isSymmetric(left.right, right.left) && isSymmetric(left.left, right.right);
        }
    }
}
```

[566 Reshape the Matrix \(link\)](#)

Description

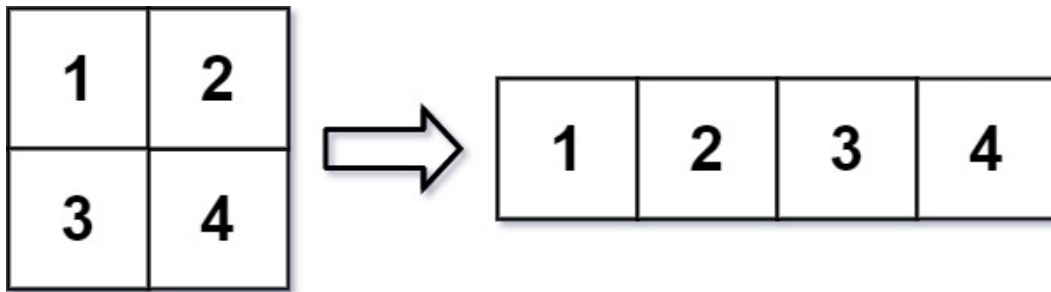
In MATLAB, there is a handy function called `reshape` which can reshape an $m \times n$ matrix into a new one with a different size $r \times c$ keeping its original data.

You are given an $m \times n$ matrix `mat` and two integers `r` and `c` representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

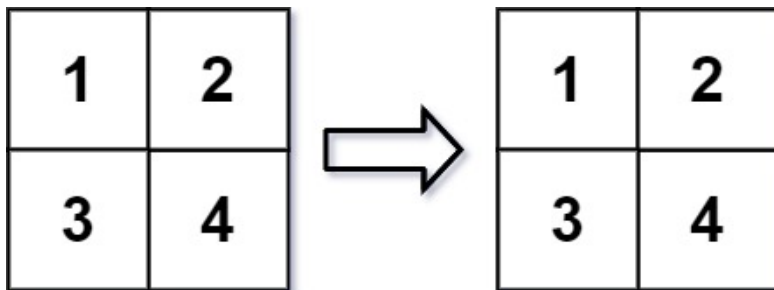
If the `reshape` operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:



Input: `mat = [[1,2],[3,4]]`, `r = 1`, `c = 4`
Output: `[[1,2,3,4]]`

Example 2:



Input: `mat = [[1,2],[3,4]]`, `r = 2`, `c = 4`
Output: `[[1,2],[3,4]]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 100`
- `-1000 <= mat[i][j] <= 1000`
- `1 <= r, c <= 300`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[][] matrixReshape(int[][] mat, int r, int c) {
        int m = mat.length, n = mat[0].length;
        if(m*n != r*c)
            return mat;

        int k=0, l=0;
        int i=0, j=0;
        int[][] result = new int[r][c];
        while(i < m && k < r) {
            result[k][l++] = mat[i][j++];
            if(l == c) {
                l=0;
                k++;
            }

            if(j == n) {
                j=0;
                i++;
            }
        }
        return result;
    }
}
```

387 First Unique Character in a String [\(link\)](#)

Description

Given a string `s`, find the **first** non-repeating character in it and return its index. If it **does not** exist, return `-1`.

Example 1:

Input: `s = "leetcode"`

Output: `0`

Explanation:

The character `'l'` at index `0` is the first character that does not occur at any other index.

Example 2:

Input: `s = "loveleetcode"`

Output: `2`

Example 3:

Input: `s = "aabb"`

Output: `-1`

Constraints:

- $1 \leq s.length \leq 10^5$
- `s` consists of only lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int firstUniqChar(String s) {
        int[] cmap = new int[26];
        for(int i=0; i< s.length(); i++) {
            cmap[s.charAt(i)-'a']++;
        }

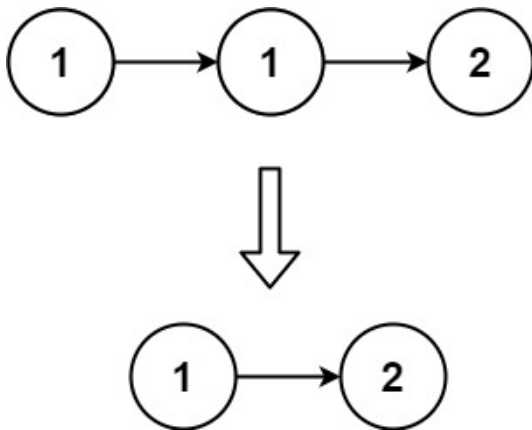
        for(int i=0; i < s.length();i++)
            if(cmap[s.charAt(i)-'a'] == 1)
                return i;
        return -1;
    }
}
```

83 Remove Duplicates from Sorted List [\(link\)](#)

Description

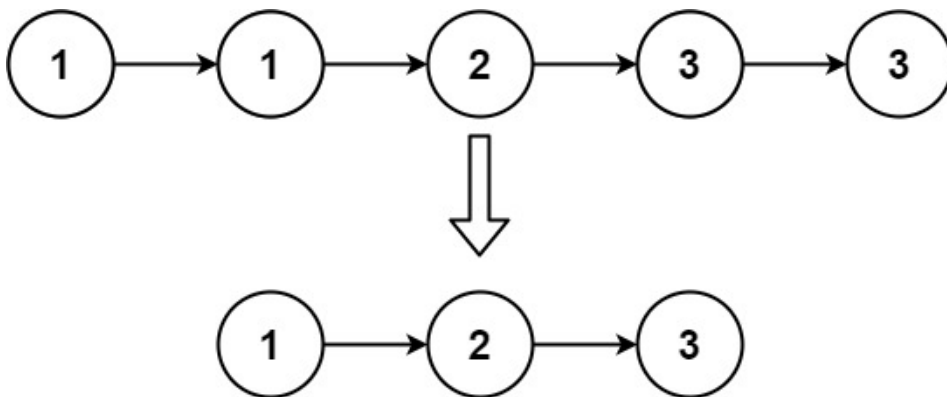
Given the head of a sorted linked list, *delete all duplicates such that each element appears only once*. Return the linked list **sorted** as well.

Example 1:



Input: head = [1,1,2]
Output: [1,2]

Example 2:



Input: head = [1,1,2,3,3]
Output: [1,2,3]

Constraints:

- The number of nodes in the list is in the range $[0, 300]$.
- $-100 \leq \text{Node.val} \leq 100$
- The list is guaranteed to be **sorted** in ascending order.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode dHead = new ListNode(-101, head), next = head;
        head = dHead;

        while(next != null) {
            if(head.val == next.val) {
                head.next = next.next;
            } else {
                head = next;
            }
            next = head.next;
        }

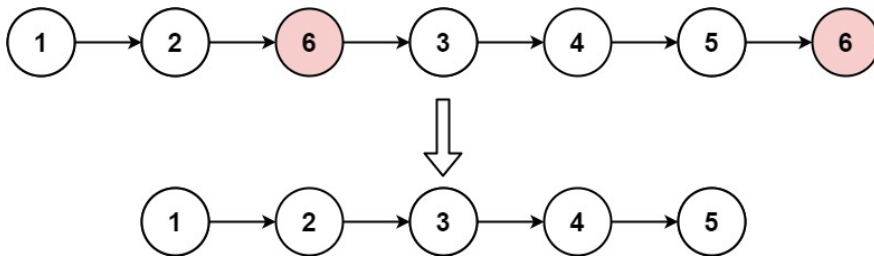
        return dHead.next;
    }
}
```

203 Remove Linked List Elements (link)

Description

Given the head of a linked list and an integer `val`, remove all the nodes of the linked list that has `Node.val == val`, and return *the new head*.

Example 1:



Input: head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]

Example 2:

Input: head = [], val = 1
Output: []

Example 3:

Input: head = [7,7,7,7], val = 7
Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 10^4]$.
- $1 \leq \text{Node.val} \leq 50$
- $0 \leq \text{val} \leq 50$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode dummy = new ListNode(-1, head);
        ListNode prev = dummy;
        ListNode node = dummy.next;

        while (node != null) {
            if (node.val == val) {
                prev.next = node.next;
            } else {
                prev = node;
            }
            node = prev.next;
        }
        return dummy.next;
    }
}
```

141 Linked List Cycle (link)

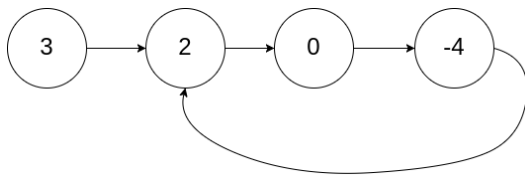
Description

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:

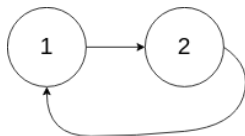


Input: `head = [3,2,0,-4]`, `pos = 1`

Output: `true`

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:



Input: `head = [1,2]`, `pos = 0`

Output: `true`

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: `head = [1]`, `pos = -1`

Output: `false`

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- `pos` is `-1` or a **valid index** in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    public boolean hasCycle(ListNode head) {
        if(head == null) {
            return false;
        }
        ListNode slow = head;
        ListNode fast = head;
        while(fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if(slow == fast) return true;
        }
        return false;
    }
}
```

[350 Intersection of Two Arrays II \(link\)](#)

Description

Given two integer arrays `nums1` and `nums2`, return *an array of their intersection*. Each element in the result must appear as many times as it shows in both arrays and you may return the result in **any order**.

Example 1:

Input: `nums1 = [1,2,2,1]`, `nums2 = [2,2]`
Output: `[2,2]`

Example 2:

Input: `nums1 = [4,9,5]`, `nums2 = [9,4,9,8,4]`
Output: `[4,9]`
Explanation: `[9,4]` is also accepted.

Constraints:

- `1 <= nums1.length, nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 1000`

Follow up:

- What if the given array is already sorted? How would you optimize your algorithm?
- What if `nums1`'s size is small compared to `nums2`'s size? Which algorithm is better?
- What if elements of `nums2` are stored on disk, and the memory is limited such that you cannot load all elements into the memory at once?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] intersect(int[] nums1, int[] nums2) {
        Map<Integer, Integer> map = new HashMap<>();
        for(int num: nums1)
            map.put(num, map.getOrDefault(num,0)+1);

        List<Integer> list = new ArrayList<>();
        for(int num: nums2) {
            if(map.containsKey(num)) {
                if(map.get(num) > 0) {
                    map.put(num, map.get(num)-1);
                    list.add(num);
                }
            }
        }
        return list.stream().mapToInt(i->i).toArray();
    }
}
```


88 Merge Sorted Array [\(link\)](#)

Description

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be *stored inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`
Output: `[1,2,2,3,5,6]`
Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.
The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

Example 2:

Input: `nums1 = [1]`, `m = 1`, `nums2 = []`, `n = 0`
Output: `[1]`
Explanation: The arrays we are merging are `[1]` and `[]`.
The result of the merge is `[1]`.

Example 3:

Input: `nums1 = [0]`, `m = 0`, `nums2 = [1]`, `n = 1`
Output: `[1]`
Explanation: The arrays we are merging are `[]` and `[1]`.
The result of the merge is `[1]`.
Note that because `m = 0`, there are no elements in `nums1`. The `0` is only there to ensure the merge result

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-109 <= nums1[i], nums2[j] <= 109`

Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i=m-1, j=n-1,k=nums1.length-1;
        while(i > -1 && j > -1) {
            if(nums1[i] > nums2[j])
                nums1[k--] = nums1[i--];
            else
                nums1[k--] = nums2[j--];
        }

        while(i>-1)
            nums1[k--]=nums1[i--];

        while(j>-1)
            nums1[k--]=nums2[j--];
    }
}
```

53 Maximum Subarray (link)

Description

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum=nums[0],sum=nums[0];
        for(int i=1;i<nums.length;i++) {
            sum = Math.max(nums[i], sum+nums[i]);
            maxSum = Math.max(sum, maxSum);
        }
        return maxSum;
    }
}
```

1774 Add Two Polynomials Represented as Linked Lists **(link)**

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for polynomial singly-linked list.
 * class PolyNode {
 *     int coefficient, power;
 *     PolyNode next = null;
 *
 *     PolyNode() {}
 *     PolyNode(int x, int y) { this.coefficient = x; this.power = y; }
 *     PolyNode(int x, int y, PolyNode next) { this.coefficient = x; this.power = y; this.next = next; }
 * }
 */

class Solution {
    public PolyNode addPoly(PolyNode poly1, PolyNode poly2) {
        PolyNode dummy = new PolyNode(), node = dummy;

        while(poly1 != null && poly2 != null) {
            if(poly1.power > poly2.power) {
                node.next = new PolyNode(poly1.coefficient, poly1.power);
                poly1 = poly1.next;
            } else if(poly1.power < poly2.power) {
                node.next = new PolyNode(poly2.coefficient, poly2.power);
                poly2 = poly2.next;
            } else {
                if(poly1.coefficient + poly2.coefficient != 0)
                    node.next = new PolyNode(poly1.coefficient + poly2.coefficient, poly1.power);
                poly1 = poly1.next;
                poly2 = poly2.next;
            }

            if(node.next != null)
                node = node.next;
        }

        if(poly1 != null)
            node.next = poly1;

        if(poly2 != null) {
            node.next = poly2;
        }

        return dummy.next;
    }
}
```

260 Single Number III (link)

Description

Given an integer array `nums`, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in **any order**.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Example 1:

Input: `nums = [1,2,1,3,2,5]`
Output: `[3,5]`
Explanation: `[5, 3]` is also a valid answer.

Example 2:

Input: `nums = [-1,0]`
Output: `[-1,0]`

Example 3:

Input: `nums = [0,1]`
Output: `[1,0]`

Constraints:

- $2 \leq \text{nums.length} \leq 3 * 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- Each integer in `nums` will appear twice, only two integers will appear once.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] singleNumber(int[] nums) {
        int xored=0, x=0;
        for(int i=0; i < nums.length; i++)
            xored ^= nums[i];

        int rightMostSetBit = xored & (-xored) ;
        for(int num: nums)
            if((rightMostSetBit & num) == 0)
                x ^= num;

        return new int[]{x^xored, x};
    }
}
```


[371 Sum of Two Integers \(link\)](#)

Description

Given two integers a and b, return *the sum of the two integers without using the operators + and -*.

Example 1:

Input: a = 1, b = 2
Output: 3

Example 2:

Input: a = 2, b = 3
Output: 5

Constraints:

- $-1000 \leq a, b \leq 1000$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int getSum(int a, int b) {  
        int carry;  
        while(b != 0) {  
            carry = a&b;  
            a ^= b;  
            b = carry << 1;  
        }  
        return a;  
    }  
}
```

[1762 Furthest Building You Can Reach \(link\)](#)

Description

You are given an integer array `heights` representing the heights of buildings, some bricks, and some ladders.

You start your journey from building 0 and move to the next building by possibly using bricks or ladders.

While moving from building i to building $i+1$ (**0-indexed**),

- If the current building's height is **greater than or equal** to the next building's height, you do **not** need a ladder or bricks.
- If the current building's height is **less than** the next building's height, you can either use **one ladder** or $(h[i+1] - h[i])$ **bricks**.

Return the furthest building index (0-indexed) you can reach if you use the given ladders and bricks optimally.

Example 1:



Input: `heights = [4,2,7,6,9,14,12]`, `bricks = 5`, `ladders = 1`

Output: 4

Explanation: Starting at building 0, you can follow these steps:

- Go to building 1 without using ladders nor bricks since $4 \geq 2$.
- Go to building 2 using 5 bricks. You must use either bricks or ladders because $2 < 7$.

- Go to building 3 without using ladders nor bricks since $7 \geq 6$.
- Go to building 4 using your only ladder. You must use either bricks or ladders because $6 < 9$.
It is impossible to go beyond building 4 because you do not have any more bricks or ladders.

Example 2:

Input: heights = [4,12,2,7,3,18,20,3,19], bricks = 10, ladders = 2
Output: 7

Example 3:

Input: heights = [14,3,19,3], bricks = 17, ladders = 0
Output: 3

Constraints:

- $1 \leq \text{heights.length} \leq 10^5$
- $1 \leq \text{heights}[i] \leq 10^6$
- $0 \leq \text{bricks} \leq 10^9$
- $0 \leq \text{ladders} \leq \text{heights.length}$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int furthestBuilding(int[] heights, int bricks, int ladders) {
        int lo = 0, hi = heights.length-1;
        while(lo < hi) {
            int mid=lo+(hi-lo+1)/2;
            if(isReachable(heights, mid, bricks, ladders)){
                lo = mid;
            } else {
                hi=mid-1;
            }
        }
        return lo;
    }

    private boolean isReachable(int[] heights, int bIndex, int bricks, int ladders) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        int climb;
        for(int i=0; i < bIndex; i++) {
            climb = heights[i+1]-heights[i];
            if(climb > 0)
                minHeap.add(climb);
        }

        while(!minHeap.isEmpty()) {
            climb = minHeap.poll();
            if(climb <= bricks)
                bricks -= climb;
            else if(ladders > 0)
                ladders--;
            else
                return false;
        }
        return true;
    }
}
```

1126 Minimum Cost to Connect Sticks (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int connectSticks(int[] sticks) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for(int i=0; i < sticks.length; i++)
            minHeap.add(sticks[i]);

        int cost=0,temp;
        while(minHeap.size()>1) {
            temp = minHeap.poll()+minHeap.poll();
            cost+=temp;
            minHeap.add(temp);
        }
        return cost;
    }
}
```

1463 The K Weakest Rows in a Matrix (link)

Description

You are given an $m \times n$ binary matrix `mat` of 1's (representing soldiers) and 0's (representing civilians). The soldiers are positioned **in front** of the civilians. That is, all the 1's will appear to the **left** of all the 0's in each row.

A row i is **weaker** than a row j if one of the following is true:

- The number of soldiers in row i is less than the number of soldiers in row j .
- Both rows have the same number of soldiers and $i < j$.

Return *the indices of the k **weakest** rows in the matrix ordered from weakest to strongest*.

Example 1:

```
Input: mat =
[[1,1,0,0,0],
 [1,1,1,1,0],
 [1,0,0,0,0],
 [1,1,0,0,0],
 [1,1,1,1,1]],
k = 3
Output: [2,0,3]
Explanation:
The number of soldiers in each row is:
- Row 0: 2
- Row 1: 4
- Row 2: 1
- Row 3: 2
- Row 4: 5
The rows ordered from weakest to strongest are [2,0,3,1,4].
```

Example 2:

```
Input: mat =
[[1,0,0,0],
 [1,1,1,1],
 [1,0,0,0],
 [1,0,0,0]],
k = 2
Output: [0,2]
Explanation:
The number of soldiers in each row is:
- Row 0: 1
- Row 1: 4
- Row 2: 1
- Row 3: 1
The rows ordered from weakest to strongest are [0,2,3,1].
```

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $2 \leq n, m \leq 100$
- $1 \leq k \leq m$
- `matrix[i][j]` is either 0 or 1.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] kWeakestRows(int[][] mat, int k) {
        int[] rank = new int[mat.length];
        for (int i = 0; i < mat.length; i++) {
            rank[i] = rank(mat[i]) * 101 + i;
        }

        Arrays.sort(rank);
        int[] result = new int[k];
        for (int i = 0; i < result.length; i++) {
            result[i] = rank[i] % 101;
        }
        return result;
    }

    private final int rank(int[] row) {
        int i = 0;
        for(; i < row.length && row[i] == 1; i++) {
        }
        return i;
    }
}
```

789 Kth Largest Element in a Stream (link)

Description

You are part of a university admissions office and need to keep track of the k^{th} highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k , maintains a stream of test scores and continuously returns the k^{th} highest test score **after** a new score has been submitted. More specifically, we are looking for the k^{th} highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer k and the stream of test scores `nums`.
- `int add(int val)` Adds a new test score `val` to the stream and returns the element representing the k^{th} largest element in the pool of test scores so far.

Example 1:

Input:

```
["KthLargest", "add", "add", "add", "add", "add"]  
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]
```

Output: [null, 4, 5, 5, 8, 8]

Explanation:

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);  
kthLargest.add(3); // return 4  
kthLargest.add(5); // return 5  
kthLargest.add(10); // return 5  
kthLargest.add(9); // return 8  
kthLargest.add(4); // return 8
```

Example 2:

Input:

```
["KthLargest", "add", "add", "add", "add"]  
[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]
```

Output: [null, 7, 7, 7, 8]

Explanation:

```
KthLargest kthLargest = new KthLargest(4, [7, 7, 7, 7, 8, 3]);  
kthLargest.add(2); // return 7  
kthLargest.add(10); // return 7  
kthLargest.add(9); // return 7  
kthLargest.add(9); // return 8
```

Constraints:

- $0 \leq \text{nums.length} \leq 10^4$
- $1 \leq k \leq \text{nums.length} + 1$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

- $-10^4 \leq \text{val} \leq 10^4$
- At most 10^4 calls will be made to `add`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class KthLargest {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>((a, b) -> (a - b));
    int k;

    public KthLargest(int k, int[] nums) {
        this.k = k;
        for(int i=0; i < nums.length;i++)
            add(nums[i]);

        while(minHeap.size() > k)
            minHeap.poll();
    }

    public int add(int val) {
        minHeap.add(val);
        if(minHeap.size()>k)
            minHeap.poll();
        return minHeap.peek();
    }
}

/**
 * Your KthLargest object will be instantiated and called as such:
 * KthLargest obj = new KthLargest(k, nums);
 * int param_1 = obj.add(val);
 */
```

1168 Duplicate Zeros (link)

Description

Given a fixed-length integer array `arr`, duplicate each occurrence of zero, shifting the remaining elements to the right.

Note that elements beyond the length of the original array are not written. Do the above modifications to the input array in place and do not return anything.

Example 1:

Input: `arr = [1,0,2,3,0,4,5,0]`

Output: `[1,0,0,2,3,0,0,4]`

Explanation: After calling your function, the input array is modified to: `[1,0,0,2,3,0,0,4]`

Example 2:

Input: `arr = [1,2,3]`

Output: `[1,2,3]`

Explanation: After calling your function, the input array is modified to: `[1,2,3]`

Constraints:

- $1 \leq \text{arr.length} \leq 10^4$
- $0 \leq \text{arr}[i] \leq 9$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public void duplicateZeros(int[] arr) {
        int dups=0, k = arr.length -1;
        for(int i=0; i <= k-dups;i++) {
            if(arr[i]==0) {
                if(i == k - dups) {
                    arr[k--] = 0;
                    break;
                }
                dups++;
            }
        }

        for(int i=k-dups; i>-1;i--) {
            arr[k--]=arr[i];
            if(arr[i]==0) {
                arr[k--]=0;
            }
        }
    }
}
```

[487 Max Consecutive Ones II \(link\)](#)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findMaxConsecutiveOnes(int[] arr) {
        int maxLength=0, left=0, maxCountOfOnes=0;
        for(int right=0; right < arr.length; right++) {
            maxCountOfOnes+= arr[right];
            if(right-left+1-maxCountOfOnes > 1)
                maxCountOfOnes-=arr[left++];
            maxLength = Math.max(maxLength, right-left+1);
        }
        return maxLength;
    }
}
```

[1137 Height Checker \(link\)](#)

Description

A school is trying to take an annual photo of all the students. The students are asked to stand in a single file line in **non-decreasing order** by height. Let this ordering be represented by the integer array `expected` where `expected[i]` is the expected height of the i^{th} student in line.

You are given an integer array `heights` representing the **current order** that the students are standing in. Each `heights[i]` is the height of the i^{th} student in line (**0-indexed**).

Return *the number of indices where* `heights[i] != expected[i]`.

Example 1:

```
Input: heights = [1,1,4,2,1,3]
Output: 3
Explanation:
heights:  [1,1,4,2,1,3]
expected: [1,1,1,2,3,4]
Indices 2, 4, and 5 do not match.
```

Example 2:

```
Input: heights = [5,1,2,3,4]
Output: 5
Explanation:
heights:  [5,1,2,3,4]
expected: [1,2,3,4,5]
All indices do not match.
```

Example 3:

```
Input: heights = [1,2,3,4,5]
Output: 0
Explanation:
heights:  [1,2,3,4,5]
expected: [1,2,3,4,5]
All indices match.
```

Constraints:

- `1 <= heights.length <= 100`
- `1 <= heights[i] <= 100`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int heightChecker(int[] heights) {
        int count=0, length=heights.length;
        int[] expected = new int[length];
        for(int i=0; i < length;i++)
            expected[i] = heights[i];
        Arrays.sort(expected);

        for(int i=0;i<length;i++)
            if(expected[i] != heights[i])
                count++;

        return count;
    }
}
```

1231 Replace Elements with Greatest Element on Right Side (link)

Description

Given an array `arr`, replace every element in that array with the greatest element among the elements to its right, and replace the last element with `-1`.

After doing so, return the array.

Example 1:

Input: `arr = [17,18,5,4,6,1]`

Output: `[18,6,6,6,1,-1]`

Explanation:

- index 0 --> the greatest element to the right of index 0 is index 1 (18).
- index 1 --> the greatest element to the right of index 1 is index 4 (6).
- index 2 --> the greatest element to the right of index 2 is index 4 (6).
- index 3 --> the greatest element to the right of index 3 is index 4 (6).
- index 4 --> the greatest element to the right of index 4 is index 5 (1).
- index 5 --> there are no elements to the right of index 5, so we put -1.

Example 2:

Input: `arr = [400]`

Output: `[-1]`

Explanation: There are no elements to the right of index 0.

Constraints:

- $1 \leq \text{arr.length} \leq 10^4$
- $1 \leq \text{arr}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {  
    public int[] replaceElements(int[] arr) {  
        int max = -1, right;  
        for(int j=arr.length-1; j>=0; j--) {  
            right = arr[j];  
            arr[j] = max;  
            max = Math.max(max, right);  
        }  
        return arr;  
    }  
}
```

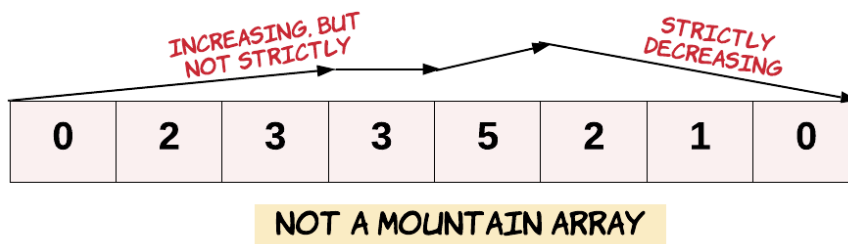
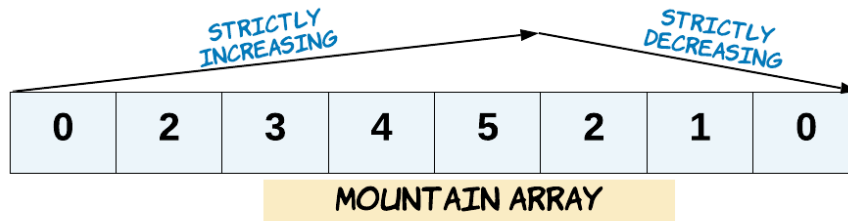
978 Valid Mountain Array (link)

Description

Given an array of integers `arr`, return *true* if and only if it is a valid mountain array.

Recall that `arr` is a mountain array if and only if:

- `arr.length >= 3`
- There exists some `i` with $0 < i < arr.length - 1$ such that:
 - `arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`
 - `arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`



Example 1:

Input: `arr = [2,1]`
Output: `false`

Example 2:

Input: `arr = [3,5,5]`
Output: `false`

Example 3:

Input: `arr = [0,3,2,1]`
Output: `true`

Constraints:

- $1 \leq arr.length \leq 10^4$
- $0 \leq arr[i] \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean validMountainArray(int[] arr) {
        int length = arr.length;
        if(length < 3)
            return false;

        int i=1,j=0;
        while(i < length && arr[j] < arr[i])
            j=i++;

        if(j==0 || i==length)
            return false;

        while(i < length)
            if(arr[i++] >= arr[j++])
                return false;

        return true;
    }
}
```

1468 Check If N and Its Double Exist (link)

Description

Given an array `arr` of integers, check if there exist two indices `i` and `j` such that :

- `i != j`
- `0 <= i, j < arr.length`
- `arr[i] == 2 * arr[j]`

Example 1:

Input: `arr = [10,2,5,3]`

Output: `true`

Explanation: For `i = 0` and `j = 2`, `arr[i] == 10 == 2 * 5 == 2 * arr[j]`

Example 2:

Input: `arr = [3,1,7,11]`

Output: `false`

Explanation: There is no `i` and `j` that satisfy the conditions.

Constraints:

- `2 <= arr.length <= 500`
- `-103 <= arr[i] <= 103`

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean checkIfExist(int[] arr) {
        int half, dbl;
        Set<Integer> set = new HashSet<>();
        for(int i=0; i < arr.length; i++) {
            half = (arr[i]&1)==0?arr[i]>>1:-1001;
            dbl = arr[i] <<1;
            if(set.contains(half) || set.contains(dbl)) {
                return true;
            } else {
                set.add(arr[i]);
            }
        }
        return false;
    }
}
```

1079 Sum of Root To Leaf Binary Numbers (link)

Description

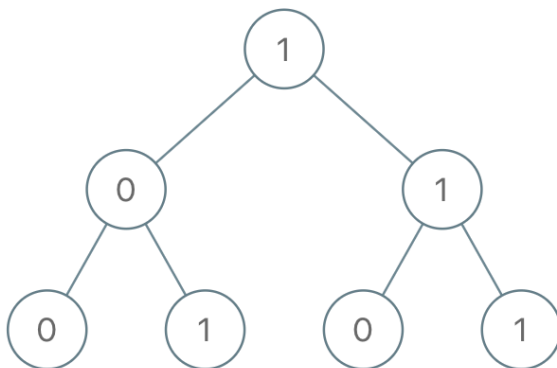
You are given the `root` of a binary tree where each node has a value `0` or `1`. Each root-to-leaf path represents a binary number starting with the most significant bit.

- For example, if the path is `0 -> 1 -> 1 -> 0 -> 1`, then this could represent `01101` in binary, which is `13`.

For all leaves in the tree, consider the numbers represented by the path from the root to that leaf. Return *the sum of these numbers*.

The test cases are generated so that the answer fits in a **32-bits** integer.

Example 1:



Input: `root = [1,0,1,0,1,0,1]`

Output: `22`

Explanation: $(100) + (101) + (110) + (111) = 4 + 5 + 6 + 7 = 22$

Example 2:

Input: `root = [0]`

Output: `0`

Constraints:

- The number of nodes in the tree is in the range $[1, 1000]$.
- `Node.val` is `0` or `1`.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode() {}
 *     TreeNode(int val) { this.val = val; }
 *     TreeNode(int val, TreeNode left, TreeNode right) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
    public int sumRootToLeaf(TreeNode root) {
        if(root == null)
            return 0;

        return sumRootToLeaf(root, 0);
    }

    public int sumRootToLeaf(TreeNode node, int sum) {
        if(node == null)
            return 0;

        sum <<= 1;
        if(node.left == null && node.right == null)
            return sum+node.val;

        return sumRootToLeaf(node.left, sum+node.val)
            + sumRootToLeaf(node.right, sum+node.val);
    }
}
```

252 Meeting Rooms (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        int length = intervals.length;
        Arrays.sort(intervals, (a,b)->(a[0] - b[0]));
        for(int i=1; i<length;i++)
            if(intervals[i-1][1] > intervals[i][0])
                return false;
        return true;
    }
}
```

16 3Sum Closest (link)

Description

Given an integer array `nums` of length `n` and an integer `target`, find three integers in `nums` such that the sum is closest to `target`.

Return *the sum of the three integers*.

You may assume that each input would have exactly one solution.

Example 1:

Input: `nums = [-1,2,1,-4]`, `target = 1`
Output: `2`
Explanation: The sum that is closest to the target is 2. $(-1 + 2 + 1 = 2)$.

Example 2:

Input: `nums = [0,0,0]`, `target = 1`
Output: `0`
Explanation: The sum that is closest to the target is 0. $(0 + 0 + 0 = 0)$.

Constraints:

- $3 \leq \text{nums.length} \leq 500$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^4 \leq \text{target} \leq 10^4$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int threeSumClosest(int[] arr, int target) {
        int closestSum = Integer.MAX_VALUE;
        int smallestDiff = Integer.MAX_VALUE;
        int diff;

        Arrays.sort(arr);
        for (int i = 0; i < arr.length - 2; i++) {
            int left = i + 1, right = arr.length - 1;
            while (left < right) {
                int sum = arr[i] + arr[left] + arr[right];
                diff = Math.abs(target - sum);
                if (diff == 0)
                    return sum;

                // handle smallest difference
                if (diff < smallestDiff) {
                    smallestDiff = diff;
                    closestSum = sum;
                }

                if (sum < target) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return closestSum;
    }
}
```

[1320 Remove All Adjacent Duplicates in String II \(link\)](#)

Description

You are given a string s and an integer k , a k **duplicate removal** consists of choosing k adjacent and equal letters from s and removing them, causing the left and the right side of the deleted substring to concatenate together.

We repeatedly make k **duplicate removals** on s until we no longer can.

Return *the final string after all such duplicate removals have been made*. It is guaranteed that the answer is **unique**.

Example 1:

Input: $s = \text{"abcd"}$, $k = 2$
Output: "abcd"
Explanation: There's nothing to delete.

Example 2:

Input: $s = \text{"deeedbbcccbdaa"}$, $k = 3$
Output: "aa"
Explanation:
First delete "eee" and "ccc" , get "ddbbbdaa"
Then delete "bbb" , get "dddaa"
Finally delete "ddd" , get "aa"

Example 3:

Input: $s = \text{"pbbcggtttciiippooaaais"}$, $k = 2$
Output: "ps"

Constraints:

- $1 \leq s.length \leq 10^5$
- $2 \leq k \leq 10^4$
- s only contains lowercase English letters.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String removeDuplicates(String s, int k) {
        Stack<Integer> st = new Stack();
        StringBuilder sb = new StringBuilder(s);
        int count;
        for(int i=0;i < sb.length();i++) {
            if(i==0 || sb.charAt(i)!=sb.charAt(i-1)) {
                st.push(1);
            } else {
                count = st.pop()+1;
                if(count == k) {
                    sb.delete(i-k+1, i+1);
                    i = i-k;
                } else {
                    st.push(count);
                }
            }
        }
        return sb.toString();
    }
}
```

1078 Remove Outermost Parentheses (link)

Description

A valid parentheses string is either empty `"", "(" + A + "`", or `A + B`, where `A` and `B` are valid parentheses strings, and `+` represents string concatenation.

- For example, `"", "()", "(()())",` and `"(()(()))"` are all valid parentheses strings.

A valid parentheses string `s` is primitive if it is nonempty, and there does not exist a way to split it into `s = A + B`, with `A` and `B` nonempty valid parentheses strings.

Given a valid parentheses string `s`, consider its primitive decomposition: `s = P1 + P2 + ... + Pk`, where `Pi` are primitive valid parentheses strings.

Return `s` *after removing the outermost parentheses of every primitive string in the primitive decomposition of `s`.*

Example 1:

Input: `s = "(()())(())"`

Output: `"()()()`

Explanation:

The input string is `"(()())(())"`, with primitive decomposition `"(()())" + "(())"`.
After removing outer parentheses of each part, this is `"()()" + "()" = "()()()`.

Example 2:

Input: `s = "(()())((()))(())(())(())"`

Output: `"()()()()()()"`

Explanation:

The input string is `"(()())((()))(())(())(())"`, with primitive decomposition `"(()())" + "((()))" + "(()())"`.
After removing outer parentheses of each part, this is `"()()" + "()" + "()(())" = "()()()()()"`.

Example 3:

Input: `s = "()()"`

Output: `""`

Explanation:

The input string is `"()()"`, with primitive decomposition `"()" + "()"`.
After removing outer parentheses of each part, this is `"" + "" = ""`.

Constraints:

- `1 <= s.length <= 105`
- `s[i]` is either `'('` or `')'`.
- `s` is a valid parentheses string.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String removeOuterParentheses(String S) {
        StringBuilder builder = new StringBuilder();
        int count=0;
        char ch;
        for(int i=0;i<S.length();i++) {
            ch = S.charAt(i);
            if(count==0) {
                count++;
            } else if(ch==')' && count==1) {
                count=0;
            } else {
                count+=ch=='(' ? 1: -1;
                builder.append(ch);
            }
        }
        return builder.toString();
    }
}
```

957 Minimum Add to Make Parentheses Valid (link)

Description

A parentheses string is valid if and only if:

- It is the empty string,
- It can be written as AB (A concatenated with B), where A and B are valid strings, or
- It can be written as (A) , where A is a valid string.

You are given a parentheses string s . In one move, you can insert a parenthesis at any position of the string.

- For example, if $s = "())"$, you can insert an opening parenthesis to be $"(())"$ or a closing parenthesis to be $"(())"$.

Return *the minimum number of moves required to make s valid*.

Example 1:

Input: $s = "())"$
Output: 1

Example 2:

Input: $s = "((("$
Output: 3

Constraints:

- $1 \leq s.length \leq 1000$
- $s[i]$ is either '(' or ')'

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int minAddToMakeValid(String S) {
        int ubal = 0, bal = 0;
        for (int i = 0; i < S.length(); ++i) {
            bal += S.charAt(i) == '(' ? 1 : -1;
            if (bal == -1) {
                ubal++;
                bal=0;
            }
        }
        return ubal + bal;
    }
}
```

682 Baseball Game (link)

Description

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings `operations`, where `operations[i]` is the i^{th} operation you must apply to the record and is one of the following:

- An integer x .
 - Record a new score of x .
- `'+'`.
 - Record a new score that is the sum of the previous two scores.
- `'D'`.
 - Record a new score that is the double of the previous score.
- `'C'`.
 - Invalidate the previous score, removing it from the record.

Return *the sum of all the scores on the record after applying all the operations*.

The test cases are generated such that the answer and all intermediate calculations fit in a **32-bit** integer and that all operations are valid.

Example 1:

```
Input: ops = ["5","2","C","D","+"]
Output: 30
Explanation:
"5" - Add 5 to the record, record is now [5].
"2" - Add 2 to the record, record is now [5, 2].
"C" - Invalidate and remove the previous score, record is now [5].
"D" - Add 2 * 5 = 10 to the record, record is now [5, 10].
"+" - Add 5 + 10 = 15 to the record, record is now [5, 10, 15].
The total sum is 5 + 10 + 15 = 30.
```

Example 2:

```
Input: ops = ["5","-2","4","C","D","9","+","+"]
Output: 27
Explanation:
"5" - Add 5 to the record, record is now [5].
"-2" - Add -2 to the record, record is now [5, -2].
"4" - Add 4 to the record, record is now [5, -2, 4].
"C" - Invalidate and remove the previous score, record is now [5, -2].
"D" - Add 2 * -2 = -4 to the record, record is now [5, -2, -4].
"9" - Add 9 to the record, record is now [5, -2, -4, 9].
"+" - Add -4 + 9 = 5 to the record, record is now [5, -2, -4, 9, 5].
"+" - Add 9 + 5 = 14 to the record, record is now [5, -2, -4, 9, 5, 14].
The total sum is 5 + -2 + -4 + 9 + 5 + 14 = 27.
```

Example 3:

```
Input: ops = ["1","C"]
Output: 0
Explanation:
"1" - Add 1 to the record, record is now [1].
"C" - Invalidate and remove the previous score, record is now [].
Since the record is empty, the total sum is 0.
```

Constraints:

- $1 \leq \text{operations.length} \leq 1000$
- `operations[i]` is "C", "D", "+", or a string representing an integer in the range $[-3 * 10^4, 3 * 10^4]$.
- For operation "+", there will always be at least two previous scores on the record.
- For operations "C" and "D", there will always be at least one previous score on the record.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int calPoints(String[] ops) {
        Stack<Integer> stack = new Stack<>();
        int last,curr,result=0;
        for(String op:ops) {
            switch(op) {
                case "C":
                    stack.pop();
                    break;
                case "D":
                    stack.push(stack.peek()*2);
                    break;
                case "+":
                    last=stack.pop();
                    curr=stack.peek()+last;
                    stack.push(last);
                    stack.push(curr);
                    break;
                default:
                    stack.add(Integer.parseInt(op));
                    break;
            }
        }

        for(int score:stack) {
            result+=score;
        }

        return result;
    }
}
```


[496 Next Greater Element I \(link\)](#)

Description

The **next greater element** of some element x in an array is the **first greater** element that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each $0 \leq i < \text{nums1.length}$, find the index j such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return *an array* `ans` *of length* `nums1.length` *such that* `ans[i]` *is the next greater element as described above*.

Example 1:

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.
- 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is `3`.
- 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.

Example 2:

Input: `nums1 = [2,4]`, `nums2 = [1,2,3,4]`

Output: `[3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 2 is underlined in `nums2 = [1,2,3,4]`. The next greater element is `3`.
- 4 is underlined in `nums2 = [1,2,3,4]`. There is no next greater element, so the answer is `-1`.

Constraints:

- $1 \leq \text{nums1.length} \leq \text{nums2.length} \leq 1000$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^4$
- All integers in `nums1` and `nums2` are **unique**.
- All the integers of `nums1` also appear in `nums2`.

Follow up: Could you find an $O(\text{nums1.length} + \text{nums2.length})$ solution?

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int[] nextGreaterElement(int[] findNums, int[] nums) {
        Map<Integer, Integer> map = new HashMap<>();
        Stack<Integer> stack = new Stack<>();

        for(int i=0; i < nums.length; i++) {
            while(!stack.isEmpty() && nums[i]>stack.peek()) {
                map.put(stack.pop(), nums[i]);
            }
            stack.push(nums[i]);
        }
        while(!stack.isEmpty()) {
            map.put(stack.pop(), -1);
        }

        for(int i=0; i < findNums.length;i++) {
            findNums[i] = map.get(findNums[i]);
        }
        return findNums;
    }
}
```

785 Basic Calculator III (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public static String OPERATORS="+-*/";
    public int calculate(String s) {
        //convert to postfix
        Stack<String> stack=getTokens(s);
        //evaluate postfix
        return stack.isEmpty()?0:evaluate(stack);
    }

    private Stack<String> getTokens(String s) {
        int j=0,len = s.length();
        Stack<String> tokens = new Stack<String>();
        Stack<Character> stack = new Stack<Character>();

        char ch;
        Integer tmp = null;
        for(int i=0; i<len;i++) {
            ch = s.charAt(i);
            if(Character.isDigit(ch)) {
                tmp=(tmp==null?0:tmp*10)+(ch-'0');
            }
            else {
                if(tmp != null) {
                    tokens.push(tmp+"");
                    tmp = null;
                }
                if('('== ch) {
                    stack.push(ch);
                } else if(')'== ch) {
                    while(!stack.isEmpty() && stack.peek()!='(') {
                        tokens.push(stack.pop()+"");
                    }
                    stack.pop();
                } else if(OPERATORS.indexOf(ch)>-1) {
                    int chrPrecedence=precedence(ch);
                    while(!stack.isEmpty() && precedence(stack.peek())>=chrPrecedence) {
                        tokens.push(stack.pop()+"");
                    }
                    stack.push(ch);
                }
            }
        }

        if(tmp!=null) {
            tokens.push(tmp+"");
        }
        while(!stack.isEmpty()) {
            tokens.push(stack.pop()+"");
        }
        return tokens;
    }

    private int evaluate(Stack<String> tokens) {
        Stack<Integer> stack = new Stack<Integer>();
        int tmp;
        for(String token: tokens) {
            if(OPERATORS.contains(token)) {
                int op2 = stack.pop();
                int op1 = stack.pop();
                tmp = operate(op1,op2, token);
            } else {
                tmp = Integer.parseInt(token);
            }
            stack.push(tmp);
        }
    }
}
```

```
        return stack.pop();
    }

    private int precedence(char ch) {
        switch (ch) {
            case '+':
            case '-':
                return 1;
            case '*':
            case '/':
                return 2;
            default:
                return -1;
        }
    }

    private int operate(int op1, int op2, String op) {
        switch(op) {
            case "+":
                return op1+op2;
            case "-":
                return op1-op2;
            case "*":
                return op1*op2;
            case "/":
                return op1/op2;
            default:
                return 0;
        }
    }
}
```

[150 Evaluate Reverse Polish Notation \(link\)](#)

Description

You are given an array of strings `tokens` that represents an arithmetic expression in a [Reverse Polish Notation](#).

Evaluate the expression. Return *an integer that represents the value of the expression*.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

Example 1:

Input: `tokens = ["2","1","+","3","*"]`
Output: 9
Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: `tokens = ["4","13","5","/","+"]`
Output: 6
Explanation: $(4 + (13 / 5)) = 6$

Example 3:

Input: `tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]`
Output: 22
Explanation: $((10 * (6 / ((9 + 3) * -11))) + 17) + 5$
 $= ((10 * (6 / (12 * -11))) + 17) + 5$
 $= ((10 * (6 / -132)) + 17) + 5$
 $= ((10 * 0) + 17) + 5$
 $= (0 + 17) + 5$
 $= 17 + 5$
 $= 22$

Constraints:

- $1 \leq \text{tokens.length} \leq 10^4$
- `tokens[i]` is either an operator: "+", "-", "*", or "/", or an integer in the range $[-200, 200]$.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int evalRPN(String[] tokens) {
        Stack<Integer> stack = new Stack<Integer>();
        int tmp;
        for(String token: tokens) {
            if("+*/".contains(token)) {
                int op2 = stack.pop();
                int op1 = stack.pop();
                tmp = operate(op1,op2, token);
            } else {
                tmp = Integer.parseInt(token);
            }
            stack.push(tmp);
        }
        return stack.pop();
    }

    private int operate(int op1, int op2, String op) {
        switch(op) {
            case "+":
                return op1+op2;
            case "-":
                return op1-op2;
            case "*":
                return op1*op2;
            case "/":
                return op1/op2;
            default:
                return 0;
        }
    }
}
```

[71 Simplify Path \(link\)](#)

Description

You are given an *absolute* path for a Unix-style file system, which always begins with a slash '/'. Your task is to transform this absolute path into its **simplified canonical path**.

The *rules* of a Unix-style file system are as follows:

- A single period '.' represents the current directory.
- A double period '..' represents the previous/parent directory.
- Multiple consecutive slashes such as '//' and '///' are treated as a single slash '/'.
- Any sequence of periods that does **not match** the rules above should be treated as a **valid directory or file name**. For example, '...' and '....' are valid directory or file names.

The simplified canonical path should follow these *rules*:

- The path must start with a single slash '/'.
- Directories within the path must be separated by exactly one slash '/'.
- The path must not end with a slash '/', unless it is the root directory.
- The path must not have any single or double periods ('.' and '..') used to denote current or parent directories.

Return the **simplified canonical path**.

Example 1:

Input: path = "/home/"

Output: "/home"

Explanation:

The trailing slash should be removed.

Example 2:

Input: path = "/home//foo/"

Output: "/home/foo"

Explanation:

Multiple consecutive slashes are replaced by a single one.

Example 3:

Input: path = "/home/user/Documents/../Pictures"

Output: "/home/user/Pictures"

Explanation:

A double period ".." refers to the directory up a level (the parent directory).

Example 4:

Input: path = "/../"

Output: "/"

Explanation:

Going one level up from the root directory is not possible.

Example 5:

Input: path = "/.../a/.../b/c/.../d/..."

Output: "/.../b/d"

Explanation:

"..." is a valid name for a directory in this problem.

Constraints:

- $1 \leq \text{path.length} \leq 3000$
- path consists of English letters, digits, period '.', slash '/' or '_'.
- path is a valid absolute Unix path.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public String simplifyPath(String path) {
        String DOT = ".";
        String DDOT = "..";
        String SLASH = "/";
        String EMPTY="";
        Stack<String> stack = new Stack<String>();
        String[] PARTS = path.split("/");

        for(String dir: PARTS) {
            if(dir.isEmpty() || DOT.equals(dir)) {
                continue;
            } else if(DDOT.equals(dir)) {
                if(!stack.isEmpty()) {
                    stack.pop();
                }
            } else {
                stack.push(dir);
            }
        }

        StringBuilder builder = new StringBuilder("");
        for(String dir: stack) {
            builder.append("/").append(dir);
        }
        return builder.length()==0? SLASH: builder.toString();
    }
}
```

237 Delete Node in a Linked List (link)

Description

There is a singly-linked list `head` and we want to delete a node `node` in it.

You are given the node to be deleted `node`. You will **not be given access** to the first node of `head`.

All the values of the linked list are **unique**, and it is guaranteed that the given node `node` is not the last node in the linked list.

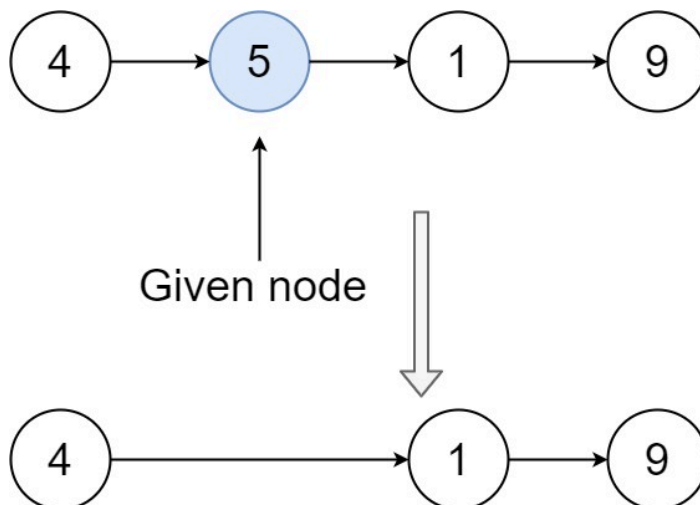
Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before `node` should be in the same order.
- All the values after `node` should be in the same order.

Custom testing:

- For the input, you should provide the entire linked list `head` and the node to be given `node`. `node` should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

Example 1:

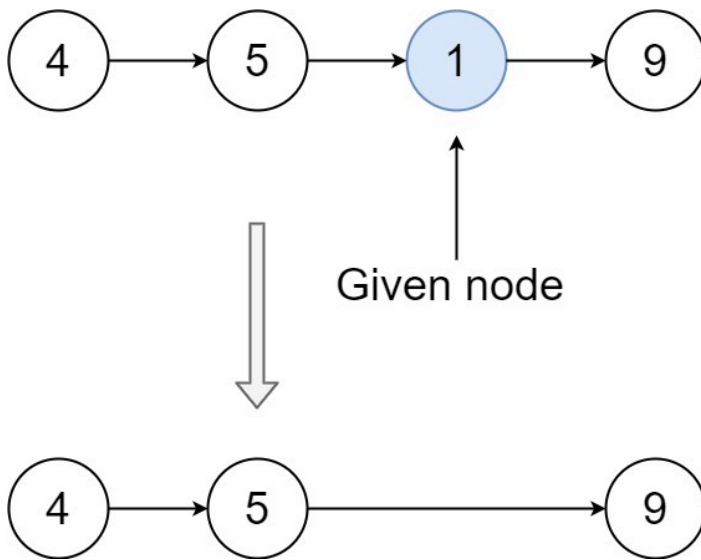


Input: `head = [4,5,1,9]`, `node = 5`

Output: `[4,1,9]`

Explanation: You are given the second node with value 5, the linked list should become `4 -> 1 -> 9` after

Example 2:



Input: head = [4,5,1,9], node = 1

Output: [4,5,9]

Explanation: You are given the third node with value 1, the linked list should become 4 -> 5 -> 9 after

Constraints:

- The number of the nodes in the given list is in the range [2, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$
- The value of each node in the list is **unique**.
- The node to be deleted is **in the list** and is **not a tail** node.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {
    public void deleteNode(ListNode node) {
        ListNode prev = null;
        while(node.next != null) {
            node.val = node.next.val;
            prev = node;
            node = node.next;
        }
        prev.next = null;
    }
}
```

1404 Print Immutable Linked List in Reverse (link)

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * // This is the ImmutableListNode's API interface.
 * // You should not implement it, or speculate about its implementation.
 * interface ImmutableListNode {
 *     public void printValue(); // print the value of this node.
 *     public ImmutableListNode getNext(); // return the next node.
 * };
 */

class Solution {
    public void printLinkedListInReverse(ImmutableListNode head) {
        int size = 0;
        ImmutableListNode node = head;
        while(node != null) {
            size++;
            node = node.getNext();
        }

        int counter = 0, block = (int) Math.ceil(Math.sqrt(size));
        Stack<ImmutableListNode> blockStart = new Stack<ImmutableListNode>();
        node = head;
        while(node != null) {
            if(counter % block == 0) {
                blockStart.push(node);
            }
            node = node.getNext();
            counter++;
        }

        ImmutableListNode start = null, end, temp;
        Stack<ImmutableListNode> stackValues = new Stack<ImmutableListNode>();
        while(!blockStart.isEmpty()) {
            end = start;
            start = blockStart.pop();
            temp = start;

            while(temp != end) {
                stackValues.push(temp);
                temp = temp.getNext();
            }

            while(!stackValues.isEmpty()) {
                stackValues.pop().printValue();
            }
        }
    }
}
```

1892 Find Total Time Spent by Each Employee (link)

Description

Table: Employees

Column Name	Type
emp_id	int
event_day	date
in_time	int
out_time	int

(emp_id, event_day, in_time) is the primary key (combinations of columns with unique values) of this table. The table shows the employees' entries and exits in an office. event_day is the day at which this event happened, in_time is the minute at which the employee entered the office, and out_time is the minute at which the employee exited the office. It is guaranteed that no two events on the same day intersect in time, and $in_time < out_time$.

Write a solution to calculate the total time **in minutes** spent by each employee on each day at the office. Note that within one day, an employee can enter and leave more than once. The time spent in the office for a single entry is $out_time - in_time$.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Employees table:

emp_id	event_day	in_time	out_time
1	2020-11-28	4	32
1	2020-11-28	55	200
1	2020-12-03	1	42
2	2020-11-28	3	33
2	2020-12-09	47	74

Output:

day	emp_id	total_time
2020-11-28	1	173
2020-11-28	2	30
2020-12-03	1	41
2020-12-09	2	27

Explanation:

Employee 1 has three events: two on day 2020-11-28 with a total of $(32 - 4) + (200 - 55) = 173$, and one on day 2020-12-03 with a total of $42 - 1 = 41$. Employee 2 has two events: one on day 2020-11-28 with a total of $33 - 3 = 30$, and one on day 2020-12-09 with a total of $74 - 47 = 27$.

(scroll down for solution)

Solution

Language: mysql

Status: Accepted

```
select event_day day, emp_id, sum(out_time - in_time) total_time
from Employees
group by event_day, emp_id
```

[1908 Recyclable and Low Fat Products \(link\)](#)

Description

Table: Products

Column Name	Type
product_id	int
low_fats	enum
recyclable	enum

product_id is the primary key (column with unique values) for this table.

low_fats is an ENUM (category) of type ('Y', 'N') where 'Y' means this product is low fat and 'N' means

recyclable is an ENUM (category) of types ('Y', 'N') where 'Y' means this product is recyclable and 'N'

Write a solution to find the ids of products that are both low fat and recyclable.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Products table:

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

Output:

product_id
1
3

Explanation: Only products 1 and 3 are both low fat and recyclable.

(scroll down for solution)

Solution

Language: mysql

Status: Accepted

```
# Write your MySQL query statement below
select product_id from Products where low_fats='Y' and recyclable='Y'
```

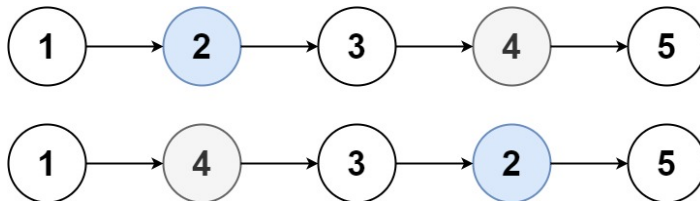
528 Swapping Nodes in a Linked List (link)

Description

You are given the head of a linked list, and an integer k .

Return the head of the linked list after **swapping** the values of the k^{th} node from the beginning and the k^{th} node from the end (the list is **1-indexed**).

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [1,4,3,2,5]

Example 2:

Input: head = [7,9,6,6,7,8,3,0,9,5], k = 5
Output: [7,9,6,6,8,7,3,0,9,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 10^5$
- $0 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode swapNodes(ListNode head, int k) {
        if(head.next == null)
            return head;

        ListNode tmp = head;
        while(k-- > 1) {
            tmp = tmp.next;
        }

        ListNode fromStart = tmp, fromEnd = head;
        while (tmp.next != null) {
            fromEnd = fromEnd.next;
            tmp = tmp.next;
        }

        if(fromEnd != fromStart) {
            int temp = fromEnd.val;
            fromEnd.val = fromStart.val;
            fromStart.val = temp;
        }
        return head;
    }
}
```

1421 Find Numbers with Even Number of Digits (link)

Description

Given an array `nums` of integers, return how many of them contain an **even number** of digits.

Example 1:

```
Input: nums = [12,345,2,6,7896]
Output: 2
Explanation:
12 contains 2 digits (even number of digits).
345 contains 3 digits (odd number of digits).
2 contains 1 digit (odd number of digits).
6 contains 1 digit (odd number of digits).
7896 contains 4 digits (even number of digits).
Therefore only 12 and 7896 contain an even number of digits.
```

Example 2:

```
Input: nums = [555,901,482,1771]
Output: 1
Explanation:
Only 1771 contains an even number of digits.
```

Constraints:

- $1 \leq \text{nums.length} \leq 500$
- $1 \leq \text{nums}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findNumbers(int[] nums) {
        int count = 0;
        for(int num: nums) {
            int noOfDigit = 1;
            while((num=num/10) != 0) ++noOfDigit;
            count+= noOfDigit%2 == 0 ? 1:0;
        }
        return count;
    }
}
```

485 Max Consecutive Ones [\(link\)](#)

Description

Given a binary array `nums`, return *the maximum number of consecutive 1's in the array*.

Example 1:

Input: `nums = [1,1,0,1,1,1]`

Output: 3

Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.

Example 2:

Input: `nums = [1,0,1,1,0,1]`

Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- `nums[i]` is either 0 or 1.

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int findMaxConsecutiveOnes(int[] nums) {
        int count = 0, max = 0;
        for(int i=0; i< nums.length;i++) {
            if(nums[i]==1) {
                count++;
            } else {
                max = Math.max(count, max);
                count=0;
            }
        }
        return Math.max(count, max);
    }
}
```

850 Insert into a Sorted Circular Linked List ([link](#))

Description

null

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
/*
// Definition for a Node.
class Node {
    public int val;
    public Node next;

    public Node() {}

    public Node(int _val) {
        val = _val;
    }

    public Node(int _val, Node _next) {
        val = _val;
        next = _next;
    }
};
*/

class Solution {
    public Node insert(Node head, int val) {
        if (head == null || head.next == head) {
            Node temp = new Node(val);
            if (head == null) {
                head = temp;
            } else {
                head.next = temp;
            }
            temp.next = head;
            return head;
        }

        Node prev, node = head;
        do {
            if (node.val <= val && node.next.val >= val) {
                node.next = new Node(val, node.next);
                return head;
            }

            if (node.val > node.next.val) {
                Node high = node, low = node.next;
                if (val >= high.val || val <= low.val) {
                    high.next = new Node(val, low);
                    return head;
                }
            }
            prev = node;
            node = node.next;
        } while (node != head);

        // implies all node values are same
        // insert at end
        prev.next = new Node(val, node);

        return head;
    }
}
```

[7 Reverse Integer \(link\)](#)

Description

Given a signed 32-bit integer x , return x *with its digits reversed*. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$
Output: 321

Example 2:

Input: $x = -123$
Output: -321

Example 3:

Input: $x = 120$
Output: 21

Constraints:

- $-2^{31} \leq x \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: java

Status: Accepted

```
class Solution {
    public int reverse(int x) {
        int maxBy10 = Integer.MAX_VALUE / 10;
        int minBy10 = Integer.MIN_VALUE / 10;

        int rev = 0;
        int rem = 0;
        while (x != 0) {
            rem = x % 10;
            x /= 10;
            if (rev > maxBy10 || (rev == maxBy10 && rem > 7) || rev < minBy10 || (rev == minBy10 && rem < -8))
                return 0;
            else {
                rev = rem + rev * 10;
            }
        }
        return rev;
    }
}
```